GitHub完全ガイド

はじめてのバージョン管理からチーム開発まで

Rhizobium-gits

すべての初学者へ

プログラミングの世界への第一歩を踏み出すあなたに

Author's Note: 本書は、Git と GitHub の全操作・全機能を日本語で網羅した完全リファレンスガイドです。プログラミングを始めたばかりの方から、チームで本格的に開発を始めたい方まで、これ 1 冊で GitHub の全てが分かることを目指しました。用語の意味から丁寧に解説し、なぜその操作が必要なのかという文脈を重視しています。

Publisher: GitHub Complete Guide Project
First edition, published in 2026

本書の作成にあたり、Claude Code の支援に感謝します

Copyright 2026
本作品が採用しているのは Creative Commons "表示-非営利-継承 4.0 国際" 利用許諾です.

目次

1 Git とは何か — バージョン管理の世界へようこそ 4

1.1 この章で学ぶこと . . . 4
1.2 バージョン管理とは何か . . . 4
1.2.1 ファイル管理の悩み . . . 4
1.2.2 バージョン管理システムという解決策 . . . 4
1.2.3 プログラミングにおけるバージョン管理 . . . 5
1.3 Git とは . . . 5
1.3.1 Git の基本的な特徴 . . . 5
1.3.2 Git が扱う基本概念 . . . 5
1.4 GitHub とは . . . 7
1.4.1 Git と GitHub の違い . . . 7
1.4.2 GitHub が提供する付加価値 . . . 8
1.5 Git の歴史 — なぜ Git は生まれたのか . . . 8
1.5.1 Linux カーネル開発と BitKeeper . . . 8
1.5.2 10 日間で作られたバージョン管理システム . . . 8
1.5.3 バージョン管理の進化の歴史 . . . 9
1.6 GitHub の歴史 — ソフトウェア開発を変えた革命 . . . 10
1.6.1 GitHub の誕生と成長 . . . 10
1.6.2 GitHub が世界にもたらした変革 . . . 10
1.6.3 GitHub の競合サービス . . . 11
1.7 Git/GitHub を使うメリット . . . 11
1.8 まとめ . . . 12

2 環境構築 — Git を自分のパソコンに導入する 13

2.1 この章で学ぶこと . . . 13
2.2 Git のインストール . . . 13
2.2.1 macOS の場合 . . . 13
2.2.2 Windows の場合 . . . 14
2.2.3 Linux（Ubuntu/Debian 系）の場合 . . . 15
2.2.4 Linux（Fedora/RHEL 系）の場合 . . . 15
2.3 インストールの確認 . . . 16
2.4 初期設定の意味と方法 . . . 16
2.4.1 なぜ名前とメールアドレスが必要なのか . . . 16
2.4.2 設定コマンド . . . 17
2.4.3 `--global` と `--local` の違い . . . 17
2.5 プライバシー保護：noreply メールアドレス . . . 17
2.5.1 GitHub の「noreply メールアドレス」機能 . . . 18
2.6 追加の推奨設定 . . . 18
2.6.1 デフォルトブランチ名の設定 . . . 18
2.6.2 エディタの設定 . . . 19
2.6.3 .DS_Store の無視（macOS のみ） . . . 19
2.6.4 カラー表示の有効化 . . . 19
2.6.5 プッシュのデフォルト動作 . . . 20
2.6.6 プル時のリベース設定 . . . 20
2.7 設定の確認と設定ファイルの階層 . . . 20

2.7.1 現在の設定を確認する . . . 20
2.7.2 設定ファイルの 3 つの階層 . . . 21
2.8 エディタとターミナルの準備 . . . 22
2.8.1 Visual Studio Code（VS Code）の導入 . . . 22
2.8.2 おすすめの VS Code 拡張機能 . . . 22
2.8.3 ターミナルの選び方 . . . 23
2.9 まとめ . . . 23

3 GitHub アカウント — 開発者としての第一歩 25
3.1 この章で学ぶこと . . . 25
3.2 アカウント作成の手順 . . . 25
3.3 ユーザー名の選び方 . . . 26
3.3.1 なぜユーザー名が重要なのか . . . 26
3.3.2 選び方のヒント . . . 27
3.3.3 ユーザー名は後から変更できるか . . . 27
3.4 二段階認証（2FA）— なぜ必要か . . . 27
3.4.1 二段階認証とは何か . . . 28
3.4.2 設定手順 . . . 28
3.4.3 推奨認証アプリ . . . 28
3.4.4 リカバリーコードの重要性 . . . 29
3.5 プロフィール設定 — 基本情報 . . . 29
3.5.1 基本プロフィールの設定 . . . 29
3.5.2 プロフィール写真の設定 . . . 30
3.6 プロフィール README — 自分だけのページを作る . . . 30
3.6.1 作り方 . . . 30
3.6.2 書き方の例 . . . 31
3.6.3 よく使われるバッジとウィジェット . . . 31
3.7 GitHub の料金プラン . . . 32
3.7.1 各プランの概要 . . . 32
3.7.2 GitHub Education（学生・教育者向け） . . . 33
3.8 通知設定のカスタマイズ . . . 33
3.8.1 通知設定の画面 . . . 33
3.8.2 おすすめの設定 . . . 33
3.9 Achievements（実績バッジ）— GitHub が認める功績 . . . 34
3.9.1 Achievements とは . . . 34
3.9.2 Achievements の表示設定 . . . 34
3.9.3 現在獲得できる Achievements 一覧 . . . 34
3.9.4 引退した Achievements（もう獲得できないバッジ） . . . 36
3.9.5 Highlights（ハイライト） . . . 37
3.10 コントリビューショングラフ（草）— 活動の可視化 . . . 37
3.10.1 「草」とは何か . . . 37
3.10.2 コントリビューションとしてカウントされる活動 . . . 37
3.10.3 コントリビューションの見せ方 . . . 38
3.11 まとめ . . . 38

4 認証と接続 — パソコンと GitHub を安全につなぐ 40
4.1 この章で学ぶこと . . . 40

4.2 なぜ認証が必要なのか . . . 40
4.3 接続方式の比較 . . . 41
4.4 SSH 鍵による接続（推奨） . . . 41
4.4.1 公開鍵暗号の仕組み — 南京錠のたとえ . . . 41
4.4.2 SSH 鍵の生成 . . . 42
4.4.3 SSH エージェントに鍵を登録する . . . 43
4.4.4 公開鍵を GitHub に登録する . . . 44
4.4.5 接続テスト . . . 44
4.4.6 SSH 接続でリポジトリを操作する . . . 45
4.5 HTTPS + Personal Access Token . . . 45
4.5.1 Personal Access Token（PAT）とは . . . 46
4.5.2 PAT の作成手順 . . . 46
4.5.3 スコープの選び方 . . . 46
4.5.4 PAT を使ったリポジトリ操作 . . . 47
4.5.5 認証情報の保存（毎回入力しなくて済むように） . . . 47
4.5.6 Fine-grained PAT（きめ細かい PAT） . . . 47
4.6 GitHub CLI . . . 48
4.6.1 インストール . . . 48
4.6.2 認証の設定 . . . 48
4.6.3 認証状態の確認 . . . 49
4.7 GPG 署名 — コミットの本人証明 . . . 50
4.7.1 なぜ署名が必要なのか . . . 50
4.7.2 GPG 鍵の生成 . . . 50
4.7.3 GitHub に GPG 鍵を登録する . . . 51
4.7.4 Git で署名を有効化する . . . 52
4.8 複数アカウントの管理 . . . 52
4.8.1 SSH config によるアカウントの使い分け . . . 52
4.8.2 使い方 . . . 53
4.8.3 リポジトリごとのユーザー情報の切り替え . . . 54
4.9 まとめ . . . 54

5 リポジトリ — プロジェクトの器を作る 56
5.1 この章で学ぶこと . . . 56
5.2 リポジトリの構造 — .git フォルダの役割 . . . 56
5.2.1 プロジェクトフォルダとリポジトリの関係 . . . 56
5.2.2 .git フォルダの中身 . . . 57
5.3 リポジトリの作成方法 . . . 57
5.3.1 方法 1: GitHub で作成してからクローン（推奨） . . . 57
5.3.2 方法 2: ローカルで作成してから GitHub にプッシュ . . . 59
5.3.3 方法 3: GitHub CLI で一発作成 . . . 60
5.4 クローン（clone）を深く理解する . . . 61
5.4.1 clone が行うこと . . . 61
5.4.2 clone の主要オプション . . . 61
5.5 リモート（remote）の管理 . . . 62
5.5.1 origin とは何か . . . 62
5.5.2 upstream とは何か . . . 63
5.5.3 リモートの追加・変更・削除 . . . 63

5.6 .gitignore の書き方と考え方 . . . 64
5.6.1 なぜファイルの無視が必要なのか . . . 64
5.6.2 .gitignore の書式ルール . . . 64
5.6.3 言語別テンプレート . . . 65
5.6.4 既に追跡されたファイルへの対処 . . . 67
5.7 リポジトリの設定 . . . 67
5.7.1 GitHub 上の Settings . . . 67
5.7.2 テンプレートリポジトリ . . . 69
5.7.3 リポジトリの削除 . . . 69
5.8 まとめ . . . 70

6 基本ワークフロー — 変更を記録し共有する 71
6.1 この章で学ぶこと . . . 71
6.2 Git の 3 つのエリアを理解する . . . 71
6.2.1 買い物にたとえて理解する . . . 72
6.2.2 なぜステージングエリアが必要なのか . . . 72
6.3 git status — 現在の状態を把握する . . . 73
6.3.1 出力の読み方 . . . 73
6.3.2 短縮表示 . . . 74
6.4 git add — ステージングの操作と考え方 . . . 74
6.4.1 基本的な使い方 . . . 74
6.4.2 なぜ全部 add せず選ぶのか . . . 75
6.4.3 -p オプション：ファイルの一部だけをステージングする . . . 75
6.4.4 ステージングの取り消し . . . 75
6.5 git commit — 変更を記録する . . . 76
6.5.1 基本的な使い方 . . . 76
6.5.2 コミットメッセージの書き方の哲学 . . . 76
6.5.3 Conventional Commits . . . 77
6.5.4 –amend：直前のコミットを修正する . . . 78
6.6 git push — ローカルの記録を GitHub に送信する . . . 78
6.6.1 基本的な使い方 . . . 78
6.6.2 -u オプション（–set-upstream）の意味 . . . 79
6.6.3 プッシュできない場合の対処 . . . 79
6.7 git pull — GitHub の変更をローカルに取り込む . . . 80
6.7.1 基本的な使い方 . . . 80
6.7.2 pull = fetch + merge . . . 80
6.7.3 fetch だけを使う理由 . . . 80
6.8 git log — 履歴を確認する . . . 81
6.8.1 基本的な使い方と各オプション . . . 81
6.8.2 きれいなログ表示のエイリアス . . . 82
6.9 git diff — 差分を確認する . . . 83
6.9.1 4 つの差分比較パターン . . . 83
6.9.2 便利なオプション . . . 84
6.10 ファイルの移動・削除・変更取り消し . . . 84
6.10.1 ファイルの移動・名前変更（git mv） . . . 84
6.10.2 ファイルの削除（git rm） . . . 84
6.10.3 変更の取り消し . . . 85

6.11 実践：一連の開発フローを体験する 85
6.12 まとめ 87

7 ブランチ — 並行開発の技法 88
7.1 この章で学ぶこと 88
7.2 ブランチとは何か 88
7.2.1 ポインタとしてのブランチ 88
7.2.2 HEAD とは何か 89
7.2.3 なぜブランチが必要なのか 89
7.3 ブランチの基本操作 90
7.3.1 ブランチの一覧表示 90
7.3.2 ブランチの作成 91
7.3.3 ブランチの切り替え（switch vs checkout） 91
7.3.4 ブランチの削除 92
7.3.5 ブランチ名の変更 92
7.4 ブランチの命名規則 93
7.4.1 プレフィックスによる分類 93
7.4.2 命名のベストプラクティス 93
7.5 マージ（merge） 94
7.5.1 Fast-forward マージ 94
7.5.2 3-way マージ 95
7.5.3 マージの取り消し 95
7.6 リベース（rebase） 96
7.6.1 リベースとは何をしているのか 96
7.6.2 merge vs rebase の判断基準 97
7.6.3 「push 済みには rebase しない」ルール 97
7.7 ブランチ戦略 98
7.7.1 GitHub Flow — シンプルで始めやすい 98
7.7.2 Git Flow — 大規模プロジェクト向けの体系的な戦略 99
7.7.3 Trunk-based Development — 高速なイテレーション 99
7.7.4 戦略の選び方 100
7.8 実践：ブランチを使った開発の一連の流れ 100
7.9 まとめ 101

8 コンフリクト — 変更の衝突を解決する 103
8.1 この章で学ぶこと 103
8.2 コンフリクトとは何か 103
8.2.1 日常のたとえで理解する 103
8.2.2 Git におけるコンフリクトの仕組み 103
8.2.3 コンフリクトは「エラー」ではない 104
8.3 コンフリクトが発生する場面 104
8.3.1 git merge でブランチを統合するとき 104
8.3.2 git pull でリモートの変更を取り込むとき 105
8.3.3 git rebase でブランチの基点を付け替えるとき 105
8.3.4 git cherry-pick で特定のコミットを取り込むとき 105
8.3.5 コンフリクト発生時の git status 105
8.4 コンフリクトマーカーの読み方 105

8.4.1 マーカーの全体像 . . . 106
8.4.2 各行の意味 . . . 106
8.4.3 マーカーの構造を表にまとめると . . . 106
8.5 手動で解決する方法 . . . 107
8.5.1 パターン 1：片方の変更を採用する . . . 107
8.5.2 パターン 2：両方の変更を統合する . . . 107
8.5.3 パターン 3：完全に書き直す . . . 108
8.5.4 解決後の操作：add と commit . . . 108
8.6 VS Code で解決する方法 . . . 108
8.6.1 コンフリクト箇所の表示 . . . 109
8.6.2 ワンクリック解決ボタン . . . 109
8.6.3 VS Code のマージエディタ . . . 109
8.7 コマンドで一括解決する方法 . . . 109
8.7.1 –ours：自分の変更をすべて採用 . . . 110
8.7.2 –theirs：相手の変更をすべて採用 . . . 110
8.7.3 注意点 . . . 110
8.8 マージやリベースの中止方法 . . . 110
8.8.1 マージの中止 . . . 111
8.8.2 リベースの中止 . . . 111
8.8.3 cherry-pick の中止 . . . 111
8.8.4 中止した後にやるべきこと . . . 111
8.9 コンフリクトを予防するプラクティス . . . 111
8.9.1 こまめに pull / rebase する . . . 111
8.9.2 小さなプルリクエストを心がける . . . 112
8.9.3 担当ファイルを分ける . . . 112
8.9.4 rerere を有効にする . . . 112
8.9.5 チーム内でコミュニケーションをとる . . . 113
8.10 まとめ . . . 113

9 履歴の操作 — 過去を自在にたどる 114
9.1 この章で学ぶこと . . . 114
9.2 なぜ履歴操作が必要か . . . 114
9.2.1 「間違えても大丈夫」という安心感 . . . 114
9.2.2 安全な操作と危険な操作 . . . 114
9.3 git revert — コミットを安全に打ち消す . . . 115
9.3.1 revert の考え方 . . . 115
9.3.2 なぜ revert は push 済みでも安全なのか . . . 115
9.3.3 使い方 . . . 115
9.4 git reset — コミットを巻き戻す . . . 116
9.4.1 reset の考え方 . . . 116
9.4.2 3 つのモード — アナロジーで理解する . . . 116
9.4.3 3 つのモードの比較表 . . . 117
9.4.4 HEAD~1 の意味 . . . 117
9.4.5 reset の注意点 . . . 118
9.5 git restore — ファイル単位の変更取り消し . . . 118
9.5.1 restore の位置づけ . . . 118
9.5.2 ワーキングツリーの変更を取り消す . . . 118

9.5.3 ステージングの取り消し . . . 118
9.5.4 特定のコミット時点の状態に戻す . . . 119
9.6 git stash — 作業の一時退避 . . . 119
9.6.1 「引き出しにしまう」アナロジー . . . 119
9.6.2 基本的な使い方 . . . 119
9.6.3 退避した内容の確認と復元 . . . 120
9.6.4 退避の削除 . . . 120
9.6.5 典型的な使い方シナリオ . . . 121
9.7 git cherry-pick — 特定のコミットだけ取り込む . . . 121
9.7.1 cherry-pick とは . . . 121
9.7.2 どんな場面で使うのか . . . 121
9.7.3 使い方 . . . 122
9.8 git reflog —「消えた」コミットを救出する . . . 122
9.8.1 reflog とは何か . . . 122
9.8.2 reflog の確認方法 . . . 122
9.8.3 reset –hard からの復旧 . . . 123
9.8.4 reflog の注意点 . . . 123
9.9 git blame — 各行の変更者を特定する . . . 124
9.9.1 blame の目的 . . . 124
9.9.2 使い方 . . . 124
9.9.3 GitHub 上での blame . . . 125
9.10 まとめ . . . 125

10 プルリクエスト — コードレビューの文化を作る 126
10.1 この章で学ぶこと . . . 126
10.2 プルリクエストとは何か . . . 126
10.2.1 「取り込んでほしい」という提案 . . . 126
10.2.2 なぜ直接 push しないのか . . . 127
10.3 PR の作成方法 . . . 127
10.3.1 Web UI から作成する . . . 127
10.3.2 GitHub CLI から作成する . . . 128
10.3.3 push と同時に URL を取得する . . . 128
10.4 PR の構成要素 . . . 129
10.4.1 Title（タイトル） . . . 129
10.4.2 Description（説明） . . . 129
10.4.3 Reviewers（レビュアー） . . . 129
10.4.4 Assignees（担当者） . . . 129
10.4.5 Labels（ラベル） . . . 129
10.4.6 Projects（プロジェクト） . . . 129
10.4.7 Milestone（マイルストーン） . . . 130
10.4.8 Linked Issues（関連 Issue） . . . 130
10.5 良い PR description の書き方 . . . 130
10.5.1 良い description の構成例 . . . 130
10.5.2 Issue との連携 — 自動クローズ . . . 131
10.6 PR の確認とレビュー . . . 131
10.6.1 Files changed タブの使い方 . . . 131
10.6.2 コメントの付け方 . . . 132

10.6.3 suggestion 機能 — 修正提案をその場で . . . 132
10.6.4 レビューの 3 つのステータス . . . 132
10.6.5 CLI でのレビュー確認 . . . 133
10.7 3 つのマージ方法 . . . 133
10.7.1 Create a merge commit（マージコミット） . . . 133
10.7.2 Squash and merge（スカッシュマージ） . . . 134
10.7.3 Rebase and merge（リベースマージ） . . . 134
10.7.4 3 つの方法の比較 . . . 134
10.7.5 CLI でのマージ . . . 134
10.8 ドラフト PR . . . 135
10.8.1 ドラフト PR とは . . . 135
10.8.2 どんな場面で使うのか . . . 135
10.8.3 作成と解除 . . . 135
10.9 PR テンプレート . . . 136
10.9.1 テンプレートの目的 . . . 136
10.9.2 テンプレートの作成方法 . . . 136
10.9.3 テンプレートの運用 Tips . . . 137
10.10 まとめ . . . 137

11 Issue — タスクとバグを管理する 139
11.1 この章で学ぶこと . . . 139
11.2 Issue とは何か . . . 139
11.2.1 チケットシステムとしての Issue . . . 139
11.2.2 Issue 番号の仕組み . . . 140
11.3 Issue の作成 . . . 140
11.3.1 Web UI での作成 . . . 140
11.3.2 GitHub CLI（gh）での作成 . . . 140
11.4 良い Issue の書き方 . . . 141
11.4.1 バグ報告のテンプレート . . . 141
11.4.2 機能要望の書き方 . . . 142
11.5 ラベル（Labels） . . . 143
11.5.1 ラベルの役割 . . . 143
11.5.2 デフォルトラベル一覧 . . . 143
11.5.3 カスタムラベルの作成 . . . 143
11.6 マイルストーン（Milestones） . . . 144
11.6.1 マイルストーンとは . . . 144
11.6.2 マイルストーンの作成と管理 . . . 144
11.7 Issue と Pull Request の連携 . . . 145
11.7.1 自動クローズキーワード . . . 145
11.7.2 コミットメッセージでの参照 . . . 146
11.8 Issue テンプレート . . . 146
11.8.1 テンプレートの必要性 . . . 146
11.8.2 バグ報告テンプレートの作成 . . . 146
11.8.3 機能要望テンプレートの作成 . . . 148
11.9 Issue の管理 . . . 149
11.9.1 CLI での操作 . . . 149
11.9.2 ピン留め（Pinned Issues） . . . 150

11.9.3 Issue の検索とフィルタリング . . . 150
11.10 まとめ . . . 150

12 GitHub Actions — 自動化で開発を加速する 152
12.1 この章で学ぶこと . . . 152
12.2 CI/CD とは何か . . . 152
12.2.1 手動テストの限界 . . . 152
12.2.2 Continuous Integration（継続的インテグレーション） . . . 152
12.2.3 Continuous Delivery / Deployment（継続的デリバリー/デプロイ） . . . 153
12.3 GitHub Actions の基本用語 . . . 153
12.3.1 Workflow（ワークフロー） . . . 153
12.3.2 Event（イベント） . . . 153
12.3.3 Job（ジョブ） . . . 153
12.3.4 Step（ステップ） . . . 154
12.3.5 Action（アクション） . . . 154
12.3.6 Runner（ランナー） . . . 154
12.4 最初のワークフローを作ってみる . . . 154
12.5 トリガー（Event）の種類 . . . 156
12.5.1 push — コードがプッシュされた時 . . . 156
12.5.2 pull_request — Pull Request に関するイベント . . . 157
12.5.3 schedule — 定期実行 . . . 157
12.5.4 workflow_dispatch — 手動実行 . . . 157
12.5.5 release — リリース公開時 . . . 158
12.6 実用的なワークフロー例 . . . 158
12.6.1 Python プロジェクトの本格的な CI . . . 158
12.6.2 Node.js プロジェクトの CI . . . 159
12.6.3 タグプッシュによる自動リリース . . . 160
12.7 Secrets（シークレット） . . . 161
12.7.1 なぜ Secrets が必要か . . . 161
12.7.2 Secrets の設定方法 . . . 161
12.7.3 ワークフローでの使い方 . . . 162
12.8 料金と制限 . . . 162
12.8.1 パブリックリポジトリ . . . 162
12.8.2 プライベートリポジトリ . . . 162
12.8.3 その他の制限 . . . 163
12.9 ワークフローの確認とデバッグ . . . 163
12.9.1 CLI での操作 . . . 163
12.9.2 デバッグのコツ . . . 164
12.10 まとめ . . . 164

13 GitHub Pages — Web サイトを無料で公開する 166
13.1 この章で学ぶこと . . . 166
13.2 GitHub Pages とは . . . 166
13.2.1 静的サイトホスティングの概念 . . . 166
13.3 公開 URL の形式 . . . 167
13.3.1 ユーザーサイト（User Site） . . . 167
13.3.2 プロジェクトサイト（Project Site） . . . 167

13.3.3 カスタムドメイン . . . 167
13.4 セットアップ方法 . . . 168
13.4.1 方法 1: ブランチから公開（最もシンプル） . . . 168
13.4.2 方法 2: GitHub Actions から公開（柔軟な方法） . . . 168
13.4.3 方法 3: ユーザーサイトの作成 . . . 170
13.5 Jekyll テーマの適用 . . . 171
13.5.1 Jekyll とは . . . 171
13.5.2 テーマの設定 . . . 171
13.5.3 Markdown でページを作成する . . . 172
13.6 カスタムドメインの設定 . . . 172
13.6.1 なぜカスタムドメインを使うのか . . . 172
13.6.2 DNS レコードの設定 . . . 173
13.6.3 GitHub 側の設定 . . . 173
13.6.4 HTTPS 対応 . . . 174
13.7 まとめ . . . 174

14 リリース管理 — ソフトウェアを世に届ける 175
14.1 この章で学ぶこと . . . 175
14.2 タグ（Tag）とは . . . 175
14.2.1 コミットに名前を付ける . . . 175
14.2.2 軽量タグと注釈付きタグ . . . 175
14.2.3 タグの基本操作 . . . 176
14.3 セマンティックバージョニング . . . 177
14.3.1 MAJOR.MINOR.PATCH の意味 . . . 177
14.3.2 各桁の判断基準 . . . 177
14.3.3 プレリリースバージョン . . . 178
14.3.4 v を付けるか付けないか . . . 178
14.4 リリースの作成 . . . 178
14.4.1 Web UI での作成 . . . 179
14.4.2 CLI での作成 . . . 179
14.4.3 ドラフトリリースの活用 . . . 180
14.5 リリースのカスタマイズ . . . 181
14.5.1 リリースノート自動生成のカテゴリ分け . . . 181
14.5.2 exclude セクション . . . 182
14.6 まとめ . . . 182

15 Fork — オープンソースに貢献する 184
15.1 この章で学ぶこと . . . 184
15.2 Fork とは何か . . . 184
15.2.1 他の人のリポジトリを自分のアカウントにコピーする . . . 184
15.2.2 clone との違い . . . 185
15.3 OSS への貢献の全体像 . . . 185
15.4 実践手順 . . . 186
15.4.1 ステップ 1: Fork する . . . 186
15.4.2 ステップ 2: upstream リモートを設定する . . . 187
15.4.3 ステップ 3: Fork 元の最新の状態と同期する . . . 187
15.4.4 ステップ 4: ブランチを作成して作業する . . . 188

15.4.5 ステップ 5: Pull Request を送る . . . 188
15.4.6 ステップ 6: レビュー対応 . . . 189
15.5 貢献の前に確認すべきこと . . . 190
15.5.1 CONTRIBUTING.md . . . 190
15.5.2 Code of Conduct（行動規範） . . . 190
15.5.3 good first issue を探す . . . 190
15.5.4 ライセンスの確認 . . . 191
15.6 コード以外のコントリビューション . . . 191
15.6.1 ドキュメントの改善 . . . 191
15.6.2 翻訳 . . . 191
15.6.3 Issue 報告 . . . 191
15.6.4 レビュー . . . 192
15.6.5 デザイン . . . 192
15.7 まとめ . . . 192

16 Organization — チームでリポジトリを管理する 193
16.1 この章で学ぶこと . . . 193
16.2 Organization とは . . . 193
16.2.1 個人アカウントとの違い . . . 193
16.2.2 チーム管理の必要性 . . . 194
16.3 Organization の作成手順 . . . 194
16.4 メンバーの管理 . . . 195
16.4.1 権限ロールの詳細 . . . 195
16.4.2 メンバーの招待方法 . . . 196
16.5 Teams（チーム） . . . 196
16.5.1 なぜ Teams が必要なのか . . . 196
16.5.2 チームの作成 . . . 196
16.5.3 チームの権限レベル . . . 197
16.5.4 ネストされたチーム . . . 197
16.6 CODEOWNERS . . . 198
16.6.1 ファイルやディレクトリの担当者を定義する仕組み . . . 198
16.6.2 CODEOWNERS ファイルの書き方 . . . 198
16.6.3 ブランチ保護ルールとの連携 . . . 199
16.7 まとめ . . . 199

17 コードレビュー — 品質を高め合う文化 201
17.1 この章で学ぶこと . . . 201
17.2 コードレビューとは . . . 201
17.2.1 なぜレビューするのか . . . 201
17.2.2 コードレビューは「検査」ではなく「対話」 . . . 202
17.3 レビューの進め方 . . . 202
17.3.1 Files changed タブ . . . 202
17.3.2 コメントの追加方法 . . . 202
17.3.3 コメントの種類 . . . 202
17.3.4 レビュー結果の送信 . . . 203
17.4 ブランチ保護ルール . . . 204
17.4.1 設定方法 . . . 204

17.4.2 主な設定項目の解説 204
17.4.3 CLI での設定 205
17.5 レビュアーとしてのベストプラクティス 206
17.5.1 建設的なフィードバックを心がける 206
17.5.2 コメントのプレフィックスを活用する 206
17.5.3 迅速にレビューする 207
17.5.4 全体像を把握してからレビューする 207
17.6 PR 作成者としてのベストプラクティス 207
17.6.1 小さな Pull Request を作る 207
17.6.2 セルフレビューを行う 207
17.6.3 テストを含める 208
17.6.4 説明文を丁寧に書く 208
17.6.5 コンテキストを共有する 208
17.7 まとめ 208

18 Projects — プロジェクトの進捗を可視化する 209
18.1 この章で学ぶこと 209
18.2 GitHub Projects とは 209
18.2.1 プロジェクト管理ツールとしての位置づけ 209
18.3 Project の作成 210
18.3.1 作成手順 210
18.3.2 3 つのビュー 210
18.3.3 複数のビューを組み合わせる 211
18.4 カスタムフィールド 212
18.4.1 Text（テキスト） 212
18.4.2 Number（数値） 212
18.4.3 Date（日付） 212
18.4.4 Single select（単一選択） 212
18.4.5 Iteration（イテレーション） 213
18.4.6 フィールド設計のヒント 213
18.5 ワークフロー自動化 213
18.5.1 設定方法 213
18.5.2 主なワークフロー 214
18.5.3 より高度な自動化 214
18.6 フィルタリングとソート 214
18.6.1 フィルタの記法 214
18.6.2 ソート 215
18.6.3 ビューの保存 215
18.6.4 グループ化 216
18.7 Organization レベルのプロジェクト 216
18.7.1 複数リポジトリにまたがるプロジェクト管理 216
18.7.2 Organization プロジェクトの作成 216
18.7.3 実践的な活用パターン 217
18.8 まとめ 217

19 ファイルの扱い方 — Raw URL・画像・LFS 218
19.1 この章で学ぶこと 218

19.2 GitHub 上のファイル URL 構造 — blob/tree/raw/blame/commits の違いを詳しく 218
19.3 特定行へのリンクとパーマリンク 219
19.3.1 行番号リンク 219
19.3.2 パーマリンク — なぜ永続リンクが重要なのか 220
19.4 Raw URL — 生データの取得 220
19.4.1 Raw URL とは 220
19.4.2 curl でのダウンロード 221
19.4.3 プログラムからの読み込み 221
19.5 画像の保存と表示 221
19.5.1 方法 1: リポジトリに画像ファイルを保存する 222
19.5.2 方法 2: Issue にドラッグアンドドロップして CDN を利用する 222
19.5.3 HTML タグでのサイズ指定 223
19.6 Git LFS — 大容量ファイルの管理 223
19.6.1 なぜ大きなファイルが問題になるのか 223
19.6.2 LFS の仕組み — ポインタファイル 224
19.6.3 セットアップ手順 224
19.6.4 容量制限 225
19.7 サブモジュール — リポジトリの中に別リポジトリを含める 225
19.7.1 サブモジュールとは 225
19.7.2 サブモジュールの追加 226
19.7.3 サブモジュール付きリポジトリのクローン 226
19.7.4 サブモジュールの更新 226
19.8 GitHub 上でのファイル操作 227
19.8.1 ファイルの作成 227
19.8.2 ファイルの編集 227
19.8.3 キーボードショートカットによる検索 228
19.9 まとめ 228

20 README とドキュメント — プロジェクトの顔を作る 229
20.1 この章で学ぶこと 229
20.2 README とは何か 229
20.2.1 名前の由来 — 1970 年代からの伝統 229
20.2.2 GitHub での役割 — 自動表示の仕組み 229
20.2.3 `.md` の意味 — Markdown 形式 230
20.2.4 README が表示される場所一覧 230
20.3 README の書き方 231
20.3.1 最低限の README 231
20.3.2 標準的な README の構成 231
20.4 Markdown 記法 233
20.4.1 見出し 233
20.4.2 太字・斜体・取り消し線 233
20.4.3 リスト 233
20.4.4 リンクと画像 234
20.4.5 テーブル 234
20.4.6 コードブロック 234
20.4.7 タスクリスト 235

20.4.8 折りたたみ（details/summary） . . . 235
20.4.9 注意・警告（Alerts） . . . 235
20.4.10 Mermaid 図 . . . 236
20.4.11 数式（LaTeX） . . . 236
20.5 ジャンル別 README テンプレート . . . 237
20.5.1 CLI ツール . . . 237
20.5.2 Web アプリ / API . . . 238
20.5.3 ライブラリ / パッケージ . . . 239
20.5.4 研究プロジェクト . . . 240
20.6 README 作成のコツ . . . 241
20.6.1 最初の 3 秒で勝負が決まる . . . 241
20.6.2 コピーアンドペーストで動くコードを載せる . . . 242
20.6.3 スクリーンショットや GIF を使う . . . 242
20.6.4 目次を付ける（長い README の場合） . . . 242
20.7 バッジの付け方 . . . 242
20.7.1 shields.io — 動的バッジ . . . 242
20.7.2 技術スタックバッジ . . . 243
20.8 Wiki . . . 244
20.8.1 有効化 . . . 244
20.8.2 クローンと編集 . . . 244
20.9 よく使うファイル . . . 244
20.10 ライセンスの選び方 . . . 245
20.10.1 判断基準 . . . 245
20.11 まとめ . . . 246

21 カスタマイズ — GitHub を自分好みに整える 248
21.1 この章で学ぶこと . . . 248
21.2 テーマ設定 — ダークモード/ライトモード . . . 248
21.3 キーボードショートカット . . . 249
21.3.1 グローバルショートカット（どのページでも使える） . . . 249
21.3.2 リポジトリ内ショートカット . . . 249
21.3.3 ファイル表示時のショートカット . . . 249
21.3.4 Pull Request と Issue のショートカット . . . 249
21.4 GitHub CLI (gh) のカスタマイズ — エイリアスの設定と活用例 . . . 250
21.4.1 エイリアスの設定 . . . 250
21.4.2 gh の設定 . . . 251
21.5 Git のカスタマイズ — エイリアスと.gitconfig . . . 252
21.5.1 エイリアスの設定 . . . 252
21.5.2 .gitconfig の直接編集 . . . 253
21.6 通知のカスタマイズ . . . 254
21.6.1 通知設定の場所 . . . 254
21.6.2 リポジトリごとの通知設定 . . . 255
21.7 GitHub Mobile . . . 255
21.8 Saved Replies . . . 256
21.8.1 設定方法 . . . 256
21.8.2 便利なテンプレート例 . . . 256
21.9 まとめ . . . 257

22 セキュリティ — リポジトリを安全に保つ 258
22.1 この章で学ぶこと . . . 258
22.2 絶対にコミットしてはいけないもの . . . 258
22.2.1 なぜ秘密情報のコミットが危険なのか . . . 258
22.2.2 コミットしてはならない情報の一覧 . . . 258
22.3 万が一コミットしてしまったときの対処 . . . 259
22.3.1 最優先事項：秘密情報を無効化する . . . 259
22.3.2 履歴からの完全削除 . . . 260
22.4 Dependabot — 依存パッケージの脆弱性を自動検出する . . . 261
22.4.1 なぜ依存パッケージの管理が重要なのか . . . 261
22.4.2 Dependabot とは . . . 261
22.4.3 有効化の手順 . . . 261
22.4.4 設定ファイルによるカスタマイズ . . . 262
22.5 Secret Scanning — リポジトリ内の秘密情報を自動検出する . . . 263
22.5.1 Secret Scanning とは . . . 263
22.5.2 検出できる秘密情報の種類 . . . 263
22.5.3 有効化と動作 . . . 263
22.5.4 Push protection . . . 263
22.6 セキュリティポリシー — SECURITY.md の作成 . . . 264
22.6.1 なぜセキュリティポリシーが必要か . . . 264
22.6.2 SECURITY.md の書き方 . . . 264
22.7 セキュリティのベストプラクティス 7 箇条 . . . 265
22.7.1 第 1 条：二段階認証（2FA）を必ず有効にする . . . 265
22.7.2 第 2 条：.gitignore を徹底する . . . 265
22.7.3 第 3 条：GitHub Secrets で秘密情報を管理する . . . 265
22.7.4 第 4 条：最小権限の原則を守る . . . 265
22.7.5 第 5 条：Dependabot のアラートに迅速に対応する . . . 266
22.7.6 第 6 条：ブランチ保護ルールを設定する . . . 266
22.7.7 第 7 条：定期的にアクセス権限を見直す . . . 266
22.8 まとめ . . . 266

23 外部連携 — 開発ツールとつなげる 267
23.1 この章で学ぶこと . . . 267
23.2 VS Code との連携 . . . 267
23.2.1 なぜ VS Code と GitHub の連携が重要なのか . . . 267
23.2.2 拡張機能一覧と用途 . . . 267
23.2.3 ソース管理パネルの使い方 . . . 268
23.2.4 インライン blame . . . 268
23.2.5 github.dev — ブラウザ版 VS Code . . . 269
23.3 Claude Code との連携 . . . 269
23.3.1 Claude Code とは何か . . . 269
23.3.2 インストールと認証 . . . 269
23.3.3 GitHub リポジトリでの使用方法 . . . 270
23.3.4 Git 操作の対話的な依頼 . . . 270
23.3.5 VS Code 拡張としての使用 . . . 271
23.4 GitHub Apps / OAuth Apps . . . 271
23.4.1 GitHub Marketplace の概要 . . . 271

23.4.2 カテゴリ別の代表的なアプリ . . . 271
23.4.3 インストール方法 . . . 272
23.4.4 管理方法 . . . 272
23.5 Slack 連携 . . . 272
23.5.1 なぜ Slack 連携が必要なのか . . . 272
23.5.2 GitHub for Slack の設定 . . . 272
23.5.3 通知カスタマイズ . . . 273
23.6 Webhook . . . 273
23.6.1 仕組みの解説 . . . 273
23.6.2 設定方法 . . . 274
23.6.3 よく使うイベント . . . 274
23.7 GitHub API . . . 275
23.7.1 REST API と GraphQL API の概要 . . . 275
23.7.2 gh api コマンドでの使い方 . . . 275
23.7.3 GraphQL API の使い方 . . . 276
23.7.4 実用例 . . . 276
23.8 Gist . . . 277
23.8.1 コードスニペット共有の仕組み . . . 277
23.8.2 作成・一覧・表示のコマンド . . . 277
23.9 まとめ . . . 278

24 トラブルシューティング — よくある問題の処方箋 279
24.1 この章で学ぶこと . . . 279
24.2 認証エラー . . . 279
24.2.1 Permission denied (publickey) の原因と解決法 . . . 279
24.2.2 Invalid username or password の原因と解決法 . . . 280
24.2.3 403 Forbidden の原因と解決法 . . . 281
24.3 push/pull エラー . . . 281
24.3.1 rejected non-fast-forward の原因と解決法 . . . 281
24.3.2 refusing to merge unrelated histories の原因と解決法 . . . 282
24.3.3 Your branch is behind . . . 283
24.4 ブランチ関連の問題 . . . 283
24.4.1 local changes would be overwritten（未コミット変更時のブランチ切替） . . . 283
24.4.2 branch not found（リモートブランチの取得） . . . 284
24.5 コミット関連の問題 . . . 285
24.5.1 間違ったブランチにコミットした場合 . . . 285
24.5.2 コミットメッセージを間違えた場合 . . . 286
24.5.3 大きなファイルをコミットしてしまった場合 . . . 286
24.6 .gitignore が効かない問題 . . . 287
24.6.1 既に追跡されたファイルの対処 . . . 287
24.7 ネットワーク関連 . . . 288
24.7.1 Connection timed out（ポート 443 の使用） . . . 288
24.7.2 クローンが遅い（shallow clone） . . . 288
24.8 その他の問題 . . . 289
24.8.1 detached HEAD . . . 289
24.8.2 LF/CRLF の改行コード問題 . . . 289
24.8.3 not a git repository . . . 290

24.9 困ったときのコマンド集 291
24.10 まとめ 292

25 ファイル拡張子とプログラミング言語 294
25.1 この章で学ぶこと 294
25.2 拡張子とは何か 294
25.2.1 ファイル名の末尾の .xxx の役割 294
25.2.2 GitHub でのシンタックスハイライト 295
25.3 プログラミング言語と拡張子 295
25.3.1 Web 開発の拡張子 295
25.3.2 汎用プログラミング言語の拡張子 296
25.3.3 データサイエンス・科学計算の拡張子 296
25.3.4 シェルスクリプトの拡張子 297
25.4 データ・設定ファイルの拡張子 297
25.5 ドキュメントファイルの拡張子 298
25.6 ビルド・パッケージ管理ファイル 298
25.7 GitHub 特有のファイルとフォルダ 299
25.7.1 リポジトリルートの特別なファイル 299
25.7.2 .github/ ディレクトリ内の各種ファイル 300
25.8 GitHub の言語判定の仕組み 300
25.8.1 linguist による自動判定 300
25.8.2 .gitattributes でのカスタマイズ 301
25.9 ファイルの文字コード 301
25.9.1 UTF-8 が標準 301
25.9.2 .editorconfig 302
25.10 拡張子のないファイル 302
25.11 まとめ 303

26 GitHub アプリの比較 — Web・Desktop・CLI・Mobile・IDE 305
26.1 この章で学ぶこと 305
26.2 GitHub を操作する 5 つの方法の概観 305
26.3 GitHub Web UI 306
26.3.1 できること（全機能） 306
26.3.2 Web 上でのコード編集 306
26.3.3 向いている人 307
26.4 GitHub Desktop 307
26.4.1 インストール 307
26.4.2 特徴（GUI で Git 操作） 307
26.4.3 画面の見方 308
26.4.4 他の Git GUI ツールとの比較 308
26.4.5 向いている人 309
26.5 GitHub CLI (gh) 309
26.5.1 git コマンドと gh コマンドの違い 309
26.5.2 インストール 310
26.5.3 よく使うコマンド集 310
26.5.4 エイリアス 311
26.5.5 向いている人 312

26.6 GitHub Mobile . . . 312
26.6.1 インストール . . . 312
26.6.2 できること（通知/Issue/PR/コード閲覧） . . . 312
26.6.3 できないこと（編集/設定） . . . 312
26.6.4 向いている人 . . . 313
26.7 IDE 拡張機能 . . . 313
26.7.1 VS Code 標準 Git 機能の解説 . . . 313
26.7.2 おすすめ拡張機能 . . . 313
26.7.3 Claude Code . . . 314
26.7.4 JetBrains IDE の Git 連携 . . . 314
26.7.5 向いている人 . . . 315
26.8 どれを使うべきか . . . 315
26.8.1 初心者の場合 . . . 315
26.8.2 開発者の日常 . . . 315
26.8.3 レベル別おすすめ組み合わせ . . . 316
26.9 まとめ . . . 316

A Git コマンド早見表 318
この付録の使い方 . . . 318
A.1 初期設定（→ 第 2 章） . . . 318
A.2 リポジトリの作成と取得（→ 第 5 章） . . . 318
A.3 基本操作（→ 第 6 章） . . . 319
A.4 差分と履歴の確認（→ 第 6 章、第 9 章） . . . 320
A.5 ブランチ操作（→ 第 7 章） . . . 321
A.6 リモート操作（→ 第 5 章、第 6 章） . . . 322
A.7 変更の取り消しと復元（→ 第 9 章） . . . 323
A.8 一時退避（→ 第 9 章） . . . 324
A.9 タグ（→ 第 14 章） . . . 324
A.10 ファイル操作（→ 第 6 章） . . . 325
A.11 GitHub CLI（gh）コマンド（→ 第 23 章） . . . 325
A.12 逆引きリファレンス —「やりたいこと」から探す . . . 326

B 用語集 328
この付録の使い方 . . . 328
A . . . 328
B . . . 328
C . . . 329
D . . . 330
F . . . 331
G . . . 331
H . . . 332
I . . . 332
L . . . 332
M . . . 333
O . . . 333
P . . . 334
R . . . 334

S 336
T 337
U 337
V 337
W 337

Gitとは何か — バージョン管理の世界へようこそ

1.1. この章で学ぶこと

本章では、GitとGitHubという2つの言葉の意味と違いを明らかにし、「バージョン管理」とはそもそも何なのかを理解します。また、Gitがどのような経緯で生まれ、GitHubがソフトウェア開発の世界をどのように変えたのかについても触れます。技術的な操作に入る前に、全体像をしっかりと把握しておきましょう。

1.2. バージョン管理とは何か

1.2.1. ファイル管理の悩み

レポートや論文を書いた経験がある方なら、次のような状況に心当たりがあるのではないでしょうか。

```
レポート_最終版.docx
レポート_最終版_修正.docx
レポート_最終版_修正2_これが本当の最終版.docx
レポート_提出用_本当にこれで最後.docx
```

ファイル名に「最終版」と付けたはずなのに修正が入り、どれが本当の最終版なのか分からなくなる。ある変更を元に戻したいのに、どのファイルがその時点の状態だったか思い出せない——こうした問題は、ファイルの変更履歴を体系的に管理していないことから生じます。

1.2.2. バージョン管理システムという解決策

バージョン管理システム（Version Control System、略してVCS）とは、ファイルの変更履歴を自動的に記録・管理するソフトウェアのことです。バージョン管理システムを使えば、ファイルの名前は常にひとつのままで、過去のあらゆる時点の状態にいつでも戻ることができます。

```
レポート.docx  ←  ファイルはひとつだけ
  ├── バージョン1: 初稿作成 (2024-01-10)
  ├── バージョン2: 第2章を追加 (2024-01-15)
  ├── バージョン3: 図表を修正 (2024-01-20)
  └── バージョン4: 最終校正 (2024-01-25)  ←  現在の状態
```

バージョン管理システムは、「いつ」「誰が」「何を」「なぜ」変更したのかを全て記録します。これにより、次のようなことが可能になります。

- **過去の状態に戻る**: 3日前の状態に戻したい、というときにワンコマンドで実現できます
- **変更の追跡**: どの行がいつ変更されたかを正確に把握できます
- **並行作業**: 複数の人が同じプロジェクトで同時に作業しても、それぞれの変更を後から統合で

きます

- **バックアップ**: 変更履歴ごとデータが保存されるため、データの消失を防げます

1.2.3. プログラミングにおけるバージョン管理

プログラムのソースコードはテキストファイルであるため、バージョン管理との相性が抜群です。例えば、新しい機能を追加しようとしたがうまくいかなかった場合、バージョン管理があれば「機能追加前」の状態に簡単に戻せます。また、チーム開発では複数の開発者が同時にコードを書きますが、バージョン管理システムがそれぞれの変更を追跡し、矛盾なく統合してくれます。

現代のソフトウェア開発において、バージョン管理システムを使わずに開発するということは、まずありません。そして、今日世界で最も広く使われているバージョン管理システムが「Git」です。

1.3. Git とは

1.3.1. Git の基本的な特徴

Git（ギット）は、2005 年にリーナス・トーバルズ（Linus Torvalds）によって開発された**分散型バージョン管理システム**です。リーナス・トーバルズは、世界で最も広く使われているオペレーティングシステムである Linux の作者としても知られています。

Git の大きな特徴は「分散型」であるという点です。これは、プロジェクトの全ての変更履歴が各開発者のパソコンに完全にコピーされることを意味します。インターネットに接続していなくても、ローカル（自分のパソコン）だけで変更履歴の記録や確認ができるのです。

Git は無料で使えるオープンソースソフトウェアであり、Windows、macOS、Linux の全てで動作します。

1.3.2. Git が扱う基本概念

Git を理解するうえで欠かせない概念がいくつかあります。ここでは最も基本的なものを紹介します。それぞれの詳しい使い方は後続の章で改めて解説しますので、まずは「こういうものがあるのだな」という程度の理解で構いません。

1.3.2.1. リポジトリ（Repository）

リポジトリとは、プロジェクトのファイル群とその変更履歴の全てを格納する場所のことです。一般的には「リポ」や「repo（レポ）」と略されます。フォルダ（ディレクトリ）のようなものだと思ってください。

リポジトリには 2 種類あります。

- **ローカルリポジトリ**: 自分のパソコン上にあるリポジトリ。日々の開発作業はここで行います。
- **リモートリポジトリ**: インターネット上のサーバー（GitHub など）に置かれたリポジトリ。ローカルで行った変更をここに送信することで、他の開発者と共有できます。

1.3.2.2. コミット（Commit）

コミットとは、ある時点でのファイルの状態を履歴に記録する操作のことです。ゲームの「セーブポイント」に例えると分かりやすいでしょう。コミットを行うたびに、プロジェクトのスナップショット（その瞬間の状態）が保存されます。

```
コミット1 → コミット2 → コミット3 → コミット4（現在）
 "初期作成"   "機能A追加"  "バグ修正"   "機能B追加"
```

ひとつひとつのコミットには、次の情報が記録されます。

- **誰が** 変更したか（Author）
- **いつ** 変更したか（Date）
- **何を** 変更したか（Diff: 変更差分）
- **なぜ** 変更したか（Commit Message: コミットメッセージ）
- **一意の ID**（ハッシュ値: `a1b2c3d4e5f6...` のような文字列）

コミットメッセージは、将来の自分や他の開発者が「なぜこの変更を行ったのか」を理解するための手がかりになります。分かりやすいメッセージを書く習慣をつけることが大切です。

1.3.2.3. ブランチ（Branch）

ブランチは「枝」を意味する英語で、開発の流れを枝分かれさせる機能のことです。メインの開発ライン（多くの場合「main」というブランチ）に影響を与えることなく、新しい機能の開発やバグの修正を独立して進められます。

```
main:        A → B → C ————————————→ F（統合）
                  \              /
feature:           D → E ————————
                  "新機能の開発"
```

この図では、main ブランチの C の時点で「feature」という新しいブランチが分岐しています。feature ブランチで D と E という作業を行い、完成したら main ブランチに統合（マージ）しています。もし feature ブランチでの作業がうまくいかなかったとしても、main ブランチには何の影響もありません。

1.3.2.4. ステージング（Staging）

ステージングとは、コミットに含めるファイルを選択する操作のことです。変更したファイルの全てをコミットするのではなく、「今回のコミットにはこのファイルとこのファイルだけを含める」といった選択ができます。

この仕組みは、変更を意味のあるまとまりに分けてコミットするために重要です。例えば、バグの修正と新機能の追加を同時に行った場合、ステージングを使えばそれぞれを別のコミットとして

記録できます。

```
作業ディレクトリ              ステージングエリア              リポジトリ
（ファイルを編集する場所）    （コミット対象を選ぶ場所）      （履歴が記録される場所）
┌──────────────────────┐          ┌──────────────────────────────────────┐
│  ファイルを編集  │  add →   │  コミット対象を  │  commit →  │  履歴に記録  │
│                  │          │  選んで準備      │            │
└──────────────────────┘          └──────────────────────────────────────┘
```

1.4. GitHubとは

1.4.1. GitとGitHubの違い

Gitと GitHub は名前が似ていますが、全く別のものです。この違いを正しく理解することが非常に重要です。

	Git	GitHub
正体	バージョン管理ソフトウェア	Gitリポジトリのホスティングサービス
動作する場所	自分のパソコン（ローカル）	インターネット上（クラウド）
開発元	Linus Torvalds（個人→コミュニティ）	GitHub社（現在はMicrosoft傘下）
料金	完全無料	基本無料（有料プランあり）
主な役割	ファイルの変更履歴をローカルで管理する	Gitの履歴をオンラインで保存・共有する

Gitは自分のパソコンの中だけで動くソフトウェアです。一方、GitHubはGitで管理されたプロジェクトをインターネット上に保存し、他の人と共有するためのWebサービスです。

両者の関係は次の図のように表せます。

```
あなたのPC（Git）                          インターネット上（GitHub）
┌──────────────────────────┐    push →    ┌──────────────────────────┐
│  ローカル          │                │  リモートリポジトリ      │
│  リポジトリ        │   ← pull       │  （みんなが見られる）    │
│  (.git フォルダ)   │                │  github.com/user/repo    │
└──────────────────────────┘              └──────────────────────────┘
```

ローカルで行ったコミットを `push`（プッシュ）でGitHubに送信し、逆にGitHub上の変更を `pull`

（プル）でローカルに取り込みます。この往復のやり取りによって、複数の開発者が同じプロジェクトを共有できるのです。

1.4.2. GitHub が提供する付加価値

GitHub は単なる Git リポジトリの保管場所ではありません。次のような豊富な機能を備えた開発プラットフォームです。

- **プルリクエスト（Pull Request）**：コードの変更を提案し、レビューしてもらう仕組み
- **Issue**: バグ報告や機能要望を管理するチケットシステム
- **GitHub Actions**: テストやデプロイを自動化する CI/CD 機能
- **GitHub Pages**: 静的 Web サイトを無料でホスティング
- **Projects**: カンバンボードによるプロジェクト管理

これらの機能については第 3 部以降で詳しく解説します。

1.5. Git の歴史 — なぜ Git は生まれたのか

1.5.1. Linux カーネル開発と BitKeeper

Git が生まれた背景を理解するには、Linux カーネルの開発史を知る必要があります。

Linux カーネルは世界中の何千人もの開発者が参加する巨大なオープンソースプロジェクトです。2002 年から 2005 年にかけて、このプロジェクトでは BitKeeper という商用のバージョン管理ツールが無償で提供されており、開発者たちはそれを使ってコードを管理していました。

ところが 2005 年、BitKeeper の開発元がライセンスポリシーを変更し、無償利用ができなくなってしまいます。この出来事が Git 誕生の直接のきっかけとなりました。

1.5.2. 10 日間で作られたバージョン管理システム

Linus Torvalds は、既存のどのバージョン管理ツールも Linux カーネルのような巨大プロジェクトには不十分だと考え、自分で新しいシステムを作ることを決意します。彼が掲げた設計目標は次のようなものでした。

- **高速であること**: 何万ものファイルと長大な履歴を持つプロジェクトでも快適に動くこと
- **分散型であること**: 中央サーバーがなくても、各開発者の PC だけで完全に動作すること
- **データの完全性**: 一度記録した履歴が壊れたり改ざんされたりしないこと
- **非線形な開発への対応**: 何千ものブランチを同時に扱えること

驚くべきことに、Git の最初のバージョンはわずか **10 日間**で完成しました。2005 年 4 月に開発が始まり、同年 6 月には Linux カーネルの管理が Git に移行されています。

「Git」という名前の由来について、Linus 自身はユーモアを込めてこう語っています。「僕は自分のプロジェクトには自分にちなんだ名前を付けるんだ。最初が Linux、次が Git だ」。英語のスラング

で「git」は「嫌なやつ」「ろくでなし」を意味する言葉で、自嘲的なネーミングです。一方で、”Global Information Tracker”（グローバル情報追跡装置）の頭文字だという解釈もあります。

1.5.3. バージョン管理の進化の歴史

Git の革新性を理解するために、バージョン管理システムの歴史を振り返ってみましょう。

年代	システム	特徴
1972 年	SCCS	最初のバージョン管理システム。ファイル単位で管理
1982 年	RCS	SCCS の改良版。依然としてファイル単位
1990 年	CVS	ディレクトリ単位で管理可能に。中央集権型
2000 年	Subversion（SVN）	CVS の後継。中央サーバーでの管理を洗練
2005 年	**Git**	**分散型。各 PC に完全な履歴を保持。ここが大きな転換点**

2005 年以前のバージョン管理システムは「中央集権型」と呼ばれ、変更履歴は中央のサーバーに一元管理されていました。開発者のパソコンには最新のファイルしかなく、履歴を確認したりブランチを切り替えたりするにはサーバーへの接続が必要でした。サーバーが故障すれば、全ての履歴が失われるリスクもありました。

Git の「分散型」というアプローチでは、全ての開発者のパソコンにリポジトリの完全なコピー（全履歴を含む）が存在します。これにより、オフラインでも全ての操作が可能になり、サーバーが故障しても誰かの PC からデータを復元できるようになりました。

```
中央集権型（SVN等）：                    分散型（Git）：
      ┌──────────┐                          ┌──────────┐     ┌────
        ───┐
      │ サーバー │                          │ PC A │     │ PC B │
      │ （唯一） │                          │ （完全） │     │ （完全） │
      └───┬────┘                          └───┬────┘     └───┬
        ───┘
    ┌───┼───┐                                  │           │
    ↓   ↓   ↓                              ↓           ↓
  PC  PC  PC                          ┌──────────┐     ┌──────────┐
（部分）（部分）（部分）                    │ GitHub │     │ PC C │
                                      │ （完全） │     │ （完全） │
サーバーが壊れたら終わり                    └──────────┘     └──────────┘
                                      どのPCにも完全な履歴がある
```

現在、Git はバージョン管理システムの事実上の標準となっており、世界中の開発者の 95%以上が Git を使用していると言われています。

1.6. GitHub の歴史 — ソフトウェア開発を変えた革命

1.6.1. GitHub の誕生と成長

Git は強力なバージョン管理ツールですが、コマンドラインでの操作が中心で、他の開発者との協業の仕組みは Git 自体には含まれていませんでした。この課題を解決したのが GitHub です。

2008 年 2 月、トム・プレストン=ワーナー（Tom Preston-Werner）、クリス・ワンストラス（Chris Wanstrath）、PJ・ハイエット（PJ Hyett）、スコット・チャコン（Scott Chacon）の 4 人が GitHub を設立し、サービスを公開しました。

以下は GitHub の歩みを示す年表です。

年	出来事
2008 年	GitHub 設立・公開。公開 1 年で 10 万ユーザーを獲得
2009 年	ホストするリポジトリが 10 万を突破
2011 年	ユーザー数 100 万人を突破
2012 年	著名ベンチャーキャピタルの Andreessen Horowitz から 1 億ドル（約 130 億円）の資金調達
2013 年	リポジトリ数が 1,000 万を突破
2018 年	Microsoft が 75 億ドル（約 1 兆円）で GitHub を買収。当初は懸念の声もあったが、GitHub の独立性は維持された
2019 年	GitHub Actions（CI/CD 機能）リリース。プライベートリポジトリが無料プランでも利用可能に
2020 年	npm（Node.js のパッケージ管理システム）を買収。ユーザー数 5,600 万人
2021 年	GitHub Copilot（AI コーディング支援）のプレビューを開始
2022 年	GitHub Copilot を一般公開
2023 年	ユーザー数 1 億人を突破。リポジトリ数は 3 億 7,200 万超
2024 年	Copilot Workspace、GitHub Models など、AI を活用した新機能を次々と発表
2025 年	ユーザー数 1 億 5,000 万人を超え、世界最大の開発プラットフォームとして不動の地位を確立

1.6.2. GitHub が世界にもたらした変革

GitHub は単に Git リポジトリをホストするサービスを超えて、ソフトウェア開発の文化そのものを変革しました。

1.6.2.1. オープンソースの民主化

GitHub 以前の時代、オープンソースソフトウェア（OSS）に貢献するのは敷居の高い行為でした。プロジェクトのメーリングリストを見つけ、パッチファイルを作成し、メールで送信し、メンテナの返事を待つ——こうした手順は、経験の浅い開発者にとって大きな壁でした。

GitHub は「Fork（フォーク）」と「Pull Request（プルリクエスト）」という仕組みを洗練させる

ことで、この壁を取り払いました。気になるプロジェクトを見つけたら、ボタンひとつで自分のアカウントにコピーし、修正を加え、元のプロジェクトに提案する——このシンプルな流れが、世界中の何百万人もの開発者が OSS に貢献するきっかけを作りました。

1.6.2.2. ソーシャルコーディングの概念

GitHub は開発者のための「SNS」としての側面も持っています。他の開発者をフォローしたり、気に入ったリポジトリにスター（星マーク）を付けたり、コントリビューショングラフ（通称「草」と呼ばれる、1 年間の活動量を緑色の濃淡で表示する図）で日々の活動を可視化したり——こうした機能が開発者同士のつながりを促進しています。

1.6.2.3. 世界を動かすプロジェクトの集積地

今日、世界を動かす多くのソフトウェアが GitHub 上でホストされています。

プロジェクト	説明
Linux	世界のサーバーの大半、Android の基盤となる OS カーネル
React	Meta 社（旧 Facebook）が開発した Web フロントエンドライブラリ
VS Code	Microsoft 製のソースコードエディタ
TensorFlow	Google 製の機械学習フレームワーク
Python (CPython)	Python 言語の公式実装
Kubernetes	コンテナ化されたアプリケーションの運用管理ツール

これらのプロジェクトは全てオープンソースとして GitHub 上に公開されており、世界中の開発者がコードを読み、バグを報告し、改善を提案しています。

1.6.2.4. 開発者の「履歴書」としての GitHub

技術者の採用面接において、GitHub アカウントが「技術力の証明」として活用されるようになりました。どんなプロジェクトに貢献しているか、どの程度の頻度でコードを書いているか、コードの品質はどうか——GitHub のプロフィールページがこれらを雄弁に語ってくれます。

1.6.3. GitHub の競合サービス

GitHub が圧倒的なシェア（推定 90%以上）を持つ一方で、他にも Git リポジトリのホスティングサービスは存在します。

組織のセキュリティポリシーや既存のツール群との相性によって、GitLab や Bitbucket が選ばれるケースもありますが、本書では GitHub の使い方に焦点を当てて解説していきます。

1.7. Git/GitHub を使うメリット

ここまでの内容を踏まえて、Git/GitHub を使うことのメリットを整理しましょう。

サービス	特徴
GitLab	セルフホスト（自社サーバーでの運用）が可能。CI/CD 機能が充実。DevOps 全般をカバー
Bitbucket	Atlassian 社製。同社の Jira（プロジェクト管理）や Confluence（ドキュメント）と密に連携
Gitea	軽量なセルフホスト型。Go 言語で書かれており、小規模チーム向け
Azure DevOps	Microsoft 製のエンタープライズ向け開発プラットフォーム

1. **変更履歴が全て残る** — いつでも過去の状態に戻れるため、「あの変更を元に戻したい」という場面で困ることがありません。
2. **複数人で同時作業できる** — ブランチを使って独立した作業を進め、後から統合できます。同じファイルを複数人が触ったとしても、Git が変更点を追跡してくれます。
3. **バックアップになる** — GitHub にコードが保存されるため、パソコンが壊れてもプロジェクトを失うことがありません。
4. **コードレビューができる** — プルリクエストを通じて、他の人にコードを読んでもらい、改善点を指摘してもらえます。
5. **オープンソースへの参加** — 世界中の優れたプロジェクトのコードを読み、学び、貢献することができます。
6. **ポートフォリオとして活用** — GitHub のプロフィールとリポジトリが、あなたの技術力を示すポートフォリオになります。

1.8. まとめ

本章では、以下の内容を学びました。

- **バージョン管理**とは、ファイルの変更履歴を体系的に記録・管理する仕組みのことである
- **Git** はローカルで動作する分散型バージョン管理ソフトウェアであり、**GitHub** は Git リポジトリをオンラインでホスティングするサービスである。両者は補完関係にある
- Git の基本概念として、**リポジトリ**（プロジェクトの器）、**コミット**（変更の記録）、**ブランチ**（開発の分岐）、**ステージング**（コミット対象の選択）がある
- Git は 2005 年に Linux カーネルの管理のために生まれ、GitHub は 2008 年の設立以来、オープンソースの民主化やソーシャルコーディングの概念を通じてソフトウェア開発の世界を大きく変えた

次の章では、実際に Git を自分のパソコンにインストールする手順を見ていきます。

環境構築 — Gitを自分のパソコンに導入する

2.1. この章で学ぶこと

本章では、Gitを自分のパソコンにインストールし、使い始めるための準備を整えます。Gitは優れたバージョン管理システムですが、ただインストールするだけでは十分ではありません。自分の名前やメールアドレスを登録する「初期設定」を行い、エディタやターミナルといった周辺ツールも整えて初めて、快適な開発環境が完成します。

具体的には、以下の内容を扱います。

- macOS・Windows・Linuxそれぞれでの**Gitのインストール方法**
- インストールが成功したかどうかの**確認方法**
- コミットに記録される**名前とメールアドレスの設定**とその意味
- プライバシーを守るための**noreplyメールアドレス**の活用
- 日々の作業を快適にする**推奨設定**の数々
- 設定ファイルの**階層構造**と優先順位の仕組み
- **VS Code**やターミナルなど、Git操作に必要な周辺ツールの準備

この章を終えれば、いつでもGitを使い始められる状態になっているはずです。焦らず一つずつ進めていきましょう。

2.2. Gitのインストール

Gitはオープンソースのソフトウェアであり、macOS・Windows・Linuxのいずれの環境でも無料で利用できます。ただし、インストールの方法はOSによって異なります。ここでは、各OSごとに推奨される方法を紹介します。

2.2.1. macOSの場合

macOSでGitを導入するには、主に2つの方法があります。

方法1：Xcode Command Line Tools（推奨）

macOSには、Appleが提供する開発者向けツール群「Xcode Command Line Tools」があり、この中にGitが含まれています。多くのmacOSユーザーにとって、これが最も手軽な方法です。ターミナルを開いて次のコマンドを実行してください。

```
xcode-select --install
```

このコマンドを実行すると、画面にダイアログが表示されます。「インストール」をクリックし、利用規約に同意すれば、自動的にダウンロードとインストールが始まります。インストールには数分かかることがありますので、気長に待ちましょう。

スクリーンショット: *xcode-select* インストールダイアログ

この方法の利点は、Git だけでなく、`make` や `gcc` といった C 言語のコンパイラなど、開発に必要な基本的なツールが一式揃う点にあります。将来的にプログラミングの幅を広げたいと考えている方にも適しています。

方法 2：Homebrew を使う方法

Homebrew（ホームブリュー）は、macOS 向けの非公式パッケージマネージャーです。「パッケージマネージャー」とは、ソフトウェアのインストール・更新・削除を一元管理してくれるツールのことです。スマートフォンでいえば「App Store」のような存在だと思ってください。

Homebrew がすでにインストールされている場合は、次のコマンドだけで Git を導入できます。

```
brew install git
```

Homebrew の利点は、Git を常に最新バージョンに保ちやすいことです。Xcode Command Line Tools に含まれる Git は、Apple のアップデートサイクルに依存するため、最新版より少し古いことがあります。Homebrew なら `brew upgrade git` で簡単に更新できます。

Homebrew をまだインストールしていない場合は、*https://brew.sh* にアクセスし、記載されているインストールコマンドをターミナルに貼り付けて実行してください。

2.2.2. Windows の場合

Windows には Git が標準搭載されていないため、別途インストールする必要があります。

方法 1：Git for Windows（推奨）

最もポピュラーな方法は、「Git for Windows」の公式インストーラーを使うことです。

1. *https://gitforwindows.org/* にアクセスします
2. 「Download」ボタンをクリックしてインストーラーをダウンロードします
3. ダウンロードしたファイルを実行します

スクリーンショット: *Git for Windows* インストーラー

インストーラーはいくつかの設定画面を表示しますが、初心者の方は基本的にデフォルト（初期値）のまま進めて問題ありません。ただし、以下の 3 つの設定については意識しておくとよいでしょう。

- **デフォルトエディタの選択**：コミットメッセージを書く際に使用するエディタです。Vim に馴染みがなければ、Visual Studio Code（後述）を選択することをおすすめします。Vim は強力なエディタですが、操作方法が独特で、初心者が最初に触れるには少しハードルが高いためです。

- **PATH 環境変数の設定**：「Git from the command line and also from 3rd-party software」を選択してください。これにより、コマンドプロンプトや PowerShell など、Windows 標準のターミナルからも Git を使えるようになります。
- **改行コードの設定**：「Checkout Windows-style, commit Unix-style line endings」を選択してください。Windows とそれ以外の OS では改行コードが異なるため（Windows は `CR+LF`、macOS/Linux は `LF`）、この設定により共同作業時のトラブルを防げます。

Git for Windows をインストールすると、「Git Bash」というターミナルも一緒にインストールされます。Git Bash は Linux 風のコマンドを使える環境で、本書に掲載しているコマンドの多くはそのまま実行できます。

方法 2：winget を使う方法（Windows 10/11）

Windows 10 以降には、Microsoft 公式のパッケージマネージャー「winget」が搭載されています。PowerShell またはコマンドプロンプトを開き、次のコマンドを実行します。

```
winget install --id Git.Git -e --source winget
```

`--id Git.Git` はインストールするソフトウェアの ID、`-e` は「完全一致（exact match）」を意味し、`--source winget` は Microsoft 公式のリポジトリから取得することを指定しています。この方法は、コマンド一行で完結するため、手早く済ませたい方に向いています。

2.2.3. Linux（Ubuntu/Debian 系）の場合

Ubuntu や Debian などの Debian 系ディストリビューションでは、`apt` パッケージマネージャーを使います。

```
sudo apt update
sudo apt install git
```

`sudo apt update` はパッケージの一覧（カタログ）を最新化するコマンドです。これを先に実行しておかないと、古いバージョンの Git がインストールされてしまうことがあります。料理のレシピに例えれば、「まず冷蔵庫の中身を確認してから買い物に行く」ようなものです。

2.2.4. Linux（Fedora/RHEL 系）の場合

Fedora や Red Hat Enterprise Linux（RHEL）系では、`dnf` を使います。

```
sudo dnf install git
```

`dnf` は `apt` と同じくパッケージマネージャーですが、Red Hat 系のディストリビューションで

使われるものです。

2.3. インストールの確認

インストールが完了したら、Git が正しく導入されたかを確認しましょう。ターミナル（macOS/Linux）またはコマンドプロンプト/PowerShell/Git Bash（Windows）を開いて、次のコマンドを実行します。

```
git --version
```

成功していれば、次のようにバージョン番号が表示されます。

```
git version 2.44.0
```

バージョン番号は環境やインストール時期によって異なりますが、表示されていれば問題ありません。もし `command not found`（コマンドが見つかりません）というエラーが出た場合は、インストールがうまくいっていないか、ターミナルを再起動する必要がある可能性があります。Windows の場合は、インストール後にターミナルを一度閉じて開き直してみてください。

スクリーンショット: *git –version* の出力例

2.4. 初期設定の意味と方法

Git を使い始める前に、必ず行わなければならない設定が 2 つあります。それは**ユーザー名**と**メールアドレス**の登録です。この設定を怠ると、コミット（変更の記録）を作成できません。

2.4.1. なぜ名前とメールアドレスが必要なのか

Git は「誰が」「いつ」「何を」変更したかを記録するツールです。コミットのたびに、変更者の名前とメールアドレスがその記録に刻み込まれます。これは、チーム開発において「この変更は誰が行ったのか」を追跡するために不可欠な情報です。

たとえば、あるファイルにバグが混入したとき、`git log` コマンドでコミット履歴を確認すれば、「このコミットは山田さんが 3 日前に行った」とわかります。名前やメールアドレスが設定されていなければ、こうした追跡ができなくなってしまいます。

重要なのは、ここで設定する名前とメールアドレスは**認証情報ではない**という点です。つまり、この設定によって本人確認が行われるわけではありません。あくまで「コミットに付与するラベル」のようなものです。本人証明が必要な場合は、第 4 章で解説する GPG 署名を使います。

2.4.2. 設定コマンド

ターミナルで次のコマンドを実行します。

```
git config --global user.name "あなたの名前"
git config --global user.email "your-email@example.com"
```

`user.name` には、GitHub で使用しているユーザー名を設定するのが一般的です。もちろん本名でも構いません。日本語でも入力できますが、海外のツールやサービスとの互換性を考えると、英数字（ローマ字）を使うことをおすすめします。

`user.email` には、GitHub アカウントに登録したメールアドレスを設定します。GitHub は、コミットに記録されたメールアドレスを使って、そのコミットを GitHub 上のアカウントと紐づけます。もしここで別のメールアドレスを設定すると、GitHub 上でコミットが自分のアカウントに関連付けられず、Contribution（貢献活動）のカウントにも反映されないことがあります。

2.4.3. `--global` と `--local` の違い

`--global` オプションは、「このパソコンのすべてのリポジトリに対してこの設定を適用する」という意味です。一度設定すれば、新しいリポジトリを作成するたびに設定し直す必要はありません。

一方、`--local`（または `--global` を省略した場合）は、そのリポジトリだけに適用される設定です。たとえば、個人プロジェクトでは個人のメールアドレスを使い、会社のプロジェクトでは会社のメールアドレスを使いたい場合に便利です。

```
# 特定のリポジトリでだけ会社のメールアドレスを使う
cd /path/to/work/project
git config user.email "tsubasa@company.com"
```

この仕組みについては、2.7 節で設定ファイルの階層構造として詳しく解説します。

2.5. プライバシー保護：noreply メールアドレス

前節で「GitHub に登録したメールアドレスを Git の設定に使いましょう」と書きました。しかし、ここで一つ注意すべきことがあります。公開リポジトリにコミットすると、そのコミットに記録されたメールアドレスは**世界中の誰でも閲覧できる**状態になるということです。

これはプライバシーの観点から問題になり得ます。メールアドレスが公開されると、迷惑メール（スパム）の標的になったり、個人を特定する手がかりになったりする可能性があるためです。

2.5.1. GitHub の「noreply メールアドレス」機能

この問題に対処するため、GitHub は各ユーザーに専用の「noreply メールアドレス」を提供しています。これは `xxxxxxx+username@users.noreply.github.com` という形式のアドレスで、メールの受信はできませんが、GitHub がコミットをアカウントに紐づけるためには十分に機能します。

設定手順は次のとおりです。

1. GitHub にログインし、右上のアイコンから **Settings** を開きます
2. 左側メニューの **Emails** をクリックします
3. 「**Keep my email addresses private**」のチェックボックスをオンにします
4. その下に表示される `xxxxxxx+username@users.noreply.github.com` という形式のアドレスをコピーします

スクリーンショット: *GitHub Email* 設定画面で *noreply* アドレスが表示されている状態

次に、ターミナルで Git の設定を更新します。

```
git config --global user.email "xxxxxxx+username@users.noreply.
   github.com"
```

`xxxxxxx` の部分は GitHub が自動的に割り当てる数字の ID で、ユーザーごとに異なります。必ず自分の Settings 画面に表示されたアドレスをそのまま使ってください。

さらに、同じ設定画面の「**Block command line pushes that expose my email**」にもチェックを入れておくと、万が一、個人のメールアドレスが含まれたコミットをプッシュしようとした際に GitHub が拒否してくれます。これは二重の安全策として有効です。

2.6. 追加の推奨設定

Git には非常に多くの設定項目がありますが、ここでは最初に設定しておくと日々の作業が快適になるものを厳選して紹介します。それぞれの設定について、「なぜこの設定が必要なのか」を理解してから設定することを心がけましょう。

2.6.1. デフォルトブランチ名の設定

```
git config --global init.defaultBranch main
```

なぜ必要か： `git init` で新しいリポジトリを作成するとき、最初のブランチ（枝）には自動的に名前が付けられます。古いバージョンの Git ではこの名前が `master` でしたが、近年、業界全体

で `main` に移行する動きが広がっています。GitHub も 2020 年以降、新規リポジトリのデフォルトブランチ名を `main` に変更しました。

ローカル（自分の PC）で作成したリポジトリのデフォルトブランチ名が `master` のままだと、GitHub の `main` と食い違いが生じ、プッシュ時に混乱の原因になります。この設定を行うことで、ローカルと GitHub の両方で `main` に統一され、不要なトラブルを避けられます。

2.6.2. エディタの設定

```
git config --global core.editor "code --wait"
```

なぜ必要か：Git は、コミットメッセージの入力やリベース操作など、いくつかの場面でテキストエディタを自動的に起動します。デフォルトでは多くの環境で Vim が使われますが、Vim の操作に慣れていない方にとっては「エディタが開いたけれど、文字を入力する方法がわからない」「エディタを閉じる方法がわからない」という事態に陥りがちです。

`"code --wait"` は VS Code を起動し、ファイルが閉じられるまで Git が待機するという意味です。`--wait` がないと、VS Code が起動した瞬間に Git が「エディタが終了した」と判断してしまい、空のコミットメッセージで処理が進んでしまいます。

2.6.3. .DS_Store の無視（macOS のみ）

```
echo ".DS_Store" >> ~/.gitignore_global
git config --global core.excludesfile ~/.gitignore_global
```

なぜ必要か：macOS の Finder は、フォルダを開くたびに `.DS_Store` という隠しファイルを自動的に作成します。このファイルにはフォルダの表示設定（アイコンの配置やウィンドウサイズなど）が記録されており、macOS 独自のものです。

このファイルが Git の管理対象に入ってしまうと、リポジトリに不要なファイルが紛れ込み、Windows や Linux を使っているチームメンバーにとっては無意味な変更として表示されます。`~/.gitignore_global` にこのファイル名を登録し、すべてのリポジトリで自動的に無視されるように設定しておきましょう。

2.6.4. カラー表示の有効化

```
git config --global color.ui auto
```

なぜ必要か：Git の出力は、デフォルトではすべて同じ色（白または黒）で表示されることがあります。`color.ui auto` を設定すると、`git diff` で追加行が緑、削除行が赤で表示されるなど、情報

が視覚的に区別しやすくなります。`auto` は「ターミナルがカラー表示に対応している場合にのみ色を付ける」という意味で、ファイルにリダイレクトする場合などには色コードが混入しません。

2.6.5. プッシュのデフォルト動作

```
git config --global push.default current
```

なぜ必要か： `git push` コマンドを実行したとき、「どのブランチをどこにプッシュするか」を明示しなかった場合の動作を指定します。`current` は「現在チェックアウトしているブランチと同じ名前のリモートブランチにプッシュする」という設定です。

この設定がないと、Git のバージョンによっては想定と異なるブランチにプッシュされたり、「どのブランチをプッシュするか指定してください」というエラーが出たりすることがあります。`current` にしておけば、`git push` だけで直感的にプッシュが完了します。

2.6.6. プル時のリベース設定

```
git config --global pull.rebase true
```

なぜ必要か： `git pull` はリモートの変更を取り込むコマンドですが、取り込み方には「マージ」と「リベース」の 2 つの方式があります。デフォルトの「マージ」方式では、自分の変更とリモートの変更を統合するたびに「マージコミット」という余分なコミットが作られ、履歴が複雑になりがちです。

`pull.rebase true` を設定すると、リモートの変更を取り込む際に「リベース」方式が使われます。リベースは、自分のコミットをリモートの最新コミットの上に載せ直す操作で、コミット履歴が一直線に保たれます。これは後から履歴を読み返すときに大変わかりやすくなります。ただし、リベースの概念は第 7 章で詳しく解説しますので、今はひとまず「履歴をきれいに保つための設定」と理解しておいてください。

2.7. 設定の確認と設定ファイルの階層

2.7.1. 現在の設定を確認する

ここまでの設定が正しく反映されているかを確認してみましょう。

```
# すべての設定を一覧表示
git config --list

# 特定の設定項目だけを確認
git config user.name
```

```
git config user.email
```

`git config --list` は、現在有効なすべての Git 設定を表示します。設定項目が多い場合は画面がスクロールしますので、`q` キーを押して表示を終了できます。

2.7.2. 設定ファイルの 3 つの階層

Git の設定は、3 つのレベルの設定ファイルに分かれています。これはちょうど「国の法律」「都道府県の条例」「市区町村のルール」のような関係です。より狭い範囲のルールが、より広い範囲のルールよりも優先されます。

レベル	オプション	ファイルの場所	適用範囲
System	`--system`	`/etc/gitconfig`	その PC 上のすべてのユーザー
Global	`--global`	`~/.gitconfig`	現在のユーザーのすべてのリポジトリ
Local	`--local`	`.git/config`	そのリポジトリのみ

優先順位は Local > Global > System です。つまり、Global で `user.email` を個人アドレスに設定していても、特定のリポジトリの Local で会社のアドレスを設定すれば、そのリポジトリでは Local の設定が使われます。

この階層構造を理解していれば、「普段は個人アドレスだけど、仕事のリポジトリだけは会社のアドレスを使う」といった使い分けが自在にできるようになります。

設定ファイルの中身を直接編集したい場合は、次のコマンドでエディタを開けます。

```
# Global設定ファイルをエディタで開く
git config --global --edit
```

設定ファイルは INI 形式のテキストファイルで、次のような構造になっています。

```
[user]
    name = あなたの名前
    email = your-email@example.com
[init]
    defaultBranch = main
[core]
    editor = code --wait
```

コマンドで設定するのも、このファイルを直接編集するのも、結果は同じです。最初はコマンドで設定し、慣れてきたらファイルを直接編集するほうが効率的な場合もあるでしょう。

2.8. エディタとターミナルの準備

Git の操作はターミナル（コマンドライン）で行うのが基本ですが、現代の開発では、コードを書くためのエディタとターミナルが統合された環境を使うのが一般的です。ここでは、本書を通して使用するエディタとターミナルの準備について解説します。

2.8.1. Visual Studio Code（VS Code）の導入

本書では、エディタとして **Visual Studio Code**（通称 VS Code）を推奨しています。VS Code は Microsoft が開発した無料のコードエディタで、世界中の開発者に圧倒的に支持されています。Git との統合機能も優れており、コードの編集から Git 操作まで、一つの画面で完結できます。

インストール手順：

1. *https://code.visualstudio.com/* にアクセスします
2. OS に合ったインストーラーをダウンロードします
3. ダウンロードしたファイルを実行し、画面の指示に従ってインストールします

スクリーンショット：*VS Code* 公式サイトのダウンロードページ

コマンドラインから VS Code を起動できるようにする：

VS Code をインストールしたら、ターミナルから `code` コマンドで起動できるように設定しましょう。これは 2.6 節で紹介した `core.editor` の設定で必要になります。

- **macOS**：VS Code を開き、`Cmd+Shift+P` でコマンドパレットを表示し、「Shell Command: Install 'code' command in PATH」を検索して実行します。
- **Windows**：Git for Windows のインストール時に自動的に設定されていることが多いですが、されていない場合は VS Code のインストーラーを再実行し、「Add to PATH」のチェックボックスを有効にしてください。
- **Linux**：パッケージマネージャー経由でインストールした場合は、自動的に PATH に追加されます。

2.8.2. おすすめの VS Code 拡張機能

VS Code は「拡張機能（Extensions）」を追加することで、機能を自由に拡張できます。Git/GitHub 関連でぜひ導入しておきたい拡張機能を紹介します。

- **GitLens**：ファイル内の各行がいつ、誰によって変更されたかを表示してくれます。`git blame` の視覚的な拡張版のような存在で、コードの変更履歴を理解するのに非常に役立ちます。
- **GitHub Pull Requests and Issues**：VS Code の中からプルリクエストの作成・レビュー・マージや Issue の管理ができるようになります。わざわざブラウザに切り替えなくて済むため、

作業効率が大きく向上します。

- **GitHub Copilot**：AI がコードの補完や提案を行ってくれます。有料サービスですが、学生は GitHub Education で無料利用できます。

拡張機能のインストールは、VS Code の左側メニューにある四角いアイコン（Extensions）をクリックし、検索バーに名前を入力して「Install」ボタンを押すだけです。

スクリーンショット: *VS Code* の拡張機能インストール画面

2.8.3. ターミナルの選び方

Git 操作はターミナル上でコマンドを入力して行います。各 OS で使いやすいターミナルを選びましょう。

OS	推奨ターミナル	特徴
macOS	Terminal.app（標準）または iTerm2	Terminal.app は最初から入っています。iTerm2 は分割表示やカスタマイズ性に優れた高機能ターミナルです。
Windows	Windows Terminal + Git Bash	Windows Terminal は Microsoft 公式の高機能ターミナルで、タブ機能があります。Git for Windows をインストールすると使える Git Bash を Windows Terminal のプロファイルに追加すると便利です。
Linux	標準ターミナル	Ubuntu なら GNOME Terminal、Fedora なら GNOME Console など、ディストリビューションに含まれるもので十分です。

また、VS Code には**統合ターミナル**と呼ばれる機能があり、エディタの画面内でターミナルを開くことができます。`Ctrl+` `（バッククォート）を押すと表示されます。コードを編集しながら Git コマンドを実行できるため、非常に便利です。本書の操作では、この統合ターミナルを使っても、独立したターミナルアプリを使っても、どちらでも構いません。

2.9. まとめ

本章では、Git を自分のパソコンにインストールし、使い始めるための一連の準備を行いました。振り返ってみましょう。

- **Git のインストール**：macOS では Xcode Command Line Tools または Homebrew、Windows では Git for Windows または winget、Linux では apt または dnf を使ってインストールしました。
- **インストールの確認**：`git --version` コマンドでバージョン番号が表示されれば成功です。

- **初期設定**：`user.name` と `user.email` を `--global` で設定しました。この情報はすべてのコミットに記録されます。
- **プライバシー保護**：GitHub が提供する noreply メールアドレスを使うことで、個人のメールアドレスが公開されるのを防ぎました。
- **推奨設定**：デフォルトブランチ名、エディタ、カラー表示などを設定し、日々の作業を快適にしました。
- **設定の階層構造**：System < Global < Local の 3 段階で設定が管理され、より狭い範囲の設定が優先されることを学びました。
- **ツールの準備**：VS Code と推奨拡張機能をインストールし、ターミナル環境を整えました。

これで、Git を使い始めるための土台が完成しました。次の章では、いよいよ GitHub にアカウントを作成し、オンラインでの活動を始める準備をします。

GitHub アカウント — 開発者としての第一歩

3.1. この章で学ぶこと

本章では、GitHub にアカウントを作成し、プロフィールを整え、開発者としてのオンライン上の「顔」を作り上げていきます。GitHub のアカウントは、単なるログイン情報ではありません。それは、あなたのコードやプロジェクトへの貢献を蓄積していく「デジタルなポートフォリオ」であり、世界中の開発者とつながるための「名刺」でもあります。

具体的には、以下の内容を学びます。

- GitHub アカウントの**作成手順**と各入力項目の意味
- 長く使い続けるための**ユーザー名の選び方**
- アカウントを不正アクセスから守る**二段階認証（2FA）**の設定
- 自分をアピールする**プロフィール**の整え方
- 自分だけのページを作る**プロフィール README** の仕組み
- 用途に応じた**料金プランの選び方**
- 情報の洪水に溺れないための**通知設定のカスタマイズ**

GitHub は、世界で 1 億人を超える開発者が利用するプラットフォームです。アカウントの作成は、そのコミュニティへの参加証を手に入れるようなものだと考えてください。

3.2. アカウント作成の手順

GitHub のアカウント作成は無料で、数分で完了します。以下の手順に沿って進めましょう。

ステップ 1：GitHub のサイトにアクセスする

ウェブブラウザで *https://github.com* にアクセスします。トップページが表示されたら、画面右上の「**Sign up**」ボタンをクリックしてください。

> スクリーンショット: *GitHub トップページの Sign up ボタン*

ステップ 2：メールアドレスを入力する

最初に求められるのはメールアドレスです。このアドレスは、アカウントの認証やパスワードリセット、セキュリティ通知の受信に使われます。日常的に確認するメールアドレスを使用してください。

ここで入力したメールアドレスは、後から変更したり、複数のアドレスを追加したりすることも可能です。ただし、最初に登録するアドレスが「プライマリメールアドレス」となり、GitHub からの重要な通知はこのアドレスに届きます。

ステップ 3：パスワードを設定する

パスワードは、以下のいずれかの条件を満たす必要があります。

- **15 文字以上**（制約なし）
- **8 文字以上**で、数字と英小文字をそれぞれ 1 つ以上含む

セキュリティの観点から、15 文字以上のパスフレーズ（たとえば複数の単語をつなげたもの）を使うことを強くおすすめします。「correct-horse-battery-staple」のような、ランダムな単語を組み合わせたパスフレーズは、短くて複雑なパスワードよりも覚えやすく、かつ破られにくいとされています。パスワードマネージャー（1Password、Bitwarden など）を使って生成・管理するのも良い方法です。

ステップ 4：ユーザー名を選ぶ

ユーザー名は、GitHub 上でのあなたの識別子です。`github.com/ユーザー名` があなたのプロフィールページの URL になります。ユーザー名の選び方については次の 3.3 節で詳しく解説しますが、ここでは「慎重に選ぶ価値がある」ということだけ覚えておいてください。

ステップ 5：メール配信の設定

GitHub からの製品アップデートやお知らせをメールで受け取るかどうかを選択します。これは後からでも変更できるので、気軽に選んでください。

ステップ 6：CAPTCHA の認証

ロボットではないことを証明するためのパズルが表示されます。画面の指示に従って操作してください。

ステップ 7：メール認証を完了する

「Create account」ボタンを押すと、入力したメールアドレスに認証コードが送信されます。メールを確認し、記載されている数字のコードを GitHub の画面に入力してください。メールが届かない場合は、迷惑メールフォルダを確認するか、数分待ってから再送信を試みてください。

スクリーンショット: メール認証コード入力画面

ステップ 8：パーソナライゼーション（任意）

アカウント作成直後に、チームの規模や利用目的などに関する質問が表示されることがあります。これらは GitHub がサービス改善のために収集する情報であり、スキップしても問題ありません。

以上でアカウントの作成は完了です。おめでとうございます。GitHub の世界へようこそ。

3.3. ユーザー名の選び方

GitHub のユーザー名は、あなたのオンラインアイデンティティの一部になります。軽い気持ちで決めてしまいがちですが、少し立ち止まって考える価値があります。

3.3.1. なぜユーザー名が重要なのか

ユーザー名は、以下のさまざまな場面であなたを代表します。

- **プロフィール URL**：`github.com/ユーザー名` があなたのページになります
- **リポジトリ URL**：`github.com/ユーザー名/リポジトリ名` で公開されます
- **コミット履歴**：他の人のリポジトリに貢献したとき、ユーザー名が記録されます
- **Issue やプルリクエスト**：あなたの発言にはすべてユーザー名が表示されます
- **`@ユーザー名` によるメンション**：他のユーザーがあなたに言及するときに使われます

つまり、ユーザー名は GitHub 上でのあなたの「名前」そのものなのです。

3.3.2. 選び方のヒント

- **短くて覚えやすいもの**：URL の一部になるため、あまりに長いユーザー名は扱いにくくなります。
- **本名でもハンドルネームでも OK**：技術コミュニティでは、本名を使う人もハンドルネームを使う人もいます。どちらが良いかに正解はありません。
- **プロフェッショナルさを意識する**：将来、就職活動やオープンソースへの貢献を考えているなら、あまりにカジュアルすぎるユーザー名は避けたほうが無難です。GitHub のプロフィールを「技術者としての履歴書」として見る採用担当者も少なくありません。
- **使える文字**：英数字とハイフン（`-`）のみ使用可能です。アンダースコア（`_`）は使えません。ハイフンで始めたり終わったりすることもできません。

3.3.3. ユーザー名は後から変更できるか

はい、変更は可能です。Settings → Account → Change username から変更できます。ただし、変更にはいくつかの注意点があります。

- 旧ユーザー名を含む**すべての URL が変更**されます。他のサイトやドキュメントからリンクされている場合、それらは無効になります。
- GitHub は一定期間、旧 URL からのリダイレクトを提供しますが、永久ではありません。
- 旧ユーザー名は**他のユーザーが取得できるようになります**。つまり、あなたの旧ユーザー名で別の人がアカウントを作成する可能性があります。

このように、ユーザー名の変更は可能ですがリスクも伴うため、最初の段階でよく考えて決めることをおすすめします。

3.4. 二段階認証（2FA）— なぜ必要か

GitHub では、2023 年以降、すべてのアカウントに対して**二段階認証（Two-Factor Authentication, 2FA）が必須**となっています。これはセキュリティ上の要請であり、選択の余地はありません。しかし、「面倒な手続き」として捉えるのではなく、「自分のコードと活動を守るための盾」として前向きに設定しましょう。

3.4.1. 二段階認証とは何か

通常のログインでは、ユーザー名とパスワードの 2 つの情報を入力します。二段階認証は、これに加えて「もう一つの認証手段」を要求する仕組みです。

たとえるなら、銀行の ATM を思い浮かべてください。キャッシュカード（持っているもの）と暗証番号（知っていること）の両方がなければ、お金を引き出せません。二段階認証もこれと同じで、パスワード（知っていること）とスマートフォンに表示される認証コード（持っているもの）の両方がなければ、ログインできないようにしています。

仮にパスワードが漏洩した場合でも、攻撃者があなたのスマートフォンを持っていなければログインできないため、アカウントの安全性が大幅に高まります。

3.4.2. 設定手順

1. GitHub にログインし、右上のアイコンをクリックして **Settings** を開きます
2. 左側のメニューから **Password and authentication** を選択します
3. 「Two-factor authentication」のセクションにある **Enable** ボタンをクリックします
4. 認証方法を選択します。**認証アプリ（Authenticator app）**の使用が推奨されています

スクリーンショット: *2FA* 設定画面

認証アプリを選択すると、QR コードが表示されます。この QR コードを認証アプリでスキャンすると、アプリに GitHub のエントリが追加され、30 秒ごとに変わる 6 桁の認証コードが表示されるようになります。表示されたコードを GitHub の画面に入力して、設定を完了してください。

3.4.3. 推奨認証アプリ

認証アプリにはさまざまな選択肢がありますが、代表的なものを紹介します。

アプリ	対応 OS	特徴
1Password	macOS, Windows, iOS, Android	パスワード管理と二段階認証を一つのアプ で統合できます。有料ですが、セキュリティ 理の一元化という点で非常に優れています
Authy	macOS, Windows, iOS, Android	無料で、複数デバイス間の同期に対応して ます。スマートフォンを買い替えた際にも 行が容易です。
Google Authenticator	iOS, Android	Google が提供するシンプルな認証アプリ す。必要最低限の機能に絞られており、使 方に迷うことがありません。
Microsoft Authenticator	iOS, Android	Microsoft 製品（Azure, Microsoft 365 など を使う環境で便利です。顔認証や指紋認証 よるロック機能もあります。

どのアプリを選んでも基本的な機能は同じです。すでにパスワードマネージャーを使っている場合は、そのアプリに認証コード機能が含まれていないか確認してみてください。

3.4.4. リカバリーコードの重要性

二段階認証を設定すると、**リカバリーコード**（復旧コード）が表示されます。これは、スマートフォンを紛失したり、認証アプリにアクセスできなくなったりした場合の「最後の砦」です。

リカバリーコードは必ず安全な場所に保管してください。これを紛失すると、認証アプリが使えない状況でアカウントにアクセスする手段がなくなります。GitHub のサポートに問い合わせても、本人確認に時間がかかり、最悪の場合アカウントにアクセスできなくなる可能性もあります。

保管場所の例を挙げます。

- **パスワードマネージャーのセキュアノート**に保存する（おすすめ）
- **印刷して金庫や鍵付きの引き出し**に保管する
- **暗号化されたファイル**としてクラウドストレージに保存する

紙に書いてモニターに貼る、スマートフォンのメモ帳に平文で保存する、といった方法は避けてください。リカバリーコードはパスワードと同等の機密情報です。

3.5. プロフィール設定 — 基本情報

GitHub のプロフィールは、他の開発者があなたを知るための最初の窓口です。設定は任意ですが、プロフィールが充実しているアカウントは、コミュニティにおいて信頼感を与えやすくなります。

3.5.1. 基本プロフィールの設定

Settings → **Public profile** で、以下の項目を設定できます。

項目	説明	記入のヒント
Name	表示名。ユーザー名とは別に設定できます。	本名でもニックネームでも OK。
Bio	自己紹介文。160 文字以内です。	自分の専門分野や興味を簡潔に書き しょう。
URL	個人サイトやブログの URL です。	ポートフォリオサイトやブログがあ ばリンクしましょう。
Twitter/X username	X（旧 Twitter）のアカウント名です。	技術発信を行っている場合は連携す と便利です。
Company	所属する組織や企業名です。	`@組織名` の形式で入力すると Organ tion にリンクされます。
Location	所在地です。	「Tokyo, Japan」など。詳細な住所は かないでください。
Pronouns	代名詞（人称）です。	he/him、she/her、they/them など

スクリーンショット: プロフィール設定画面

3.5.2. プロフィール写真の設定

プロフィール写真は、GitHub 上のあらゆる場所であなたのアイコンとして表示されます。コミット一覧、Issue、プルリクエストのコメントなど、活動の痕跡にはすべてこのアイコンが添えられます。

設定は Settings → **Public profile** → **Profile picture** の「Edit」ボタンから行います。推奨されるサイズは正方形で、最低でも 200×200 ピクセルあると鮮明に表示されます。

写真を設定しない場合、GitHub が自動生成する幾何学模様のアイコン（Identicon）が使われます。これは他のユーザーと視覚的に区別しにくいため、何でも構わないので写真やイラストを設定しておくことをおすすめします。自分の顔写真でなくても、イラストやロゴなど、あなたを識別できるものであれば十分です。

3.6. プロフィール README — 自分だけのページを作る

GitHub には、自分のプロフィールページを Markdown で自由にカスタマイズできる「プロフィール README」という機能があります。`github.com/ユーザー名` にアクセスしたとき、リポジトリ一覧の上にこの README の内容が表示されます。いわば、GitHub における自分の「ホームページ」を作れる機能です。

3.6.1. 作り方

プロフィール README を作るには、**自分のユーザー名と同じ名前のリポジトリ**を作成する必要があります。たとえば、ユーザー名が `tsubasa` であれば、`tsubasa` という名前のリポジトリを作成します。

1. GitHub の画面右上の「+」ボタンから「**New repository**」を選択します
2. Repository name に**自分のユーザー名**をそのまま入力します
3. 入力すると「You found a secret! tsubasa/tsubasa is a special repository...」というメッセージが表示されます。これが表示されれば正しく設定できています
4. 「**Public**」を選択します（プロフィール README はパブリックリポジトリでのみ機能します）
5. 「**Add a README file**」にチェックを入れます
6. 「**Create repository**」をクリックします

作成後、このリポジトリ内の `README.md` を編集すれば、その内容がプロフィールページに反映されます。

スクリーンショット: プロフィール *README* が表示されたプロフィールページ

3.6.2. 書き方の例

Markdown の書式（見出し、箇条書き、リンクなど）を自由に使えます。以下は、よくあるパターンの一例です。

```
# Hi there

## About Me
- Bioinformatics researcher
- Currently learning metagenomics
- Python, R, Shell scripting
- How to reach me: example@example.com

## GitHub Stats
![GitHub Stats](https://github-readme-stats.vercel.app/api?username
   =YOUR_USERNAME&show_icons=true&theme=default)

## Top Languages
![Top Langs](https://github-readme-stats.vercel.app/api/top-langs/?
   username=YOUR_USERNAME&layout=compact&theme=default)
```

`YOUR_USERNAME` の部分を自分のユーザー名に置き換えてください。上記の「GitHub Stats」や「Top Languages」は、外部サービス（github-readme-stats）が自動生成する画像カードで、あなたの GitHub での活動状況やよく使うプログラミング言語を視覚的に表示してくれます。

3.6.3. よく使われるバッジとウィジェット

プロフィール README をさらに充実させるために、さまざまなバッジやウィジェットを埋め込むことができます。これらは Markdown の画像記法を使って表示します。

```
<!-- GitHub Streak（連続貢献日数） -->
![GitHub Streak](https://streak-stats.demolab.com/?user=
   YOUR_USERNAME&theme=default)

<!-- 訪問者カウンター -->
![Visitors](https://visitor-badge.laobi.icu/badge?page_id=
   YOUR_USERNAME)

<!-- スキルバッジ -->
```

```
![Python](https://img.shields.io/badge/Python-3776AB?style=for-the-
   badge&logo=python&logoColor=white)
![R](https://img.shields.io/badge/R-276DC3?style=for-the-badge&logo
   =r&logoColor=white)
![Git](https://img.shields.io/badge/Git-F05032?style=for-the-badge&
   logo=git&logoColor=white)
```

スキルバッジは *https://shields.io/* で自由にカスタマイズでき、色やロゴ、スタイルを選ぶことができます。自分の得意な技術をバッジとして並べると、一目で技術スタックが伝わるプロフィールになります。

ただし、バッジやウィジェットを詰め込みすぎると、かえって読みにくくなることもあります。伝えたい情報を絞り、シンプルで見やすいプロフィールを心がけましょう。

3.7. GitHub の料金プラン

GitHub には複数の料金プランがありますが、個人で学習や開発を始める段階では **Free プラン**で十分です。ここでは各プランの概要を紹介しますので、将来的なニーズに応じて参考にしてください。

3.7.1. 各プランの概要

プラン	料金	主な機能
Free	無料	無制限のパブリック・プライベートリポジトリ、GitHub Actions 月 2,000 分、GitHub Packages 500MB
Pro	$4/月	Free の全機能に加え、プライベートリポジトリでの高度な機能
Team	$4/ユーザー/月	組織向けのチーム管理機能、コードオーナー、ドラフト PR
Enterprise	$21/ユーザー/月	SAML SSO、詳細な監査ログ、高度なセキュリティ機能、専用サポート

Free プランで使えることをもう少し具体的に見てみましょう。

- パブリックリポジトリもプライベートリポジトリも**無制限**に作成できます
- プライベートリポジトリでも **3 人まで**のコラボレーターを招待できます
- GitHub Actions（自動化機能）は**月 2,000 分**まで無料で利用できます
- GitHub Pages（Web サイト公開機能）もパブリックリポジトリで利用可能です

個人学習や小規模なプロジェクトであれば、Free プランの制約に引っかかることはまずありません。

3.7.2. GitHub Education（学生・教育者向け）

大学や高等教育機関に所属する学生であれば、**GitHub Education** プログラムを通じて、Pro 相当の機能を無料で利用できます。さらに、「GitHub Student Developer Pack」として、GitHub 以外の多くのサービス（JetBrains IDE、Namecheap ドメイン、DigitalOcean クレジットなど）の特典も付属します。

申請手順は以下のとおりです。

1. *https://education.github.com* にアクセスします
2. 「**Get benefits**」または「**Join Global Campus**」をクリックします
3. 学校のメールアドレスを使用するか、学生証の写真をアップロードします
4. 審査が行われ、承認されると特典が適用されます

審査には数日かかることがありますが、有効な学生証や在学証明があれば、ほとんどの場合承認されます。学生の方は、ぜひ活用してください。

3.8. 通知設定のカスタマイズ

GitHub を使い始めると、さまざまな通知が届くようになります。自分が参加しているリポジトリの Issue にコメントがついたとき、プルリクエストにレビューが付いたとき、自分がメンションされたときなど、活動に応じて通知が発生します。

通知は便利な反面、設定を適切にしないと情報の洪水に飲まれてしまいます。メールの受信ボックスが GitHub の通知で埋め尽くされるのは、多くの開発者が経験する「あるある」です。ここでは、快適に作業を続けるための通知設定を紹介します。

3.8.1. 通知設定の画面

Settings → **Notifications** で、通知に関する詳細な設定ができます。主な設定項目は以下のとおりです。

Default notifications email：通知メールの送信先アドレスを選択します。複数のメールアドレスを登録している場合は、通知を受け取りたいアドレスを指定できます。

Automatically watch repositories：新しくリポジトリの権限を得たとき（作成したとき、アクセス権を付与されたとき）に、自動的にそのリポジトリを「Watching（監視）」状態にするかどうかの設定です。Watching にすると、そのリポジトリのすべての活動が通知されます。

Subscriptions：現在購読中の Issue やプルリクエストを管理できます。不要な通知が来ている場合は、ここで購読を解除できます。

3.8.2. おすすめの設定

初期設定では通知が多すぎることがあるため、以下のように調整することをおすすめします。

- **Web notifications**（GitHub 上での通知）：**有効**のままにしておきます。画面上部のベルアイ

コンに通知が表示されるだけなので、邪魔になりません。

- **Email notifications**：必要なものだけに絞ります。すべての活動に対してメールが来ると、重要な通知が埋もれてしまいます。「自分がメンションされたとき」や「レビューをリクエストされたとき」だけメールを受け取るように設定するのがバランスが良いでしょう。
- **Automatically watch repositories**：**無効**にすることを強くおすすめします。多くのリポジトリに関わるようになると、この設定が有効のままだとあっという間に通知が爆発的に増えます。本当に監視したいリポジトリだけを手動で Watching に設定する方が、ノイズを抑えられます。

スクリーンショット: 通知設定画面

通知の管理は、GitHub を快適に使い続けるための重要なスキルです。最初から完璧な設定を目指す必要はありません。使い続ける中で「この通知は不要だな」「この通知はメールでも欲しいな」と感じたら、その都度調整していきましょう。

3.9. Achievements（実績バッジ）— GitHubが認める功績

3.9.1. Achievements とは

GitHub には、プラットフォーム上での活動に応じて自動的に付与される **Achievements（アチーブメント、実績バッジ）**という仕組みがあります。これはゲームの「実績解除」に似た機能で、特定の条件を満たすとプロフィールページにバッジが表示されます。

Achievements は 2022 年 6 月にパブリックベータとして導入され、現在も進化を続けています。プロフィールページの左側カラムに表示され、他のユーザーがあなたのページを訪れたときに、あなたの GitHub 上での活動の幅と深さを一目で伝えることができます。

スクリーンショット: プロフィールページの *Achievements* バッジ表示エリア

3.9.2. Achievements の表示設定

Achievements の表示は、Settings → Public profile で制御できます。すべてのバッジを表示することも、特定のバッジだけを選んで表示することも、すべてを非表示にすることもできます。

3.9.3. 現在獲得できる Achievements 一覧

以下は、現在アクティブな Achievements の一覧です。多くの Achievements には**ティア（段階）**が設けられており、条件を多く達成するほど上位のバッジにグレードアップします。

3.9.3.1. Pull Shark（プルシャーク）

Pull Shark は、**マージされた Pull Request の数**に基づいて付与されるバッジです。GitHub における最も基本的な貢献活動である PR 作成を評価するものであり、多くの開発者が最初に獲得する Achievement のひとつです。

ティア	条件
デフォルト	マージされた PR を 2 件達成
ブロンズ	マージされた PR を 16 件達成
シルバー	マージされた PR を 128 件達成
ゴールド	マージされた PR を 1,024 件達成

ゴールドの Pull Shark を獲得するには 1,024 件もの PR がマージされる必要があり、長期にわたって活発に開発を続けている証です。

3.9.3.2. Galaxy Brain（ギャラクシーブレイン）

Galaxy Brain は、**GitHub の Discussions（ディスカッション）で回答が「Accepted Answer（採用された回答）」として選ばれた回数**に基づいて付与されます。技術的な質問に的確に回答し、コミュニティを助けた証です。

ティア	条件
デフォルト	2 件の回答が採用
ブロンズ	8 件の回答が採用
シルバー	16 件の回答が採用
ゴールド	32 件の回答が採用

3.9.3.3. Starstruck（スターストラック）

Starstruck は、**自分が作成したリポジトリが獲得したスター（Star）の数**に基づいて付与されます。スターとは、他のユーザーがリポジトリを「気に入った」「後で見返したい」と思ったときに付けるブックマークのようなものです。多くのスターを集めるということは、多くの人に価値を認められたプロジェクトを作ったことを意味します。

ティア	条件
デフォルト	16 スター
ブロンズ	128 スター
シルバー	512 スター
ゴールド	4,096 スター

ゴールドの Starstruck は、世界的に注目される OSS プロジェクトの作者であることを示す非常に名誉あるバッジです。

3.9.3.4. Pair Extraordinaire（ペア・エクストラオーディネール）

Pair Extraordinaire は、**マージされた Pull Request において共同著者（co-author）としてコミットに記名された回数**に基づいて付与されます。ペアプログラミングやチーム開発において、他のメンバーと協力してコードを書いたことを示します。

ティア	条件
デフォルト	1 件
ブロンズ	10 件
シルバー	24 件
ゴールド	48 件

コミットメッセージに `Co-authored-by: Name <email>` のトレーラーを付けることで、共同著者として記録されます。

3.9.3.5. YOLO（ヨロ）

YOLO は、**レビューなしで Pull Request をマージした**ときに付与されるバッジです。”You Only Live Once”（人生は一度きり）の略で、「レビューなしでマージするなんて大胆だね」というユーモアが込められています。

ティア	条件
デフォルトのみ	レビューなしで PR を 1 件マージ

ティアはデフォルトの 1 段階のみです。このバッジは少し皮肉を込めた「不名誉」な実績ともいえますが、個人プロジェクトでは珍しくない操作です。

3.9.3.6. Quickdraw（クイックドロー）

Quickdraw は、**Issue または Pull Request を開いてから 5 分以内にクローズした**ときに付与されます。素早い判断力を評価するバッジです。

ティア	条件
デフォルトのみ	5 分以内に Issue/PR をクローズ

3.9.3.7. Public Sponsor（パブリックスポンサー）

Public Sponsor は、**GitHub Sponsors を通じてオープンソースの開発者やプロジェクトに金銭的な支援を行った**ときに付与されます。コードを書くだけでなく、OSS の持続可能性に貢献する姿勢を示すバッジです。

3.9.4. 引退した Achievements（もう獲得できないバッジ）

以下のバッジは特定の歴史的イベントに関連しており、現在は新たに獲得することはできません。すでに獲得したユーザーのプロフィールには引き続き表示されます。

ティア	条件
デフォルトのみ	GitHub Sponsors で 1 回以上のスポンサー

Arctic Code Vault Contributor（北極コード保管庫コントリビューター）は、2020 年に GitHub が実施した「GitHub Archive Program」に貢献したユーザーに付与されました。このプログラムでは、世界中のオープンソースコードのスナップショットが北極圏のスバールバル諸島にある保管庫にフィルムとして保存されました。対象リポジトリに 2020 年 2 月 2 日時点でコントリビューションがあったユーザーがこのバッジを獲得しています。

Mars 2020 Contributor（火星 2020 コントリビューター）は、NASA の火星探査ヘリコプター「Ingenuity」のミッションに関連するリポジトリに貢献したユーザーに付与されました。GitHub 上でホストされていたオープンソースプロジェクトのコードが実際に火星で使われたという、歴史的な出来事を記念するバッジです。

3.9.5. Highlights（ハイライト）

Achievements とは別に、**Highlights** と呼ばれる特別なバッジもプロフィールに表示されることがあります。

バッジ	条件
Pro	GitHub Pro プランを利用中
Developer Program Member	GitHub Developer Program に登録
Security Bug Bounty Hunter	GitHub のセキュリティバグバウンティに参加
GitHub Campus Expert	GitHub Campus Program のエキスパートに認定
Security Advisory Credit	GitHub Advisory Database にセキュリティアドバイザリが採用

3.10. コントリビューショングラフ（草）— 活動の可視化

3.10.1. 「草」とは何か

GitHub のプロフィールページには、過去 1 年間の活動量を緑色の濃淡で示す**コントリビューショングラフ**が表示されます。日本の開発者コミュニティでは、緑色のマスが芝生のように見えることから、これを「草」と呼ぶ文化があります。「草を生やす」とは、GitHub で活動してグラフを緑色にすることを意味します。

スクリーンショット: コントリビューショングラフの表示例

3.10.2. コントリビューションとしてカウントされる活動

以下の活動がコントリビューションとしてカウントされ、グラフの色に反映されます。

- **コミット**: デフォルトブランチまたは `gh-pages` ブランチへのコミット

- **Issue**: 新しい Issue の作成
- **Pull Request**: 新しい PR の作成
- **Pull Request レビュー**: PR へのレビュー投稿

注意点として、**Fork したリポジトリへのコミットはカウントされません**（Fork から元のリポジトリへの PR がマージされた場合は除く）。また、コミットに設定されているメールアドレスが GitHub アカウントに紐づいていない場合もカウントされません。第 2 章で設定したメールアドレスが正しく設定されているか確認しましょう。

3.10.3. コントリビューションの見せ方

プロフィールページの Contributions セクションでは、コントリビューションの表示方法を切り替えることができます。デフォルトでは「Contribution activity」が表示されますが、設定によりプライベートリポジトリへのコントリビューションも含めるかどうかを選択できます。

Settings → Public profile → **Contributions & Activity** セクションで、「Private contributions」の表示をオンにすると、プライベートリポジトリでの活動もグラフに反映されます（ただし、どのリポジトリで活動したかは公開されません。カウント数のみが反映されます）。

3.11. まとめ

本章では、GitHub にアカウントを作成し、開発者としてのオンラインプレゼンスを整えました。振り返ってみましょう。

- **アカウントの作成**：メールアドレス、パスワード、ユーザー名を入力し、メール認証を完了してアカウントを作成しました
- **ユーザー名の選び方**：URL の一部になり、変更にはリスクが伴うため、慎重に選ぶことの重要性を学びました
- **二段階認証（2FA）**：パスワードだけでなく認証アプリのコードも必要にすることで、アカウントのセキュリティを大幅に強化しました。リカバリーコードの安全な保管も忘れてはなりません
- **プロフィール設定**：名前、自己紹介、写真などを設定し、他の開発者から信頼されるプロフィールを整えました
- **プロフィール README**：自分のユーザー名と同じ名前のリポジトリを作り、プロフィールページをカスタマイズする方法を学びました
- **料金プラン**：個人利用なら Free プランで十分であること、学生には GitHub Education という強力な支援があることを確認しました
- **通知設定**：情報の洪水に溺れないよう、通知を適切にカスタマイズする方法を学びました
- **Achievements**：Pull Shark、Galaxy Brain、Starstruck などの実績バッジの仕組みと獲得条件を学びました

- **コントリビューショングラフ**：「草」と呼ばれる活動可視化の仕組みと、カウント対象の活動を確認しました

アカウントとプロフィールが整ったところで、次はいよいよ自分のパソコンと GitHub を「つなぐ」作業に入ります。パソコンから GitHub にコードを送ったり、GitHub からコードを取得したりするためには、安全な接続方法を設定する必要があります。

認証と接続 — パソコンとGitHubを安全につなぐ

4.1. この章で学ぶこと

本章では、自分のパソコンとGitHubの間に「安全な通信路」を確立する方法を学びます。前章までGitのインストールとGitHubアカウントの作成が完了しましたが、それだけではパソコンからGitHubにコードを送ったり、GitHubからコードを取得したりすることはできません。「このパソコンからの操作は、確かにこのGitHubアカウントの持ち主によるものである」という証明、つまり**認証**が必要です。

具体的には、以下の内容を学びます。

- そもそも**なぜ認証が必要なのか**というセキュリティの基本的な考え方
- **SSH**、**HTTPS + Personal Access Token**、**GitHub CLI** という3つの接続方式の比較
- 最も広く推奨されている**SSH鍵による接続**の設定方法（公開鍵暗号の仕組みも含めて）
- HTTPS接続で使う**Personal Access Token（PAT）**の作成と管理
- コマンド一つで認証を完了できる**GitHub CLI**の活用
- コミットが本当に自分のものであることを証明する**GPG署名**
- 仕事用と個人用など、**複数アカウントの使い分け**方

認証の設定は、一度行えば日常的にやり直す必要はありません。しかし、仕組みを理解しておくことで、トラブルが起きたときに自分で対処できるようになります。少し難しく感じるかもしれませんが、一つずつ丁寧に進めていきましょう。

4.2. なぜ認証が必要なのか

GitHubにコードをアップロード（プッシュ）したり、プライベートリポジトリのコードをダウンロード（クローン）したりする操作を考えてみましょう。もし認証の仕組みがなかったらどうなるでしょうか。

誰でも他人のリポジトリにコードを書き込めてしまいます。悪意のある人がプロジェクトのコードを改ざんしたり、プライベートなコードを盗み見たりすることが可能になります。これは、鍵のかかっていない家に等しい状態です。

認証とは、「あなたが本当にあなたであること」を確認するプロセスです。GitHubにパソコンからアクセスする際、「このアクセスは、GitHubアカウント `tsubasa` の持ち主からのものです」ということを、何らかの方法で証明する必要があります。

以前は、ユーザー名とパスワードを入力するだけで認証できましたが、GitHubは2021年にパスワード認証を廃止しました。パスワードは盗まれやすく、使い回しのリスクもあるためです。現在は、より安全な認証方式のみが利用可能になっています。

4.3. 接続方式の比較

GitHub との接続方法は、大きく分けて 3 つあります。それぞれに特徴があり、状況に応じて使い分けることができます。

方式	セキュリティ	手軽さ	推奨度	特徴
SSH 鍵	非常に高い	初回設定にやや手間	最も推奨	一度設定すれば認証不要
HTTPS + PAT	高い	トークン管理が必要	状況に応じて	ファイアウォール内でも可
GitHub CLI	非常に高い	最も簡単	推奨	コマンド一つで設定

初心者の方へのおすすめは、GitHub CLI（`gh`）を使う方法です。対話形式で質問に答えていくだけで、裏側で SSH 鍵の生成や GitHub への登録まで自動的に行ってくれます。

一方、**SSH 鍵の仕組みを理解しておくこと**は、GitHub CLI を使う場合でも重要です。トラブルシューティングの際や、GitHub 以外の Git サービス（GitLab、Bitbucket など）を使う際にも必ず役立ちます。そのため、本章では SSH 鍵の解説を最も詳しく行います。

4.4. SSH 鍵による接続（推奨）

4.4.1. 公開鍵暗号の仕組み — 南京錠のたとえ

SSH 鍵を理解するために、まず「公開鍵暗号」という仕組みを平易に説明します。

公開鍵暗号は、2 つの鍵を使う暗号方式です。たとえ話で説明しましょう。

あなたが誰かに秘密のメッセージを送りたいとします。従来の方法（共通鍵暗号）では、一つの鍵で箱を施錠し、同じ鍵で開錠します。しかし、その鍵をどうやって相手に安全に渡すかという問題があります。鍵を配送中に盗まれたら、メッセージも読まれてしまいます。

公開鍵暗号は、この問題を巧妙に解決します。仕組みはこうです。

1. あなたは「**南京錠**」（公開鍵）と「**南京錠を開ける鍵**」（秘密鍵）のペアを作ります
2. 南京錠（公開鍵）は世界中にばらまいても構いません。誰でも持って行って OK です
3. メッセージを送りたい人は、あなたの南京錠で箱を施錠して送ります
4. その箱を開けられるのは、南京錠を開ける鍵（秘密鍵）を持っているあなただけです

SSH 鍵もこれと同じ原理です。

- **公開鍵**（`id_ed25519.pub`）：GitHub に登録します。「南京錠」に相当します
- **秘密鍵**（`id_ed25519`）：自分のパソコンに保管します。「南京錠を開ける鍵」に相当します。**絶対に他人に渡してはいけません**

```
あなたのPC                                GitHub
┌──────────────────────────────┐                    ┌──────────────
  ──────────────────────┐
```

```
│ 秘密鍵          │ ←── 認証 ──→  │ 公開鍵          │
│ (~/.ssh/        │               │ (Settings →     │
│  id_ed25519)    │               │  SSH keys)      │
└─────────────────┘               └─────────────────
  ────────────────┘
```

GitHub にアクセスするとき、GitHub は「この公開鍵に対応する秘密鍵を持っていますか？」と確認します。あなたのパソコンが秘密鍵を使って正しく応答できれば、「本人である」と認められます。秘密鍵そのものがネットワーク上を流れることはないため、非常に安全です。

4.4.2. SSH 鍵の生成

それでは、実際に SSH 鍵を生成しましょう。ターミナルを開いて次のコマンドを実行します。

```
ssh-keygen -t ed25519 -C "your-email@example.com"
```

各オプションの意味を説明します。

- `-t ed25519`：鍵の種類（アルゴリズム）を指定しています。Ed25519 は現在最も推奨されているアルゴリズムで、RSA よりも短い鍵長で同等以上のセキュリティを提供します。古い環境で Ed25519 がサポートされていない場合は、`-t rsa -b 4096` を使ってください。
- `-C "your-email@example.com"`：コメントを付加します。このコメントは鍵の識別に使われるもので、認証には影響しません。GitHub に登録したメールアドレスを入れておくのが慣例です。

コマンドを実行すると、いくつかの質問が表示されます。

```
Generating public/private ed25519 key pair.
Enter file in which to save the key (/Users/you/.ssh/id_ed25519):
```

鍵の保存場所を聞かれています。特に理由がなければ、そのまま Enter キーを押してデフォルトの場所に保存してください。

```
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
```

次に**パスフレーズ**の設定を求められます。パスフレーズは、秘密鍵に追加のパスワードをかけるものです。パスフレーズを設定しておくと、万が一パソコンが盗まれたり、秘密鍵ファイルが流出したりしても、パスフレーズを知らなければ鍵を使うことができません。

セキュリティの観点から、パスフレーズの設定を強くおすすめします。後述する SSH エージェントを使えば、パスフレーズを毎回手入力する手間も省けます。

鍵の生成が完了すると、`~/.ssh/` ディレクトリに以下の 2 つのファイルが作成されます。

- `id_ed25519`：秘密鍵。このファイルは絶対に他人と共有しないでください。
- `id_ed25519.pub`：公開鍵。このファイルの内容を GitHub に登録します。

4.4.3. SSH エージェントに鍵を登録する

SSH エージェントは、秘密鍵をメモリ上に保持してくれるプログラムです。エージェントに鍵を登録しておくと、SSH 接続のたびにパスフレーズを入力する必要がなくなります。自動車でいえば、エンジンをかけるたびにキーを回す代わりに、スマートキーで自動的にエンジンがかかるようなイメージです。

まず、SSH エージェントをバックグラウンドで起動します。

```
eval "$(ssh-agent -s)"
```

`Agent pid 12345` のような出力が表示されれば、エージェントが起動しています。

macOS の場合は、SSH 設定ファイルを作成（または編集）して、Keychain との連携を設定します。

```
cat >> ~/.ssh/config << 'EOF'
Host github.com
  AddKeysToAgent yes
  UseKeychain yes
  IdentityFile ~/.ssh/id_ed25519
EOF
```

この設定の意味を解説します。

- `Host github.com`：以下の設定が `github.com` への接続時に適用されることを示します
- `AddKeysToAgent yes`：鍵を使用する際に自動的に SSH エージェントに追加します
- `UseKeychain yes`：macOS の Keychain（パスワード管理機能）にパスフレーズを保存します。これにより、パソコンを再起動してもパスフレーズの再入力が不要になります
- `IdentityFile ~/.ssh/id_ed25519`：使用する秘密鍵のファイルパスを指定します

次に、鍵をエージェントに追加します。

```
# macOSの場合
ssh-add --apple-use-keychain ~/.ssh/id_ed25519

# Linuxの場合
ssh-add ~/.ssh/id_ed25519
```

パスフレーズを設定している場合、ここで入力を求められます。一度入力すれば、以降はエージェントが自動的に処理してくれます。

4.4.4. 公開鍵を GitHub に登録する

生成した公開鍵を GitHub に登録します。まず、公開鍵の内容をクリップボードにコピーします。

```
# macOSの場合
pbcopy < ~/.ssh/id_ed25519.pub

# Linuxの場合（xclipがインストールされている場合）
xclip -selection clipboard < ~/.ssh/id_ed25519.pub

# または、内容を表示して手動でコピー
cat ~/.ssh/id_ed25519.pub
```

公開鍵は `ssh-ed25519 AAAA...` で始まる 1 行のテキストです。

次に、GitHub の Web サイトで以下の操作を行います。

1. GitHub にログインし、右上のアイコンから **Settings** を開きます
2. 左側のメニューから **SSH and GPG keys** を選択します
3. 「**New SSH key**」ボタンをクリックします
4. **Title**：この鍵を識別するための名前を入力します。「MacBook Pro 2024」や「自宅デスクトップ」など、どのパソコンの鍵かがわかる名前にしましょう
5. **Key type**：「Authentication Key」を選択します
6. **Key**：コピーした公開鍵を貼り付けます
7. 「**Add SSH key**」をクリックします

スクリーンショット: *SSH 鍵の追加画面*

GitHub のパスワード入力を求められることがありますが、これはセキュリティ上の確認です。

4.4.5. 接続テスト

設定が正しくできたかどうかを確認しましょう。

```
ssh -T git@github.com
```

初めて接続する場合、以下のようなメッセージが表示されることがあります。

```
The authenticity of host 'github.com (20.27.177.113)' can't be
   established.
ED25519 key fingerprint is SHA256:+DiY3wvvSqA3npr5lFzhzg6hcJkiGk
   ....
Are you sure you want to continue connecting (yes/no/[fingerprint
   ])?
```

これは「このサーバーに初めて接続しますが、本当に接続しますか？」という確認です。`yes` と入力してください。GitHub のフィンガープリント（サーバーの指紋）は、GitHub の公式ドキュメントで公開されていますので、心配な場合は照合することもできます。

接続に成功すると、以下のメッセージが表示されます。

```
Hi username! You've been successfully authenticated, but GitHub
   does not provide shell access.
```

「shell access」（シェルへのアクセス）は提供されないと書かれていますが、これは正常です。GitHub は Git リポジトリのホスティングサービスであり、サーバーにログインして操作するサービスではないためです。認証自体は成功しています。

4.4.6. SSH 接続でリポジトリを操作する

SSH 接続が確立できたら、以降は SSH 形式の URL を使ってリポジトリを操作します。

```
# SSHのURLでリポジトリをクローン
git clone git@github.com:username/repo.git

# 既存リポジトリのリモートURLをSSHに変更する場合
git remote set-url origin git@github.com:username/repo.git
```

SSH 形式の URL は `git@github.com:ユーザー名/リポジトリ名.git` という形式です。HTTPS 形式の `https://github.com/...` とは異なることに注意してください。GitHub のリポジトリページで「Code」ボタンをクリックすると、「SSH」タブでこの URL を確認できます。

4.5. HTTPS + Personal Access Token

SSH 鍵の代わりに、HTTPS 接続と Personal Access Token（PAT）を使う方法もあります。企業のネットワーク環境では SSH ポート（22 番）がファイアウォールでブロックされていることがあり、その場合は HTTPS（443 番ポート）を使うこの方式が有効です。

4.5.1. Personal Access Token（PAT）とは

PAT は、パスワードの代わりに使用する「トークン（文字列）」です。GitHub のパスワードとは別に生成され、有効期限や権限の範囲を細かく設定できます。パスワードがアカウント全体の「万能鍵」だとすれば、PAT は「特定の部屋だけ開けられる鍵」のようなものです。

4.5.2. PAT の作成手順

1. GitHub にログインし、右上のアイコンから **Settings** を開きます
2. 左側メニューの一番下にある **Developer settings** をクリックします
3. **Personal access tokens** → **Tokens (classic)** を選択します
4. 「**Generate new token (classic)**」をクリックします

スクリーンショット: *PAT 作成画面*

以下の項目を設定します。

Note（メモ）：このトークンの用途を書きます。「MacBook 開発用」「CI/CD パイプライン用」など、後から見て何のために作ったトークンかがわかる名前にしてください。

Expiration（有効期限）：トークンの有効期間を設定します。90 日程度が推奨されます。有効期限を設定しないこともできますが、セキュリティの観点からおすすめしません。期限が切れたら、新しいトークンを生成して差し替えます。

Scopes（スコープ ＝ 権限範囲）：このトークンにどの操作を許可するかを選択します。

4.5.3. スコープの選び方

スコープ	用途	必要な場面
`repo`	リポジトリの読み書き	ほぼ必須。プライベートリポジトリへのアクセスに必要です
`workflow`	GitHub Actions ワークフローの管理	CI/CD を使う場合に必要です
`admin:org`	Organization（組織）の管理	組織のメンバー管理などを行う場合
`gist`	Gist の作成・編集	Gist を使う場合のみ
`delete_repo`	リポジトリの削除	通常は不要です。付与しないことを推奨

最小権限の原則に従い、必要最低限のスコープだけを選択してください。日常的な開発作業であれば、`repo` と `workflow` の 2 つで十分です。

「**Generate token**」をクリックすると、トークンが表示されます。**このトークンは一度しか表示されません**。画面を閉じると二度と確認できなくなるため、必ずコピーしてパスワードマネージャーなどの安全な場所に保存してください。

4.5.4. PAT を使ったリポジトリ操作

```
# HTTPS URLでクローン
git clone https://github.com/username/repo.git
```

ユーザー名とパスワードを聞かれたら、パスワードの代わりに PAT を入力します。

4.5.5. 認証情報の保存（毎回入力しなくて済むように）

HTTPS 接続では、操作のたびにユーザー名とトークンの入力を求められることがあります。これを省略するために、OS の認証情報管理機能を活用しましょう。

```
# macOSの場合：Keychainに保存
git config --global credential.helper osxkeychain

# Windowsの場合：Windows Credential Managerに保存
git config --global credential.helper wincred

# Linuxの場合：一定時間メモリに保存（秒数で指定、以下は1時間）
git config --global credential.helper 'cache --timeout=3600'
```

macOS の `osxkeychain` と Windows の `wincred` は、OS 内蔵のパスワード管理機能に認証情報を安全に保存します。一度入力すれば、以降は自動的に認証が行われます。

Linux の `cache` は、メモリ上に一時的に保存する方式で、指定した秒数が経過すると認証情報は消えます。永続的に保存したい場合は `store` ヘルパーもありますが、トークンが平文（暗号化なし）でファイルに保存されるため、セキュリティ上のリスクがあります。可能であれば、`libsecret` や `pass` などの暗号化された認証情報ストアの利用を検討してください。

4.5.6. Fine-grained PAT（きめ細かい PAT）

GitHub は、従来の PAT（Classic）に加えて、**Fine-grained PAT**（きめ細かい PAT）という新しい方式も提供しています。

Fine-grained PAT では、以下のようなより詳細な権限制御が可能です。

- **リポジトリ単位**で権限を設定できる（特定のリポジトリだけにアクセスを限定）
- **読み取りのみ・読み書き**などの粒度で権限を設定できる
- 組織のセキュリティポリシーに準拠したトークンを作成できる

作成は **Developer settings → Personal access tokens → Fine-grained tokens** から行います。より安全なトークン管理が可能なため、新規にトークンを作成する場合は Fine-grained PAT

の使用を検討してみてください。

4.6. GitHub CLI

GitHub CLI（コマンド名：`gh`）は、GitHub が公式に提供するコマンドラインツールです。認証の設定をはじめ、リポジトリの作成、プルリクエストの操作、Issue の管理など、GitHub 上のほぼすべての操作をターミナルから行えます。

認証に関して言えば、GitHub CLI は最も簡単な方法です。対話形式で質問に答えていくだけで、SSH 鍵の生成から GitHub への登録までを自動的に行ってくれます。

4.6.1. インストール

```
# macOS (Homebrew)
brew install gh

# Windows (winget)
winget install --id GitHub.cli

# Linux (Debian/Ubuntu)
sudo apt install gh

# Linux (Fedora)
sudo dnf install gh
```

4.6.2. 認証の設定

インストールが完了したら、次のコマンドで認証を開始します。

```
gh auth login
```

対話形式でいくつかの質問が表示されます。

```
? What account do you want to log into?  GitHub.com
? What is your preferred protocol for Git operations?  SSH
? Generate a new SSH key to add to your GitHub account?  Yes
? Title for your SSH key:  MacBook Pro
? How would you like to authenticate GitHub CLI?  Login with a web
   browser
```

各質問の意味を解説します。

- **アカウントの種類**：`GitHub.com`（通常の GitHub）か `GitHub Enterprise Server`（企業向け自社ホスト版）かを選びます。ほとんどの場合は `GitHub.com` です。
- **Git 操作のプロトコル**：`SSH` を選択することをおすすめします。この選択により、GitHub CLI が SSH 鍵の設定も併せて行ってくれます。
- **SSH 鍵の生成**：まだ SSH 鍵を作成していなければ `Yes` を選びます。すでに 4.4 節の手順で鍵を作成済みの場合は、既存の鍵を使うことも可能です。
- **SSH 鍵のタイトル**：GitHub に登録する際の識別名です。どのパソコンの鍵かがわかる名前を付けましょう。
- **認証方法**：`Login with a web browser` を選ぶと、ブラウザが自動的に開き、GitHub のログインページが表示されます。ターミナルに表示されるコードをブラウザに入力して認証を完了してください。

スクリーンショット: *gh auth login* の対話画面

ブラウザでの認証が完了すると、ターミナルに認証成功のメッセージが表示されます。これで設定は完了です。

4.6.3. 認証状態の確認

現在の認証状態を確認するには、次のコマンドを使います。

```
gh auth status
```

出力例：

```
github.com
  Logged in to github.com account username (keyring)
  - Active account: true
  - Git operations protocol: ssh
  - Token: gho_****
  - Token scopes: 'admin:org', 'gist', 'repo', 'workflow'
```

この出力から、ログインしているアカウント、Git 操作で使われるプロトコル、トークンのスコープなどを確認できます。もし認証に問題がある場合は、`gh auth login` を再度実行して設定をやり直すことができます。

4.7. GPG 署名 — コミットの本人証明

4.7.1. なぜ署名が必要なのか

第 2 章で `user.name` と `user.email` を設定しましたが、実はこの情報は**誰でも自由に設定できます**。極端な話、他人のユーザー名とメールアドレスを設定してコミットすることも技術的には可能です。つまり、コミットの「著者情報」だけでは、そのコミットが本当にその人によるものかどうかを保証できないのです。

GPG 署名は、この問題を解決します。GPG（GNU Privacy Guard）という暗号ツールを使ってコミットに電子署名を付与することで、「このコミットは確かに私が作成したものです」という暗号的な証明を提供します。

GPG 署名されたコミットは、GitHub 上で「**Verified**」（検証済み）という緑色のバッジが表示されます。このバッジがあるコミットは、GitHub に登録された GPG 公開鍵と照合され、本人によるものであることが確認済みです。

スクリーンショット：*Verified* バッジ付きコミット

オープンソースプロジェクトやセキュリティが重視される環境では、GPG 署名が求められることがあります。必須ではありませんが、設定しておくとコミットの信頼性が高まります。

4.7.2. GPG 鍵の生成

まず、GPG をインストールします。

```
# macOS（Homebrew）
brew install gnupg

# Linux（Ubuntu/Debian）
sudo apt install gnupg

# Windows：Git for Windows に同梱されている場合があります
```

次に、GPG 鍵を生成します。

```
gpg --full-generate-key
```

いくつかの質問が表示されます。

- **鍵の種類：** `RSA and RSA`（デフォルト）を選択します
- **鍵のビット数：** `4096` を入力します。2048 ビットでも十分ですが、4096 ビットにしておくとよ

り長期間安全です

- **有効期限**：`0`（無期限）でも構いませんが、セキュリティを重視するなら 1〜2 年に設定し、期限が来たら更新することも検討してください
- **名前**：GitHub の表示名と一致させてください
- **メールアドレス**：GitHub に登録したメールアドレス（または noreply アドレス）を入力してください
- **パスフレーズ**：GPG 鍵を保護するパスフレーズを設定します

4.7.3. GitHub に GPG 鍵を登録する

まず、生成した鍵の ID を確認します。

```
gpg --list-secret-keys --keyid-format=long
```

出力例：

```
/Users/you/.gnupg/pubring.kbx
-------------------------------
sec   rsa4096/3AA5C34371567BD2 2024-01-01 [SC]
      A1B2C3D4E5F6G7H8I9J0K1L2M3N4O5P6Q7R8S9T0
uid                 [ultimate] Your Name <your-email@example.com>
ssb   rsa4096/1234567890ABCDEF 2024-01-01 [E]
```

`sec` 行の `rsa4096/` の後に続く `3AA5C34371567BD2` が鍵 ID です。

次に、公開鍵をテキスト形式でエクスポートします。

```
gpg --armor --export 3AA5C34371567BD2
```

`-----BEGIN PGP PUBLIC KEY BLOCK-----` から `-----END PGP PUBLIC KEY BLOCK-----` までの全文をコピーし、GitHub に登録します。

1. GitHub → Settings → **SSH and GPG keys** を開きます
2. 「**New GPG key**」ボタンをクリックします
3. コピーした公開鍵の全文を貼り付けます
4. 「**Add GPG key**」をクリックします

4.7.4. Git で署名を有効化する

最後に、Git がコミット時に自動的に GPG 署名を付与するように設定します。

```
# 使用するGPG鍵のIDを指定
git config --global user.signingkey 3AA5C34371567BD2

# すべてのコミットを自動的に署名する
git config --global commit.gpgsign true

# GPGプログラムのパスを指定（macOSで必要な場合）
git config --global gpg.program gpg
```

`commit.gpgsign true` を設定すると、以降のすべてのコミットに自動的に GPG 署名が付与されます。手動で署名したい場合は、この設定を省略し、コミット時に `-S` オプションを付けてください（`git commit -S -m "メッセージ"`）。

macOS で GPG 署名が動作しない場合は、以下の設定が必要なことがあります。

```
echo 'export GPG_TTY=$(tty)' >> ~/.zshrc
source ~/.zshrc
```

これは、GPG がパスフレーズの入力を受け付けるために、現在のターミナルの情報を必要とするためです。

4.8. 複数アカウントの管理

仕事用と個人用など、複数の GitHub アカウントを持っている場合、それぞれのアカウントに対して異なる SSH 鍵を使い分ける必要があります。一つの SSH 公開鍵を複数の GitHub アカウントに登録することはできないためです。

4.8.1. SSH config によるアカウントの使い分け

SSH 設定ファイル（`~/.ssh/config`）を編集して、接続先ごとに異なる鍵を使うように設定します。

まず、アカウントごとに SSH 鍵を生成します。

```
# 個人アカウント用の鍵
ssh-keygen -t ed25519 -C "personal@example.com" -f ~/.ssh/
   id_ed25519_personal

# 仕事アカウント用の鍵
```

```
ssh-keygen -t ed25519 -C "work@company.com" -f ~/.ssh/
   id_ed25519_work
```

`-f` オプションでファイル名を指定することで、デフォルトの `id_ed25519` とは異なる名前で鍵を保存できます。

次に、`~/.ssh/config` ファイルを編集します。

```
# 個人アカウント
Host github.com-personal
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_personal

# 仕事アカウント
Host github.com-work
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_work
```

ここでのポイントは、`Host` の値です。実際のホスト名は `github.com` ですが、`github.com-personal` や `github.com-work` という**エイリアス（別名）**を設定しています。`HostName github.com` により、どちらのエイリアスでも実際には `github.com` に接続されますが、使用する鍵が異なります。

それぞれの公開鍵を、対応する GitHub アカウントに登録してください。

4.8.2. 使い方

リポジトリの操作時は、目的のアカウントに対応するエイリアスを使います。

```
# 個人アカウントのリポジトリをクローン
git clone git@github.com-personal:personal-user/my-project.git

# 仕事アカウントのリポジトリをクローン
git clone git@github.com-work:work-user/company-project.git
```

URL の `github.com` の部分が、`github.com-personal` や `github.com-work` に置き換わっている点に注目してください。SSH はこのエイリアスを見て、どの鍵を使うかを自動的に判断します。

4.8.3. リポジトリごとのユーザー情報の切り替え

SSH 鍵の使い分けに加えて、コミットに記録されるユーザー名とメールアドレスもアカウントごとに切り替える必要があります。

```
# 仕事用のリポジトリに移動
cd /path/to/work/project

# このリポジトリだけで使うユーザー情報を設定（--localがデフォルト）
git config user.name "Tsubasa (Work)"
git config user.email "tsubasa@company.com"
```

第 2 章で解説した `--global` と `--local` の仕組みがここで活きてきます。`--global` で個人アカウントの情報を設定しておき、仕事用のリポジトリでは `--local` で上書きするという運用が一般的です。

設定を忘れて個人のメールアドレスで仕事のコミットをしてしまうミスを防ぐために、Git の `includeIf` 機能を使う方法もあります。`~/.gitconfig` に以下のように記述すると、特定のディレクトリ以下のリポジトリに対して自動的に設定を切り替えられます。

```
[includeIf "gitdir:~/work/"]
    path = ~/.gitconfig-work
```

`~/.gitconfig-work` ファイルに仕事用の設定を書いておけば、`~/work/` ディレクトリ以下のリポジトリでは自動的にその設定が適用されます。

```
# ~/.gitconfig-work の内容
[user]
    name = Tsubasa (Work)
    email = tsubasa@company.com
```

この方法なら、リポジトリごとに毎回設定する手間が省け、設定忘れのリスクも軽減されます。

4.9. まとめ

本章では、パソコンと GitHub を安全に接続するための認証方法を学びました。振り返ってみましょう。

- **認証の必要性**：パスワード認証は廃止されており、より安全な方法で GitHub に接続する必要があります。
- **3 つの接続方式**：SSH 鍵、HTTPS + PAT、GitHub CLI の 3 つの方式があり、それぞれに適し

た場面があります。

- **SSH 鍵**：公開鍵暗号の仕組みを利用した認証方式で、秘密鍵は自分の PC に、公開鍵は GitHub に登録します。一度設定すれば、以降の認証は自動的に行われます。
- **Personal Access Token（PAT）**：HTTPS 接続で使用するトークンで、有効期限や権限の範囲を細かく設定できます。Fine-grained PAT ではリポジトリ単位の権限制御も可能です。
- **GitHub CLI**：`gh auth login` コマンドで対話的に認証を設定できる、最も手軽な方法です。
- **GPG 署名**：コミットに電子署名を付与し、本人が作成したものであることを暗号的に証明します。GitHub 上で「Verified」バッジとして表示されます。
- **複数アカウントの管理**：SSH config のエイリアス機能と `--local` 設定、`includeIf` を組み合わせることで、複数の GitHub アカウントをスムーズに使い分けられます。

認証の設定は、最初の一回だけ乗り越えれば、あとは快適に GitHub を使い続けることができます。もし途中でエラーが出たり、うまくいかなかったりした場合は、第 24 章のトラブルシューティングも参照してください。

次の章では、いよいよ GitHub 上にリポジトリを作成し、コードを管理するための「器」を用意します。

リポジトリ — プロジェクトの器を作る

5.1. この章で学ぶこと

ソフトウェア開発において、プロジェクトのファイルを管理するための「入れ物」が必要です。Gitではこの入れ物を**リポジトリ（repository）**と呼びます。略して「リポ」や「repo」と呼ばれることもあります。リポジトリは単なるフォルダではなく、ファイルの現在の状態だけでなく、過去のすべての変更履歴を保持する特別な構造を持っています。

この章では、以下のことを学びます。

- リポジトリの内部構造と `.git` フォルダの役割
- リポジトリを作成する3つの方法と、それぞれの適切な使い分け
- `git clone` コマンドの詳細な動作とオプション
- リモートリポジトリ（remote）の概念と管理方法
- `.gitignore` によるファイル除外の仕組みと実践的な書き方
- GitHub上でのリポジトリ設定（可視性、ブランチ保護、テンプレート）

リポジトリの作成はGitを使ったすべての作業の出発点です。この章の内容をしっかり理解することで、以降の章で扱うブランチ操作やコミットの意味がより深く理解できるようになります。

5.2. リポジトリの構造 — .gitフォルダの役割

5.2.1. プロジェクトフォルダとリポジトリの関係

まず、「リポジトリ」と「普通のフォルダ」の違いを理解しましょう。たとえば、あなたが `my-project` というフォルダを作り、その中にソースコードやドキュメントを置いたとします。この段階では、それはただのフォルダです。ここで `git init` コマンドを実行すると、フォルダの中に `.git` という隠しディレクトリが作成されます。この `.git` ディレクトリが存在することで、普通のフォルダが「リポジトリ」に変わるのです。

```
my-project/                  ← プロジェクトフォルダ全体がリポジトリ
 ├── .git/                     ← Gitの管理データ（この中にすべての
   履歴が格納される）
 │   ├── HEAD                   ← 現在チェックアウトしているブラン
   チへの参照
 │   ├── config                 ← このリポジトリ固有の設定
 │   ├── objects/               ← コミット、ファイル内容などのオブ
   ジェクトデータベース
 │   ├── refs/                  ← ブランチやタグのポインタ
 │   └── hooks/                 ← コミット前後に実行されるスクリプ
   ト
```

```
├── .gitignore              ← Gitに無視させるファイルの指定
├── README.md               ← プロジェクト説明
├── src/                    ← ソースコード
│   └── main.py
└── tests/                  ← テスト
    └── test_main.py
```

5.2.2. .git フォルダの中身

`.git` フォルダは「リポジトリの心臓部」とも言える存在です。この中には、プロジェクトの過去から現在に至るまでのすべての変更履歴が格納されています。通常、この中のファイルを手動で操作する必要はありませんが、その構造を理解しておくと、Git の動作原理がより明確になります。

HEAD ファイルには、現在作業しているブランチの情報が書かれています。たとえば `ref: refs/heads/main` という内容であれば、今 `main` ブランチにいることを意味します。

objects/ ディレクトリは、Git のデータベースそのものです。ファイルの内容（blob）、ディレクトリ構造（tree）、コミット情報（commit）がすべてハッシュ値で管理され、この中に保存されています。Git が「分散型バージョン管理」と呼ばれるのは、この `objects/` ディレクトリにリポジトリの完全な履歴が含まれているからです。GitHub のサーバーがダウンしても、あなたのローカルに完全な履歴が残っています。

refs/ ディレクトリには、ブランチやタグの「ポインタ」が保存されています。ブランチとは、特定のコミットを指し示す軽量なポインタに過ぎないのです（これについては第 7 章で詳しく解説します）。

config ファイルには、このリポジトリに固有の設定が保存されます。リモートリポジトリの URL や、ブランチの追跡情報などがここに記録されます。

> **重要**: `.git` フォルダを削除すると、すべての変更履歴が失われます。プロジェクトのファイル自体は残りますが、*Git* の管理下ではなくなります。`.git` フォルダには絶対に触れないようにしましょう。

5.3. リポジトリの作成方法

リポジトリを作成する方法は大きく分けて 3 つあります。それぞれの方法には適した場面がありますので、状況に応じて使い分けましょう。

5.3.1. 方法 1: GitHub で作成してからクローン（推奨）

最も一般的で、初心者にも推奨される方法です。この方法では、まず GitHub の Web インターフェース上でリポジトリを作成し、それをローカルにコピー（クローン）します。

なぜこの方法が推奨されるのでしょうか。それは、GitHub で作成する際に README ファイルや `.gitignore`、ライセンスファイルなどの初期ファイルを自動生成できるからです。また、リモートリポジトリとの接続設定が自動的に行われるため、設定ミスが起きにくいという利点もあります。

GitHub 上での操作手順:

1. GitHub にログインし、画面右上の「+」ボタンをクリックします
2. ドロップダウンメニューから「**New repository**」を選択します

スクリーンショット: *GitHub 右上の+ボタンとドロップダウンメニュー*

3. リポジトリの作成画面で、以下の項目を設定します

項目	説明	推奨設定
Repository name	リポジトリ名。URL の一部になる	プロジェクト内容がわかる簡潔な名前
Description	リポジトリの簡単な説明文（任意）	1 行で内容がわかる説明を書く
Public / Private	公開・非公開の選択	学習目的なら Public で OK
Add a README file	README.md を自動生成するか	チェック推奨
Add .gitignore	言語に応じた.gitignore テンプレートを選択	使用する言語を選択
Choose a license	オープンソースライセンスの選択	OSS の場合は MIT などを選択

スクリーンショット: *リポジトリ作成画面の各項目*

4.「**Create repository**」ボタンをクリックして作成完了です

ローカルにクローンする:

GitHub でリポジトリが作成できたら、次はそれをローカルの PC にコピーします。リポジトリのページにある「**Code**」ボタンをクリックすると、クローン用の URL が表示されます。

スクリーンショット: *Code ボタンをクリックした状態のクローン URL 表示*

```
# SSH接続を使う場合（第4章で設定済みなら推奨）
git clone git@github.com:username/my-project.git

# HTTPS接続を使う場合
git clone https://github.com/username/my-project.git

# GitHub CLIを使う場合（最もシンプル）
gh repo clone username/my-project
```

クローンが完了したら、作成されたディレクトリに移動して作業を始めます。

```
cd my-project
```

5.3.2. 方法2: ローカルで作成してからGitHubにプッシュ

既にローカルで開発を始めているプロジェクトをGitHub管理に移行したい場合や、ローカルでの初期作業を先に行いたい場合は、この方法を使います。

この方法では、まずローカルでGitリポジトリを初期化し、最初のコミットを作成した後、GitHubにリモートリポジトリを作成して接続します。各コマンドの意味を順に見ていきましょう。

```
# ステップ1: プロジェクトフォルダを作成して移動する
mkdir my-project
cd my-project

# ステップ2: Gitリポジトリとして初期化する
# このコマンドで .git フォルダが作成される
git init

# ステップ3: README ファイルを作成する
echo "# My Project" > README.md

# ステップ4: ファイルをステージングしてコミットする
git add README.md
git commit -m "Initial commit"
```

`git init` は、カレントディレクトリに `.git` フォルダを作成し、そのフォルダをGitリポジトリとして初期化するコマンドです。この時点ではまだGitHubとの接続はありません。完全にローカルだけのリポジトリです。

次に、GitHubにリモートリポジトリを作成して接続します。GitHub CLIがインストールされている場合は、1つのコマンドで完了します。

```
# GitHub CLIを使う場合（リモート作成・接続・プッシュを一括実行）
gh repo create my-project --public --source=. --remote=origin --
   push
```

このコマンドのオプションの意味は次のとおりです。`--public` はリポジトリをPublic（公開）として作成することを指定します。`--source=.` はカレントディレクトリをソースとして使うことを意味します。`--remote=origin` はリモート接続名を `origin` に設定します。`--push` はローカル

のコミットを即座にプッシュします。

GitHub CLI を使わない場合は、まず GitHub の Web インターフェースで空のリポジトリを作成します（このとき、README や.gitignore のチェックは外してください。既にローカルにコミットがあるためです）。その後、以下のコマンドでローカルリポジトリと GitHub を接続します。

```
# リモートリポジトリのURLを登録する
git remote add origin git@github.com:username/my-project.git

# デフォルトブランチ名をmainに設定する
git branch -M main

# ローカルのコミットをGitHubにプッシュする（-uで上流ブランチを設
  定）
git push -u origin main
```

`git remote add origin` は、「origin」という名前でリモートリポジトリの URL を登録するコマンドです。`git branch -M main` は、現在のブランチ名を `main` に変更します（古いバージョンの Git では `master` がデフォルトのため）。`git push -u origin main` の `-u` は `--set-upstream` の省略形で、ローカルの `main` ブランチとリモートの `main` ブランチを関連付けます。一度設定すれば、以降は `git push` だけで同じブランチにプッシュできるようになります。

5.3.3. 方法 3: GitHub CLI で一発作成

GitHub CLI が使える環境では、コマンド 1 つでリポジトリの作成からクローンまで完了できます。素早くプロジェクトを立ち上げたいときに便利です。

```
# 対話形式で作成（質問に答えていくだけ）
gh repo create

# ワンライナーで作成してクローンまで行う
gh repo create my-project --public --clone
cd my-project
```

対話形式では、リポジトリ名、公開/非公開、説明文などを順に聞かれます。初心者の方は対話形式で始めると、各設定項目の意味を理解しやすいでしょう。

5.4. クローン（clone）を深く理解する

5.4.1. cloneが行うこと

`git clone` は、見た目にはリモートリポジトリの「コピー」を作るコマンドですが、内部的にはいくつかの操作をまとめて実行しています。書店でたとえるなら、単に本を1冊コピーするのではなく、本の全ページ、改訂履歴、著者のメモまで含めた完全な複製を作るようなものです。

具体的には、`git clone` は以下の操作を順番に行います。

1. 指定したURLのリモートリポジトリに接続する
2. ローカルに新しいディレクトリを作成する
3. `.git` ディレクトリを初期化する（`git init` 相当）
4. リモートリポジトリのすべてのデータ（全ブランチ、全コミット履歴）をダウンロードする
5. リモートリポジトリを `origin` という名前で自動登録する（`git remote add origin` 相当）
6. デフォルトブランチ（通常は `main`）をチェックアウトする

つまり、`git clone` を実行した直後の状態は、リモートリポジトリの完全なミラーがローカルに存在し、すぐに作業を開始できる状態になっているのです。

```
# 基本的なクローン
git clone git@github.com:username/repo.git

# 別名のフォルダに保存したい場合
git clone git@github.com:username/repo.git my-custom-folder-name
```

5.4.2. cloneの主要オプション

クローンにはいくつかの便利なオプションがあり、状況に応じて使い分けると効率的です。

5.4.2.1. –depth（浅いクローン）

```
git clone --depth 1 git@github.com:username/repo.git
```

`--depth` オプションは、取得するコミット履歴の深さを制限します。`--depth 1` なら最新のコミット1件だけを取得します。このオプションは「浅いクローン（shallow clone）」と呼ばれ、以下のような場面で役立ちます。

- **大規模リポジトリ**を素早くクローンしたいとき（Linux カーネルのリポジトリは完全な履歴が数GBに達します）

- **CI/CD 環境**で最新のソースコードだけが必要なとき
- とりあえずコードの中身を確認したいだけで、過去の履歴は不要なとき

ただし、浅いクローンでは `git log` で過去の履歴を遡ることができません。本格的な開発作業を行う場合は、通常の（完全な）クローンを使いましょう。

5.4.2.2. –branch（特定ブランチのクローン）

```
git clone -b develop git@github.com:username/repo.git
```

`-b`（`--branch`）オプションを指定すると、クローン後にチェックアウトされるブランチを指定できます。デフォルトではリポジトリのデフォルトブランチ（通常は `main`）がチェックアウトされますが、たとえば `develop` ブランチで開発を始めたい場合にこのオプションを使います。なお、すべてのブランチのデータ自体はダウンロードされるので、後から別のブランチに切り替えることも可能です。

5.4.2.3. –recursive（サブモジュール付きクローン）

```
git clone --recursive git@github.com:username/repo.git
```

`--recursive` オプションは、リポジトリが**サブモジュール**（他の Git リポジトリを内部に含む構造）を持っている場合に使います。このオプションを付けないと、サブモジュールのディレクトリは空の状態でクローンされます。サブモジュールを使っているかどうかは、リポジトリのルートに `.gitmodules` ファイルが存在するかどうかで判断できます。

5.5. リモート（remote）の管理

5.5.1. origin とは何か

Git では、リモートリポジトリ（GitHub などのサーバー上にあるリポジトリ）に「名前」を付けて管理します。この名前は自由に設定できますが、最初のリモートリポジトリには慣例的に **origin** という名前が使われます。

`git clone` でリポジトリを作成した場合、クローン元のリポジトリが自動的に `origin` として登録されます。つまり、`origin` とは「このリポジトリのコピー元（もしくは同期先）である GitHub 上のリポジトリ」を指す名前なのです。

```
# 登録されているリモートの一覧を確認する
git remote -v
# 出力例:
# origin  git@github.com:username/repo.git (fetch)
```

```
# origin  git@github.com:username/repo.git (push)
```

出力が2行あるのは、`fetch`（取得）と`push`（送信）で異なるURLを設定できるためです。通常は同じURLが表示されます。

5.5.2. upstreamとは何か

`origin`が「自分のリポジトリ」を指すのに対し、**upstream**は「Fork元のオリジナルリポジトリ」を指す慣例的な名前です。

たとえば、あなたがオープンソースプロジェクト`original-author/awesome-tool`をForkして`your-name/awesome-tool`として自分のアカウントにコピーしたとします。この場合の関係は次のようになります。

リモート名	指し示すリポジトリ	用途
`origin`	`your-name/awesome-tool`（自分のFork）	自分の変更をプッシュする先
`upstream`	`original-author/awesome-tool`（Fork元）	オリジナルの最新変更を取り込むため

この設定は第15章「Forkとコントリビューション」で実践的に使用しますが、ここでは概念を理解しておいてください。

5.5.3. リモートの追加・変更・削除

リモートの管理には以下のコマンドを使います。

```
# リモートを新規に追加する
# 書式: git remote add <名前> <URL>
git remote add upstream git@github.com:original-author/repo.git

# リモートのURLを変更する（SSHからHTTPSへの切り替えなど）
git remote set-url origin https://github.com/username/repo.git

# リモートを削除する
git remote remove upstream

# リモートの詳細情報を表示する
git remote show origin
```

`git remote show origin`は、リモートブランチの状態や追跡関係など、詳細な情報を表示してくれる便利なコマンドです。リモートとの同期に問題が起きたときの調査にも使えます。

5.6. .gitignoreの書き方と考え方

5.6.1. なぜファイルの無視が必要なのか

プロジェクトのフォルダには、Gitで管理すべきファイルと管理すべきでないファイルが混在しています。たとえば、Pythonプロジェクトでは `__pycache__/` ディレクトリや `.pyc` ファイルが自動生成されますが、これらはソースコードから再生成できるため、バージョン管理する意味がありません。

管理すべきでないファイルを大きく分類すると、次の4種類になります。

1. **ビルド成果物・生成物** — コンパイル結果、バンドル出力など。ソースコードから再現可能
2. **依存パッケージ** — `node_modules/` や `.venv/` など。パッケージマネージャで再インストール可能
3. **機密情報** — `.env` ファイル、APIキー、パスワード。公開リポジトリに含めると重大なセキュリティリスク
4. **OS・エディタ固有のファイル** — `.DS_Store`（macOS）、`Thumbs.db`（Windows）、`.vscode/`（VS Code設定）

これらのファイルをGitの追跡対象から除外するために、`.gitignore` ファイルを使います。

5.6.2. .gitignoreの書式ルール

`.gitignore` ファイルはリポジトリのルートディレクトリに配置するテキストファイルで、1行に1つのパターンを記述します。書式のルールを覚えておきましょう。

```
# シャープ記号で始まる行はコメント（説明を書ける）

# 特定のファイル名を指定する
secret.txt
.env

# アスタリスク（*）はワイルドカード（任意の文字列にマッチ）
*.log          # 拡張子が .log のすべてのファイル
*.tmp          # 拡張子が .tmp のすべてのファイル
*.pyc          # Pythonのコンパイル済みファイル

# スラッシュ（/）で終わるとディレクトリ全体を無視
node_modules/
__pycache__/
.venv/
dist/
build/
```

```
# ディレクトリ内の特定パターンだけ無視
docs/*.pdf      # docs/ 直下のPDFファイルのみ

# ダブルアスタリスク（**）は任意の深さのディレクトリにマッチ
**/logs/        # どの階層にあっても logs/ ディレクトリを無視
src/**/*.tmp    # src/ 以下のすべての階層の .tmp ファイルを無視

# エクスクラメーションマーク（!）で否定（無視しない）
*.log
!important.log  # important.log だけは追跡する

# OSが生成するファイル
.DS_Store       # macOS のフォルダメタデータ
Thumbs.db       # Windows のサムネイルキャッシュ
```

パターンの優先順位について補足すると、`.gitignore` の中では後に書かれたルールが優先されます。そのため、先に `*.log` で全ログファイルを無視しておき、その後に `!important.log` で特定のファイルだけ例外にする、という書き方ができます。

5.6.3. 言語別テンプレート

GitHub は多くの言語やフレームワーク向けの `.gitignore` テンプレートを公開しています。新しいリポジトリを作成する際にテンプレートを選択できますが、後からダウンロードすることもできます。

```
# GitHubのテンプレートをダウンロードする例（Python用）
curl -o .gitignore https://raw.githubusercontent.com/github/
   gitignore/main/Python.gitignore
```

以下に、よく使われる言語の `.gitignore` の主要な設定を示します。

Python の場合:

```
# バイトコードとキャッシュ
__pycache__/
*.py[cod]
*$py.class

# 仮想環境
.venv/
venv/
```

```
env/

# パッケージビルド
*.egg-info/
dist/
build/

# 環境変数（機密情報を含むことが多い）
.env

# Jupyter Notebook のチェックポイント
.ipynb_checkpoints/
```

Node.js の場合:

```
# 依存パッケージ（npm install で再現可能）
node_modules/

# ビルド出力
dist/
build/

# 環境変数
.env
.env.local
.env.production

# ログファイル
*.log
npm-debug.log*
```

R の場合:

```
# RStudio のセッション情報
.Rhistory
.RData
.Rproj.user/

# knitr / R Markdown の中間ファイル
*_cache/
/cache/
```

テンプレートの全一覧は *https://github.com/github/gitignore* で確認できます。プロジェクトで使用する言語やフレームワークに応じて、適切なテンプレートを選択しましょう。

5.6.4. 既に追跡されたファイルへの対処

よくある失敗として、「`.gitignore` にパターンを追加したのに、ファイルが無視されない」というケースがあります。これは、そのファイルが既に Git の追跡対象になっている場合に起こります。`.gitignore` は「まだ追跡されていないファイル」にのみ効果があり、一度 `git add` されて追跡が始まったファイルには効きません。

この場合は、`git rm --cached` コマンドで Git の追跡を解除する必要があります。`--cached` オプションが重要で、これを付けることで「Git の管理からは外すが、ディスク上のファイルは削除しない」という動作になります。

```
# 特定ファイルの追跡を解除する（ファイル自体はディスクに残る）
git rm --cached secret.txt

# ディレクトリごと追跡を解除する場合は -r（recursive）を付ける
git rm -r --cached node_modules/

# .gitignore に追加したことと追跡解除をまとめてコミットする
git add .gitignore
git commit -m "Remove tracked files that should be ignored"
```

この手順を忘れると、機密情報が含まれたファイルが GitHub に残り続けるリスクがあります。特に `.env` ファイルなどを誤ってコミットしてしまった場合は、速やかに対処しましょう（ただし、一度プッシュされた情報は Git の履歴に残るため、パスワードや API キーは変更する必要があります）。

5.7. リポジトリの設定

5.7.1. GitHub 上の Settings

リポジトリを作成した後、GitHub 上の **Settings** タブからさまざまな設定を行うことができます。ここでは特に重要な設定項目について解説します。

スクリーンショット: リポジトリの *Settings* タブ画面

5.7.1.1. リポジトリの可視性（Visibility）

リポジトリの可視性は、誰がリポジトリの内容を閲覧できるかを制御します。

設定	説明	適した用途
Public	インターネット上の誰でも閲覧・クローン可能	オープンソースプロジェクト、ポートフォリオ
Private	招待された共同作業者のみアクセス可能	個人プロジェクト、企業の内部コード
Internal	Organization 内のメンバーのみアクセス可能	Enterprise 向け、社内利用

可視性は Settings の「General」セクションで変更できます。また、GitHub CLI からも変更可能です。

```
# リポジトリを公開に変更する
gh repo edit --visibility public

# リポジトリを非公開に変更する
gh repo edit --visibility private
```

Public から Private への変更は即座に反映されますが、既に Fork されたリポジトリには影響しません。逆に、Private から Public に変更する場合は、リポジトリの全履歴が公開されることに注意してください。

5.7.1.2. ブランチ保護（Branch Protection Rules）

チーム開発において非常に重要な機能が**ブランチ保護ルール**です。Settings の「Branches」セクションで設定できます。

ブランチ保護ルールを設定すると、たとえば `main` ブランチに直接プッシュすることを禁止し、必ずプルリクエストを経由してマージすることを強制できます。これにより、コードレビューを必ず通す運用が実現します。

主な保護設定には以下のものがあります。

- **Require a pull request before merging**: マージ前にプルリクエストを必須にする
- **Require approvals**: 指定した人数のレビュー承認を必須にする
- **Require status checks to pass**: CI/CD のテストが成功しないとマージできなくする
- **Restrict who can push**: プッシュできるユーザーを制限する

スクリーンショット: ブランチ保護ルールの設定画面

5.7.1.3. Collaborators（共同作業者）

Private リポジトリで他の人と共同作業をする場合、Settings の「Collaborators」からユーザーを招待できます。招待されたユーザーは、リポジトリへのプッシュやプルリクエストの作成などが行

えるようになります。

5.7.2. テンプレートリポジトリ

同じような構成のプロジェクトを何度も作成する場合、**テンプレートリポジトリ**を活用すると効率的です。テンプレートリポジトリから新しいリポジトリを作成すると、ファイル構造がコピーされますが、コミット履歴はコピーされません（新しい履歴がゼロから始まります）。

テンプレートリポジトリの作成手順:

1. テンプレートにしたいリポジトリの Settings を開く
2. 「General」セクションの「**Template repository**」にチェックを入れる

テンプレートから新しいリポジトリを作成する手順:

1. テンプレートリポジトリのページで「**Use this template**」ボタンをクリック
2. 「**Create a new repository**」を選択
3. 通常のリポジトリ作成と同じように名前や設定を入力して作成

```
# GitHub CLIでテンプレートから作成する場合
gh repo create my-new-project --template username/template-repo --
   clone
```

テンプレートリポジトリは、チームの開発規約に沿ったプロジェクト構成（ディレクトリ構造、CI/CD 設定、コーディングルールのファイルなど）を統一するのに非常に有効です。

5.7.3. リポジトリの削除

リポジトリを削除する場合は、Settings ページの最下部にある「**Delete this repository**」から実行できます。

```
# GitHub CLIで削除する場合（確認プロンプトが表示される）
gh repo delete username/repo-name --yes
```

> **注意**: リポジトリの削除は取り消せません。クローンを持っている人のローカルにはデータが残りますが、*GitHub* 上からはイシュー、プルリクエスト、設定を含めてすべてが完全に削除されます。削除前に、本当に不要なリポジトリかどうかを確認してください。

5.8. まとめ

この章では、Git を使った作業の出発点であるリポジトリについて、その構造から作成・管理までを詳しく学びました。

- **リポジトリの構造**: `.git` フォルダがリポジトリの心臓部であり、すべての変更履歴を保持している
- **作成方法**: GitHub で作成してクローンする方法が最も簡単で推奨される。ローカルで先に作業を始めた場合は `git init` してから GitHub に接続する
- **クローン**: `git clone` は単なるコピーではなく、リポジトリの完全な複製を作る。`--depth` や `--branch` などのオプションで動作をカスタマイズできる
- **リモート管理**: `origin` は自分のリモートリポジトリ、`upstream` は Fork 元を指す慣例的な名前
- **.gitignore**: ビルド成果物、依存パッケージ、機密情報、OS 固有ファイルを除外する。既に追跡されたファイルは `git rm --cached` で解除が必要
- **リポジトリ設定**: 可視性の変更、ブランチ保護ルール、テンプレートリポジトリなど、GitHub 上で多くの設定が可能

次の章では、リポジトリを作成した後の最も基本的な操作 — ファイルの変更を記録し、GitHub に共有するワークフローを学びます。

基本ワークフロー — 変更を記録し共有する

6.1. この章で学ぶこと

前の章でリポジトリという「器」を作る方法を学びました。この章では、その器の中で日々の開発作業をどのように進めるか — つまり、ファイルの変更を記録し、チームメンバーと共有するための基本的なワークフローを学びます。

Git の基本ワークフローは、突き詰めると「編集する → 記録する → 共有する」というシンプルな流れです。しかし、このシンプルな流れの中に、Git が持つ強力な思想が詰まっています。この章では、以下のコマンドとその背景にある考え方を丁寧に解説します。

- `git status` — 今の状態を把握する
- `git add` — 記録したい変更を選び取る（ステージング）
- `git commit` — 変更を確定して履歴に刻む
- `git push` — ローカルの履歴を GitHub に送信する
- `git pull` — GitHub の変更をローカルに取り込む
- `git log` — 過去の履歴を振り返る
- `git diff` — 変更内容を詳しく確認する
- ファイルの移動・削除・変更の取り消し

各コマンドを「何ができるか」だけでなく「なぜそのような設計になっているのか」という観点から理解することで、応用的な場面でも自信を持って操作できるようになります。

6.2. Git の 3 つのエリアを理解する

Git を使いこなすうえで最も重要な概念が、**3 つのエリア**の理解です。Git では、ファイルの変更が最終的にリポジトリに記録されるまでに、3 つの段階を経ます。

```
ワーキングディレクトリ　　ステージングエリア　　ローカルリポジトリ　　リモートリポジトリ
(Working Directory)　　(Staging Area)　　(Local Repository)　　(GitHub)
┌──────────────────────┐　　┌──────────────────────────────────────────────────────┐　　┌──────────────────────────────────────────────────────┐
│　　　
```

```
│   自由に編集      │ ──→   │   選択する      │ ──→   │   確定
  する      │ ──→ │   共有する      │
│               │       │               │     │               │
      │               │
│               │       │               │     │               │
  pull │               │
│               │       │               │     │               │
   ←──  │               │
└──────────────────────────────┘       └──────────────────────
  ────┘     └──────────────────────────┘     └──────────
  ──────────┘
```

6.2.1. 買い物にたとえて理解する

この 3 つのエリアは、買い物のプロセスにたとえるとわかりやすくなります。

ワーキングディレクトリ（Working Directory）は、お店の中で商品棚を見ながら歩き回っている状態です。商品を手に取って見たり、棚に戻したりと、自由に行動できます。プロジェクトフォルダの中でファイルを編集している状態がこれに当たります。

ステージングエリア（Staging Area）は、「買い物カゴ」に相当します。棚から商品を選んでカゴに入れた状態です。まだ購入は確定していないので、カゴから商品を戻すこともできます。Git では `git add` でファイルをステージングエリアに追加します。インデックス（Index）とも呼ばれます。

ローカルリポジトリ（Local Repository）は、「レジで会計を済ませた」状態です。購入が確定し、レシート（コミット履歴）が発行されます。`git commit` でステージングエリアの内容が確定され、変更履歴として永続的に記録されます。

そして **リモートリポジトリ** は、「購入した商品を自宅に届ける」ようなものです。`git push` でローカルの履歴を GitHub に送信し、チームメンバーと共有します。

6.2.2. なぜステージングエリアが必要なのか

「編集したファイルを直接コミットすればいいのでは？」と思うかもしれません。しかし、ステージングエリアの存在には重要な意味があります。

たとえば、あなたが 3 つのファイルを編集したとします。そのうち 2 つはログイン機能の実装に関する変更で、残り 1 つは README の誤字修正です。これらを 1 つのコミットにまとめてしまうと、後から履歴を見たときに「このコミットは何をしたのか」がわかりにくくなります。

ステージングエリアがあることで、「ログイン機能の変更だけを選んでコミット」→「README の修正を別のコミットとして記録」という、論理的に意味のある単位でコミットを作成できます。これは第 6.5 節のコミットメッセージの話とも密接に関わってきます。

6.3. git status — 現在の状態を把握する

`git status` は、Git の作業中に最も頻繁に使うコマンドです。現在のリポジトリの状態 — どのファイルが変更されているか、どのファイルがステージングされているか、未追跡のファイルがあるか — を一目で把握できます。

6.3.1. 出力の読み方

`git status` を実行すると、以下のような出力が表示されます。1 つずつ意味を見ていきましょう。

```
git status
```

```
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working
     directory)
        modified:   src/main.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        new-file.txt
```

1 行目: `On branch main` — 現在 `main` ブランチで作業していることを示しています。ブランチについては第 7 章で詳しく学びます。

2 行目: `Your branch is up to date with 'origin/main'.` — ローカルの `main` ブランチがリモート（GitHub）の `main` ブランチと同じ状態であることを意味します。新しいコミットをプッシュしていない場合やリモートに新しいコミットがある場合は、別のメッセージが表示されます。

`Changes to be committed:` **（緑色で表示）** — ステージングエリアに追加されたファイルの一覧です。次に `git commit` を実行すると、ここに表示されているファイルの変更がコミットに含まれます。カッコ内には、ステージを取り消すためのコマンドが親切に表示されています。

`Changes not staged for commit:` **（赤色で表示）** — 変更されているがまだステージングされてい

ないファイルです。`git add` でステージングするか、`git restore` で変更を元に戻すことができます。

`Untracked files:`（赤色で表示）— Gitが一度も追跡したことのない新しいファイルです。`git add` で追跡を開始できます。

6.3.2. 短縮表示

頻繁に `git status` を実行する場合、短縮表示が便利です。

```
git status -s
```

```
M  README.md        # 左側のMは「ステージ済みの変更」
 M src/main.py      # 右側のMは「未ステージの変更」
?? new-file.txt     # ??は「未追跡ファイル」
A  added-file.txt   # Aは「新規追加してステージ済み」
```

短縮表示では2列の文字で状態を表します。左列がステージングエリアの状態、右列がワーキングディレクトリの状態を示します。慣れると情報が一目で読み取れるようになります。

6.4. git add — ステージングの操作と考え方

`git add` は、ワーキングディレクトリの変更をステージングエリアに追加するコマンドです。「次のコミットに含めたい変更を選ぶ」操作と考えてください。

6.4.1. 基本的な使い方

```
# 特定のファイルをステージングする
git add README.md

# 複数のファイルを指定する
git add file1.py file2.py file3.py

# 特定のディレクトリ内の全ファイルをステージングする
git add src/

# カレントディレクトリ以下の全変更をステージングする
git add .

# リポジトリ全体の全変更をステージングする（削除されたファイルも含
  む）
```

```
git add -A
```

6.4.2. なぜ全部addせず選ぶのか

`git add .` や `git add -A` を使えばすべての変更を一度にステージングできます。しかし、先ほど説明したように、コミットは「論理的に意味のある単位」で作成することが望ましいのです。

たとえば、バグを修正している最中に、ついでにコードのインデントを整えたとします。このとき、バグ修正のファイルとインデント修正のファイルを別々のコミットにすることで、後から「バグ修正のコミットだけを取り消す」といった操作が可能になります。すべてを1つのコミットにまとめてしまうと、このような柔軟な対応ができなくなります。

6.4.3. -pオプション：ファイルの一部だけをステージングする

`git add -p`（`--patch`）は、1つのファイルの中から一部の変更だけをステージングできる強力なオプションです。

```
git add -p src/main.py
```

このコマンドを実行すると、ファイル内の変更箇所が「ハンク（hunk）」という単位で1つずつ表示され、それぞれについて以下の選択肢が提示されます。

- `y` — このハンクをステージングする
- `n` — このハンクをスキップする（ステージングしない）
- `s` — ハンクをさらに小さく分割する
- `q` — 終了する

この機能は、1つのファイルに複数の論理的に異なる変更を加えた場合に特に有用です。

6.4.4. ステージングの取り消し

間違えてステージングしてしまった場合は、`git restore --staged` で取り消すことができます。ステージングを取り消しても、ファイルの内容自体は変更されたままです。あくまでも「ステージングエリアから外す」だけの操作です。

```
# 特定ファイルのステージングを取り消す
git restore --staged README.md

# すべてのステージングを取り消す
git restore --staged .
```

6.5. git commit — 変更を記録する

`git commit` は、ステージングエリアにある変更をローカルリポジトリに「確定」するコマンドです。コミットとは、プロジェクトのある時点の**スナップショット**（全体像の記録）に、メッセージや著者情報、タイムスタンプなどのメタデータを付けたものです。

6.5.1. 基本的な使い方

```
# メッセージを付けてコミットする
git commit -m "Add user authentication feature"

# 複数行のメッセージを書く場合
git commit -m "Add user authentication feature

- Implement login/logout endpoints
- Add password hashing with bcrypt
- Create user session management"

# エディタを開いてメッセージを書く場合（-m を省略する）
git commit

# 追跡済みファイルの変更をaddとcommitを同時に行う（新規ファイルは対象外）
git commit -am "Fix typo in README"
```

`git commit` を `-m` オプションなしで実行すると、第 2 章で設定したエディタ（VS Code や vim など）が開きます。長いコミットメッセージを書く場合や、フォーマットを丁寧に整えたい場合はこの方法が適しています。

6.5.2. コミットメッセージの書き方の哲学

コミットメッセージは、「未来の自分」や「チームメンバー」に向けた手紙のようなものです。3 か月後にバグが見つかったとき、コミット履歴をたどって原因を探ることがあります。そのとき、`"fix"` や `"update"` としか書かれていないコミットが並んでいたら、原因調査は非常に困難になります。

良いコミットメッセージには、以下のルールがあります。

構造のルール:

- **1 行目（タイトル行）**：50 文字以内で変更の要約を書く。命令形（「Add...」「Fix...」「Update...」）

で始める

- **2 行目**: 空行にする（タイトルと本文を区切るため）
- **3 行目以降（本文）**: 必要に応じて詳細な説明を書く。「なぜ」この変更が必要なのかを記述する

```
feat: Add dark mode support for the dashboard

The dashboard was difficult to use in low-light environments.
This commit adds a toggle switch in the header that switches
between light and dark themes using CSS custom properties.
```

6.5.3. Conventional Commits

チーム開発では、コミットメッセージの書き方を統一するために **Conventional Commits** という規約がよく使われます。これは、コミットメッセージの先頭に「タイプ」を示すプレフィックスを付ける方式です。

プレフィックス	用途	例
`feat:`	新しい機能の追加	`feat: Add dark mode support`
`fix:`	バグの修正	`fix: Resolve login timeout issue`
`docs:`	ドキュメントのみの変更	`docs: Update API reference`
`style:`	コードの意味に影響しない変更	`style: Format with prettier`
`refactor:`	バグ修正でも機能追加でもない変更	`refactor: Extract helper functions`
`test:`	テストの追加・修正	`test: Add unit tests for auth`
`chore:`	ビルドプロセスやツールの変更	`chore: Update dependencies`
`perf:`	パフォーマンスの改善	`perf: Optimize database queries`
`ci:`	CI/CD 設定の変更	`ci: Add GitHub Actions workflow`

Conventional Commits を採用すると、コミットの種類が一目で分かるだけでなく、コミット履歴から自動的に CHANGELOG を生成するツールとの連携も可能になります。

```
# 良い例
git commit -m "fix: Prevent crash when user input is empty"
git commit -m "feat: Add CSV export for report page"
git commit -m "docs: Add contributing guidelines"

# 悪い例（何をしたかわからない）
git commit -m "fix"
git commit -m "update"
git commit -m "WIP"
```

```
git commit -m "asdfgh"
```

6.5.4. –amend：直前のコミットを修正する

コミットした直後に、タイプミスに気づいたり、ファイルを追加し忘れたりすることがあります。そのような場合に `--amend` オプションを使うと、直前のコミットを修正できます。

```
# 直前のコミットメッセージだけを修正する
git commit --amend -m "fix: Correct the validation logic for email
   input"

# ファイルを追加し忘れた場合、追加してから amend する
git add forgotten-file.py
git commit --amend --no-edit   # --no-edit でメッセージはそのまま
```

> **重要な注意**: `--amend` はコミットの内容を書き換えます。つまり、既にプッシュ済みのコミットを `--amend` で修正すると、ローカルとリモートの履歴が食い違い、トラブルの原因になります。`--amend` **はまだプッシュしていないコミットにのみ使う**、というルールを必ず守ってください。

6.6. git push — ローカルの記録をGitHubに送信する

`git commit` で変更をローカルリポジトリに記録した後、その記録をGitHub（リモートリポジトリ）に送信するのが `git push` です。プッシュするまで、あなたのコミットは他のチームメンバー

```
# 基本のプッシュ（上流ブランチが設定済みの場合）
git push

# 初回プッシュ時（上流ブランチを設定する）
git push -u origin main
```

6.6.2. -u オプション（–set-upstream）の意味

初めてブランチをプッシュするとき、Git は「ローカルのこのブランチは、リモートのどのブランチに対応するのか？」を知りません。`-u`（`--set-upstream` の省略形）オプションを付けることで、ローカルブランチとリモートブランチの対応関係（上流ブランチ、upstream branch）を設定します。

```
# 初回: -u でローカルのmainとリモートのmainを関連付ける
git push -u origin main

# 2回目以降: -u は不要。git push だけでOK
git push
```

この設定は一度行えばリポジトリに記録されるため、同じブランチで 2 回目以降は `git push` だけで十分です。

6.6.3. プッシュできない場合の対処

チーム開発では、あなたが作業している間に他のメンバーがリモートにプッシュしていることがあります。この場合、あなたの `git push` は拒否されます。

```
git push
# → ! [rejected]        main -> main (fetch first)
# → error: failed to push some refs to 'github.com:username/repo.
   git'
```

このエラーは「リモートにあなたが持っていない新しいコミットがあるので、まず取り込んでください」という意味です。対処方法は次のとおりです。

```
# まずリモートの変更を取り込む
git pull

# コンフリクト（競合）が発生した場合は解決する（第8章で詳しく解説）

# その後、改めてプッシュする
git push
```

このように、Git は「リモートの変更を無視して上書きする」ことを防ぐ安全機構を持っています。

6.7. git pull — GitHubの変更をローカルに取り込む

`git pull` は、リモートリポジトリ（GitHub）にある最新の変更をローカルに取り込むコマンドです。チーム開発では、作業を始める前に `git pull` を実行して最新の状態にするのが良い習慣です。

6.7.1. 基本的な使い方

```
# 基本のプル（上流ブランチから取り込む）
git pull

# 特定のリモート・ブランチを指定する
git pull origin main

# リベースモードでプルする（履歴がきれいになる）
git pull --rebase
```

6.7.2. pull = fetch + merge

`git pull` は実際には2つのコマンドを連続して実行しています。まず `git fetch` でリモートの最新情報をダウンロードし、次に `git merge` でダウンロードした変更をローカルブランチに統合します。

```
# git pull は以下の2つのコマンドと同じ動作をする
git fetch           # リモートの最新情報をダウンロード（ローカルの
   ファイルは変わらない）
git merge origin/main  # ダウンロードした変更をマージする
```

6.7.3. fetchだけを使う理由

`git fetch` を単独で使う場面もあります。`fetch` はリモートの情報をダウンロードするだけで、ローカルのファイルには一切手を加えません。これにより、「リモートでどのような変更があったのか」を安全に確認してから、マージするかどうかを判断できます。

```
# リモートの最新情報を取得する（ローカルのファイルは変更されない）
git fetch

# リモートにある自分が持っていないコミットを確認する
git log HEAD..origin/main --oneline
```

```
# 内容を確認して問題なければマージする
git merge origin/main
```

この方法は、大きな変更がリモートにプッシュされている可能性があるとき、いきなりマージして自分の作業が壊れるのを避けたい場合に有用です。「まず偵察してから行動する」というアプローチです。

6.8. git log — 履歴を確認する

`git log` は、リポジトリのコミット履歴を表示するコマンドです。過去にどのような変更が、誰によって、いつ行われたのかを確認できます。

6.8.1. 基本的な使い方と各オプション

```
# 基本のログ表示（新しいコミットから順に表示される）
git log
```

基本の `git log` を実行すると、各コミットについてハッシュ値、著者、日時、コミットメッセージが表示されます。`q` キーを押すと表示を終了できます。

```
# 1行表示（ハッシュの短縮形 + コミットメッセージだけを表示）
git log --oneline

# 出力例:
# a1b2c3d feat: Add dark mode support
# d4e5f6g fix: Resolve login timeout issue
# k0l1m2n Initial commit
```

`--oneline` はコミット全体を俯瞰するのに便利です。ハッシュ値は先頭7文字に短縮されますが、通常はこれでコミットを一意に特定できます。

```
# グラフ表示（ブランチの分岐・合流を視覚的に表示）
git log --oneline --graph --all

# 出力例:
# * a1b2c3d (HEAD -> main, origin/main) feat: Add dark mode
# * d4e5f6g fix: Resolve login issue
# | * h7i8j9k (feature/api) feat: Add REST API
# |/
```

```
# * k0l1m2n Initial commit
```

`--graph` はブランチの分岐と合流を ASCII アートで描画します。`--all` はすべてのブランチを表示します（省略すると現在のブランチのみ）。

```
# 最新の5件だけ表示する
git log -5

# 特定ファイルに関するコミットだけを表示する
git log -- README.md

# 変更内容（diff）も一緒に表示する
git log -p

# ファイルごとの変更行数の統計を表示する
git log --stat

# 特定の著者のコミットだけを表示する
git log --author="username"

# 日付で絞り込む
git log --since="2024-01-01" --until="2024-06-30"

# コミットメッセージにキーワードを含むものを検索する
git log --grep="fix"
```

6.8.2. きれいなログ表示のエイリアス

日常的に使うログ表示コマンドは長くなりがちです。Git のエイリアス機能を使って、短いコマンドで呼び出せるように設定しておくと便利です。

```
# エイリアスを設定する
git config --global alias.lg "log --oneline --graph --decorate --
   all"

# 設定後は以下のように使える
git lg
```

`--decorate` オプションは、ブランチ名やタグ名をコミットの横に表示します。先ほどの出力例で `(HEAD -> main, origin/main)` のように表示されていたのが、このオプションの効果です。

6.9. git diff — 差分を確認する

`git diff` は、ファイルの変更内容を詳細に確認するコマンドです。「何が変わったのか」を正確に把握するために使います。コミットやプッシュの前に差分を確認する習慣をつけると、意図しない変更を混入させるリスクを減らせます。

6.9.1. 4つの差分比較パターン

`git diff` は、比較するエリアの組み合わせによって表示される内容が変わります。

パターン 1: ワーキングディレクトリとステージングエリアの差分（未ステージの変更）

```
git diff
```

まだ `git add` していない変更を表示します。最も基本的な使い方です。

パターン 2: ステージングエリアとリポジトリの差分（ステージ済みの変更）

```
git diff --staged
# または
git diff --cached    # --staged と同じ意味
```

`git add` してステージングしたが、まだコミットしていない変更を表示します。「次のコミットに含まれる内容」を確認するのに使います。

パターン 3: コミット間の差分

```
# 2つのコミット間の差分を表示する
git diff abc1234..def5678

# 直前のコミットと現在の差分
git diff HEAD~1..HEAD
```

特定のコミット間で何が変わったかを確認するときに使います。

パターン 4: ブランチ間の差分

```
# 2つのブランチの差分を表示する
git diff main..feature-branch
```

プルリクエストを作成する前に、main ブランチとの差分を確認するのに使えます。

6.9.2. 便利なオプション

```
# 特定ファイルの差分だけを表示する
git diff README.md

# 変更されたファイルの名前だけを一覧表示する（内容は表示しない）
git diff --name-only

# ファイルごとの変更行数の統計だけを表示する
git diff --stat

# 単語単位で差分を表示する（行全体ではなく変更箇所だけをハイライト）
git diff --word-diff
```

`--name-only` と `--stat` は、大量のファイルが変更されている場合に全体像を把握するのに便利です。まず `--stat` で概要を確認し、気になるファイルだけ `git diff ファイル名` で詳細を見る、という使い方がおすすめです。

6.10. ファイルの移動・削除・変更取り消し

日常の開発では、ファイルの名前を変えたり、不要になったファイルを削除したり、編集内容を元に戻したりすることがあります。これらの操作にも Git 専用のコマンドがあります。

6.10.1. ファイルの移動・名前変更（git mv）

```
# ファイル名を変更する
git mv old-name.py new-name.py

# ファイルを別のディレクトリに移動する
git mv src/utils.py lib/utils.py
```

`git mv` を使わず、OS のファイルマネージャやコマンド（`mv`）で名前を変えても動作しますが、Git はそれを「旧ファイルの削除 + 新ファイルの追加」として認識します。`git mv` を使うと、Git が「名前変更」として正しく追跡してくれます。

6.10.2. ファイルの削除（git rm）

```
# Gitの追跡からもディスクからも削除する
git rm unnecessary-file.py
```

```
# Gitの追跡だけ解除し、ディスク上にはファイルを残す
git rm --cached secret.env
```

`git rm --cached` は、第5章の `.gitignore` の節で解説した「既に追跡されたファイルを無視する」場面で特に重要です。

6.10.3. 変更の取り消し

ファイルを編集したが、やっぱり元に戻したい、という場面は頻繁にあります。`git restore` コマンドで元に戻すことができます。

```
# ファイルの変更を元に戻す（ワーキングディレクトリの変更を取り消
  す）
git restore README.md

# 全ファイルの変更を元に戻す
git restore .

# ステージングを取り消す（変更自体はワーキングディレクトリに残る）
git restore --staged README.md
```

> **注意**: `git restore` で元に戻した変更は、コミットされていない限り復元できません。本当に不要な変更かどうかを確認してから実行してください。

6.11. 実践：一連の開発フローを体験する

ここまで学んだコマンドを使って、実際の開発フローを一通り体験してみましょう。新しいリポジトリを作成するところから、ファイルを編集してコミット・プッシュするまでの流れをステップバイステップで進めます。

```
# ステップ1: リポジトリをクローンする
git clone git@github.com:username/my-project.git
cd my-project

# ステップ2: 現在の状態を確認する
git status
# → "nothing to commit, working tree clean" と表示されるはず

# ステップ3: 新しいブランチを作成して切り替える（詳しくは第7章）
```

```
git switch -c feature/add-login

# ステップ4: ファイルを編集する（エディタで自由に作業）
# ここでは src/auth.py を新規作成したとします

# ステップ5: 状態を確認する
git status
# → Untracked files: に src/auth.py が表示される

# ステップ6: 差分を確認する（新規ファイルの場合は内容全体が差分として表示）
git diff

# ステップ7: ファイルをステージングする
git add src/auth.py

# ステップ8: ステージングされた内容を確認する
git diff --staged

# ステップ9: コミットする
git commit -m "feat: Add login functionality"

# ステップ10: さらに作業を続けてコミットする
# ... テンプレートファイルを編集 ...
git add src/templates/login.html
git commit -m "feat: Add login page template"

# ステップ11: 履歴を確認する
git log --oneline
# → 2つのコミットが表示される

# ステップ12: GitHubにプッシュする
git push -u origin feature/add-login

# ステップ13: GitHubでプルリクエストを作成する（第10章で詳しく解説）
gh pr create --title "feat: Add login feature" --body "Login機能を追加"
```

この流れが、日常的な開発作業の基本パターンです。慣れてくると、`status` → `add` → `commit` → `push` のサイクルが自然に身についてきます。最初のうちは、各ステップの間に `git status` を挟んで状態を確認する習慣をつけると、Git の動作への理解が深まります。

6.12. まとめ

この章では、Git の日常的なワークフローの中核となるコマンド群を学びました。

- **3 つのエリア**: ワーキングディレクトリ → ステージングエリア → ローカルリポジトリという 3 段階の構造が、Git の柔軟性の源泉である
- **git status**: 作業中に最も頻繁に使うコマンド。現在の状態を把握する習慣をつけよう
- **git add**: コミットに含める変更を選択する。`-p` オプションでファイル内の一部だけをステージングすることもできる
- **git commit**: 変更を履歴に確定する。コミットメッセージは「未来の自分への手紙」として丁寧に書こう。Conventional Commits の規約が広く使われている
- **git push**: ローカルの記録を GitHub に送信する。初回は `-u` で上流ブランチを設定する
- **git pull**: GitHub の変更を取り込む。内部的には fetch + merge の 2 段階で動作する。安全を期すなら `fetch` だけで偵察してからマージするとよい
- **git log**: 履歴を確認する。`--oneline --graph --all` の組み合わせが特に便利
- **git diff**: 差分を確認する。ステージ前・ステージ後・コミット間・ブランチ間の 4 パターンを使い分ける

次の章では、チーム開発に不可欠な「ブランチ」の概念と操作を学びます。ブランチを使いこなすことで、複数の機能を同時に、安全に開発できるようになります。

ブランチ — 並行開発の技法

7.1. この章で学ぶこと

ここまでの章では、1 本の流れの上でファイルを編集し、コミットし、プッシュする方法を学びました。しかし実際の開発では、「新しい機能を追加しながら、同時にバグを修正する」「本番環境で動いているコードには触れずに、実験的な変更を試す」といった場面が頻繁に発生します。

このような並行作業を可能にするのが**ブランチ（branch）**です。ブランチは、Git の中でも最も重要な機能の 1 つであり、現代のソフトウェア開発のワークフローを支える根幹の仕組みです。

この章では、以下のことを学びます。

- ブランチの正体 — Git の内部でブランチがどのように実装されているか
- ブランチの基本操作 — 作成、切り替え、削除、名前変更
- ブランチの命名規則 — チームで統一するためのルール
- マージ（merge）— ブランチの変更を統合する 2 つの方式
- リベース（rebase）— 履歴を整理するための強力な操作
- ブランチ戦略 — チームの規模やプロジェクトの性質に応じた運用方針

ブランチの操作自体はシンプルですが、その背景にある考え方を理解しているかどうかで、チーム開発での立ち回りが大きく変わります。

7.2. ブランチとは何か

7.2.1. ポインタとしてのブランチ

多くの入門書では、ブランチを「開発の枝分かれ」と説明します。この説明は概念としては正しいのですが、Git の内部実装を知ると、ブランチの本質がより明確になります。

Git におけるブランチは、**特定のコミットを指し示すポインタ（参照）**に過ぎません。ブランチの実体は `.git/refs/heads/` ディレクトリにある小さなテキストファイルで、その中にはコミットのハッシュ値（40 文字の英数字）が 1 行だけ書かれています。

たとえば `main` ブランチの実体は `.git/refs/heads/main` というファイルで、その中身は次のようなものです。

```
a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0
```

これは、`main` ブランチが指しているコミットのハッシュ値です。新しいコミットを作ると、このポインタが最新のコミットに自動的に移動します。つまり、ブランチを作るという操作は「コミットを丸ごとコピーする」のではなく、「新しいポインタを 1 つ作る」だけなのです。これが、Git のブランチ作成が一瞬で完了する理由です。

```
main:    A → B → C          ← mainブランチはコミットCを指している
                 ↑
                 main（ポインタ）
```

ここで新しいブランチ `feature` を作ると、同じコミット C を指す別のポインタが追加されるだけです。

```
main:    A → B → C          ← mainもfeatureもコミットCを指している
                 ↑
                 main, feature（2つのポインタ）
```

`feature` ブランチに切り替えてコミットを重ねると、`feature` のポインタだけが前に進みます。

```
main:    A → B → C              ← mainはCのまま
                 \
feature:          D → E        ← featureはEを指している
```

7.2.2. HEAD とは何か

ブランチのポインタに加えて、もう 1 つ重要なポインタがあります。それが **HEAD** です。HEAD は「今、自分がどのブランチで作業しているか」を示すポインタです。`git switch main` を実行すると HEAD が main を指し、`git switch feature` を実行すると HEAD が feature を指します。

つまり、`git commit` を実行したとき、HEAD が指しているブランチのポインタが前に進むのです。

7.2.3. なぜブランチが必要なのか

ブランチの必要性を、具体的なシナリオで考えてみましょう。

シナリオ 1: 機能開発中にバグ報告が入った

あなたは新しい検索機能を開発中です。半分ほど実装が終わったところで、「本番環境でログインできなくなっている」という緊急の報告が入りました。ブランチがなければ、中途半端な検索機能のコードが含まれた状態でバグ修正をするか、検索機能のコードを手動で元に戻してからバグ修正をするか、どちらにしても大変な作業になります。

ブランチがあれば、検索機能は `feature/search` ブランチに残したまま、`main` ブランチに切り替えて `hotfix/login-bug` ブランチを作り、バグだけを修正できます。修正が終わったら `main` にマージしてデプロイし、その後 `feature/search` に戻って検索機能の開発を再開する、という

流れが自然に行えます。

シナリオ 2: 複数人での同時開発

チームの 3 人がそれぞれ異なる機能を担当しています。全員が `main` ブランチで直接作業すると、一人がプッシュするたびに他の 2 人は `git pull` でコンフリクトを解決しなければなりません。各自が専用のブランチで作業し、完成したらプルリクエストを通じてマージする、というワークフローなら、お互いの作業に干渉せずに開発を進められます。

シナリオ 3: 実験的な変更を試したい

「このライブラリを使えばパフォーマンスが改善するかもしれない」という仮説を検証したいとき、ブランチを作って実験できます。うまくいけばマージし、ダメなら単にブランチを削除するだけです。`main` ブランチの安定性は一切損なわれません。

7.3. ブランチの基本操作

7.3.1. ブランチの一覧表示

まず、現在存在するブランチを確認する方法です。

```
# ローカルブランチの一覧を表示する
git branch
# * main              ← アスタリスク（*）が付いているのが現在のブランチ
#   feature/login
#   fix/typo

# リモートブランチも含めて表示する
git branch -a
# * main
#   feature/login
#   remotes/origin/main
#   remotes/origin/develop

# リモートブランチのみ表示する
git branch -r

# 各ブランチの最新コミット情報も合わせて表示する
git branch -v
# * main           a1b2c3d feat: Add dark mode
#   feature/login d4e5f6g feat: Implement login
```

`git branch -a` で表示される `remotes/origin/main` は「リモート追跡ブランチ」と呼ばれ、最後

に `fetch` または `pull` した時点でのリモートブランチの状態を記憶しています。

7.3.2. ブランチの作成

```
# ブランチを作成する（作成するだけで、切り替えはしない）
git branch feature/new-feature

# ブランチを作成して、同時に切り替える
git switch -c feature/new-feature

# 従来の書き方（同じ動作）
git checkout -b feature/new-feature

# 特定のコミットを起点としてブランチを作成する
git switch -c hotfix/urgent abc1234

# リモートブランチを元にローカルブランチを作成する
git switch -c feature/api origin/feature/api
```

`-c` は `--create` の省略形です。「create（作成）して switch（切り替え）する」と覚えると、オプションの意味がわかりやすいでしょう。

7.3.3. ブランチの切り替え（switch vs checkout）

```
# 新しい書き方（推奨）
git switch main

# 従来の書き方
git checkout main

# 直前にいたブランチに戻る（ハイフンが「前の場所」を意味する）
git switch -
```

7.3.3.1. switch と checkout の歴史的経緯

Git 2.23（2019 年リリース）以前は、ブランチの切り替えには `git checkout` コマンドが使われていました。しかし `checkout` は、ブランチの切り替えだけでなく、ファイルの変更を元に戻す操作にも使えるコマンドでした。

```
# checkout の2つの意味（紛らわしい）
```

```
git checkout feature    # ブランチの切り替え
git checkout -- file.py # ファイルの変更を元に戻す
```

この設計は「1つのコマンドが2つの異なる役割を持つ」という点で混乱の原因になっていました。そこでGit 2.23で、ブランチ操作専用の `git switch` と、ファイル復元専用の `git restore` が新設されました。

```
# 新しいコマンドによる役割の分離（わかりやすい）
git switch feature      # ブランチの切り替え
git restore file.py     # ファイルの変更を元に戻す
```

`checkout` も引き続き使用可能ですが、新しいプロジェクトでは `switch` と `restore` を使うことが推奨されています。

7.3.4. ブランチの削除

```
# マージ済みのブランチを削除する
git branch -d feature/completed

# 強制削除（マージされていなくても削除する）
git branch -D feature/abandoned

# リモートブランチを削除する
git push origin --delete feature/old-branch

# ローカルに残っている、削除されたリモートブランチの参照をクリーン
  アップする
git fetch --prune
```

`-d`（小文字）は安全な削除です。そのブランチの変更がまだどこにもマージされていない場合、Gitが警告を出して削除を拒否します。これは、うっかり作業中のブランチを消してしまうのを防ぐ安全機構です。本当に不要なブランチを削除したい場合は `-D`（大文字）で強制削除します。

`git fetch --prune` は、チームメンバーがGitHub上で削除したブランチの参照がローカルに残っている場合に、それをクリーンアップするコマンドです。定期的に実行すると、`git branch -a` の表示がすっきりします。

7.3.5. ブランチ名の変更

```
# 現在のブランチの名前を変更する
```

```
git branch -m new-name

# 指定したブランチの名前を変更する
git branch -m old-name new-name
```

`-m` は `--move` の省略形です。UNIX の `mv` コマンド（ファイルの移動・名前変更）と同じ発想です。

7.4. ブランチの命名規則

ブランチ名は自由に付けることができますが、チーム開発では命名規則を統一しておくことが重要です。ブランチ名を見ただけで「これは何のためのブランチか」がわかるようにするのが目標です。

7.4.1. プレフィックスによる分類

最も広く使われている方法は、スラッシュ（`/`）区切りのプレフィックスを付ける方法です。

プレフィックス	用途	例
`feature/`	新しい機能の開発	`feature/user-auth`, `feature/dark-mode`
`fix/`	バグの修正	`fix/login-crash`, `fix/header-alignment`
`hotfix/`	本番環境の緊急修正	`hotfix/security-patch`
`release/`	リリース準備のための調整	`release/v2.0`, `release/2024-q1`
`docs/`	ドキュメントの追加・更新	`docs/api-reference`
`refactor/`	機能変更を伴わないコードの改善	`refactor/database-layer`
`test/`	テストの追加・改善	`test/unit-coverage`

7.4.2. 命名のベストプラクティス

プレフィックスに加えて、以下のルールを守るとブランチ名が読みやすくなります。

- **小文字とハイフン**を使う: `feature/add-user-auth`（`feature/AddUserAuth` ではなく）
- **簡潔だが具体的**に: `feature/auth` は曖昧、`feature/user-password-reset` は明確
- **イシュー番号を含める**のも良い方法: `feature/42-add-search`（イシュー#42 に対応）

- **スラッシュは 1 段階だけ**にする: `feature/auth` は良いが `feature/auth/login/form` は深すぎる

チームでの統一が最も重要です。どのルールを採用するかよりも、全員が同じルールに従うことに意味があります。プロジェクトの初期段階でルールを決め、README や開発ガイドに明文化しておきましょう。

7.5. マージ（merge）

マージは、あるブランチの変更を別のブランチに統合する操作です。たとえば、`feature/login` ブランチで開発したログイン機能を `main` ブランチに取り込む場合にマージを行います。

Git のマージには、状況に応じて 2 つの方式が自動的に選択されます。

7.5.1. Fast-forward マージ

Fast-forward（早送り）マージは、最もシンプルなマージです。分岐後に `main` ブランチに新しいコミットが追加されていない場合に発生します。

```
マージ前:
main:      A → B → C
                   \
feature:            D → E      ← featureブランチはmainの先にある

マージ後 (git merge feature):
main:      A → B → C → D → E    ← mainのポインタがEまで前に進む
   だけ
```

この場合、Git がやることは非常にシンプルです。`main` のポインタを `feature` が指しているコミットまで「早送り」するだけです。新しいコミットは作成されません。

```
# mainブランチに切り替える
git switch main

# featureブランチをマージする
git merge feature/login
# → "Fast-forward" と表示される
```

Fast-forward マージは履歴が直線的でわかりやすくなりますが、「いつブランチが作られてマージされたか」という情報は残りません。それが問題になる場合は、`--no-ff` オプションを使って明示的にマージコミットを作成することもできます。

```
# Fast-forwardが可能な場合でも、必ずマージコミットを作成する
git merge --no-ff feature/login
```

7.5.2. 3-way マージ

分岐後に `main` ブランチにも新しいコミットがある場合、Fast-forward はできません。この場合、Git は **3-way マージ**を行います。「3-way」と呼ばれるのは、以下の 3 つの状態を比較してマージ結果を決定するからです。

1. 共通の祖先（分岐点）のコミット
2. 現在のブランチ（main）の最新コミット
3. マージするブランチ（feature）の最新コミット

```
マージ前:
main:     A → B → C → F          ← mainにも新しいコミットFがある
                  \
feature:           D → E

マージ後 (git merge feature):
main:     A → B → C → F → G      ← Gはマージコミット（FとEの両
   方を親に持つ）
                  \       /
feature:           D → E
```

マージコミット G は、2 つの親コミット（F と E）を持つ特別なコミットです。このマージコミットがあることで、「feature ブランチがこの地点で main に統合された」という情報が履歴に残ります。

```
git switch main
git merge feature/login
# → "Merge made by the 'ort' strategy." と表示される
```

3-way マージでは、両方のブランチで同じ箇所が変更されている場合に**コンフリクト（衝突）**が発生することがあります。コンフリクトの解決方法は第 8 章で詳しく解説します。

7.5.3. マージの取り消し

マージ操作を取り消したい場合の対処法です。

```
# マージ中にコンフリクトが発生し、マージ自体を中止したい場合
```

```
git merge --abort

# マージが完了してコミットされた後に取り消したい場合
# -m 1 は「最初の親（mainブランチ側）の状態に戻す」を意味する
git revert -m 1 HEAD
```

`git merge --abort` はマージ操作を完全になかったことにし、マージ前の状態に戻します。コンフリクトが複雑で手に負えない場合に使います。

`git revert -m 1 HEAD` は、マージコミットを打ち消す新しいコミットを作成します。マージを取り消しつつ、「取り消した」という事実を履歴に残すのが特徴です。

7.6. リベース（rebase）

7.6.1. リベースとは何をしているのか

リベース（rebase）は「ブランチの分岐点（base）を移動させる（re-base）」操作です。マージとは異なり、コミット履歴を書き換えて直線的な歴史を作ります。

具体的に何が起きているか見てみましょう。

```
リベース前:
main:     A → B → C → F
                 \
feature:          D → E     ← featureはCから分岐している

リベース後（feature上で git rebase main):
main:     A → B → C → F
                     \
feature:              D' → E'   ← 分岐点がFに移動した
```

リベースにおいて重要な点は、D' と E' は元の D と E と同じ変更内容を持っていますが、**異なるコミット**（異なるハッシュ値を持つ）だということです。なぜなら、親コミットが変わっているため、Git は変更内容を新しい起点に「貼り直す」ことで新しいコミットを作り直しているのです。

これはちょうど、積み木を組み替えるようなものです。D と E という積み木を、C の上からいったん外して、F の上に積み直す。積み木の形（変更内容）は同じですが、位置（親コミット）が変わるため、新しい積み木（コミット）として扱われるのです。

```
# featureブランチにいる状態でリベースを実行する
git switch feature/login
git rebase main

# コンフリクトが発生した場合
```

```
# 1. ファイルを編集してコンフリクトを解決する
# 2. 解決したファイルをステージングする
git add resolved-file.py
# 3. リベースを続行する
git rebase --continue

# リベースを中止して元に戻したい場合
git rebase --abort
```

リベース中のコンフリクト解決は、マージのコンフリクト解決と基本的に同じですが、リベースでは各コミットを1つずつ貼り直すため、複数回コンフリクトを解決する必要がある場合があります。

7.6.2. merge vs rebase の判断基準

マージとリベースはどちらもブランチの変更を統合する操作ですが、結果として得られる履歴の形が異なります。

観点	merge	rebase
履歴の形	分岐と合流が残る（ネットワーク型）	直線的な履歴になる
既存コミット	変更しない（安全）	書き換える（注意が必要）
コンフリクト解決	一度で済む	コミットごとに発生しうる
マージの事実	マージコミットとして記録される	記録されない
主な用途	ブランチを main に統合する（PR）	作業ブランチを main の最新に追従させる

多くのチームでは、以下のような使い分けをしています。

- **プルリクエストのマージ**: マージを使う（GitHub 上で行う）
- **作業ブランチの更新**: リベースを使う（main の最新変更を自分のブランチに取り込む）

7.6.3. 「push 済みには rebase しない」ルール

リベースには1つの絶対的なルールがあります。**既にプッシュして他の人と共有しているブランチにはリベースしない**ことです。

なぜでしょうか。リベースはコミットを作り直す操作です。つまり、リベース後のコミットは元のコミットとは異なるハッシュ値を持ちます。あなたがリベースした後にプッシュすると、リモートにあるコミットとは別物のコミットが送られることになります。他のメンバーがリベース前のコミットを元に作業している場合、彼らのブランチとの整合性が取れなくなり、非常に面倒な事態を引き起こします。

```
安全にリベースできるケース:
- 自分だけが使っているローカルブランチ
- まだプッシュしていないコミット

リベースしてはいけないケース:
- 既にプッシュしたブランチ（他の人が参照している可能性がある）
- main や develop などの共有ブランチ
```

迷ったら、マージを使いましょう。マージは既存のコミットを変更しないため、常に安全です。

7.7. ブランチ戦略

ブランチ戦略とは、「チームとしてブランチをどのように作り、どのように管理し、どのタイミングでマージするか」という運用方針のことです。プロジェクトの規模やリリースの頻度に応じて、適切な戦略を選ぶことが重要です。

7.7.1. GitHub Flow — シンプルで始めやすい

GitHub Flow は、GitHub が提唱するシンプルなブランチ戦略で、多くの Web アプリケーション開発で採用されています。ルールは非常に明快です。

```
main ————————————————————————————————————————————————
   ————————→  （常にデプロイ可能な状態）
       \           /    \            /
        feature-A —       feature-B —
```

ルール:

1. `main` ブランチは常にデプロイ可能な状態を維持する
2. 新しい作業を始めるときは、`main` から説明的な名前のブランチを作成する
3. ブランチにコミットをプッシュし、プルリクエストを作成する
4. チームメンバーがコードレビューを行う
5. レビューが承認されたら、`main` にマージする
6. マージ後、速やかにデプロイする

GitHub Flow の最大の利点はそのシンプルさです。ルールが少ないため、チーム全員が理解しやすく、運用の負担が小さい。個人開発や小規模チームには特に適しています。

7.7.2. Git Flow — 大規模プロジェクト向けの体系的な戦略

Git Flow は、Vincent Driessen 氏が 2010 年に提唱した、より体系的なブランチ戦略です。リリースサイクルが明確に定義されているプロジェクト（モバイルアプリ、デスクトップソフトウェアなど）に適しています。

```
main     ———————●————————————————————●————————→
      （リリース済みコードのみ）
               ↑                ↑
release ———————┤    release/v2 ┤            （リリース準備）
               ↑                ↑
develop ———————●————————————————————●————————→
      （開発の中心）
          \       /    \        /
           feat-A        feat-B            （個別の機能開発）
```

ブランチの構成:

- **main**: リリース済みのコードだけが含まれる。各コミットにはバージョンタグが付く
- **develop**: 開発の中心となるブランチ。次のリリースに向けた変更が集約される
- **feature/**: 個別の機能開発。`develop` から分岐し、`develop` にマージする
- **release/**: リリース準備のためのブランチ。バージョン番号の更新やバグ修正を行う
- **hotfix/**: 本番環境の緊急修正。`main` から分岐し、`main` と `develop` の両方にマージする

Git Flow はルールが多く運用コストが高いため、継続的デプロイを行う Web サービスには向きません。しかし、複数のバージョンを同時にサポートする必要があるソフトウェアには、その体系的な管理が大きな価値を発揮します。

7.7.3. Trunk-based Development — 高速なイテレーション

Trunk-based Development（トランクベース開発）は、Google や Facebook などの大規模テック企業で採用されている戦略です。「trunk（幹）」とは `main` ブランチのことを指します。

```
main ————●————●————●————●————→    （頻繁に小
   さな変更がマージされる）
        ↑    ↑    ↑    ↑
    短命ブランチ（数時間〜1-2日で完了）
```

特徴:

- ブランチは極めて短命（数時間から長くても 1〜2 日）で `main` にマージする

- 大きな機能は**フィーチャーフラグ**（機能の有効/無効をコードで切り替える仕組み）で制御し、未完成でもマージする
- `main` は常にデプロイ可能な状態を維持する
- CI/CD パイプラインによる自動テストが必須

この戦略の核心は「長命なブランチを作らない」ことにあります。ブランチが長く存在するほど `main` との差分が大きくなり、マージ時のコンフリクトリスクが高まります。短命なブランチでこまめにマージすることで、このリスクを最小化します。

7.7.4. 戦略の選び方

戦略	適した状況	チーム規模
GitHub Flow	Web サービス、個人開発、小規模チーム	1〜10 人
Git Flow	リリースサイクルが明確な製品、複数バージョンのサポート	5〜50 人
Trunk-based	継続的デプロイ、高い自動テストカバレッジがある環境	10 人以上

迷ったら、まず GitHub Flow から始めることをお勧めします。シンプルな戦略で経験を積み、プロジェクトの成長に合わせてより複雑な戦略に移行すればよいのです。

7.8. 実践：ブランチを使った開発の一連の流れ

ここまで学んだ内容を使って、GitHub Flow に基づく実際の開発フローを体験しましょう。ダークモード機能を追加する、という架空のタスクを題材にします。

```
# ステップ1: mainブランチを最新の状態にする
# 作業を始める前に、必ずmainの最新を取得する
git switch main
git pull

# ステップ2: 機能ブランチを作成して切り替える
git switch -c feature/dark-mode

# ステップ3: 機能を実装する（ファイルの編集はエディタで行う）
# ... CSSファイルにダークモードのスタイルを追加 ...
# ... JavaScriptにトグルスイッチの機能を追加 ...

# ステップ4: 状態を確認する
git status

# ステップ5: 変更をステージングしてコミットする
git add src/styles/dark-mode.css
git commit -m "feat: Add dark mode CSS styles"
```

```
# ステップ6: さらに作業を続けてコミットする
git add src/scripts/theme-toggle.js
git commit -m "feat: Add dark mode toggle switch"

# ステップ7: mainの最新変更を取り込む（コンフリクトの早期発見）
# 他のメンバーがmainに変更をマージしている可能性があるため
git fetch origin
git rebase origin/main

# ステップ8: リモートにプッシュする
git push -u origin feature/dark-mode

# ステップ9: プルリクエストを作成する（第10章で詳しく解説）
gh pr create --title "feat: Add dark mode support" \
  --body "ダークモードのCSSスタイルとトグルスイッチを追加"

# ステップ10: レビューを受けてマージされた後、ブランチを掃除する
git switch main
git pull
git branch -d feature/dark-mode
```

スクリーンショット: プルリクエスト作成後の *GitHub* 画面

このフローのポイントは、ステップ7の `git fetch` + `git rebase` です。自分の作業ブランチを定期的に `main` の最新に追従させておくことで、プルリクエスト作成時のコンフリクトを最小限に抑えることができます。

7.9. まとめ

この章では、Git のブランチ機能を包括的に学びました。

- **ブランチの正体**: ブランチはコミットを指すポインタに過ぎない。そのためブランチの作成は一瞬で完了する
- **基本操作**: `git switch -c` で作成と切り替え、`git branch -d` で削除。`switch` は `checkout` に代わる新しいコマンド
- **命名規則**: `feature/`、`fix/`、`hotfix/` などのプレフィックスで目的を明示する。チーム内での統一が重要
- **マージ**: Fast-forward マージは分岐がない場合のシンプルな統合。3-way マージは分岐がある場

合にマージコミットを作成して統合する

- **リベース**: ブランチの分岐点を移動させて直線的な履歴を作る。ただし、プッシュ済みのブランチには絶対にリベースしない
- **ブランチ戦略**: GitHub Flow（シンプル）、Git Flow（体系的）、Trunk-based（高速）の 3 つの戦略があり、プロジェクトの特性に合わせて選択する

ブランチを使った開発では、異なるブランチで同じファイルの同じ箇所を変更した場合に「コンフリクト（衝突）」が発生します。次の章では、コンフリクトが発生する仕組みと、その解決方法を学びます。

コンフリクト — 変更の衝突を解決する

8.1. この章で学ぶこと

チーム開発において、複数人が同時に作業を進めることはごく日常的な光景です。しかし、2人以上の開発者が同じファイルの同じ箇所をそれぞれ異なる形で変更してしまった場合、Gitはどちらの変更を採用すべきか判断できなくなります。この状態を**コンフリクト（conflict、競合）**と呼びます。

コンフリクトという言葉を聞くと、「怖い」「壊れたらどうしよう」と身構える初学者が少なくありません。しかし、コンフリクトはGitが「ここは人間の判断が必要です」と教えてくれているだけであり、正しく対処すれば何も失われることはありません。むしろ、コンフリクトを適切に解決できるスキルは、チーム開発において非常に重要な能力の一つです。

この章では、以下の内容を学びます。

- コンフリクトとは何か、なぜ発生するのかを具体例で理解する
- コンフリクトが発生する4つの典型的な場面を知る
- コンフリクトマーカーの読み方を1行ずつ丁寧に学ぶ
- 手動・VS Code・コマンドの3通りの解決方法を習得する
- マージやリベースの中止方法を知る
- コンフリクトを未然に防ぐチーム運営のプラクティスを身につける

8.2. コンフリクトとは何か

8.2.1. 日常のたとえで理解する

コンフリクトを日常生活に例えてみましょう。あなたとチームメンバーの田中さんが、同じ共有ドキュメントを編集しているとします。あなたは「会議の日程は火曜日」と書き換え、同じ時間帯に田中さんは「会議の日程は木曜日」と書き換えました。このとき、最終的な正解は「火曜日」なのか「木曜日」なのか、コンピュータには判断のしようがありません。これがコンフリクトです。

8.2.2. Gitにおけるコンフリクトの仕組み

Gitは非常に賢いマージアルゴリズムを持っており、ほとんどの場合は自動的に変更を統合してくれます。たとえば、あなたが `README.md` の1行目を変更し、田中さんが同じファイルの50行目を変更した場合、Gitは「別々の箇所の変更だから両方取り込めばよい」と判断して自動マージを行います。

しかし、**同じファイルの同じ行（または非常に近い行）を異なる内容に変更した場合**、Gitは自動マージを諦め、人間に判断を委ねます。これがコンフリクトの正体です。

```
main:     "Hello World"  →  "Hello GitHub"    ← mainブランチで変
    更
```

```
                    \
feature:            →  "Hello Git"                ← featureブランチで
   も同じ箇所を変更

→ どちらを採用すべきかGitには判断できない = コンフリクト発生
```

この図の例では、元々 `"Hello World"` だった文字列を、main ブランチでは `"Hello GitHub"` に変更し、feature ブランチでは `"Hello Git"` に変更しています。Git にとって、どちらが「正しい」変更なのかを知る手がかりはありません。そのため、開発者自身に「どうしますか？」と問いかけるわけです。

8.2.3. コンフリクトは「エラー」ではない

ここで強調しておきたいのは、コンフリクトはエラーやバグではないということです。コンフリクトは、チーム開発における正常な出来事です。Git が「私には判断できないので、あなたが決めてください」と言っているだけであり、リポジトリが壊れたわけでも、データが失われたわけでもありません。落ち着いて解決すれば、何の問題もないのです。

8.3. コンフリクトが発生する場面

コンフリクトは、2 つの異なる変更履歴を統合しようとするあらゆる場面で発生する可能性があります。具体的には、以下の 4 つの操作が代表的です。

8.3.1. git merge でブランチを統合するとき

最も一般的なコンフリクトの発生場面です。feature ブランチでの作業が完了し、main ブランチにマージしようとしたとき、main ブランチ側でも同じ箇所が変更されていると、コンフリクトが発生します。

```
git switch main
git merge feature/login
# Auto-merging src/app.py
# CONFLICT (content): Merge conflict in src/app.py
# Automatic merge failed; fix conflicts and then commit the result.
```

このメッセージを見たら、慌てる必要はありません。Git は「`src/app.py` で自動マージを試みたが、コンフリクトが発生した。コンフリクトを解決してからコミットしてください」と親切に教えてくれています。

8.3.2. git pull でリモートの変更を取り込むとき

チームメンバーがリモートリポジトリに push した変更と、あなたがローカルで行った変更が同じ箇所に及んでいる場合、`git pull` を実行した際にコンフリクトが発生します。これは本質的には `git merge` と同じ現象です（`git pull` は内部的に `git fetch` + `git merge` を行っているため）。

8.3.3. git rebase でブランチの基点を付け替えるとき

リベース操作では、あなたのコミットを 1 つずつ新しい基点の上に適用し直します。この過程で、いずれかのコミットが基点側の変更と衝突した場合、コンフリクトが発生します。リベースの場合、複数のコミットを順番に適用するため、1 回のリベースで複数回コンフリクトが発生する可能性がある点に注意が必要です。

8.3.4. git cherry-pick で特定のコミットを取り込むとき

別のブランチから特定のコミットだけを取り込む `git cherry-pick` でも、取り込もうとするコミットが現在のブランチの内容と衝突すれば、コンフリクトが発生します。

8.3.5. コンフリクト発生時の git status

コンフリクトが発生したら、まず `git status` で状況を確認しましょう。

```
git status
# On branch main
# You have unmerged paths.
#   (fix conflicts and run "git commit")
#
# Unmerged paths:
#   (use "git add <file>..." to mark resolution)
#       both modified:   src/app.py
```

`both modified`（両方で変更されている）という表示が、コンフリクトが起きているファイルを示しています。複数のファイルでコンフリクトが発生している場合は、ここに複数のファイルが列挙されます。

8.4. コンフリクトマーカーの読み方

コンフリクトが発生すると、Git は該当ファイルの中に**コンフリクトマーカー**と呼ばれる特別な記号を挿入します。このマーカーは、「ここから先が衝突している部分ですよ」と示す目印です。初めて見ると少し面食らうかもしれませんが、構造はシンプルです。

8.4.1. マーカーの全体像

```
def greet():
<<<<<<< HEAD
    return "Hello GitHub"
=======
    return "Hello Git"
>>>>>>> feature/login
```

このコードを 1 行ずつ読み解いていきましょう。

8.4.2. 各行の意味

`<<<<<<< HEAD`（7 つの `<` と「HEAD」）

この行は「ここからが、あなたが今いるブランチ（マージ先、つまり HEAD）の変更内容ですよ」という開始マーカーです。HEAD とは、現在チェックアウトしているブランチの最新コミットを指す Git の用語です。マージ操作の場合、これは「マージ先」のブランチ、つまりあなたが `git merge` を実行したときにいたブランチの内容です。

`return "Hello GitHub"`

これが HEAD 側（マージ先ブランチ）の実際のコード内容です。この例では、main ブランチ上で greet 関数が `"Hello GitHub"` を返すように変更されていたことがわかります。

`=======`（7 つの `=`）

この行は区切り線です。ここを境にして、上がマージ先の変更、下がマージ元の変更となります。いわば、2 つの変更を隔てる「仕切り」の役割を果たしています。

`return "Hello Git"`

これがマージ元ブランチの実際のコード内容です。feature ブランチでは、greet 関数が `"Hello Git"` を返すように変更されていたことがわかります。

`>>>>>>> feature/login`（7 つの `>` とブランチ名）

この行は「ここまでがマージ元ブランチ（feature/login）の変更内容ですよ」という終了マーカーです。ブランチ名が表示されるので、どのブランチの変更なのかを識別できます。

8.4.3. マーカーの構造を表にまとめると

このマーカーの読み方を理解しておけば、どんなに長いファイルでコンフリクトが発生しても、落ち着いて対処できます。なお、1 つのファイル内に複数のコンフリクト箇所がある場合、マーカーのセットが複数回登場します。すべてのマーカーを解決しなければ、コンフリクトの解決は完了しません。

マーカー	意味	覚え方
`<<<<<<< HEAD`	マージ先（現在のブランチ）の変更の開始	`<` は「こちら側」を指す矢印
`=======`	2 つの変更の区切り線	「ここから先は相手側」
`>>>>>>> branch-name`	マージ元ブランチの変更の終了	`>` は「あちら側」を指す矢印

スクリーンショット: テキストエディタで表示されたコンフリクトマーカーの例

8.5. 手動で解決する方法

コンフリクトの解決は、突き詰めれば「コンフリクトマーカーを取り除いて、最終的にあるべきコードにする」という作業です。テキストエディタでファイルを開き、自分の判断でコードを編集します。解決のパターンは大きく 3 つあります。

8.5.1. パターン 1：片方の変更を採用する

最もシンプルなパターンです。2 つの変更のうち、どちらか一方が正しいと判断できる場合、一方を残してもう一方とすべてのマーカーを削除します。

たとえば、main ブランチの変更（`"Hello GitHub"`）を採用する場合、コンフリクト箇所を以下のように書き換えます。

```
def greet():
    return "Hello GitHub"
```

マーカー行（`<<<<<<<`、`=======`、`>>>>>>>`）はすべて削除し、採用する側のコードだけを残す点がポイントです。マーカーが 1 つでも残っていると、プログラムとして正しく動作しません。

8.5.2. パターン 2：両方の変更を統合する

2 つの変更がどちらも必要な場合、両方の意図を活かした新しいコードを書きます。

```
def greet():
    return "Hello GitHub and Git"
```

この例では、「GitHub」と「Git」のどちらも盛り込む形で統合しています。両方の変更者の意図を尊重しつつ、整合性のあるコードにすることが求められます。

8.5.3. パターン3：完全に書き直す

どちらの変更もそのままでは不十分だと判断した場合、コンフリクト箇所をまったく新しいコードに書き直すことも可能です。

```
def greet(target="World"):
    return f"Hello {target}"
```

この例では、固定文字列を返すのではなく引数で切り替えられる設計に変更しています。コンフリクトをきっかけに、より良い実装に改善することは、実際のチーム開発でもよくあることです。

8.5.4. 解決後の操作：add と commit

どのパターンで解決したとしても、最終的な手順は共通です。

```
# 1. 解決したファイルをステージに追加する
git add src/app.py

# 2. マージコミットを作成する
git commit -m "Merge feature/login: resolve greeting conflict"
```

`git add` は、「このファイルのコンフリクトは解決しました」と Git に伝える役割も担っています。コンフリクトが残ったままのファイルを `git add` しようとしても、マーカーが残っている限り問題の根本は解決していませんので、必ずマーカーをすべて削除してから add してください。

複数のファイルでコンフリクトが発生している場合は、すべてのファイルを解決してから一括でコミットします。

```
git add src/app.py src/utils.py src/config.py
git commit -m "Merge feature/login: resolve all conflicts"
```

8.6. VS Code で解決する方法

コンフリクトマーカーを手動で編集するのは確実な方法ですが、Visual Studio Code（VS Code）を使えば、より視覚的にわかりやすく、効率的にコンフリクトを解決できます。VS Code は現在、最も広く使われているコードエディタの一つであり、Git のコンフリクト解決機能が標準で組み込まれています。

8.6.1. コンフリクト箇所の表示

VS Code でコンフリクトが発生しているファイルを開くと、コンフリクトマーカーの部分が色分けして表示されます。通常、HEAD 側（現在のブランチ）の変更は緑色の背景で、マージ元の変更は青色の背景でハイライトされます。

スクリーンショット: VS Code のコンフリクト解決 UI（色分けされたコンフリクト箇所とボタン）

8.6.2. ワンクリック解決ボタン

コンフリクト箇所の上部には、以下の 4 つのボタン（リンク）が表示されます。

Accept Current Change（現在の変更を採用）

HEAD 側、つまりあなたが今いるブランチの変更を採用します。マージ元の変更は破棄されます。「自分の変更が正しい」と確信できる場合に使います。

Accept Incoming Change（入ってくる変更を採用）

マージ元ブランチの変更を採用します。HEAD 側の変更は破棄されます。「相手の変更が正しい」「相手の変更のほうが新しい」という場合に使います。

Accept Both Changes（両方の変更を採用）

両方の変更を上下に並べてそのまま残します。ただし、この操作は機械的に 2 つのコードブロックを連結するだけなので、結果として文法的に正しくないコードになることもあります。必要に応じて、ボタンを押した後に手動で微調整してください。

Compare Changes（変更を比較）

2 つの変更を横に並べて比較表示します。どちらを採用するか判断に迷うとき、差分を見比べるために使います。

8.6.3. VS Code のマージエディタ

VS Code 1.70 以降では、より高度な「3-way マージエディタ」も搭載されています。これは、左側に HEAD 側の変更、右側にマージ元の変更、下部に最終結果を表示する 3 ペイン構成のエディタです。複雑なコンフリクトを解決する際に特に便利です。

コンフリクト解決後は、ターミナルに戻って通常どおり `git add` と `git commit` を実行します。

8.7. コマンドで一括解決する方法

ファイル数が多い場合や、「すべてのコンフリクトで一方の変更を採用する」と明確に決められる場合は、コマンドで一括解決するのが効率的です。Git には `--ours` と `--theirs` というオプションが用意されています。

```
git checkout --ours .
git add .
```

`--ours` は「自分（マージ先、HEAD 側）の変更を採用する」という意味です。ピリオド（`.`）はカレントディレクトリ以下のすべてのファイルを対象にすることを示します。この操作により、コンフリクトが発生しているすべてのファイルで、HEAD 側の内容が採用されます。

```
git checkout --theirs .
git add .
```

`--theirs` は「相手（マージ元ブランチ側）の変更を採用する」という意味です。たとえば、main ブランチの最新状態に feature ブランチを合わせたい場合などに使います。

8.7.3. 注意点

`--ours` と `--theirs` は非常に便利ですが、すべてのコンフリクト箇所で一律に片方を採用してしまうため、意図しない変更の喪失が起こりうる点に注意が必要です。「このコンフリクトは ours、このコンフリクトは theirs」と個別に判断したい場合は、ファイル単位で指定するか、手動で解決してください。

```
# 特定のファイルだけ相手の変更を採用する
git checkout --theirs src/config.py
git add src/config.py
```

また、`--ours` と `--theirs` の意味はリベース中に逆転する点にも注意が必要です。リベースでは、まず HEAD がリベース先のブランチに移動し、その上にあなたのコミットを 1 つずつ適用する仕組みのため、`--ours` がリベース先（相手側）、`--theirs` があなたのコミット（自分側）になります。混乱しやすいポイントなので覚えておきましょう。

8.8. マージやリベースの中止方法

コンフリクトが複雑で手に負えない場合、あるいは「そもそもこのマージをやるべきではなかった」と気づいた場合、操作を中止して元の状態に戻すことができます。中止しても何かが壊れることはありませんので、安心して使ってください。

8.8.1. マージの中止

```
git merge --abort
```

`git merge --abort` は、マージ操作を中止し、マージを開始する前の状態に完全に戻します。コンフリクトマーカーが挿入されたファイルも元に戻ります。「やっぱりやめよう」と思ったら、迷わずこのコマンドを実行してください。

8.8.2. リベースの中止

```
git rebase --abort
```

リベース中にコンフリクトが発生した場合も同様に、`git rebase --abort` で中止できます。リベースでは複数のコミットを順番に適用するため、途中でコンフリクトが何度も発生することがあります。「これは想定以上に複雑だ」と感じたら、一旦中止して戦略を練り直すのも賢い判断です。

8.8.3. cherry-pick の中止

```
git cherry-pick --abort
```

`git cherry-pick` 中にコンフリクトが発生した場合も、`--abort` で中止できます。

8.8.4. 中止した後にやるべきこと

操作を中止した後は、なぜコンフリクトが発生したのかを振り返りましょう。チームメンバーと会話して、作業の分担を見直したり、先に main ブランチの最新を取り込んでからマージし直したりすると、スムーズに解決できることが多いです。

8.9. コンフリクトを予防するプラクティス

コンフリクトは完全に防ぐことはできませんが、発生頻度を大幅に減らし、発生したとしても小規模に抑えるための工夫があります。以下のプラクティスをチームで実践すると、日々の開発がぐっとスムーズになります。

8.9.1. こまめに pull / rebase する

長期間 main ブランチの変更を取り込まずに作業を続けると、main との差分がどんどん大きくなり、いざマージしようとしたときに大量のコンフリクトが発生しがちです。毎日の作業開始時に、main の最新を取り込む習慣をつけましょう。

```
# 毎朝の習慣にしたいコマンド
git switch main
git pull
git switch feature/my-work
git rebase main
```

この 4 行を日課にすることで、main ブランチとの乖離を最小限に抑え、コンフリクトが発生しても小さなうちに対処できます。

8.9.2. 小さなプルリクエストを心がける

変更行数が多ければ多いほど、他の開発者の変更と衝突する確率は高まります。1 つのプルリクエストは可能な限り小さく保ちましょう。目安として、変更が 200〜300 行を超えるようであれば、機能を分割して複数のプルリクエストに分けることを検討してください。小さなプルリクエストはレビューもしやすく、マージまでの時間も短くなるため、コンフリクトの発生リスクが下がります。

8.9.3. 担当ファイルを分ける

チームメンバーが同時に同じファイルを編集する機会が多いほど、コンフリクトは発生しやすくなります。作業開始前にチームで簡単に共有し、「このファイルは今日は〇〇さんが触る」といった暗黙の了解を作ることで、衝突を減らせます。

もちろん、コードの構造自体を工夫して、1 つの巨大なファイルに機能を詰め込むのではなく、適切にファイルを分割しておくことも重要です。

8.9.4. rerere を有効にする

`rerere`（reuse recorded resolution）は、Git の隠れた便利機能の一つです。一度解決したコンフリクトのパターンを Git が記憶し、同じパターンのコンフリクトが再度発生した際に自動的に同じ解決を適用してくれます。

```
# rererereを有効化（一度設定すればOK）
git config --global rerere.enabled true
```

たとえば、リベースを繰り返す運用をしているチームでは、同じコンフリクトが何度も発生することがあります。rerere を有効にしておけば、2 回目以降は自動的に解決されるため、作業の手間が大幅に減ります。

rerere の名前は「re-use recorded resolution」の略で、文字どおり「記録した解決を再利用する」という機能です。地味ですが、長期的に見ると非常に大きな効果があります。

8.9.5. チーム内でコミュニケーションをとる

技術的な対策も重要ですが、最も効果的なのはチーム内のコミュニケーションです。「今からこのファイルを大きく変更する」「このモジュールのリファクタリングを始める」といった情報を事前に共有することで、意図しない衝突を避けることができます。Slack や Teams などのチャットツール、デイリースタンドアップミーティングなどを活用しましょう。

8.10. まとめ

この章では、Git におけるコンフリクト（変更の衝突）について、発生の仕組みから解決方法、予防策までを学びました。

コンフリクトは、同じファイルの同じ箇所を複数の開発者が異なる形で変更した際に発生します。Git はコンフリクトマーカー（`<<<<<<<` 、 `=======` 、 `>>>>>>>` ）を使って衝突箇所を示してくれるので、このマーカーの読み方を覚えておけば、どんなコンフリクトにも対処できます。

解決方法としては、テキストエディタでの手動解決、VS Code の視覚的な GUI、`--ours` / `--theirs` を使ったコマンドでの一括解決の 3 通りがあります。状況に応じて使い分けてください。また、コンフリクトが手に負えない場合は `--abort` で安全に中止できることも覚えておきましょう。

そして何より、こまめな pull、小さなプルリクエスト、担当の分担、rerere の活用、チーム内のコミュニケーションといったプラクティスを日常的に実践することで、コンフリクトの発生頻度と深刻度を大幅に抑えることができます。

コンフリクトは怖いものではなく、チーム開発における自然な現象です。この章で学んだ知識があれば、自信を持って対処できるはずです。

次の章では、Git の履歴操作コマンドを学びます。間違えてしまったコミットを取り消す方法、作業を一時退避する方法、消えてしまったコミットを救出する方法など、「過去を自在にたどる」テクニックを身につけましょう。

履歴の操作 — 過去を自在にたどる

9.1. この章で学ぶこと

Git を使う最大のメリットの一つは、「間違えても大丈夫」という安心感です。ファイルを誤って上書きしてしまった、コミットメッセージを間違えた、そもそもこの変更は不要だった——こうした「やり直したい」場面は、開発作業において日常的に発生します。Git は、こうした場面に対応するための豊富な履歴操作コマンドを備えています。

この章では、以下のコマンドと概念を学びます。

- `git revert` — 過去のコミットを安全に打ち消す
- `git reset` — コミットを巻き戻す（3 つのモードの違い）
- `git restore` — ファイル単位で変更を取り消す
- `git stash` — 作業を一時退避して後で復元する
- `git cherry-pick` — 特定のコミットだけを取り込む
- `git reflog` —「消えた」と思ったコミットを救出する
- `git blame` — 各行の変更者を特定する

これらのコマンドを使いこなせるようになると、Git が単なるバックアップツールではなく、過去を自在にたどり、操作できるタイムマシンのような存在であることを実感するでしょう。

9.2. なぜ履歴操作が必要か

9.2.1. 「間違えても大丈夫」という安心感

プログラミングにおいて、「何かを壊してしまうかもしれない」という恐怖は、大胆なリファクタリングや新しいアプローチの試行を躊躇させます。しかし、Git の履歴操作を知っていれば、「いつでも元に戻せる」という安心感を持って作業に臨めます。

たとえるなら、Git の履歴操作は文書作成ソフトの「元に戻す（Undo）」機能の超強力版です。文書ソフトの Undo は、ソフトを閉じれば履歴は失われますが、Git の履歴はリポジトリが存在する限り永続的に残ります。しかも、単純な「1 つ戻す」だけでなく、「3 つ前のコミットだけ取り消す」「あのブランチのあのコミットだけこちらに取り込む」といった、柔軟な操作が可能です。

9.2.2. 安全な操作と危険な操作

履歴操作のコマンドは、大きく 2 つに分類できます。

安全な操作（履歴を書き換えない）：`git revert` は、過去のコミットを打ち消す新しいコミットを作成するだけで、既存の履歴には一切手を加えません。チームで共有済みの履歴に対しても安心

して使えます。

注意が必要な操作（履歴を書き換える）：`git reset` は、HEAD の位置を過去に戻すことで、コミットをなかったことにします。ローカルでのみ使う分には問題ありませんが、既に push 済みのコミットに対して使うと、チームメンバーの履歴と食い違いが生じるため、原則として避けるべきです。

この「安全かどうか」の判断基準を理解しておくことが、履歴操作を正しく使いこなすための第一歩です。

9.3. git revert — コミットを安全に打ち消す

9.3.1. revert の考え方

`git revert` は、指定したコミットで行われた変更を「逆方向に」適用する新しいコミットを作成するコマンドです。元のコミットは履歴にそのまま残りますが、その変更内容は実質的に取り消されます。

```
Before: A → B → C → D (HEAD)
After:  A → B → C → D → D' (Dの変更を打ち消す新しいコミット)
```

この図を見ると、コミット D は履歴に残ったままで、新たに D’（D を打ち消すコミット）が追加されていることがわかります。履歴を書き換えるのではなく、「上書き」するイメージです。

9.3.2. なぜ revert は push 済みでも安全なのか

`git revert` が安全である理由は、既存の履歴を一切変更しないからです。チームメンバーが既に手元に持っているコミット（A, B, C, D）はそのまま有効であり、新しいコミット D’ が追加されるだけです。チームメンバーが `git pull` すると、D’ が追加されて、結果的に D の変更が取り消された状態になります。履歴の整合性が保たれるため、チーム開発でも安心して使えるのです。

9.3.3. 使い方

```
# 直前のコミットを打ち消す
git revert HEAD

# 特定のコミットを打ち消す（コミットハッシュを指定）
git revert abc1234

# コミットメッセージの編集をスキップ（自動生成メッセージをそのまま
  使う）
git revert --no-edit abc1234
```

```
# マージコミットを打ち消す（-m 1 は最初の親を「正」とする指定）
git revert -m 1 abc1234
```

`git revert HEAD` を実行すると、テキストエディタが開いてコミットメッセージの入力を求められます。デフォルトで「Revert "元のコミットメッセージ"」というメッセージが用意されているので、そのまま保存して閉じれば完了です。`--no-edit` オプションをつけると、エディタを開かずにデフォルトメッセージでコミットが作成されます。

マージコミットの revert は少し特殊で、`-m 1` というオプションが必要です。これは「マージの 2 つの親のうち、1 番目の親（通常は main ブランチ側）をベースとして、もう一方の変更を打ち消す」という意味です。マージコミットには 2 つの親があるため、どちらを基準に打ち消すかを Git に伝える必要があるのです。

9.4. git reset — コミットを巻き戻す

9.4.1. reset の考え方

`git reset` は、HEAD の位置（現在のブランチが指しているコミット）を過去に移動させるコマンドです。revert が「打ち消すコミットを追加する」のに対し、reset は「そもそもコミットしなかったことにする」操作です。

```
Before: A → B → C → D (HEAD)
              ↑
          ここに戻したい

After:  A → B → C (HEAD)
              (D はなかったことに)
```

9.4.2. 3 つのモード — アナロジーで理解する

`git reset` には 3 つのモードがあり、「どこまで巻き戻すか」が異なります。これを理解するために、原稿の執筆作業にたとえてみましょう。

あなたは原稿を書き（ワーキングツリー）、校正済みの原稿を「提出トレイ」に入れ（ステージング）、最終的に印刷所に送った（コミット）とします。ここで「やり直したい」と思ったとき、3 つの選択肢があります。

`--soft`：印刷所への送付だけ取り消す

原稿の内容はそのまま、校正済みトレイにも残っている。ただし「印刷所に送った」という事実だけが取り消される。つまり、コミットだけが取り消され、ステージングもファイルもそのまま残ります。「コミットメッセージを書き直したい」「複数のコミットを 1 つにまとめたい」といった場合

に使います。

```
git reset --soft HEAD~1
```

`--mixed`（デフォルト）：印刷所への送付と、校正済みトレイからの取り出しを行う

原稿の内容はそのまま残っているが、「校正済み」のマークは外れる。つまり、コミットとステージングが取り消され、ファイルの変更内容だけがワーキングツリーに残ります。「add し直すファイルを選び直したい」場合に使います。

```
git reset HEAD~1
# --mixed はデフォルトなので省略可能
```

`--hard`：すべてなかったことにする

原稿の内容自体を破棄し、前の版に完全に差し替える。コミット、ステージング、ファイルの変更内容のすべてが消えます。「この変更は全部不要だった」と判断した場合に使います。

```
git reset --hard HEAD~1
```

9.4.3. 3 つのモードの比較表

モード	コミット	ステージ	ファイル内容	主な用途
`--soft`	取り消す	残る	残る	コミットをやり直したい
`--mixed`	取り消す	取り消す	残る	add からやり直したい
`--hard`	取り消す	取り消す	消える	全て無かったことにしたい

9.4.4. HEAD~1 の意味

`HEAD~1` は「HEAD の 1 つ前のコミット」を意味する表記法です。`~` の後の数字を変えることで、さらに過去のコミットを指定できます。

- `HEAD~1`（または `HEAD~`）：1 つ前のコミット
- `HEAD~2`：2 つ前のコミット
- `HEAD~3`：3 つ前のコミット

コミットハッシュを直接指定することも可能です。

```
# 特定のコミットまで巻き戻す
```

```
git reset --hard abc1234
```

9.4.5. reset の注意点

`git reset`、特に `--hard` モードは、変更内容が完全に失われる可能性があるコマンドです。以下の点を必ず守ってください。

1. **push 済みのコミットには原則使わない**：reset で履歴を書き換えた後に push しようとすると、リモートとの整合性が崩れます。push 済みのコミットを取り消したい場合は `git revert` を使ってください。
2. **`--hard` は慎重に使う**：ワーキングツリーの変更が完全に失われます。ただし、次の節で紹介する `git reflog` を使えば、`--hard` で消したコミットも救出できる場合があります。

9.5. git restore — ファイル単位の変更取り消し

9.5.1. restore の位置づけ

`git restore` は、Git 2.23 で導入された比較的新しいコマンドです。それ以前は `git checkout` がブランチの切り替えとファイルの復元の両方を担っていたため紛らわしかったのですが、`git restore` はファイルの復元に特化しており、意図がより明確になっています。

9.5.2. ワーキングツリーの変更を取り消す

ファイルを編集したが、その変更を破棄して最後のコミット時点の状態に戻したい場合に使います。

```
# 特定のファイルの変更を取り消す
git restore README.md

# 複数ファイルの変更を一度に取り消す
git restore src/app.py src/utils.py

# カレントディレクトリ以下のすべてのファイルの変更を取り消す
git restore .
```

この操作はワーキングツリーの変更を破棄するため、未コミットの変更が失われます。実行前に本当に不要な変更かどうかを確認してください。

9.5.3. ステージングの取り消し

`git add` でステージに上げたが、やはりステージから外したい場合は `--staged` オプションを使

います。

```
# 特定のファイルをステージから外す（ファイルの変更内容は保持される）
git restore --staged README.md

# すべてのファイルをステージから外す
git restore --staged .
```

ここで重要なのは、`--staged` はステージングを解除するだけであり、ファイル自体の変更内容には影響しないという点です。つまり、編集した内容はワーキングツリーにそのまま残ります。

9.5.4. 特定のコミット時点の状態に戻す

```
# 特定のコミット時点のファイル内容を復元
git restore --source=abc1234 README.md
```

`--source` オプションを使うと、任意のコミット時点のファイル内容をワーキングツリーに復元できます。「3 つ前のコミットの時点の config.py を確認したい」といった場面で便利です。

9.6. git stash — 作業の一時退避

9.6.1. 「引き出しにしまう」アナロジー

`git stash` は、現在の作業中の変更を一時的に「引き出し」にしまっておくコマンドです。机の上（ワーキングツリー）に広げていた書類をいったん引き出しにしまい、机をきれいにしてから別の作業を行い、終わったら引き出しから書類を取り出して元の作業に戻る——そんなイメージです。

なぜこの機能が必要なのでしょうか。たとえば、新機能の開発中に緊急のバグ報告が入ったとします。バグ修正のためにブランチを切り替えたいのですが、現在の作業中の変更がコミットするにはまだ中途半端な状態です。かといって、変更を破棄するわけにもいきません。こうした「今の変更を保存しておいて、一旦別のことをしたい」という場面で、`git stash` が威力を発揮します。

9.6.2. 基本的な使い方

```
# 現在の変更（ステージ済み＋未ステージ）を退避する
git stash

# メッセージ付きで退避する（後で何の変更か思い出しやすい）
git stash save "WIP: login feature"
```

```
# 未追跡ファイル（新規作成ファイル）も含めて退避する
git stash -u
```

`git stash` を実行すると、ワーキングツリーとステージングエリアの変更がすべて退避され、最後のコミット時点のきれいな状態に戻ります。退避した内容はスタック（積み重ね）構造で管理されるため、複数回 stash することも可能です。

9.6.3. 退避した内容の確認と復元

```
# 退避リストを表示する
git stash list
# stash@{0}: On feature/login: WIP: login feature
# stash@{1}: On main: temporary change

# 最新の退避を復元し、リストから削除する
git stash pop

# 最新の退避を復元するが、リストには残す
git stash apply

# 特定の退避を指定して復元する
git stash apply stash@{1}

# 退避した内容の差分を確認する
git stash show -p stash@{0}
```

`pop` と `apply` の違いは、退避リストから削除するかどうかです。`pop` は復元と同時にリストから削除しますが、`apply` はリストに残します。「復元に成功したことを確認してから削除したい」場合は、`apply` で復元してから手動で `drop` する方法が安全です。

9.6.4. 退避の削除

```
# 特定の退避を削除する
git stash drop stash@{0}

# すべての退避を削除する
git stash clear
```

9.6.5. 典型的な使い方シナリオ

以下は、stash が活躍する典型的な場面です。

```
# 1. 新機能を開発中...（まだコミットする段階ではない）
git stash save "WIP: ユーザー認証のバリデーション実装中"

# 2. 緊急のバグ修正に対応する
git switch main
git switch -c hotfix/urgent-fix
# ... バグを修正する ...
git commit -am "fix: Critical security bug in auth module"
git push

# 3. 元の作業に戻る
git switch feature/login
git stash pop
# 退避していた変更が復元され、中断前の状態に戻れる
```

このように、stash を使えば作業の流れを中断しても、スムーズに再開できます。

9.7. git cherry-pick — 特定のコミットだけ取り込む

9.7.1. cherry-pick とは

`git cherry-pick`（チェリーピック）は、その名のとおり「さくらんぼを摘む」ように、別のブランチから特定のコミットだけを選んで現在のブランチに取り込むコマンドです。ブランチ全体をマージするのではなく、必要なコミットだけをピンポイントで適用できます。

```
Before:
main:    A → B → C
feature: A → B → D → E → F

After (cherry-pick E):
main:    A → B → C → E'    ← Eの変更内容だけを取り込んだ新しい
   コミット
feature: A → B → D → E → F ← featureブランチはそのまま
```

9.7.2. どんな場面で使うのか

cherry-pick が役立つ典型的な場面をいくつか紹介します。

バグ修正の先行適用：feature ブランチでバグ修正を含む複数のコミットを行ったが、まだ feature ブランチ全体をマージする準備ができていない。しかし、バグ修正だけは先に main に取り込みたい——こうした場面で cherry-pick が活躍します。

誤って別のブランチにコミットした場合の救済：main ブランチにいるつもりで作業していたが、実は feature ブランチにいたことに気づいた場合、正しいブランチに切り替えて cherry-pick で取り込み、元のブランチからは reset で取り消す、という手順で修正できます。

9.7.3. 使い方

```
# 特定のコミットを現在のブランチに取り込む
git cherry-pick abc1234

# 複数のコミットを指定する
git cherry-pick abc1234 def5678

# 範囲指定（abc1234は含まず、def5678まで）
git cherry-pick abc1234..def5678

# コミットせずに変更内容だけをワーキングツリーに適用する
git cherry-pick --no-commit abc1234
```

cherry-pick で取り込まれたコミットには新しいハッシュが付与されます。元のコミットとは内容は同じですが、異なるコミットとして扱われる点に注意してください。

cherry-pick 中にコンフリクトが発生した場合は、第 8 章で学んだ手順でコンフリクトを解決してから `git cherry-pick --continue` で続行するか、`git cherry-pick --abort` で中止できます。

9.8. git reflog —「消えた」コミットを救出する

9.8.1. reflog とは何か

`git reflog`（リフログ）は、HEAD の移動履歴を記録している仕組みです。通常の `git log` はコミットの親子関係をたどって履歴を表示しますが、reflog は HEAD が「どこからどこへ移動したか」を時系列で記録しています。

この違いが重要になるのは、`git reset --hard` で「消してしまった」コミットを救出する場面です。reset で HEAD を過去に巻き戻すと、それより先のコミットは `git log` には表示されなくなります。しかし、reflog には HEAD の移動記録が残っているため、「消えた」コミットのハッシュを見つけ出すことができるのです。

9.8.2. reflog の確認方法

```
git reflog
# abc1234 HEAD@{0}: reset: moving to HEAD~3
# def5678 HEAD@{1}: commit: Add feature X
# ghi9012 HEAD@{2}: commit: Fix bug Y
# jkl3456 HEAD@{3}: checkout: moving from main to feature/login
```

この出力を読み解くと、以下のことがわかります。

- 最新の操作（`HEAD@{0}`）：HEAD~3 に reset した
- その前（`HEAD@{1}`）：「Add feature X」というコミットを行った
- さらにその前（`HEAD@{2}`）：「Fix bug Y」というコミットを行った

9.8.3. reset –hard からの復旧

たとえば、`git reset --hard HEAD~3` を実行した後で「やっぱりあのコミットが必要だった！」と気づいた場合、reflog を使って救出できます。

```
# 1. reflogで消えたコミットのハッシュを確認
git reflog

# 2. 消えたコミットに戻る
git reset --hard def5678
```

あるいは、元のブランチの状態は変えずに、消えたコミットの内容だけを新しいブランチとして復元することもできます。

```
# 消えたコミットから新しいブランチを作成
git branch recover-branch def5678
```

9.8.4. reflog の注意点

reflog について知っておくべき重要な点が 2 つあります。

ローカル専用である：reflog はローカルリポジトリにのみ存在し、リモートリポジトリには共有されません。つまり、`git clone` したばかりのリポジトリには reflog の履歴はありません。

保持期間がある：デフォルトでは、reflog のエントリは 90 日間保持された後に自動的に削除されます。つまり、3 か月以上前の操作は追跡できない可能性があります。

reflog は「最後の砦」として非常に心強い存在です。`git reset --hard` を使う際も、「最悪の場合

はreflogから復旧できる」という安心感を持てます。ただし、reflogに頼りすぎるのではなく、まずは安全な操作（`git revert` など）を優先することを心がけましょう。

9.9. git blame — 各行の変更者を特定する

9.9.1. blameの目的

`git blame` は、ファイルの各行が「誰によって」「いつ」「どのコミットで」最後に変更されたかを表示するコマンドです。「blame」という名前から「犯人捜し」のような印象を受けるかもしれませんが、実際にはコードの経緯を理解するための調査ツールとして使われます。

たとえば、「この設定値はなぜ100になっているのだろう？」「このロジックは誰がいつ追加したのだろう？」といった疑問を解決するために、blameでコードの来歴を調べ、該当するコミットメッセージやプルリクエストを確認する、というのが典型的な使い方です。

```
# ファイルの各行の変更者を表示
git blame README.md

# 出力例:
# abc1234 (Alice 2024-01-10 10:00:00 +0900  1) # My Project
# def5678 (Bob   2024-01-15 14:30:00 +0900  2) Description here
# abc1234 (Alice 2024-01-10 10:00:00 +0900  3)
# ghi9012 (Carol 2024-02-01 09:15:00 +0900  4) ## Installation
```

出力の各列は、左から順に「コミットハッシュ」「変更者名」「変更日時」「行番号」「ファイルの内容」です。

```
# 特定の行範囲だけを表示する（10行目から20行目まで）
git blame -L 10,20 README.md

# 特定のコミット時点での blame を表示する
git blame abc1234 -- README.md
```

`-L` オプションは、大きなファイルで特定の箇所だけを調べたい場合に便利です。ファイル全体のblameは出力が膨大になるため、調べたい行範囲がわかっている場合は積極的に `-L` を使いましょう。

9.9.3. GitHub 上での blame

GitHub の Web インターフェースでも blame 機能が利用できます。ファイルを表示した状態で「Blame」ボタンをクリックすると、各行の変更者とコミット情報が色分けして表示されます。さらに、コミットハッシュをクリックすると、そのコミットの全体像を確認できるため、コードの経緯を追跡するのに非常に便利です。

スクリーンショット: *GitHub の Blame 表示画面*

9.10. まとめ

この章では、Git の履歴を操作するための 7 つのコマンドを学びました。それぞれの用途と特徴を振り返りましょう。

コマンド	用途	安全性
`git revert`	コミットを打ち消す新しいコミットを作成	push 済みでも安全
`git reset`	HEAD を過去に巻き戻す	`--hard` は変更が消える。push 済みには使わない
`git restore`	ファイル単位で変更を取り消す	ワーキングツリーの変更が消える場合あり
`git stash`	作業を一時退避する	安全（退避は明示的に削除しない限り残る）
`git cherry-pick`	特定のコミットだけ取り込む	安全（新しいコミットが作られる）
`git reflog`	HEAD の移動履歴を参照する	参照のみなので安全
`git blame`	各行の最終変更者を表示する	参照のみなので安全

「間違えても大丈夫」——これが Git を使う最大の安心感です。revert で安全に打ち消し、reset で巻き戻し、reflog で消えたコミットを救出する。これらのテクニックを知っていれば、どんな失敗も恐れることなく、大胆にコードを書き進めることができます。

ただし、安全性のレベルはコマンドによって異なります。迷ったときは、まず安全な操作（`git revert`）を選び、必要に応じてより強力な操作（`git reset`）を使うという方針を心がけてください。

次の章では、チーム開発の要であるプルリクエストについて学びます。コードレビューの文化を作り、チーム全体のコード品質を高めるための仕組みと運用方法を理解しましょう。

プルリクエスト — コードレビューの文化を作る

10.1. この章で学ぶこと

プルリクエスト（Pull Request、以下 PR）は、GitHub におけるチーム開発の中心的な仕組みです。PR は単なる「コードを統合するための手順」ではなく、チームメンバーが互いのコードを確認し合い、知識を共有し、コード品質を高めるための「場」を提供します。

オープンソースプロジェクトにおいては、見知らぬ開発者同士が安全にコードを統合するための不可欠な仕組みであり、企業のチーム開発においても、バグの早期発見や設計方針の統一のために広く採用されています。

この章では、以下の内容を学びます。

- プルリクエストとは何か、なぜ必要なのか
- PR の作成方法（Web UI、GitHub CLI、push と同時に）
- PR を構成する各要素の意味と使い方
- 良い PR description の書き方と Issue との連携
- PR のレビュー方法と suggestion 機能
- 3 つのマージ方法の違いと使い分け
- ドラフト PR の活用方法
- PR テンプレートによるチームの運用標準化

10.2. プルリクエストとは何か

10.2.1. 「取り込んでほしい」という提案

プルリクエストを一言で表現するなら、「自分のブランチで行った変更を、main ブランチに取り込んでほしい」という**提案**です。英語の「Pull Request」は「（自分の変更を）引っ張って（pull）取り込んでほしいというお願い（request）」という意味です。

この仕組みがなぜ必要なのでしょうか。仮にチームの全員が main ブランチに直接 push できるとしたら、どうなるでしょう。テストされていないコードが本番環境に入り込む、他のメンバーの変更と意図せず衝突する、いつ・誰が・なぜその変更を行ったのかが追跡しにくくなる——こうした問題が日常的に発生します。

PR を使うことで、以下の流れが実現します。

```
1. ブランチで開発 → 2. PRを作成 → 3. レビュー → 4. 修正 → 5.
   マージ
```

この流れの中で、PR は「変更を議論し、品質を確認し、合意のうえで main に統合する」ための場として機能します。PR のページには変更の差分、コメントのやり取り、CI（自動テスト）の結

果がすべて集約されるため、後から「なぜこの変更が行われたのか」を振り返ることもできます。

10.2.2. なぜ直接 push しないのか

小規模な個人プロジェクトであれば、main ブランチへの直接 push でも問題ないかもしれません。しかし、チーム開発においては、PR を経由する運用が事実上の標準となっています。その理由は以下の通りです。

コードレビューの機会を確保するため：他のメンバーの目を通すことで、バグの早期発見、コードの可読性向上、設計方針の統一が図れます。

変更の意図を記録するため：PR の description に「なぜこの変更を行ったのか」を記述することで、将来のメンテナンス時に経緯を追跡できます。

CI による自動テストのため：PR の作成をトリガーにして自動テストを実行し、テストが通ったことを確認してからマージする運用が可能です。

main ブランチの保護のため：GitHub の「Branch protection rules」を設定すると、PR を経由しないと main に push できないように制限できます。

10.3. PR の作成方法

PR を作成する方法は主に 3 つあります。どの方法でも結果は同じですので、自分の好みやワークフローに合った方法を選んでください。

10.3.1. Web UI から作成する

最も視覚的にわかりやすい方法です。

1. GitHub のリポジトリページで「**Pull requests**」タブをクリック
2. 「**New pull request**」ボタンをクリック
3. **base** に `main`（マージ先）、**compare** に自分の作業ブランチ（マージ元）を選択
4. 差分が表示されるので、意図通りの変更が含まれているか確認
5. 「**Create pull request**」ボタンをクリック
6. タイトルと description（説明）を入力
7. 必要に応じて Reviewers、Labels、Linked Issues などを設定
8. 「**Create pull request**」ボタンをクリックして作成完了

スクリーンショット: GitHub 上の PR 作成画面（base、compare 選択と差分表示）

この手順で重要なのは、ステップ 4 で差分を確認することです。意図しないファイルの変更や、デバッグ用のコードが混入していないかを、この段階でチェックしましょう。

10.3.2. GitHub CLIから作成する

コマンドライン操作が好みの方には、GitHub CLI（`gh` コマンド）を使った方法が便利です。ブラウザを開かずにターミナルからPRを作成できるため、作業の流れを中断せずに済みます。

```
# 対話形式で作成する（タイトル、本文、レビュアーなどを順番に聞かれる）
gh pr create

# ワンライナーで作成する
gh pr create --title "feat: Add dark mode" --body "ダークモードを実装しました"

# ドラフトPRとして作成する
gh pr create --draft --title "WIP: New feature"

# レビュアーを指定して作成する
gh pr create --title "Fix auth bug" --reviewer alice,bob

# ラベルを付けて作成する
gh pr create --title "Fix auth bug" --label "bug,priority:high"
```

`gh pr create` を引数なしで実行すると、対話形式でタイトル、本文、レビュアーなどを入力できます。Git初心者には、まず対話形式で試してみることをお勧めします。

10.3.3. pushと同時にURLを取得する

ブランチをリモートにpushすると、GitHubがPR作成用のURLを表示してくれます。このURLをブラウザで開けば、すぐにPR作成画面にアクセスできます。

```
git push -u origin feature/my-branch
# 出力にPR作成用のURLが表示される:
# remote: Create a pull request for 'feature/my-branch' on GitHub by visiting:
# remote:   https://github.com/username/repo/pull/new/feature/my-branch
```

この方法は「まずpushして、それからPRを作成する」という自然な流れに合っており、多くの開発者が日常的に使っています。

10.4. PRの構成要素

PRにはさまざまな構成要素があり、それぞれがチーム開発を円滑にするための役割を担っています。ここでは、各要素を1つずつ解説します。

10.4.1. Title（タイトル）

PRの概要を一行で表現します。チームメンバーがPR一覧を見たとき、タイトルだけで変更の内容がわかるように、簡潔かつ具体的に書きましょう。慣例として、Conventional Commitsの形式（`feat:`, `fix:`, `docs:` など）を使うチームも多いです。

良い例：`feat: Add dark mode toggle to settings page` 悪い例：`Update files`（何をどう変更したのか不明）

10.4.2. Description（説明）

変更の背景、目的、具体的な変更内容、テスト方法などを記述する欄です。PRの中で最も重要な要素であり、後述の「良いPR descriptionの書き方」で詳しく解説します。

10.4.3. Reviewers（レビュアー）

PRのレビューを依頼するメンバーを指定します。指定されたメンバーには通知が届き、レビュータスクとしてダッシュボードに表示されます。チームの規模や方針によりますが、通常1〜2名のレビュアーを指定します。

10.4.4. Assignees（担当者）

PRの作業担当者を指定します。通常はPR作成者自身を設定しますが、「このPRは○○さんが引き続き対応する」といった場合に別のメンバーを指定することもあります。

10.4.5. Labels（ラベル）

PRを分類するためのタグです。`bug`、`enhancement`、`documentation`、`breaking change` などのラベルを付けることで、PRの種類を一目で識別できます。ラベルはリポジトリのSettingsから自由に作成・カスタマイズできます。

10.4.6. Projects（プロジェクト）

GitHub Projectsのボードにリンクするための設定です。プロジェクト管理をGitHub Projectsで行っている場合、PRを特定のプロジェクトボードに紐づけることで、進捗状況を一元管理できます。

10.4.7. Milestone（マイルストーン）

リリース計画やスプリントに対応するマイルストーンを指定します。「v2.0 リリース」「Sprint 3」などのマイルストーンに PR を紐づけることで、リリースに含まれる変更を把握しやすくなります。

10.4.8. Linked Issues（関連 Issue）

PR がどの Issue（課題）を解決するものなのかを紐づけます。これは非常に重要な機能で、次節で詳しく解説するように、特定のキーワードを使うことで PR のマージ時に Issue を自動的にクローズすることもできます。

10.5. 良い PR description の書き方

PR description は、レビュアーが変更の意図を理解し、効率的にレビューを行うための最も重要な手がかりです。コードの差分だけでは「何が変わったか」はわかっても、「なぜ変えたのか」「どういう判断でこの実装にしたのか」はわかりません。そうした情報を言語化して伝えるのが、description の役割です。

10.5.1. 良い description の構成例

```
## 概要
ユーザー認証機能を実装しました。

## 背景・目的
Issue #42 で報告されたとおり、現在のアプリケーションにはユーザー認
   証の仕組みが
ありません。本PRでは、JWTベースの認証機構を導入し、保護されたエンド
   ポイントへの
不正アクセスを防止します。

## 変更内容
- JWT ベースの認証ミドルウェアを追加（`src/middleware/auth.js`）
- ログイン/ログアウト API エンドポイント（`src/routes/auth.js`）
- パスワードハッシュ化に bcrypt を採用
- 認証関連のユニットテストを追加（カバレッジ 92%）

## テスト方法
1. `npm test` で全テスト通過を確認
2. `POST /api/login` に正しい認証情報を送信し、JWTが返ることを確認
3. 不正なトークンで保護されたエンドポイントにアクセスし、401が返る
   ことを確認

## スクリーンショット
```

```
（UIの変更がある場合に添付）

## 関連Issue
Closes #42
```

10.5.2. Issueとの連携 — 自動クローズ

PR description に特定のキーワードと Issue 番号を記述すると、その PR がマージされたタイミングで対応する Issue が自動的にクローズされます。手動で Issue をクローズする手間が省けるだけでなく、「この PR によってこの Issue が解決された」という関連性が明確に記録されます。

使用できるキーワードは以下の通りです。

```
Closes #42
Fixes #42
Resolves #42
```

いずれも同じ効果ですが、バグ修正の場合は `Fixes`、機能要望の対応は `Closes` や `Resolves` を使うのが慣例です。複数の Issue を一度にクローズすることも可能です。

```
Closes #42, Closes #43
```

description の本文中であればどこに書いても有効ですが、わかりやすさのために「関連 Issue」セクションを設けてまとめて記述するのがお勧めです。

10.6. PRの確認とレビュー

PR が作成されたら、レビュアーは変更内容を確認してフィードバックを行います。GitHub の Web インターフェースには、コードレビューを効率的に行うための機能が豊富に用意されています。

10.6.1. Files changedタブの使い方

PR ページの「**Files changed**」タブをクリックすると、変更されたすべてのファイルの差分が表示されます。追加された行は緑色、削除された行は赤色で色分けされるため、視覚的に変更箇所を把握できます。

スクリーンショット: *Files changed* タブの差分表示画面

差分の表示方法は、「Unified（統合表示）」と「Split（左右分割表示）」の 2 種類から選べます。Unified は変更前と変更後が上下に並んで表示され、Split は画面左に変更前、右に変更後が表示されます。

好みに応じて切り替えてください。

10.6.2. コメントの付け方

特定の行にコメントを付けるには、その行の行番号の左にある「+」ボタンをクリックします。コメント入力欄が表示されるので、質問、提案、指摘などを記述できます。複数行にまたがるコメントを付けたい場合は、行番号をドラッグして範囲を選択してから「+」をクリックします。

コメントには通常の Markdown が使えるので、コードブロックやリスト、リンクなどを自由に使って、わかりやすいフィードバックを心がけましょう。

10.6.3. suggestion 機能 — 修正提案をその場で

GitHub のレビューコメントでは、**suggestion** 機能を使ってコードの修正を直接提案できます。レビュアーが「こう書き換えた方がよい」と思った場合に、言葉で説明するだけでなく、実際のコード修正案を提示できる非常に便利な機能です。

コメント入力欄で以下の形式を使います。

```
```suggestion
const result = items.filter(item => item.isActive);
```
```

PR 作成者は、この提案を「Apply suggestion」ボタンひとつで適用できます。わざわざローカルでファイルを編集してコミット・push する必要がないため、軽微な修正に最適です。複数の suggestion をまとめて「Batch changes」として一度にコミットすることもできます。

10.6.4. レビューの 3 つのステータス

すべてのファイルを確認し終わったら、「**Review changes**」ボタンをクリックして、レビュー全体のステータスを選択します。

Comment（コメント）：特に承認や修正要求をせず、コメントだけを残します。軽い確認や質問のみの場合に使います。

Approve（承認）：変更に問題がないことを確認し、マージを承認します。リポジトリの設定によっては、一定数の Approve が得られないとマージできないように制限を設けることもできます。

Request changes（修正を要求）：変更に問題がある場合に、修正を求めます。修正が完了して再度レビューを受けるまで、マージがブロックされます（設定による）。

スクリーンショット: レビュー送信時の 3 つのステータス選択画面

10.6.5. CLIでのレビュー確認

レビューの確認はコマンドラインからも行えます。

```
# PRの一覧を表示する
gh pr list

# 特定のPRの詳細を表示する
gh pr view 42

# PRの差分を表示する
gh pr diff 42

# ブラウザでPRを開く
gh pr view 42 --web

# PRのブランチをローカルにチェックアウトして動作確認する
gh pr checkout 42
```

特に `gh pr checkout 42` は、レビュー対象のPRをローカル環境で実際に動かして動作確認する際に欠かせないコマンドです。差分を読むだけではわからないUIの挙動やパフォーマンスの問題を発見できます。

10.7. 3つのマージ方法

PRの内容に問題がなく、レビューが承認されたら、いよいよmainブランチにマージします。GitHubでは3つのマージ方法が用意されており、それぞれ異なる履歴構造を生み出します。チームの方針や状況に応じて使い分けましょう。

10.7.1. Create a merge commit（マージコミット）

これはGitの標準的なマージ方法です。featureブランチの全コミットを保持しつつ、マージコミットと呼ばれる特別なコミットを作成してmainブランチに統合します。

```
main:    A → B → C ————————————————→ M（マージコミット）
                 \              /
feature:          D → E → F →
```

特徴：ブランチの分岐・統合の歴史がそのまま残るため、「このfeatureブランチではどんな作業が行われたか」を後から追跡しやすくなります。一方で、featureブランチが多い大規模プロジェクトでは、履歴のグラフが複雑になりがちです。

向いている場面：ブランチの経緯を詳細に残したいプロジェクト、大きなfeatureの統合。

10.7.2. Squash and merge（スカッシュマージ）

feature ブランチ上のすべてのコミットを 1 つのコミットにまとめて（squash）、main ブランチに追加します。

```
main:    A → B → C → DEF (D+E+Fが1つのコミットにまとまる)
```

特徴：feature ブランチ上での細かいコミット（「WIP」「typo 修正」「レビュー対応」など）が main の履歴に表示されないため、main ブランチの履歴が非常にきれいになります。1 つの PR が 1 つのコミットとして記録されるので、`git log` で main の履歴を追いやすくなります。

向いている場面：main ブランチの履歴をきれいに保ちたい場合、feature ブランチに細かい WIP コミットが多い場合。多くのチームでこの方法がデフォルトとして採用されています。

10.7.3. Rebase and merge（リベースマージ）

feature ブランチのコミットをリベースして main ブランチの先端に直線的に追加します。マージコミットは作成されません。

```
main:    A → B → C → D' → E' → F'
```

特徴：feature ブランチの各コミットが個別に main の履歴に残りますが、マージコミットが存在しないため、完全に直線的な履歴になります。各コミットのハッシュは変更されます（`D` が `D'` になる）。

向いている場面：各コミットが意味のある単位で整理されている場合、直線的な履歴を好むプロジェクト。

10.7.4. 3 つの方法の比較

方法	main の履歴	元コミットの保持	マージコミット
Merge commit	分岐・統合が残る	全て残る	あり
Squash and merge	直線的（1PR = 1 コミット）	まとまる	なし
Rebase and merge	直線的（個別コミット）	個別に残る（ハッシュ変更）	なし

10.7.5. CLI でのマージ

```
# マージコミットで統合する
gh pr merge 42 --merge

# スカッシュマージで統合する
gh pr merge 42 --squash
```

```
# リベースマージで統合する
gh pr merge 42 --rebase

# マージ後にリモートのfeatureブランチを削除する
gh pr merge 42 --squash --delete-branch
```

`--delete-branch` オプションは、マージ後に不要になった feature ブランチをリモートから自動削除してくれます。ブランチが溜まって管理が煩雑になるのを防ぐため、積極的に使うことをお勧めします。

10.8. ドラフト PR

10.8.1. ドラフト PR とは

ドラフト PR（Draft Pull Request）は、「まだレビュー準備ができていないが、作業の進捗をチームに共有したい」場合に使う PR の形態です。通常の PR とは異なり、ドラフト PR は以下の特徴を持ちます。

- PR ページに「Draft」というラベルが表示される
- マージボタンが無効化され、誤ってマージされることを防ぐ
- レビュアーへの通知は送られるが、正式なレビュー依頼とは区別される

10.8.2. どんな場面で使うのか

早期フィードバックの取得：実装の方向性が正しいかどうかを、完成前の段階でチームメンバーに確認してもらいたい場合に使います。「この設計方針で進めてよいか」というレベルの相談ができます。

進捗の可視化：長期間にわたる機能開発の場合、途中経過をドラフト PR として公開することで、チームが進捗を把握できます。

CI/CD の事前確認：ドラフト PR でも CI は実行されるため、テストが通るかどうかを早い段階で確認できます。

10.8.3. 作成と解除

```
# ドラフトPRを作成する
gh pr create --draft --title "WIP: New authentication system"

# ドラフトを解除してレビュー準備完了とする
gh pr ready 42
```

スクリーンショット: ドラフト *PR* の表示（*Draft* ラベルとマージ不可の状態）

Web UI からは、PR 作成時に「Create pull request」ボタンのプルダウンメニューから「Create draft pull request」を選択します。ドラフト PR を正式な PR に変更するには、PR ページ下部の「Ready for review」ボタンをクリックします。

10.9. PR テンプレート

10.9.1. テンプレートの目的

チームで多くの PR が作成されると、description の品質にばらつきが出ることがあります。「概要だけで詳細がない」「テスト方法が書かれていない」「関連 Issue が紐づけられていない」といった問題が頻発すると、レビューの効率が低下します。

PR テンプレートは、PR 作成時に description 欄に自動的に挿入されるテンプレートです。チームで統一したフォーマットを用意しておくことで、全員が必要な情報を漏れなく記述できるようになります。

10.9.2. テンプレートの作成方法

リポジトリのルートまたは `.github` ディレクトリに `pull_request_template.md` というファイルを作成します。

```
# .github ディレクトリを作成（なければ）
mkdir -p .github

# テンプレートファイルを作成
touch .github/pull_request_template.md
```

以下は、実用的なテンプレートの例です。

```
<!-- .github/pull_request_template.md -->

## 概要
<!-- 変更の概要を簡潔に書いてください -->

## 背景・目的
<!-- なぜこの変更が必要なのかを説明してください -->

## 変更の種類
- [ ] バグ修正
- [ ] 新機能
```

```
- [ ] 破壊的変更（既存の機能に影響がある変更）
- [ ] ドキュメント更新
- [ ] リファクタリング（機能に影響のないコード改善）

## 変更内容の詳細
<!-- 具体的に何をどう変更したかを記述してください -->

## テスト方法
<!-- レビュアーがこの変更を検証するための手順を書いてください -->
1.
2.
3.

## スクリーンショット
<!-- UIの変更がある場合は、変更前と変更後のスクリーンショットを添付
   してください -->

## チェックリスト
- [ ] テストを追加/更新した
- [ ] ドキュメントを更新した（必要な場合）
- [ ] セルフレビューを行った
- [ ] 動作確認を行った

## 関連Issue
<!-- Closes #XX のように記述してください -->
```

このテンプレートファイルをリポジトリにコミットしておくと、以降の PR 作成時に description 欄にテンプレートの内容が自動的に挿入されます。PR 作成者は、各セクションの指示に従って内容を埋めていくだけで、必要な情報が網羅された description が完成します。

10.9.3. テンプレートの運用 Tips

テンプレートはチームの成長に合わせて改善していくものです。最初から完璧なテンプレートを作ろうとするよりも、実際に運用しながら「この情報が足りない」「このセクションは不要」といったフィードバックを反映して、徐々にブラッシュアップしていくのが現実的です。

また、テンプレートが長すぎると記入が面倒になり、形骸化してしまうリスクがあります。チームにとって本当に必要な情報に絞り、適度な長さに保つことを心がけましょう。

10.10. まとめ

この章では、GitHub におけるチーム開発の中心的な仕組みであるプルリクエストについて、作成から運用までを包括的に学びました。

PR は「自分の変更を main に取り込んでほしい」という提案であり、コードレビュー、議論、CI テストの場として機能します。Web UI と GitHub CLI の両方から作成でき、Title、Description、Reviewers、Labels、Linked Issues などの構成要素を適切に設定することで、レビュアーに必要な情報を過不足なく伝えられます。

description では「Closes #42」のようなキーワードを使って Issue と連携させ、マージ時に自動クローズする仕組みを活用しましょう。レビューでは、Files changed タブでの差分確認、行単位のコメント、suggestion 機能を使った具体的なコード修正の提案が可能です。

マージ方法は 3 種類あり、Merge commit（分岐の履歴を残す）、Squash and merge（1 つのコミットにまとめる）、Rebase and merge（直線的に並べる）から、チームの方針に合ったものを選択してください。多くのチームでは Squash and merge が好まれています。

さらに、ドラフト PR を活用して早期フィードバックを得る方法、PR テンプレートを使ってチーム内の description 品質を標準化する方法も紹介しました。

PR は単なる技術的な仕組みではなく、チームのコミュニケーションツールでもあります。良い PR を書き、丁寧にレビューし合う文化を築くことが、プロジェクト全体の品質とチームの成長につながります。

Issue — タスクとバグを管理する

11.1. この章で学ぶこと

この章では、GitHub の **Issue（イシュー）**機能について、その基本的な考え方から実践的な活用方法までを体系的に学びます。具体的には、以下の内容を扱います。

- Issue とは何か——プロジェクト管理における「チケット」としての役割
- Issue 番号の仕組みと、それがリポジトリ全体でどのように管理されるか
- Web UI と CLI（GitHub CLI）の両方を使った Issue の作成方法
- バグ報告や機能要望を的確に伝えるための「良い Issue の書き方」
- ラベルやマイルストーンを活用した整理・分類の手法
- Issue と Pull Request を連携させて、作業の流れを自動化する方法
- テンプレートを用いて、チーム全体の Issue 品質を底上げする方法

ソフトウェア開発では、コードを書くことと同じくらい「何をすべきか」「何が問題か」を正確に記録・共有することが重要です。Issue はまさにその「記録と共有」を担う中核的な機能です。

11.2. Issue とは何か

11.2.1. チケットシステムとしての Issue

ソフトウェア開発の現場では、日々さまざまな「やるべきこと」が発生します。ユーザーから「この画面が正しく表示されない」というバグ報告が届くこともあれば、チームメンバーから「ダークモードに対応したい」という機能提案が出ることもあります。こうした情報をメールやチャットだけでやり取りしていると、どの問題が未解決で、誰が担当していて、いつまでに対応すべきなのかが分からなくなってしまいます。

この課題を解決するのが**チケットシステム**（Issue Tracking System）です。Jira、Redmine、Backlog といったツールを聞いたことがある方もいるでしょう。GitHub の Issue は、こうしたチケットシステムの機能を GitHub の中に統合したものです。コードリポジトリと同じ場所でタスクやバグを管理できるため、「コードの変更」と「その変更が何のために行われたか」を自然に紐づけることができます。

Issue は大きく分けて以下の用途で使われます。

- **バグ報告** —「この機能が正しく動かない」という問題の記録
- **機能要望** —「こんな機能がほしい」という提案
- **タスク管理** —「次のリリースまでにやること」のリスト化
- **議論・相談** —「この設計方針でよいか」というチーム内の検討

11.2.2. Issue番号の仕組み

GitHubでは、IssueとPull Requestが**同じ番号空間**を共有しています。つまり、最初に作成したIssueが `#1`、次に作成したPull Requestが `#2`、その次のIssueが `#3` というように、リポジトリ内で通し番号が振られます。

この番号はリポジトリごとに独立しており、一度割り当てられた番号は削除しても再利用されません。これは重要な設計上の判断です。なぜなら、コミットメッセージやドキュメントの中で `#42` と書けば、それが常に同じIssue（またはPR）を指すことが保証されるからです。もし番号が再利用されてしまうと、過去の参照が別のIssueを指してしまい、混乱の原因になります。

たとえば、コミットメッセージに `Fix login bug (#42)` と書いておけば、GitHub上でこの `#42` は自動的にリンクになり、クリックすると該当のIssueに飛ぶことができます。この仕組みのおかげで、コードの変更履歴とIssueが自然につながるのです。

11.3. Issueの作成

11.3.1. Web UIでの作成

最も直感的なのは、GitHubのWebインターフェースからIssueを作成する方法です。手順は以下のとおりです。

1. リポジトリのページを開き、上部にある「**Issues**」タブをクリックします
2. 右上の「**New issue**」ボタンをクリックします
3. **タイトル**を入力します。タイトルは問題の要約を簡潔に書きます（例：「ログインページで404エラーが発生する」）
4. **本文**（Description）にMarkdown形式で詳細を記述します
5. 右側のサイドバーから、必要に応じて**ラベル**、**担当者（Assignees）**、**マイルストーン**を設定します
6. 「**Submit new issue**」をクリックして作成完了です

> スクリーンショット: *Issue*作成画面。タイトル入力欄、本文の*Markdown*エディタ、右サイドバーのラベル・担当者設定が見える状態

本文ではMarkdownが使えるため、見出し、箇条書き、コードブロック、画像の貼り付けなど、リッチな記述が可能です。また、画像はドラッグ&ドロップで簡単に添付できます。

11.3.2. GitHub CLI（gh）での作成

ターミナルで作業している場合、ブラウザに切り替えることなくIssueを作成できるのがGitHub CLIの利点です。

```
# 対話形式で作成（タイトルや本文を順に聞かれる）
gh issue create

# タイトルと本文を直接指定
gh issue create --title "ログイン画面が表示されない" --body "トップ
   ページからログインボタンを押すと画面が白くなる"

# ラベルと担当者も同時に指定
gh issue create --title "ダークモード対応" --label "enhancement" --
   assignee "@me"

# マイルストーンも指定する場合
gh issue create --title "パフォーマンス改善" --label "enhancement"
   --milestone "v2.0"
```

対話形式（`gh issue create` を引数なしで実行）では、エディタが開いて本文を編集できるため、長い説明を書く場合にも便利です。コマンドライン派の開発者にとっては、コードを書いている最中に思いついたバグや改善点をすぐに Issue 化できるため、作業の流れを中断せずに済みます。

11.4. 良いIssueの書き方

Issue は「書けば終わり」ではなく、他の人（あるいは未来の自分）が読んで理解できることが大切です。曖昧な Issue は対応の遅れや誤解の原因になります。ここでは、バグ報告と機能要望のそれぞれについて、効果的な書き方を解説します。

11.4.1. バグ報告のテンプレート

バグ報告で最も重要なのは**再現性**です。開発者がそのバグを自分の環境で再現できなければ、修正は困難です。そのため、以下の 4 つの要素を必ず含めるようにしましょう。

1. **再現手順（Steps to Reproduce）**— バグに至るまでの操作を、誰でもたどれるように順を追って書く
2. **期待する動作（Expected Behavior）**— 本来はどうなるべきだったか
3. **実際の動作（Actual Behavior）**— 実際に何が起きたか
4. **環境（Environment）**— OS、ブラウザ、アプリのバージョンなど

具体的な例を見てみましょう。

```
## バグの概要
ログインページで「パスワードを忘れた」リンクが機能しない
```

```
## 再現手順
1. https://example.com/login にアクセスする
2. 「パスワードを忘れた方はこちら」リンクをクリックする
3. 404エラーが表示される

## 期待する動作
パスワードリセットページが表示され、メールアドレスの入力フォームが
   現れる

## 実際の動作
404 Not Found エラーページが表示される

## 環境
- OS: macOS 14.2
- ブラウザ: Chrome 120.0.6099.109
- アプリバージョン: v2.3.1

## スクリーンショット
（ここにエラー画面のスクリーンショットを貼付）
```

ポイントは、「パスワードリセットが壊れている」のような曖昧な報告ではなく、具体的な操作手順と、期待と実際の差分を明示することです。スクリーンショットやエラーログがあれば、さらに診断が容易になります。

11.4.2. 機能要望の書き方

機能要望では、「何がほしいか」だけでなく「なぜほしいか」を伝えることが重要です。開発チームは限られたリソースの中で優先順位を判断する必要があるため、背景や理由が明確であるほど、提案が採用されやすくなります。

```
## 概要
ダークモードに対応してほしい

## 背景・理由
- 夜間に長時間作業するユーザーが多く、目の疲れが報告されている
- 競合製品（ProductX, ProductY）はすでに対応済み
- アクセシビリティの観点からも、表示モードの選択肢があることが望ま
   しい

## 提案する実装
- システムのダーク/ライト設定に連動する自動切り替え
- ユーザー設定画面での手動切り替えオプション
- 各UIコンポーネントでのカラーテーマ対応
```

```
## 参考情報
- デザインガイドライン案: https://example.com/dark-mode-guide
- 関連Issue: #28（カラーテーマの要望）
```

11.5. ラベル（Labels）

11.5.1. ラベルの役割

ラベルは、Issue に付ける**色付きのタグ**です。ラベルを付けることで、大量の Issue の中から特定の種類のものをすばやく絞り込むことができます。たとえば、「バグだけを見たい」「フロントエンド関連の未対応タスクだけを一覧したい」といった場面で威力を発揮します。

本のしおりやファイルのインデックスタブを想像してください。ラベルはまさにそのデジタル版であり、Issue を多角的に分類するための仕組みです。1 つの Issue に複数のラベルを付けることもできるため、「バグ」かつ「フロントエンド」かつ「高優先度」のように、複数の軸で分類が可能です。

11.5.2. デフォルトラベル一覧

GitHub でリポジトリを新規作成すると、以下のラベルが自動的に用意されます。

ラベル名	色	意味・用途
`bug`	赤	ソフトウェアのバグ報告
`enhancement`	水色	既存機能の改善や新機能の追加提案
`documentation`	紫	ドキュメントの追加・修正
`good first issue`	緑	初心者が取り組みやすい課題
`help wanted`	黄	外部からの助けを求めている課題
`question`	ピンク	質問や不明点の相談
`duplicate`	グレー	既存の Issue と重複している
`invalid`	グレー	有効でない（再現できない、仕様どおりなど）
`wontfix`	白	対応しないと判断された課題

11.5.3. カスタムラベルの作成

デフォルトラベルだけでは足りない場合、プロジェクトの特性に合わせたカスタムラベルを作成できます。Web UI では Issues タブ → Labels →「New label」から作成できます。CLI では以下のように操作します。

```
# 優先度ラベル
gh label create "priority:high" --color "FF0000" --description "高
```

```
    優先度 — 即座に対応が必要"
gh label create "priority:medium" --color "FFA500" --description "
    中優先度"
gh label create "priority:low" --color "0E8A16" --description "低優
    先度"

# 領域ラベル
gh label create "frontend" --color "1D76DB" --description "フロント
    エンド関連"
gh label create "backend" --color "E4E669" --description "バックエ
    ンド関連"
gh label create "infrastructure" --color "D4C5F9" --description "イ
    ンフラ・DevOps関連"
```

スクリーンショット: ラベル管理画面。色付きのラベルが一覧表示されている状態

ラベルの命名規則として、`priority:high` や `area:frontend` のように**カテゴリ:値**の形式を使うと、多数のラベルがあっても整理しやすくなります。色も、同じカテゴリのラベルには同系色を使うと視覚的にグルーピングできて便利です。

11.6. マイルストーン（Milestones）

11.6.1. マイルストーンとは

マイルストーンは、複数の Issue を**リリースや期限といった大きなゴールでグループ化**する機能です。たとえば「v2.0 リリース」というマイルストーンを作り、そのリリースに必要な Issue を紐づけておくと、進捗がパーセンテージで表示され、リリースに向けてあとどれくらいの作業が残っているかが一目で分かります。

登山の「マイルストーン（里程標）」と同じ発想です。頂上（リリース）に至るまでの中間地点を設定しておくことで、今どこまで進んでいるのかを把握できるようになります。

11.6.2. マイルストーンの作成と管理

Web UI では、Issues タブ → Milestones →「New milestone」から作成します。CLI からは GitHub API を直接呼び出します。

```
# マイルストーンの作成
gh api repos/{owner}/{repo}/milestones \
  -f title="v2.0" \
  -f description="バージョン2.0リリース — ダークモードと多言語対応
    を含む" \
```

```
  -f due_on="2026-06-30T00:00:00Z"
```

マイルストーンには以下の情報を設定できます。

- **タイトル** — `v2.0`、`Sprint 3`、`2026年Q2` など
- **期限（Due date）** — オプションですが、設定すると期限超過の際に警告が表示されます
- **説明** — このマイルストーンのゴールや含まれる主要な変更を記述します

Issueをマイルストーンに紐づけるには、Issueの右サイドバーから設定するか、CLIでIssue作成時に `--milestone "v2.0"` を指定します。マイルストーン内のIssueがすべてクローズされると、進捗バーが100%になり、マイルストーン自体をクローズできます。

> スクリーンショット: マイルストーンの進捗画面。進捗バーと *Open/Closed* の件数が表示されている状態

11.7. IssueとPull Requestの連携

11.7.1. 自動クローズキーワード

GitHubの非常に便利な機能の1つが、**Pull Requestのマージによる Issueの自動クローズ**です。PRの本文（Description）やコミットメッセージに特定のキーワードとIssue番号を書いておくと、そのPRがデフォルトブランチにマージされた時点で、該当のIssueが自動的にクローズされます。

使えるキーワードは以下のとおりです（大文字・小文字は区別されません）。

キーワード	例
`Closes`	`Closes #42`
`Fixes`	`Fixes #42`
`Resolves`	`Resolves #42`

これらはすべて同じ動作をしますが、ニュアンスの違いで使い分けると読みやすくなります。バグ修正なら `Fixes #42`、機能実装なら `Closes #42`、議論の解決なら `Resolves #42` といった具合です。

複数のIssueを同時にクローズすることもできます。

```
## このPRで対応した内容

- ログイン画面の404エラーを修正（Fixes #42）
- パスワードリセット機能のUIを改善（Closes #43）
- 関連するテストを追加（Closes #44）
```

この連携の重要なポイントは、「なぜこのコード変更が行われたか」が自動的に追跡可能になることです。Issue `#42` を見れば修正 PR へのリンクがあり、PR を見ればどの Issue に対応したかが分かる。この双方向のトレーサビリティが、プロジェクト管理の質を大きく向上させます。

11.7.2. コミットメッセージでの参照

PR 本文だけでなく、コミットメッセージにも Issue 番号を含めることが推奨されます。

```
git commit -m "Fix password reset link returning 404 (#42)"
```

`#42` と書くだけで（キーワードなしでも）GitHub 上で自動的にリンクが生成され、該当 Issue のタイムラインにも参照が表示されます。ただし、自動クローズされるのはキーワード付きの場合のみです。

11.8. Issue テンプレート

11.8.1. テンプレートの必要性

チームでの開発では、Issue の品質にばらつきが出がちです。ある人は詳細なバグ報告を書いてくれますが、別の人は「動かない」としか書いてくれないかもしれません。Issue テンプレートを用意しておくと、「New issue」をクリックした時点であらかじめ必要な項目が表示されるため、記入漏れを防ぐことができます。

テンプレートは `.github/ISSUE_TEMPLATE/` ディレクトリに YAML ファイルとして配置します。複数のテンプレートを用意すると、Issue 作成時に「バグ報告」「機能要望」などから選択する画面が表示されます。

スクリーンショット: *Issue* テンプレート選択画面。「バグ報告」「機能要望」の選択肢が表示されている状態

11.8.2. バグ報告テンプレートの作成

以下は、YAML 形式のバグ報告テンプレートの例です。GitHub のフォーム機能を使うことで、自由記述だけでなくドロップダウンやチェックボックスも利用できます。

```
# .github/ISSUE_TEMPLATE/bug_report.yml
name: バグ報告
description: バグを報告する
title: "[Bug]: "
labels: ["bug"]
body:
```

```
  - type: textarea
    id: description
    attributes:
      label: バグの概要
      description: 何が起きていますか？ 簡潔に説明してください。
    validations:
      required: true
  - type: textarea
    id: steps
    attributes:
      label: 再現手順
      description: バグを再現するための手順を記述してください。
      value: |
        1. '...' に移動する
        2. '...' をクリックする
        3. エラーが表示される
    validations:
      required: true
  - type: textarea
    id: expected
    attributes:
      label: 期待する動作
      description: 本来はどのように動作すべきですか？
    validations:
      required: true
  - type: textarea
    id: actual
    attributes:
      label: 実際の動作
      description: 実際にはどのように動作しましたか？
    validations:
      required: true
  - type: dropdown
    id: os
    attributes:
      label: OS
      options:
        - macOS
        - Windows
        - Linux
        - iOS
        - Android
    validations:
      required: true
```

```
  - type: input
    id: version
    attributes:
      label: アプリバージョン
      placeholder: "例: v2.3.1"
    validations:
      required: false
```

11.8.3. 機能要望テンプレートの作成

```
# .github/ISSUE_TEMPLATE/feature_request.yml
name: 機能要望
description: 新機能を提案する
title: "[Feature]: "
labels: ["enhancement"]
body:
  - type: textarea
    id: problem
    attributes:
      label: 解決したい課題
      description: この機能がないことで、どのような問題がありますか?
    validations:
      required: true
  - type: textarea
    id: solution
    attributes:
      label: 提案する解決策
      description: どのような機能を実装すれば解決しますか?
    validations:
      required: true
  - type: textarea
    id: alternatives
    attributes:
      label: 検討した代替案
      description: 他に検討した方法があれば記述してください。
    validations:
      required: false
```

`validations: required: true` を設定すると、その項目が未記入のままではIssueを送信できなくなります。これにより、必要最低限の情報が常に揃ったIssueが作成されるようになります。

11.9. Issueの管理

11.9.1. CLIでの操作

日常的なIssue管理は、GitHub CLIを使うとブラウザを開かずに素早く行えます。

```
# Issue一覧を表示（デフォルトではOpen状態のIssueが表示される）
gh issue list

# 特定のラベルが付いたIssueだけを表示
gh issue list --label "bug"

# 自分に割り当てられたIssueを表示
gh issue list --assignee "@me"

# クローズ済みのIssueも含めて表示
gh issue list --state all

# 特定のIssueの詳細を表示
gh issue view 42

# ブラウザで開く（詳細な確認やコメントの画像添付が必要な場合に便
  利）
gh issue view 42 --web

# Issueをクローズする
gh issue close 42

# Issueを再オープンする
gh issue reopen 42

# Issueにコメントを追加する
gh issue comment 42 --body "修正PRを作成しました: #45 を確認してく
  ださい"

# Issueにラベルを追加する
gh issue edit 42 --add-label "priority:high"

# Issueの担当者を変更する
gh issue edit 42 --add-assignee "alice"
```

11.9.2. ピン留め（Pinned Issues）

特に重要な Issue——たとえばプロジェクトの行動規範、コントリビューションガイドライン、既知の重大バグなど——は、Issues タブの Top に固定表示（ピン留め）することができます。最大 3 つまでピン留めが可能です。

ピン留めの方法は、該当 Issue を開き、右サイドバーの「**Pin issue**」をクリックするだけです。ピン留めされた Issue は Issues 一覧の最上部に表示されるため、リポジトリを訪れた人が最初に目にすることになります。

> スクリーンショット: ピン留めされた *Issue* が *Issues* 一覧の上部に表示されている状態

11.9.3. Issue の検索とフィルタリング

Issue が増えてくると、効率的な検索が不可欠です。GitHub Web の検索バーでは、以下のようなフィルタが使えます。

フィルタ	例	意味
`is:open`	`is:open label:bug`	Open 状態のバグ
`is:closed`	`is:closed milestone:v2.0`	v2.0 マイルストーンのクローズ済み Issue
`assignee:`	`assignee:alice`	alice に割り当てられた Issue
`author:`	`author:bob`	bob が作成した Issue
`no:label`	`no:label is:open`	ラベルが付いていない未対応 Issue

これらのフィルタを組み合わせることで、必要な Issue をすばやく見つけることができます。

11.10. まとめ

この章では、GitHub の Issue 機能を包括的に学びました。重要なポイントを振り返ります。

- **Issue はチケットシステム**であり、バグ報告・機能要望・タスク管理・議論のすべてを 1 箇所で管理できます
- **Issue 番号は PR と共通の通し番号**であり、コミットメッセージや PR から `#番号` で参照できます
- **良い Issue** は、再現手順・期待動作・実際の動作・環境情報を含む具体的なものです
- **ラベル**で Issue を分類し、**マイルストーン**でリリースや期限ごとにグループ化することで、プロジェクトの全体像を把握しやすくなります
- `Closes #42` などのキーワードにより、**PR のマージで Issue を自動クローズ**できます
- **テンプレート**を用意することで、チーム全体の Issue 品質を一定に保つことができます

Issueを適切に活用することは、単にタスクを管理するだけでなく、プロジェクトの意思決定の記録を残すことでもあります。「なぜこの機能をこのように実装したのか」という判断の背景が、Issueの議論の中に残っていれば、後からプロジェクトに参加したメンバーも文脈を理解できます。

GitHub Actions — 自動化で開発を加速する

12.1. この章で学ぶこと

この章では、GitHub に組み込まれた自動化プラットフォームである **GitHub Actions** について学びます。具体的には、以下の内容を扱います。

- CI/CD（継続的インテグレーション/継続的デリバリー）とは何か、なぜ必要なのか
- GitHub Actions を構成する基本用語（Workflow、Event、Job、Step、Action、Runner）
- YAML ファイルによるワークフローの定義方法を 1 行ずつ理解する
- トリガー（Event）の種類と使い分け
- Python、Node.js、自動リリースなど実用的なワークフロー例
- API キーなどの秘密情報をワークフローで安全に扱う方法
- 料金体系と無料枠の制限
- ワークフローの実行結果の確認とデバッグ方法

「コードを書いたら、あとは自動でテスト・ビルド・デプロイしてほしい」——この願いを実現するのが GitHub Actions です。

12.2. CI/CD とは何か

12.2.1. 手動テストの限界

ソフトウェア開発の初期段階では、コードを変更するたびに手動でテストを実行し、問題がなければ手動でサーバーにデプロイする、というワークフローでも何とかなります。しかし、プロジェクトが成長し、チームメンバーが増え、1 日に何度もコードが変更されるようになると、手動での確認は現実的ではなくなります。

たとえば、ある開発者がログイン機能を修正したとします。手動テストでは、ログイン機能自体が正しく動くかは確認できても、その変更が商品一覧ページや決済機能に影響を与えていないかまでは、なかなか確認しきれません。こうした「意図せず他の機能を壊してしまう」問題を**リグレッション（退行）**と呼びます。

12.2.2. Continuous Integration（継続的インテグレーション）

CI（Continuous Integration）とは、開発者がコードを共有リポジトリに統合（push/merge）するたびに、自動でビルドとテストを実行する手法です。「継続的」という言葉がついているのは、これを一度だけでなく、コードが変更されるたびに繰り返し行うためです。

CI を導入すると、以下のようなサイクルが自動的に回ります。

1. 開発者がコードを変更してプッシュする
2. CI システムが自動的にテストを実行する

3. テスト結果が Pull Request に表示される

4. テストが通れば安心してマージできる。失敗すれば修正してから再挑戦する

この仕組みがあることで、リグレッションを早期に発見でき、常にメインブランチが「動く状態」に保たれます。

12.2.3. Continuous Delivery / Deployment（継続的デリバリー/デプロイ）

CD は CI の延長にある概念で、テストに通ったコードを自動で本番環境にデプロイ（公開）する仕組みです。Continuous Delivery は「いつでもデプロイ可能な状態を保つ」こと、Continuous Deployment は「テストが通ったら自動的にデプロイまで行う」ことを指します。

GitHub Actions は、この CI/CD の両方を GitHub の中で実現できるツールです。外部の CI/CD サービス（Jenkins、CircleCI、Travis CI など）をわざわざ契約・設定する必要がなく、リポジトリの中に YAML ファイルを 1 つ置くだけで始められます。

12.3. GitHub Actions の基本用語

GitHub Actions を理解するには、6 つの基本用語を押さえる必要があります。これらは相互に関係しているので、全体像を把握しながら 1 つずつ見ていきましょう。

12.3.1. Workflow（ワークフロー）

ワークフローは、自動化の全体を定義する**設計図**です。`.github/workflows/` ディレクトリに配置する YAML ファイル 1 つが、1 つのワークフローに対応します。たとえば、テスト用のワークフロー（`ci.yml`）とデプロイ用のワークフロー（`deploy.yml`）を別々に作ることができます。

12.3.2. Event（イベント）

ワークフローを起動する**トリガー**です。「コードがプッシュされたら」「Pull Request が作成されたら」「毎日午前 0 時になったら」など、さまざまな条件を指定できます。1 つのワークフローに複数のイベントを設定することも可能です。

12.3.3. Job（ジョブ）

ワークフロー内で実行される**タスクの塊**です。1 つのワークフローに複数のジョブを含めることができ、デフォルトでは各ジョブは**並列に**実行されます。ジョブ間に依存関係を設定すれば、「テストジョブが成功したらデプロイジョブを実行する」のように順序を制御できます。

12.3.4. Step（ステップ）

ジョブ内の**個々の処理**です。ステップは上から順に逐次実行されます。各ステップでは、シェルコマンドを実行するか、既存の Action を呼び出すかのいずれかを行います。

12.3.5. Action（アクション）

再利用可能な**処理の部品**です。GitHub Marketplace には、コードのチェックアウト、言語のセットアップ、キャッシュの管理など、よく使う処理が Action として公開されています。自分でゼロからスクリプトを書かなくても、`uses: actions/checkout@v4` のように参照するだけで利用できます。

12.3.6. Runner（ランナー）

ワークフローを実際に実行する**サーバー（仮想マシン）**です。GitHub が提供する Runner（GitHub-hosted runner）には、Ubuntu、macOS、Windows の 3 種類があります。`runs-on: ubuntu-latest` のように指定します。自分のサーバーを Runner として使う「self-hosted runner」を設定することも可能です。

これらの関係を階層構造で整理すると、以下のようになります。

```
Workflow（YAMLファイル全体）
 ├── Event（トリガー: push, pull_request, schedule...）
 ├── Job 1（例: テスト）
 │    ├── Step 1: actions/checkout を使ってコードを取得
 │    ├── Step 2: actions/setup-python を使ってPythonをインストール
 │    ├── Step 3: pip install で依存関係をインストール（run コマンド）
 │    └── Step 4: pytest でテストを実行（run コマンド）
 └── Job 2（例: デプロイ ← Job 1に依存）
      ├── Step 1: ビルド
      └── Step 2: デプロイ
```

12.4. 最初のワークフローを作ってみる

ここでは、Python プロジェクトの簡単な CI ワークフローを例に、YAML ファイルの各行が何を意味しているかを 1 つずつ解説します。

まず、リポジトリのルートに `.github/workflows/` ディレクトリを作成し、その中に `ci.yml` というファイルを作成します。

```
name: CI                          # (1) ワークフローの名前。Actions
```

```
    タブに表示される

on:                                 # (2) トリガーの定義
  push:                             #     コードがプッシュされた時
    branches: [main]                #     main ブランチへのプッシュ
       のみ対象
  pull_request:                     #     PRが作成・更新された時
    branches: [main]                #     main ブランチへのPRのみ対
       象

jobs:                               # (3) ジョブの定義
  test:                             # (4) ジョブ名（任意の名前）
    runs-on: ubuntu-latest          # (5) 実行環境（Ubuntu最新版のラ
       ンナー）

    steps:                          # (6) ステップの定義
      - name: Checkout code         # (7) ステップ名（ログに表示され
         る）
        uses: actions/checkout@v4   # (8) 公式Actionを使ってコードを
           チェックアウト

      - name: Setup Python
        uses: actions/setup-python@v5  # Pythonをインストールする公
           式Action
        with:                          # Actionに渡すパラメータ
          python-version: '3.12'       # Python 3.12を使用

      - name: Install dependencies
        run: pip install -r requirements.txt  # (9) シェルコマンド
           を直接実行

      - name: Run tests
        run: pytest                           # pytestでテストを実
           行
```

それぞれのポイントを補足します。

(1) `name` — ワークフローの名前です。GitHub Actions タブで一覧表示されるときに使われます。省略すると、ファイル名がそのまま表示されます。

(2) `on` —「いつこのワークフローを実行するか」を定義します。この例では、main ブランチへの push と、main ブランチに対する PR が対象です。

(5) `runs-on` — ジョブを実行する Runner の種類です。`ubuntu-latest` を指定すると、GitHub が

管理する Ubuntu の最新 LTS 版が使われます。macOS が必要な場合は `macos-latest`、Windows なら `windows-latest` を指定します。

(8) `uses` — 既存の Action を利用する記法です。`actions/checkout@v4` は「GitHub の公式 organization が公開している checkout アクションの v4」を意味します。`@v4` のようにバージョンを固定することで、Action の更新による予期しない動作変更を防ぎます。

(9) `run` — シェルコマンドを直接実行します。複数行のコマンドを書きたい場合は、パイプ記号 `|` を使います。

```
      - name: Install and test
        run: |
          pip install -r requirements.txt
          pytest --verbose
```

このファイルをリポジトリにプッシュするだけで、GitHub Actions が自動的にワークフローを認識し、次回の push や PR 作成時にテストが実行されるようになります。

スクリーンショット: *GitHub Actions* タブでワークフローの実行結果が表示されている状態。緑のチェックマークが付いたジョブ一覧

12.5. トリガー（Event）の種類

ワークフローをいつ実行するかを決めるのがトリガーです。用途に応じてさまざまなトリガーが用意されています。

12.5.1. push — コードがプッシュされた時

最も基本的なトリガーです。ブランチやファイルパスで条件を絞り込めます。

```
on:
  push:
    branches: [main, develop]       # mainまたはdevelopへのpush
    paths: ['src/**', 'tests/**']   # src/またはtests/配下の変更時のみ
```

`paths` フィルタは特に便利です。たとえば、ドキュメントだけを修正したのにテストが走るのは無駄です。`paths` でソースコードのディレクトリだけを指定しておけば、不要な実行を防げます。

12.5.2. pull_request — Pull Requestに関するイベント

```
on:
  pull_request:
    branches: [main]
    types: [opened, synchronize, reopened]  # PR作成時、更新時、再
      オープン時
```

`types` を省略すると、`opened`、`synchronize`、`reopened` がデフォルトで適用されます。

12.5.3. schedule — 定期実行

cron式でスケジュール実行を設定できます。セキュリティスキャンや依存関係の更新チェックなど、定期的に行いたいタスクに適しています。

```
on:
  schedule:
    - cron: '0 0 * * *'   # 毎日UTC 0時（日本時間 午前9時）
    - cron: '0 9 * * 1'   # 毎週月曜UTC 9時
```

cron式の各フィールドは左から「分 時 日 月 曜日」です。時刻はUTC基準であることに注意してください。

12.5.4. workflow_dispatch — 手動実行

ボタン1つでワークフローを手動実行できるトリガーです。入力パラメータを定義すると、実行時にドロップダウンやテキストフィールドで値を指定できます。

```
on:
  workflow_dispatch:
    inputs:
      environment:
        description: 'デプロイ先の環境'
        required: true
        default: 'staging'
        type: choice
        options:
          - staging
          - production
      dry_run:
        description: 'ドライランモード（実際にはデプロイしない）'
        required: false
```

```
        type: boolean
        default: true
```

スクリーンショット: *workflow_dispatch* の手動実行画面。入力フィールドと *Run workflow* ボタンが表示されている状態

12.5.5. release — リリース公開時

```
on:
  release:
    types: [published]  # リリースが公開された時
```

このトリガーは、第 14 章で学ぶリリース機能と組み合わせて、バイナリの自動ビルドやパッケージの自動公開に使われます。

12.6. 実用的なワークフロー例

12.6.1. Python プロジェクトの本格的な CI

複数の Python バージョンでテストを実行し、リンターによるコード品質チェックも行う例です。`strategy.matrix` を使うと、指定した組み合わせのすべてに対してジョブが自動的に実行されます。

```
name: Python CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ['3.10', '3.11', '3.12']  # 3バージョンでテスト

    steps:
      - uses: actions/checkout@v4
```

```
      - name: Setup Python ${{ matrix.python-version }}
        uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}

      - name: Cache pip
        uses: actions/cache@v4
        with:
          path: ~/.cache/pip
          key: ${{ runner.os }}-pip-${{ hashFiles('requirements.txt
             ') }}

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Lint with ruff
        run: ruff check .

      - name: Test with pytest
        run: pytest --cov=src --cov-report=xml

      - name: Upload coverage
        uses: codecov/codecov-action@v4
```

`actions/cache@v4` はビルド時間を短縮するための重要な Action です。`requirements.txt` が変わらない限り、前回インストールしたパッケージのキャッシュが再利用されます。大規模なプロジェクトでは、キャッシュの有無でビルド時間が数分変わることもあります。

12.6.2. Node.js プロジェクトの CI

```
name: Node.js CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
```

```
  build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'          # npm のキャッシュを自動管理

      - name: Install dependencies
        run: npm ci             # npm install より厳密で高速

      - name: Lint
        run: npm run lint

      - name: Test
        run: npm test

      - name: Build
        run: npm run build
```

`npm ci` は `npm install` と異なり、`package-lock.json` に基づいて厳密にパッケージをインストールします。CI 環境では、環境ごとにバージョンが異なるという問題を防ぐために `npm ci` が推奨されます。

12.6.3. タグプッシュによる自動リリース

```
name: Release

on:
  push:
    tags: ['v*']          # v で始まるタグがプッシュされた時

jobs:
  release:
    runs-on: ubuntu-latest
    permissions:
      contents: write     # リリース作成に必要な権限
```

```
    steps:
      - uses: actions/checkout@v4

      - name: Create Release
        uses: softprops/action-gh-release@v2
        with:
          generate_release_notes: true  # PRラベルから自動でリリー
             スノート生成
```

12.7. Secrets（シークレット）

12.7.1. なぜ Secrets が必要か

ワークフローの中で外部サービスの API キーやデプロイ用のパスワードを使いたい場面があります。しかし、これらの秘密情報を YAML ファイルに直接書いてしまうと、リポジトリにアクセスできる全員に見えてしまいます。パブリックリポジトリの場合は、世界中の誰からも見える状態になります。

GitHub Secrets は、こうした秘密情報を**暗号化して安全に保存**し、ワークフローの中からだけ参照できるようにする仕組みです。Secrets の値はログに出力される場合も自動的に `***` にマスクされるため、誤って表示されることはありません。

12.7.2. Secrets の設定方法

Web UI では、リポジトリの Settings → Secrets and variables → Actions →「New repository secret」から設定します。

スクリーンショット: リポジトリの *Secrets* 設定画面。シークレット名の一覧が表示されている状態（値は表示されない）

CLI からも設定できます。

```
# 対話的に値を入力（画面に表示されない）
gh secret set API_KEY

# ファイルから値を読み込む
gh secret set API_KEY < api_key.txt

# 環境変数から設定
echo "$MY_API_KEY" | gh secret set API_KEY
```

一度設定した Secret の値は、Web UI からも CLI からも**読み取ることはできません**。値を確認したい場合は、再度設定し直す必要があります。これはセキュリティ上の意図的な設計です。

12.7.3. ワークフローでの使い方

Secrets はワークフロー内で `${{ secrets.SECRET_NAME }}` という構文で参照します。

```
steps:
  - name: Deploy to production
    env:
      API_KEY: ${{ secrets.API_KEY }}
      DEPLOY_TOKEN: ${{ secrets.DEPLOY_TOKEN }}
    run: |
      echo "Deploying with API key..."
      ./deploy.sh
```

`env` でシェルの環境変数にマッピングしてからスクリプト内で使うのが一般的なパターンです。`${{ secrets.API_KEY }}` を `run` の中に直接書くこともできますが、`env` を使うほうがスクリプトの可読性が高く、別環境での再利用もしやすくなります。

12.8. 料金と制限

GitHub Actions の料金は、リポジトリの種類と Runner の OS によって異なります。

12.8.1. パブリックリポジトリ

パブリックリポジトリでは、GitHub Actions は**完全に無料**で、実行時間の制限もありません。これはオープンソースプロジェクトにとって非常に大きなメリットです。

12.8.2. プライベートリポジトリ

プライベートリポジトリでは、プランに応じた月間の無料枠が設けられています。

プラン	Linux	macOS	Windows
Free	2,000 分/月	200 分/月	500 分/月
Pro	3,000 分/月	300 分/月	750 分/月
Team	3,000 分/月	300 分/月	750 分/月
Enterprise	50,000 分/月	5,000 分/月	10,000 分/月

注意すべき点として、macOS ランナーは Linux ランナーの **10 倍**の分数として計算されます。つまり、macOS で 10 分間実行すると、Linux の 100 分ぶんに相当します。Windows は 2 倍です。特別な理由がなければ、`runs-on: ubuntu-latest` を使用することでコストを抑えられます。

無料枠を超過した場合は追加料金が発生しますが、デフォルトでは支出制限が$0 に設定されているため、意図せず高額な請求が来ることはありません。ただし、設定を変更している場合は注意が必要です。

12.8.3. その他の制限

- ワークフロー 1 回の実行時間: 最大 **6 時間**（ジョブ単位）
- ワークフローの同時実行数: プランにより異なる
- API リクエスト: 1 時間あたり 1,000 リクエスト

12.9. ワークフローの確認とデバッグ

12.9.1. CLI での操作

ワークフローの実行結果は、Web UI の Actions タブで確認できますが、CLI でもすべての操作が可能です。

```
# 最近のワークフロー実行一覧を表示
gh run list

# 特定のワークフローだけ表示
gh run list --workflow ci.yml

# 実行の詳細を確認（各ステップの成否が分かる）
gh run view 1234567890

# ログを表示（エラーの原因を調査する際に必須）
gh run view 1234567890 --log

# 失敗したステップのログだけ表示
gh run view 1234567890 --log-failed

# 失敗したワークフローを再実行
gh run rerun 1234567890

# 失敗したジョブだけ再実行（時間短縮）
gh run rerun 1234567890 --failed

# ワークフローの実行をブラウザで確認
gh run view 1234567890 --web
```

スクリーンショット: *GitHub Actions* タブで *CI* ワークフローの実行結果が表示されている状態。各ステップの実行時間と成否が確認できる

12.9.2. デバッグのコツ

ワークフローが失敗した場合、まずは `--log-failed` でエラーログを確認しましょう。よくある失敗の原因には以下のようなものがあります。

- **依存関係のインストール失敗** — `requirements.txt` や `package.json` に存在しないパッケージが指定されている
- **テストの失敗** — コードのバグ。ローカルで再現してから修正する
- **権限の不足** — `permissions` の設定が不足している（特にリリース作成や Pages デプロイ時）
- **Secrets の未設定** — 必要な Secret が設定されていない（Secret 名の typo にも注意）

ワークフローの開発中にデバッグ用のステップを追加することもできます。

```
      - name: Debug info
        run: |
          echo "Runner OS: $RUNNER_OS"
          echo "Working directory: $(pwd)"
          ls -la
          python --version
```

ただし、Secrets の値をデバッグ用に echo すると自動的にマスクされるため、値そのものは確認できません。

12.10. まとめ

この章では、GitHub Actions によるワークフロー自動化の基礎から実践までを学びました。

- **CI/CD** は、コード変更のたびに自動でテスト・ビルド・デプロイを行う手法であり、ソフトウェアの品質を維持しながら開発速度を上げるために不可欠です
- GitHub Actions は **Workflow**、**Event**、**Job**、**Step**、**Action**、**Runner** の 6 つの概念で構成されており、`.github/workflows/` ディレクトリに YAML ファイルを置くだけで利用できます
- **トリガー**には push、pull_request、schedule、workflow_dispatch などがあり、用途に応じて適切なものを選択します
- `strategy.matrix` を使えば、複数のバージョンや環境での**マトリクステスト**が簡単に設定できます

- **Secrets** を使うことで、API キーやパスワードなどの秘密情報をワークフロー内で安全に利用できます
- パブリックリポジトリでは**無料・無制限**、プライベートリポジトリでもプランに応じた無料枠があります

GitHub Actions は、最初の設定こそ YAML の記法に慣れる必要がありますが、一度設定してしまえば、その後のすべての push や PR に対して自動的にテストが走り、チーム全体の開発効率が大きく向上します。「まずは簡単なテスト実行から始めて、徐々にワークフローを充実させていく」というアプローチが、導入を成功させるコツです。

GitHub Pages — Webサイトを無料で公開する

13.1. この章で学ぶこと

この章では、GitHub リポジトリから直接 Web サイトを公開できる **GitHub Pages** について学びます。具体的には、以下の内容を扱います。

- 静的サイトホスティングとは何か、GitHub Pages はどのような仕組みで動いているのか
- ユーザーサイト、プロジェクトサイト、カスタムドメインの 3 種類の公開 URL の違い
- ブランチベース、GitHub Actions ベース、ユーザーサイトの 3 つのセットアップ方法
- Jekyll テーマを使ったサイトの見た目のカスタマイズ
- 独自ドメイン（カスタムドメイン）の設定方法と HTTPS 対応

ポートフォリオサイト、プロジェクトのドキュメント、ブログ、技術メモなど、GitHub Pages は「ちょっとした Web サイトを手軽に公開したい」という場面で非常に便利なサービスです。サーバーの契約や管理が不要で、GitHub にコードをプッシュするだけでサイトが公開されます。

13.2. GitHub Pagesとは

13.2.1. 静的サイトホスティングの概念

Web サイトには大きく分けて**動的サイト**と**静的サイト**があります。動的サイトとは、ユーザーのリクエストに応じてサーバー側で HTML を生成するサイトです。EC サイトの商品検索結果ページや SNS のタイムラインなどが典型例です。動的サイトにはサーバーサイドのプログラム（PHP、Python、Node.js など）とデータベースが必要です。

一方、**静的サイト**は、あらかじめ用意された HTML、CSS、JavaScript ファイルをそのままブラウザに配信するサイトです。内容が「静的」つまり固定であるため、サーバー側での処理が最小限で済みます。個人のポートフォリオ、プロジェクトのドキュメント、ブログなどは静的サイトとして十分に実現できます。

GitHub Pages は、この**静的サイトのホスティング（公開）サービス**です。リポジトリに置いた HTML/CSS/JS ファイルを、GitHub のサーバーが自動的に Web サイトとして配信してくれます。レンタルサーバーを契約する必要も、AWS や GCP のようなクラウドサービスの設定を行う必要もありません。

GitHub Pages の特徴をまとめると、以下のとおりです。

- **完全無料** — パブリックリポジトリであれば無料で利用できる（プライベートリポジトリでも有料プランで利用可能）
- **HTTPS 対応** — SSL 証明書が自動的に発行され、セキュアな通信が標準で有効
- **Git ベースの更新** — コードをプッシュするだけでサイトが自動更新される
- **Jekyll 統合** — Markdown ファイルから自動的に HTML サイトを生成できる
- **カスタムドメイン対応** — 独自のドメイン名でサイトを公開できる

ただし、静的サイトのホスティングに限定されるため、データベースを使った動的な処理やサーバーサイドのスクリプト実行はできません。また、サイトのサイズには制限があり（推奨 1GB 以下）、帯域幅にも月間 100GB のソフトリミットが設けられています。

13.3. 公開 URL の形式

GitHub Pages で公開される Web サイトの URL は、サイトの種類によって 3 つの形式があります。それぞれの特徴と使い分けを理解しておきましょう。

13.3.1. ユーザーサイト（User Site）

GitHub アカウントごとに 1 つだけ作成できる特別なサイトです。`username.github.io` というリポジトリ名でリポジトリを作成すると、自動的にそのリポジトリの内容が `https://username.github.io` という URL で公開されます。

ユーザーサイトは、自分自身の紹介ページやポートフォリオとして使われることが多いです。ルート URL（`/` で始まる）でアクセスできるため、最も短くて覚えやすい URL になります。

13.3.2. プロジェクトサイト（Project Site）

個々のリポジトリごとに作成できるサイトです。リポジトリの Settings → Pages から有効化すると、`https://username.github.io/repository-name` という URL で公開されます。

たとえば、`my-library` というリポジトリで GitHub Pages を有効化すると、`https://username.github.io/my-li` でアクセスできます。ライブラリやツールのドキュメントサイトとして広く使われています。プロジェクトサイトは数の制限がなく、リポジトリごとに 1 つずつ作成できます。

13.3.3. カスタムドメイン

`https://username.github.io` の代わりに、自分で取得した独自ドメイン（例: `https://example.com`）でサイトを公開することもできます。ドメインレジストラ（お名前.com、Google Domains、Cloudflare など）でドメインを取得し、DNS レコードを設定する必要があります。設定方法は後述の 13.6 節で詳しく解説します。

サイトの種類	URL 形式	リポジトリ名の条件
ユーザーサイト	`https://username.github.io`	`username.github.io`（固定）
プロジェクトサイト	`https://username.github.io/repo-name`	任意
カスタムドメイン	`https://your-domain.com`	任意

13.4. セットアップ方法

GitHub Pages のセットアップには 3 つの方法があります。プロジェクトの規模や要件に応じて使い分けましょう。

13.4.1. 方法 1: ブランチから公開（最もシンプル）

リポジトリの特定のブランチ・ディレクトリをそのまま Web サイトとして公開する方法です。最も手軽で、設定ファイルをほとんど書く必要がありません。

1. リポジトリの「**Settings**」タブを開く
2. 左サイドバーの「**Pages**」をクリック
3. 「Build and deployment」セクションで、Source を「**Deploy from a branch**」に設定
4. **Branch** でブランチを選択（通常は `main`）。公開するフォルダは `/`（ルート）または `/docs` を選択
5. 「**Save**」をクリック

> *スクリーンショット: GitHub Pages 設定画面。Source ドロップダウンで Deploy from a branch が選択されている状態*

数分後には、`https://username.github.io/repo-name` でサイトにアクセスできるようになります。リポジトリのルートに `index.html` が存在すれば、それがトップページとして表示されます。`/docs` フォルダを選択した場合は、`docs/index.html` がトップページになります。

`/docs` フォルダの指定は、ソースコードとドキュメントを同じリポジトリで管理したい場合に便利です。ソースコードはリポジトリのルートに、ドキュメント（Web サイト）は `/docs` ディレクトリに置くことで、両者を明確に分離できます。

13.4.2. 方法 2: GitHub Actions から公開（柔軟な方法）

静的サイトジェネレーター（Hugo、Astro、Next.js など）を使ってビルドしたサイトを公開する場合や、デプロイ前に何らかの処理を行いたい場合は、GitHub Actions を使ったデプロイが適しています。

まず、Settings → Pages で Source を「**GitHub Actions**」に変更します。次に、以下のようなワークフローファイルを作成します。

```
# .github/workflows/deploy-pages.yml
name: Deploy to GitHub Pages
```

```yaml
on:
  push:
    branches: [main]

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: "pages"
  cancel-in-progress: false

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Pages
        uses: actions/configure-pages@v4

      - name: Upload artifact
        uses: actions/upload-pages-artifact@v3
        with:
          path: './public'    # デプロイしたいフォルダを指定

      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```

このワークフローのポイントをいくつか補足します。

`permissions` — GitHub Pages へのデプロイには、`pages: write` と `id-token: write` の権限が必要です。これを明示的に記述しないと、デプロイステップで権限エラーが発生します。

`concurrency` — 複数のデプロイが同時に走ることを防ぐ設定です。`cancel-in-progress: false` にすると、先に走っているデプロイが完了するまで次のデプロイは待機します。

`path: './public'` — デプロイするフォルダを指定します。静的サイトジェネレーターの出力ディレクトリに合わせて変更してください（Hugo なら `./public`、Next.js の Static Export なら `./out` など）。

13.4.3. 方法 3: ユーザーサイトの作成

自分の GitHub アカウントのポートフォリオサイトを作成する場合は、以下の手順が最も簡単です。

1. `username.github.io` という名前のリポジトリを新規作成します（`username` は自分の GitHub ユーザー名に置き換えてください）
2. リポジトリに `index.html` を配置します
3. main ブランチにプッシュすると、自動的に `https://username.github.io` で公開されます

簡単な `index.html` の例を示します。

```
<!DOCTYPE html>
<html lang="ja">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
       scale=1.0">
    <title>My Portfolio</title>
    <style>
        body {
            font-family: -apple-system, BlinkMacSystemFont, 'Segoe
               UI', sans-serif;
            max-width: 800px;
            margin: 0 auto;
            padding: 2rem;
            line-height: 1.6;
        }
        h1 { color: #333; }
    </style>
</head>
<body>
    <h1>Welcome!</h1>
    <p>This is my portfolio site, hosted on GitHub Pages.</p>
    <h2>Projects</h2>
    <ul>
        <li><a href="https://github.com/username/project-a">Project
            A</a></li>
        <li><a href="https://github.com/username/project-b">Project
```

```
            B</a></li>
    </ul>
</body>
</html>
```

ユーザーサイトは特別な設定なしに公開されるため、GitHub Pages の動作を試す最も手軽な方法でもあります。

13.5. Jekyll テーマの適用

13.5.1. Jekyll とは

Jekyll は、Ruby 製の静的サイトジェネレーターです。Markdown で書いた文章を、テーマに基づいて HTML サイトに自動変換してくれます。GitHub Pages には Jekyll が組み込まれているため、Ruby を自分のマシンにインストールしなくても、Markdown ファイルをプッシュするだけでサイトが生成されます。

HTML や CSS を一から書かなくても、プロフェッショナルな見た目のサイトが作れるのが Jekyll の魅力です。技術ドキュメントやブログに特に適しています。

13.5.2. テーマの設定

リポジトリのルートに `_config.yml` を作成し、使いたいテーマを指定します。

```
title: My Project Documentation
description: プロジェクトの公式ドキュメント
theme: minima

# リモートテーマ（GitHub Pagesが標準サポートしていないテーマ）を使
  う場合
# remote_theme: pages-themes/cayman@v0.2.0
```

GitHub Pages が標準でサポートしているテーマには、`minima`、`architect`、`cayman`、`dinky`、`hacker`、`leap-day`、`merlot`、`midnight`、`minimal`、`modernist`、`slate`、`tactile`、`time-machine` などがあります。

設定画面からテーマを選ぶこともできます。Settings → Pages →「Theme Chooser」から、プレビューを見ながらテーマを選択できます。

スクリーンショット: *Jekyll* テーマ選択画面。複数のテーマのプレビューが表示されている状態

13.5.3. Markdown でページを作成する

テーマを設定したら、あとは Markdown ファイルを追加するだけです。ファイルの先頭に **Front Matter**（YAML 形式のメタデータ）を記述することで、Jekyll がそのファイルをページとして認識します。

```
---
layout: default
title: About
---

# About This Project

This project provides a simple library for data processing.

## Features

- Fast CSV parsing
- Built-in data validation
- Export to multiple formats
```

`layout: default` は、テーマが提供するデフォルトのレイアウト（ヘッダー、フッター、ナビゲーションなどを含む HTML の枠組み）を使用することを指示しています。`title` はページのタイトルで、ブラウザのタブやナビゲーションに表示されます。

リポジトリのルートに置いた `README.md` も、Jekyll によって自動的にトップページに変換されます。すでに `README.md` があるリポジトリであれば、テーマを設定するだけでそれなりのサイトが出来上がるのです。

13.6. カスタムドメインの設定

13.6.1. なぜカスタムドメインを使うのか

`username.github.io` の URL でも十分に機能しますが、プロフェッショナルなプロジェクトや個人ブランディングのためには、独自ドメイン（例: `docs.myproject.com`）を使いたい場面があります。カスタムドメインを設定すると、GitHub Pages のサイトに自分のドメインでアクセスできるようになります。

13.6.2. DNS レコードの設定

カスタムドメインを使うには、ドメインの DNS レコードを設定して、そのドメインへのアクセスを GitHub Pages のサーバーに向ける必要があります。設定方法は、Apex ドメイン（`example.com`）とサブドメイン（`www.example.com` や `docs.example.com`）で異なります。

サブドメインの場合（推奨）

ドメインレジストラの DNS 設定画面で、`CNAME` レコードを追加します。

タイプ	ホスト名	値
CNAME	`www`	`username.github.io`

たとえば、`www.example.com` を使いたい場合は、ホスト名に `www`、値に `username.github.io` を設定します。

Apex ドメインの場合

Apex ドメイン（`example.com`）を使いたい場合は、`A` レコードで GitHub Pages の IP アドレスを指定します。

タイプ	ホスト名	値
A	`@`	`185.199.108.153`
A	`@`	`185.199.109.153`
A	`@`	`185.199.110.153`
A	`@`	`185.199.111.153`

4 つの IP アドレスすべてを登録することで、冗長性が確保されます。

13.6.3. GitHub 側の設定

DNS レコードを設定したら、GitHub 側でもカスタムドメインを登録します。

1. リポジトリの Settings → Pages を開く
2. 「**Custom domain**」欄にドメイン名を入力する（例: `www.example.com`）
3. 「**Save**」をクリックする
4. GitHub が DNS の設定を確認します。正しく設定されていれば、数分以内にチェックマークが表示されます
5. 「**Enforce HTTPS**」にチェックを入れる

スクリーンショット: *GitHub Pages の Custom domain 設定画面。ドメイン名が入力され DNS check が成功している状態*

「Save」をクリックすると、リポジトリのルートに `CNAME` という名前のファイルが自動的に作成されます。このファイルにはドメイン名が 1 行だけ書かれています。

```
www.example.com
```

このファイルは手動で作成しても構いません。重要なのは、このファイルがリポジトリに存在することです。

13.6.4. HTTPS 対応

GitHub Pages では、カスタムドメインに対しても**無料で SSL/TLS 証明書が自動発行**されます。「Enforce HTTPS」を有効にすると、HTTP でのアクセスが自動的に HTTPS にリダイレクトされます。

DNS の設定が反映されてから証明書が発行されるまでに、最大 24 時間かかることがあります。「Enforce HTTPS」のチェックボックスがグレーアウトしている場合は、DNS 設定の反映を待っている状態です。しばらく待ってからページを再読み込みしてみてください。

13.7. まとめ

この章では、GitHub Pages を使った Web サイトの公開方法を学びました。

- **GitHub Pages** は、GitHub リポジトリから直接静的 Web サイトをホスティングできる無料サービスです
- **ユーザーサイト**（`username.github.io`）は 1 つだけ、**プロジェクトサイト**（`username.github.io/repo`）はリポジトリごとに作成できます
- セットアップは**ブランチベース**（最も簡単）、**GitHub Actions ベース**（柔軟）、**ユーザーサイト**（特定リポジトリ名）の 3 つの方法があります
- **Jekyll** テーマを適用すると、Markdown ファイルからプロフェッショナルな見た目のサイトが自動生成されます
- **カスタムドメイン**を使う場合は、DNS レコード（CNAME または A レコード）を設定し、GitHub 側でドメインを登録します。HTTPS は無料で自動対応されます

GitHub Pages は、「サーバー管理の知識がなくても、コードをプッシュするだけで Web サイトが公開できる」という手軽さが最大の魅力です。プロジェクトのドキュメント公開から個人のポートフォリオまで、幅広い用途に活用できます。

リリース管理 — ソフトウェアを世に届ける

14.1. この章で学ぶこと

この章では、GitHub におけるソフトウェアの**リリース管理**について学びます。具体的には、以下の内容を扱います。

- タグ（Tag）とは何か——特定のコミットに名前を付ける仕組み
- 軽量タグと注釈付きタグの違いと使い分け
- セマンティックバージョニング（SemVer）——バージョン番号の付け方の標準的な規約
- GitHub リリースの作成方法（Web UI と CLI の両方）
- リリースノートの自動生成とカスタマイズ

ソフトウェア開発は「作って終わり」ではありません。作ったものをユーザーに届け、バージョンごとに何が変わったかを伝え、過去のバージョンに戻れるようにしておくことが、信頼されるソフトウェアの条件です。リリース管理は、その「届ける」プロセスを体系的に行うための仕組みです。

14.2. タグ（Tag）とは

14.2.1. コミットに名前を付ける

第 9 章で学んだように、Git のコミットは `a1b2c3d` のようなハッシュ値で識別されます。しかし、「このハッシュ値がバージョン 1.0 のリリースだった」ということを覚えておくのは困難です。そこで使うのが**タグ（Tag）**です。

タグは、特定のコミットに対して人間が読みやすい名前（多くの場合バージョン番号）を付ける機能です。コミット履歴の中に「しおり」を挟むようなものだと考えてください。

```
A → B → C → D → E → F (HEAD)
        ↑       ↑
      v1.0.0  v2.0.0   ← タグ
```

上の図では、コミット C に `v1.0.0`、コミット E に `v2.0.0` というタグが付けられています。タグは後から追加することもでき、コミット履歴を変更するものではありません。あくまで「このコミットがこのバージョンに相当する」という**目印**です。

14.2.2. 軽量タグと注釈付きタグ

Git には 2 種類のタグがあります。

軽量タグ（Lightweight Tag）は、特定のコミットを指すポインタだけを作成する最もシンプルなタグです。ブランチのように「名前がついた参照」を作るだけで、タグ自体に付加情報は持ちません。

```
# 軽量タグの作成
git tag v1.0.0
```

注釈付きタグ（Annotated Tag）は、タグに対してメッセージ（注釈）、作成者の情報、作成日時を記録できるタグです。Git の内部ではタグオブジェクトとして保存されるため、「誰が、いつ、なぜこのバージョンを付けたか」という情報が残ります。

```
# 注釈付きタグの作成（-a フラグと -m でメッセージを指定）
git tag -a v1.0.0 -m "Version 1.0.0 - Initial stable release"
```

リリースに使うタグとしては、**注釈付きタグが推奨**されます。理由は以下のとおりです。

- 作成者と日時が記録されるため、「誰がこのリリースを承認したか」が追跡できる
- メッセージに変更の要約を残せる
- `git describe` コマンドなどのツールが注釈付きタグを優先的に使用する
- 署名（GPG 署名）を付与できるため、リリースの真正性を保証できる

軽量タグは、個人的な作業の目印や一時的なマーキングに使い、正式なリリースには注釈付きタグを使う、というのが一般的な使い分けです。

14.2.3. タグの基本操作

タグに関する基本的な Git コマンドを見ていきましょう。

```
# 現在のHEADに注釈付きタグを作成
git tag -a v1.0.0 -m "Version 1.0.0 - Initial release"

# 過去のコミットにタグを付ける（コミットハッシュを指定）
git tag -a v0.9.0 -m "Beta release" abc1234

# タグの一覧を表示
git tag

# パターンで絞り込み
git tag -l "v1.*"

# タグの詳細情報を表示（注釈付きタグの場合、メッセージや作成者情報
  も表示）
git show v1.0.0

# タグをリモートにプッシュ（タグは通常のpushでは送信されない）
```

```
git push origin v1.0.0

# すべてのタグをリモートにプッシュ
git push origin --tags

# ローカルのタグを削除
git tag -d v1.0.0

# リモートのタグを削除
git push origin --delete v1.0.0
```

ここで重要なのは、**タグは `git push` だけではリモートに反映されない**という点です。通常のブランチの push とは異なり、タグは明示的に `git push origin v1.0.0` または `git push origin --tags` で送信する必要があります。この仕様は、ローカルで一時的に付けたタグが意図せずリモートに送られることを防ぐための設計です。

14.3. セマンティックバージョニング

14.3.1. MAJOR.MINOR.PATCH の意味

ソフトウェアのバージョン番号は、自由に付けることもできますが、多くのプロジェクトでは**セマンティックバージョニング（Semantic Versioning、略称 SemVer）**という標準的な規約に従っています。SemVer では、バージョン番号を `MAJOR.MINOR.PATCH` の 3 つの数字で構成します。

```
v 2 . 3 . 1
  ↓   ↓   ↓
  |   |   └── PATCH: バグ修正（後方互換性あり）
  |   └──────── MINOR: 新機能追加（後方互換性あり）
  └──────────── MAJOR: 破壊的変更（後方互換性なし）
```

この 3 つの数字には明確な意味があり、バージョン番号を見るだけで「このアップデートは安全に適用できるか」を判断できるようになっています。

14.3.2. 各桁の判断基準

PATCH（パッチ）バージョン — バグ修正や小さな改善で、既存の API やインターフェースに変更がない場合に上げます。たとえば、計算結果の丸め誤差を修正した場合や、ドキュメントの typo を修正した場合などです。ユーザーは安心してアップデートできます。

```
v1.2.3 → v1.2.4   （バグ修正のみ）
```

MINOR（マイナー）バージョン — 新しい機能を追加したが、既存の機能との互換性は保たれている場合に上げます。新しい API エンドポイントの追加、新しいオプションパラメータの追加などが該当します。既存のコードを変更せずにアップデートできることが保証されます。MINOR を上げた場合、PATCH は 0 にリセットします。

```
v1.2.4 → v1.3.0   （新機能追加、既存機能は維持）
```

MAJOR（メジャー）バージョン — 既存の API やインターフェースに互換性のない変更（破壊的変更）を加えた場合に上げます。関数のシグネチャの変更、設定ファイルの形式変更、依存関係の大幅な変更などが該当します。ユーザーはアップデートの際にコードの修正が必要になる可能性があります。MAJOR を上げた場合、MINOR と PATCH は 0 にリセットします。

```
v1.3.0 → v2.0.0   （互換性のない変更あり）
```

14.3.3. プレリリースバージョン

正式リリースの前にテスト版を公開する場合は、バージョン番号にサフィックスを付けます。

サフィックス	意味	安定度
`v1.0.0-alpha`	アルファ版。初期テスト段階。機能が不完全な可能性あり	低
`v1.0.0-alpha.2`	アルファ版の 2 回目のリリース	低
`v1.0.0-beta`	ベータ版。機能は揃っているがバグが残っている可能性あり	中
`v1.0.0-beta.3`	ベータ版の 3 回目のリリース	中
`v1.0.0-rc.1`	リリース候補（Release Candidate）。重大なバグがなければ正式版になる	高

プレリリース版は SemVer の仕様上、同じ `MAJOR.MINOR.PATCH` の正式版よりも常に前（低い優先度）とされます。つまり、`v1.0.0-alpha < v1.0.0-beta < v1.0.0-rc.1 < v1.0.0` という順序です。

14.3.4. v を付けるか付けないか

タグに `v` プレフィックスを付けるかどうか（`v1.0.0` vs `1.0.0`）は、プロジェクトによって流儀が異なります。SemVer の仕様自体は `v` を含みませんが、GitHub では `v` を付ける慣習が広く採用されています。GitHub CLI の `gh release create` も `v` 付きのタグを前提としている部分があります。大事なのはプロジェクト内で統一することです。

14.4. リリースの作成

GitHub の**リリース（Release）**は、タグの上に追加情報を載せたものです。タグが「コミットに付けた名前」なのに対し、リリースは以下の情報を含みます。

- **リリースノート** — このバージョンで何が変わったかの説明（変更履歴）
- **アセット（添付ファイル）** — コンパイル済みバイナリ、インストーラー、ZIP ファイルなどの配布物
- **メタ情報** — ドラフトか公開済みか、プレリリースか正式版か

リリースを作ることで、ユーザーは「現在の安定版はどれか」「どのバージョンからどんな変更があったか」を簡単に確認できます。

14.4.1. Web UI での作成

1. リポジトリのメインページ右側にある「**Releases**」セクション（またはタグアイコン）をクリック
2. 「**Create a new release**」ボタンをクリック（既存のリリースがある場合は「Draft a new release」）
3. 「**Choose a tag**」で既存のタグを選択するか、新しいタグ名を入力して作成する
4. **リリースタイトル**を入力（通常はバージョン番号、例: `v1.0.0`）
5. **リリースノート**を記述する。「**Generate release notes**」ボタンをクリックすると、前回のリリースからの変更を PR ベースで自動生成してくれる
6. 必要に応じて、バイナリファイルやアーカイブを「Attach binaries」エリアにドラッグ&ドロップ
7. プレリリースの場合は「**Set as a pre-release**」にチェック
8. 「**Publish release**」をクリック

> スクリーンショット: リリース作成画面。タグ選択、タイトル入力、リリースノート記入欄、*Generate release notes* ボタンが見える状態

「Generate release notes」機能は非常に便利です。前回のリリース以降にマージされた PR のタイトルと作成者を自動的にリストアップしてくれるため、手動で変更履歴を書く手間が大幅に削減されます。

14.4.2. CLI での作成

GitHub CLI を使えば、コマンド 1 つでリリースを作成できます。

```
# 基本的なリリース作成（タグがまだなければ自動作成される）
gh release create v1.0.0 --title "v1.0.0" --notes "Initial release"

# リリースノートを自動生成
gh release create v1.0.0 --generate-notes

# タイトルも自動（タグ名がそのままタイトルになる）
```

```
gh release create v1.0.0 --generate-notes --title "v1.0.0 - First
   Stable Release"

# ドラフトリリース（まだ公開しない）
gh release create v1.0.0 --draft --generate-notes

# プレリリース
gh release create v1.0.0-beta.1 --prerelease --generate-notes

# ファイルを添付してリリース
gh release create v1.0.0 \
  --generate-notes \
  ./dist/app-linux-amd64 \
  ./dist/app-darwin-arm64 \
  ./dist/app-windows-amd64.exe
```

リリースの管理に使えるその他のコマンドも見ておきましょう。

```
# リリース一覧の表示
gh release list

# 特定のリリースの詳細表示
gh release view v1.0.0

# リリースのアセット（添付ファイル）をダウンロード
gh release download v1.0.0

# 特定のアセットだけダウンロード
gh release download v1.0.0 --pattern "*.tar.gz"

# リリースを削除
gh release delete v1.0.0

# ドラフトリリースを公開に変更
gh release edit v1.0.0 --draft=false
```

14.4.3. ドラフトリリースの活用

リリースを作成する際、すぐに公開せず**ドラフト**として保存しておくことができます。ドラフトリリースは、チームメンバーによるリリースノートのレビューや、QA テストの完了を待ってから公開する、といったワークフローで活用されます。

ドラフトリリースはリポジトリの Releases ページに表示されますが、「Draft」というバッジが付

き、公開リリースの一覧には含まれません。準備が整ったら「Edit」から「Publish release」をクリック（またはCLIで `gh release edit v1.0.0 --draft=false`）するだけで公開できます。

14.5. リリースのカスタマイズ

14.5.1. リリースノート自動生成のカテゴリ分け

「Generate release notes」で自動生成されるリリースノートは、デフォルトではすべてのPRが1つのリストに並びます。プロジェクトが大きくなると、「新機能」「バグ修正」「ドキュメント」などのカテゴリに分けて表示したくなります。

これを実現するのが `.github/release.yml` ファイルです。このファイルでは、PRのラベルに基づいてリリースノートをカテゴリ分けするルールを定義します。

```
# .github/release.yml
changelog:
  exclude:
    labels:
      - "skip-changelog"        # このラベルが付いたPRはリリースノートから除外
    authors:
      - "dependabot"            # dependabotによるPRも除外
  categories:
    - title: "New Features"
      labels:
        - "enhancement"
        - "feature"
    - title: "Bug Fixes"
      labels:
        - "bug"
        - "bugfix"
    - title: "Documentation"
      labels:
        - "documentation"
        - "docs"
    - title: "Maintenance"
      labels:
        - "chore"
        - "dependencies"
        - "refactor"
    - title: "Other Changes"
      labels:
        - "*"                   # 上記のいずれにも該当しないPR
```

この設定を行うと、「Generate release notes」で生成されるリリースノートは以下のようにカテゴリ分けされます。

```
## What's Changed

### New Features
* Add dark mode support by @alice in #42
* Implement export to PDF by @bob in #45

### Bug Fixes
* Fix login redirect loop by @charlie in #43
* Fix date parsing in non-English locales by @alice in #47

### Documentation
* Update API reference by @dave in #44

### Other Changes
* Update CI workflow by @eve in #46

**Full Changelog**: https://github.com/owner/repo/compare/v1.0.0...
   v2.0.0
```

ポイントは、**ラベルの活用がリリースノートの品質に直結する**ということです。第 11 章で学んだラベルの運用が、ここで生きてきます。PR を作成する際にラベルを適切に付けていれば、リリース時にはボタン 1 つで整理されたリリースノートが手に入ります。

14.5.2. exclude セクション

`exclude` セクションでは、リリースノートに含めたくない PR を指定します。典型的なのは以下のケースです。

- **dependabot**（依存関係の自動更新 bot）による PR。大量に発生するため、リリースノートが見づらくなる
- **skip-changelog** ラベルを付けた、リリースノートに記載する必要のない内部的な変更

`exclude` に指定された PR は、どのカテゴリにもリストされなくなります。

14.6. まとめ

この章では、ソフトウェアをユーザーに届けるための「リリース管理」の仕組みを学びました。

- **タグ**は特定のコミットに名前を付ける機能であり、リリースには**注釈付きタグ**を使うことが推奨されます

- **セマンティックバージョニング**（`MAJOR.MINOR.PATCH`）は、バージョン番号にルールを設けることで、アップデートの影響範囲をユーザーが判断できるようにする規約です
- **GitHub リリース**は、タグにリリースノートとアセットを追加したもので、Web UI と CLI の両方から作成できます
- 「**Generate release notes**」機能を使えば、PR ベースで変更履歴を自動生成できます
- `.github/release.yml` を設定すると、**PR のラベルに基づいてリリースノートをカテゴリ分け**でき、ユーザーにとって見やすいリリースノートを作成できます

リリース管理は、開発チームとユーザーの間の**コミュニケーション手段**です。「何が変わったか」「安全にアップデートできるか」「過去のどのバージョンを使えばよいか」——これらの疑問に答えるのが、適切に管理されたタグとリリースノートの役割です。

また、第 12 章で学んだ GitHub Actions と組み合わせることで、タグをプッシュしたら自動でビルド・テスト・リリース作成が行われる、という完全自動化されたリリースパイプラインを構築することも可能です。

Fork — オープンソースに貢献する

15.1. この章で学ぶこと

この章では、GitHub 上で最も魅力的な活動のひとつである「オープンソースソフトウェア（OSS）への貢献」について学びます。OSS とは、ソースコードが公開されており、誰でも自由に閲覧・利用・改変・再配布できるソフトウェアのことです。皆さんが普段使っている Linux、Python、VS Code、そして Git 自体も OSS です。

OSS への貢献と聞くと、「自分にはまだ早い」「上級者がやることだ」と感じるかもしれません。しかし、実際にはタイポの修正やドキュメントの改善といった小さな貢献から始められますし、GitHub にはそのための仕組みが整っています。その中心にあるのが **Fork（フォーク）**という機能です。

この章を読み終えると、以下のことができるようになります。

- Fork の仕組みと、通常の clone との違いを理解する
- OSS プロジェクトに Pull Request を送る一連の手順を実践できる
- upstream リモートを設定し、Fork 元のリポジトリと同期を保てる
- 貢献する前に確認すべきルールやマナーを理解する
- コード以外の貢献方法を知る

15.2. Fork とは何か

15.2.1. 他の人のリポジトリを自分のアカウントにコピーする

Fork とは、GitHub 上で他のユーザーや Organization が所有するリポジトリを、自分の GitHub アカウントの下にまるごとコピーする操作です。日本語では「分岐」や「派生」と訳されることもありますが、実際にはリポジトリの完全なコピーを自分のアカウントに作成する行為です。

なぜこのような仕組みが必要なのでしょうか。たとえば、あなたがある OSS プロジェクトのバグを見つけて修正したいとします。しかし、そのリポジトリに対してあなたには push 権限がありません。OSS プロジェクトは世界中の何千人もの開発者がアクセスしますが、全員に push 権限を与えてしまったら、コードの品質管理が不可能になってしまいます。

そこで Fork の出番です。Fork を使えば、自分のアカウントにリポジトリのコピーを作り、そのコピーには自由に push できます。修正が完了したら、元のリポジトリに対して Pull Request を送り、メンテナーにレビューしてもらうという流れになります。

```
元のリポジトリ (upstream)                    あなたのFork (origin)
original-author/project     ——Fork——→     your-name/project
          ↑                                        |
          |                                        | push（自由にでき
             る）
          |                                        ↓
```

```
└────────── Pull Request ──────────────────────────────
  ──┘
```

15.2.2. clone との違い

初学者がよく混同するのが、Fork と clone の違いです。この 2 つは似ているようで、まったく異なる操作です。

clone は、リモートリポジトリをローカルマシンにダウンロードする操作です。`git clone` コマンドで実行し、結果としてローカルマシンにリポジトリのコピーが作られます。clone したリポジトリのリモート（origin）は、元のリポジトリを指しています。

Fork は、GitHub のサーバー上でリポジトリを自分のアカウントにコピーする操作です。これは Git 自体の機能ではなく、GitHub が提供する Web サービスの機能です。Fork した結果、GitHub 上に新しいリポジトリが作られ、そのリポジトリはあなたが自由に操作できます。

つまり、OSS に貢献する際は、まず GitHub 上で Fork し、次に Fork したリポジトリをローカルに clone するという 2 段階の操作を行います。

操作	実行場所	コピー先	push 権限
clone	ターミナル	ローカルマシン	元のリポジトリの権限に依存
Fork	GitHub Web / CLI	自分の GitHub アカウント	自分の Fork には自由に push 可能

Fork には、もうひとつ重要な特徴があります。GitHub は Fork 元のリポジトリと Fork 先のリポジトリの関連を記憶しています。これにより、Fork 先から Fork 元への Pull Request を簡単に作成できるのです。

15.3. OSS への貢献の全体像

OSS プロジェクトに貢献する際の典型的なワークフローは、以下の 6 つのステップで構成されます。この流れは世界中の OSS プロジェクトで共通しており、一度覚えてしまえばどのプロジェクトにも応用できます。

```
┌──────────────────────────────────────────────────────────────────────────
  ──────────────────────────────────────────────────────────────┐
│                    OSSコントリビューションの流れ                          │
├──────────────────────────────────────────────────────────────────────────
  ──────────────────────────────────────────────────────────────┤
│                                                                          │
│   ① Fork        GitHub上でリポジトリをコピー                              │
│       ↓                                                                  │
│   ② Clone       Forkをローカルにダウンロード                              │
│       ↓                                                                  │
│   ③ Branch      作業用ブランチを作成                                      │
│       ↓                                                                  │
```

```
│   ④ Commit      変更を加えてコミット                          │
│       ↓                                                      │
│   ⑤ Push        自分のForkにプッシュ                          │
│       ↓                                                      │
│   ⑥ PR          元のリポジトリにPull Requestを送信            │
│                                                              │
└──────────────────────────────────────────────────────────────
  ──────────────────────────────────────────────────────────────┘
```

この流れを、図書館の本に例えてみましょう。図書館（元のリポジトリ）の本に誤植を見つけたとします。図書館の本に直接ペンで書き込むわけにはいきません。そこで、本をコピーして（Fork）、自分の机に持ち帰り（clone）、修正案を書いたメモを挟み（branch + commit）、コピーに修正を反映させ（push）、図書館に「ここを直してはどうでしょうか」と提案書を送ります（Pull Request）。図書館員（メンテナー）がその提案を確認して妥当だと判断すれば、本が修正されます。

このワークフローの優れている点は、元のリポジトリに影響を与えることなく安全に作業でき、かつ正式なレビュープロセスを経て貢献を反映できるところにあります。

15.4. 実践手順

ここからは、実際のコマンドを使って貢献の手順を一つひとつ見ていきましょう。例として、`original-author/pro` というリポジトリに貢献する場面を想定します。

15.4.1. ステップ 1: Fork する

Fork の方法は 2 通りあります。GitHub CLI を使う方法と、Web ブラウザの UI を使う方法です。

GitHub CLI の `gh repo fork` コマンドを使うと、Fork と clone を一度に行えます。`--clone` フラグを付けると、Fork した直後にローカルへの clone も自動的に行われるため、初心者にはこちらがおすすめです。

```
# CLIでFork（cloneも同時に行う）
gh repo fork original-author/project --clone
cd project
```

Web ブラウザを使う場合は、貢献したいリポジトリのページを開き、右上にある「Fork」ボタンをクリックします。Fork が完了すると、`your-name/project` というリポジトリが自分のアカウントに作成されます。その後、手動で clone します。

```
# Web UIでForkした後、自分のForkをクローン
git clone git@github.com:your-name/project.git
cd project
```

スクリーンショット: リポジトリページ右上の *Fork* ボタン

15.4.2. ステップ 2: upstream リモートを設定する

clone した直後、リモートには `origin`（自分の Fork）だけが登録されています。しかし、Fork 元のリポジトリ（upstream）も登録しておく必要があります。なぜなら、Fork 元のリポジトリは日々更新されており、自分の Fork をその最新の状態に同期させる必要があるからです。

`upstream` という名前は Git の慣習で、Fork 元のリポジトリを指す際に広く使われています。この名前を使うことで、他の開発者があなたのコマンド履歴を見たときにも意図が伝わりやすくなります。

```
# Fork元をupstreamとして登録
git remote add upstream git@github.com:original-author/project.git

# リモートの設定を確認
git remote -v
# origin    git@github.com:your-name/project.git (fetch)
# origin    git@github.com:your-name/project.git (push)
# upstream  git@github.com:original-author/project.git (fetch)
# upstream  git@github.com:original-author/project.git (push)
```

`git remote -v` の出力を見て、`origin` が自分の Fork、`upstream` が Fork 元のリポジトリを指していることを確認しましょう。もし URL が間違っていた場合は、`git remote set-url upstream 正しい URL` で修正できます。

なお、`gh repo fork --clone` で Fork と clone を同時に行った場合は、upstream の設定も自動的に行われます。

15.4.3. ステップ 3: Fork 元の最新の状態と同期する

OSS プロジェクトでは、あなたが Fork してから作業を始めるまでの間にも、他の貢献者によってコードが更新されている可能性があります。古い状態のコードに対して作業を行うと、最終的に Pull Request を送る際にコンフリクト（衝突）が発生しやすくなります。

そのため、作業を始める前に Fork 元の最新の状態を取り込む習慣をつけましょう。この操作は、長期間にわたって作業する場合は定期的に行います。

```
# upstreamの最新を取得
git fetch upstream

# mainブランチに切り替え
```

```
git switch main

# upstreamのmainブランチの内容をマージ
git merge upstream/main

# 同期した内容を自分のFork（origin）にもプッシュ
git push origin main
```

`git fetch upstream` は upstream の情報をダウンロードするだけで、ローカルのファイルには変更を加えません。その後の `git merge upstream/main` で、ダウンロードした内容を実際にローカルの main ブランチに反映します。最後に `git push origin main` で、自分の Fork 上の main ブランチも最新にしておきます。

15.4.4. ステップ 4: ブランチを作成して作業する

貢献の作業は、必ず専用のブランチで行いましょう。main ブランチで直接作業してしまうと、同期の際に問題が起きやすくなります。ブランチ名は、変更の内容が分かるような名前を付けます。多くの OSS プロジェクトでは、`fix/`、`feat/`、`docs/` といったプレフィックスを使う慣習があります。

```
# 作業ブランチを作成して切り替え
git switch -c fix/typo-in-readme

# ファイルを編集する（ここでは例としてREADME.mdのタイポ修正）
# ... エディタで編集 ...

# 変更をステージングしてコミット
git add README.md
git commit -m "fix: Fix typo in installation section"

# 自分のForkにプッシュ
git push -u origin fix/typo-in-readme
```

コミットメッセージについても、プロジェクトの規約に従うことが大切です。多くのプロジェクトでは Conventional Commits のようなルールを採用しています。CONTRIBUTING.md に記載がある場合は、必ずそれに従いましょう。

15.4.5. ステップ 5: Pull Request を送る

変更を Fork にプッシュしたら、いよいよ元のリポジトリに対して Pull Request を送ります。CLI からも Web ブラウザからも作成できます。

```
# CLIでPRを作成（Fork元のリポジトリを指定する）
gh pr create --repo original-author/project \
  --title "fix: Fix typo in README" \
  --body "READMEのインストール手順にあるタイポを修正しました。

## 変更内容
- 「instlation」を「installation」に修正

## テスト
- ドキュメントの変更のみのため、動作テストは不要です。"
```

Web ブラウザを使う場合は、自分の Fork リポジトリのページを開くと、最近プッシュしたブランチの通知バーが表示されます。そこから「Compare & pull request」をクリックするか、「Contribute」ボタンから「Open pull request」を選択します。

スクリーンショット: *Fork* リポジトリから *Pull Request* を作成する画面

Pull Request の説明文（body）は丁寧に書きましょう。メンテナーはあなたの変更の意図を理解する必要があります。何を変更したのか、なぜ変更が必要なのか、どのようにテストしたのかを明記すると、レビューがスムーズに進みます。

15.4.6. ステップ 6: レビュー対応

Pull Request を送った後、プロジェクトのメンテナーがコードをレビューしてくれます。レビューでは修正のリクエストが来ることがよくあります。これは自然なプロセスであり、否定されているわけではありません。むしろ、プロジェクトの品質を保つための重要な工程です。

修正が必要な場合は、同じブランチで作業を続けます。新しいコミットをプッシュすると、Pull Request に自動的に反映されます。

```
# 作業ブランチに切り替え（すでにいる場合は不要）
git switch fix/typo-in-readme

# レビューのフィードバックに基づいて修正
# ... エディタで修正 ...

# 修正をコミットしてプッシュ
git add .
git commit -m "fix: Address review feedback - update wording"
git push
# → Pull Requestに自動的に反映される
```

レビュアーのコメントにはGitHub上で返信し、修正した箇所については「修正しました」と報告しましょう。すべての指摘に対応したら、レビュアーに再レビューをリクエストできます。

15.5. 貢献の前に確認すべきこと

OSSプロジェクトに貢献する際は、コードを書き始める前に確認すべき重要な事項がいくつかあります。これらを事前に確認しておくことで、無駄な作業を避け、プロジェクトのメンテナーとのコミュニケーションを円滑に進められます。

15.5.1. CONTRIBUTING.md

多くのOSSプロジェクトには、`CONTRIBUTING.md`というファイルが置かれています。このファイルには、貢献の手順、コーディング規約、コミットメッセージのフォーマット、テストの方法など、そのプロジェクト独自のルールが記載されています。

```
# CONTRIBUTING.mdの内容を確認する
cat CONTRIBUTING.md
```

たとえば、あるプロジェクトでは「Pull Requestを送る前に必ずIssueを立ててください」というルールがあるかもしれません。別のプロジェクトでは「コミットメッセージはConventional Commitsに従ってください」と指定されているかもしれません。こうしたルールを無視してPull Requestを送ると、レビューの前にフォーマットの修正を求められてしまいます。

15.5.2. Code of Conduct（行動規範）

`CODE_OF_CONDUCT.md`は、コミュニティでの振る舞いに関するルールを定めた文書です。差別的な発言の禁止、建設的なコミュニケーションの奨励、ハラスメントの禁止などが記載されています。多くのプロジェクトがContributor Covenantをベースにした行動規範を採用しています。

行動規範は堅苦しいものに感じるかもしれませんが、その本質は「お互いを尊重し、安心して参加できるコミュニティを作ろう」ということです。コメントやレビューでは常に敬意を持った言葉遣いを心がけましょう。

15.5.3. good first issue を探す

OSSプロジェクトに初めて貢献する場合、何から手を付ければよいか分からないことがあります。そんなときに役立つのが、`good first issue`ラベルです。このラベルが付いたIssueは、プロジェクトのメンテナーが「初めての貢献者でも取り組みやすい」と判断したものです。

```
# good first issueラベルが付いたIssueを検索
gh issue list --repo original-author/project --label "good first
   issue"
```

スクリーンショット: *good first issue* ラベルが付いた *Issue* の一覧

作業を始める前に、その Issue にコメントを残して「この課題に取り組みたい」と宣言するのがマナーです。これにより、他の貢献者と作業が重複することを防げます。もし Issue にすでに誰かがアサインされている場合は、別の Issue を探しましょう。

15.5.4. ライセンスの確認

プロジェクトのライセンス（`LICENSE` ファイル）も確認しておきましょう。あなたが書いたコードはそのプロジェクトのライセンスの下で公開されます。一部のプロジェクトでは、CLA（Contributor License Agreement）への署名を求められることがあります。CLA とは、あなたが貢献するコードの権利関係を明確にするための同意書です。

15.6. コード以外のコントリビューション

OSS への貢献というと、まずコードを書くことを思い浮かべるかもしれません。しかし、実際にはコード以外にも非常に多くの貢献の形があります。むしろ、コード以外の貢献が足りていないプロジェクトは数多くあり、そうした領域での貢献はメンテナーにとても感謝されます。

15.6.1. ドキュメントの改善

ドキュメントの誤字脱字の修正、分かりにくい説明の書き直し、インストール手順の更新など、ドキュメントに関する貢献は常に歓迎されます。特に、初心者としてドキュメントを読んだときに「ここが分かりにくかった」という気づきは、それ自体が貴重なフィードバックです。経験豊富な開発者は自分の知識を前提にしてしまいがちなので、新鮮な目で見た改善提案は大きな価値があります。

15.6.2. 翻訳

国際的なプロジェクトでは、ドキュメントや UI の翻訳が求められることがあります。英語から日本語への翻訳は、日本語話者にとって取り組みやすい貢献のひとつです。プロジェクトによっては、Crowdin などの翻訳プラットフォームを使っている場合もあります。

15.6.3. Issue 報告

バグを発見した際に、再現手順を明確に記載したバグレポートを提出することも立派な貢献です。良いバグレポートには、使用環境（OS、ブラウザ、バージョンなど）、期待される動作、実際の動作、再現手順が含まれています。メンテナーが問題を素早く理解し修正できるよう、できるだけ具体的に書きましょう。

機能の提案（Feature Request）も同様に価値のある貢献です。ただし、提案の前に Issue の一覧を検索して、同じ提案がすでにないか確認しましょう。

15.6.4. レビュー

他の貢献者が送った Pull Request をレビューすることも、プロジェクトへの大きな貢献になります。メンテナーのレビュー負担を軽減できるだけでなく、自分自身のコードリーディング力も向上します。

15.6.5. デザイン

UI や UX の改善提案、アイコンの作成、Web サイトのデザインなど、デザインの専門知識を活かした貢献も重要です。多くの OSS プロジェクトは開発者が中心であるため、デザイン面での貢献は特に不足しがちです。

貢献の種類	具体例	必要なスキル
コード	バグ修正、新機能の実装	プログラミング
ドキュメント	README 改善、チュートリアル作成	文章力
翻訳	UI やドキュメントの多言語化	語学力
Issue 報告	バグレポート、機能提案	丁寧な記述力
レビュー	Pull Request のコードレビュー	コードリーディング力
テスト	テストケースの追加、手動テスト	テスト設計力
デザイン	UI/UX の改善、アイコン作成	デザインスキル

15.7. まとめ

この章では、Fork の仕組みと OSS への貢献の流れについて学びました。重要なポイントを振り返りましょう。

- **Fork** は、GitHub 上で他のリポジトリを自分のアカウントにコピーする操作であり、clone とは異なる GitHub 固有の機能である
- OSS への貢献は、**Fork → clone → branch → commit → push → Pull Request** という一連の流れで行う
- **upstream** リモートを設定し、Fork 元のリポジトリと定期的に同期することで、コンフリクトを防ぐ
- 貢献を始める前に、**CONTRIBUTING.md**、**Code of Conduct**、**ライセンス**を確認する
- **good first issue** ラベルの付いた Issue は、初心者にとって最適な出発点となる
- コード以外にも、**ドキュメント**、**翻訳**、**Issue 報告**、**レビュー**、**デザイン**など多様な貢献方法がある

OSS への貢献は、自分のスキルを向上させると同時に、世界中の開発者が使うソフトウェアをより良くする素晴らしい活動です。最初の一歩は小さくても構いません。まずは好きなプロジェクトの `good first issue` を覗いてみるところから始めてみてください。

Organization — チームでリポジトリを管理する

16.1. この章で学ぶこと

ここまでの章では、個人の GitHub アカウントでリポジトリを管理する方法を学んできました。個人プロジェクトや小規模な共同作業であれば、個人アカウントのリポジトリに Collaborator を追加する方法で十分対応できます。しかし、チームの人数が増えてくると、「誰がどのリポジトリにアクセスできるか」「新しいメンバーが加わったときにどのリポジトリの権限を設定すればよいか」といった管理が煩雑になってきます。

GitHub の **Organization（オーガニゼーション）**は、まさにこの問題を解決するための仕組みです。Organization を使えば、チーム単位でメンバーをグループ化し、リポジトリへのアクセス権限をまとめて管理できます。

この章を読み終えると、以下のことができるようになります。

- Organization と個人アカウントの違いを理解する
- Organization の作成とメンバーの招待ができる
- 権限ロールの違いを正しく設定できる
- Teams を使ってリポジトリ権限を効率的に管理できる
- CODEOWNERS ファイルを使って、コードの担当者を定義できる

16.2. Organization とは

16.2.1. 個人アカウントとの違い

GitHub には、**個人アカウント（Personal Account）**と **Organization** の 2 種類のアカウントがあります。個人アカウントは皆さんが最初に作成した `github.com/your-name` のアカウントです。一方、Organization は**チームや企業、プロジェクトのための共有アカウント**です。

この 2 つの違いを、オフィスビルに例えてみましょう。個人アカウントは自分の部屋のようなものです。自分の部屋にある資料（リポジトリ）は自分が管理し、必要に応じて友人（Collaborator）を招いて一緒に作業できます。一方、Organization は会社のオフィスフロアのようなものです。フロアの中に部署（Team）があり、部署ごとに会議室やキャビネット（リポジトリ）へのアクセス権が設定されています。新しい社員が入社したら、所属部署に追加するだけで、必要なリソースへのアクセス権が自動的に付与されます。

Organization の主な特徴は以下の通りです。

- **リポジトリの共有所有**: リポジトリは Organization が所有するため、特定の個人に依存しない
- **チーム管理**: メンバーを Team にグループ化し、Team 単位で権限を設定できる
- **きめ細かなアクセス制御**: リポジトリごと、ブランチごとに異なる権限を設定できる
- **監査ログ**: 誰がいつ何をしたかの記録が残る（有料プランの場合）

```
Organization: my-company
 ├── Team: frontend (Alice, Bob)
 │    ├── repo: web-app (Write権限)
 │    └── repo: design-system (Write権限)
 ├── Team: backend (Charlie, Dave)
 │    ├── repo: api-server (Write権限)
 │    └── repo: database (Admin権限)
 └── Team: everyone (全メンバー)
      └── repo: docs (Write権限)
```

16.2.2. チーム管理の必要性

「個人アカウントのリポジトリにCollaboratorを追加すれば済むのでは？」と思うかもしれません。確かに、2〜3人の小さなチームであればそれで十分です。しかし、チームが10人、20人と増えていくと、以下のような問題が生じます。

まず、**権限管理の手間**です。新しいメンバーが加わるたびに、10個のリポジトリそれぞれにCollaboratorとして追加する必要があります。Organizationなら、適切なTeamに1回追加するだけで済みます。

次に、**所有権の問題**です。個人アカウントのリポジトリは、そのアカウントが削除されるとリポジトリも消えてしまいます。Organizationが所有するリポジトリは、個人の離脱に影響されません。

そして、**可視性の問題**です。チームの全体像—誰がどのプロジェクトに関わっているか—がOrganizationのページで一覧できます。

16.3. Organizationの作成手順

Organizationの作成は、GitHubのWebインターフェースから簡単に行えます。手順を一つずつ見ていきましょう。

手順1: GitHubにログインし、右上のプロフィールアイコンの横にある「+」ボタンをクリックします。ドロップダウンメニューから「**New organization**」を選択します。

手順2: プランを選択します。Freeプランでも、パブリックリポジトリの無制限作成、チーム管理、基本的なアクセス制御など、多くの機能が利用できます。企業利用で高度なセキュリティ機能や監査ログが必要な場合は、Team プランや Enterprise プランを検討しましょう。

手順3: Organization名を入力します。この名前はGitHub上のURLの一部になります（`github.com/organizati`
チームや会社の名前を使うのが一般的です。変更は後からでも可能ですが、既存のリンクが壊れるため、最初に慎重に選びましょう。

手順4: 連絡先メールアドレスを入力し、Organizationが所属する組織の種類（個人アカウント/ビジネスなど）を選択します。

手順5: 必要に応じて、この段階でメンバーを招待できます。後から追加することも可能です。

スクリーンショット: *Organization* 作成画面でプランを選択する画面

作成が完了したら、CLI から確認してみましょう。

```
# 自分が所属するOrganizationの一覧を表示
gh org list
```

Organization にリポジトリを作成するには、通常のリポジトリ作成と同じコマンドに、Organization 名をプレフィックスとして付けます。

```
# Organizationにリポジトリを作成
gh repo create my-org/new-project --public --description "新しいプロジェクト"

# Organizationのリポジトリ一覧を確認
gh repo list my-org
```

16.4. メンバーの管理

Organization にメンバーを追加する際、そのメンバーにどの程度の権限を与えるかを決める必要があります。GitHub の Organization には、4 つの権限ロールが用意されています。それぞれの役割と責任範囲を正確に理解することが、安全なチーム運営の基礎となります。

16.4.1. 権限ロールの詳細

Owner（オーナー）は、Organization に対する完全な管理権限を持つロールです。Organization の設定変更、メンバーの追加・削除、リポジトリの作成・削除、請求情報の管理など、すべての操作が可能です。Owner は最低 1 人必要ですが、セキュリティの観点から少数に限定することが推奨されます。万が一唯一の Owner がアカウントにアクセスできなくなった場合に備えて、信頼できるメンバーを 2〜3 人 Owner に設定しておくとよいでしょう。

Member（メンバー）は、Organization の一般的なメンバーです。Member は Organization 自体の設定は変更できませんが、所属する Team に応じたリポジトリへのアクセス権を持ちます。ほとんどのメンバーはこのロールに設定します。

Outside Collaborator（外部コラボレーター）は、Organization 自体には所属せず、特定のリポジトリだけにアクセス権を付与された外部の協力者です。たとえば、業務委託先の開発者や、一時的にプロジェクトに参加するフリーランスのデザイナーなどに適しています。Outside Collaborator は Organization のメンバー一覧には表示されません。

Billing Manager（請求管理者）は、請求情報の閲覧・変更のみが可能な特殊なロールです。経理担当者など、コードにはアクセスする必要がないものの、支払いの管理が必要な人に付与します。

ロール	Org 設定	メンバー管理	リポジトリ作成	リポジトリアクセス	請求管
Owner	可能	可能	可能	全リポジトリ	可能
Member	不可	不可	設定による	Team に応じる	不可
Outside Collaborator	不可	不可	不可	指定されたリポジトリのみ	不可
Billing Manager	不可	不可	不可	なし	可能

16.4.2. メンバーの招待方法

メンバーの招待は、Web UI から行います。

1. Organization のページを開き、**People** タブをクリックします
2. 「**Invite member**」ボタンをクリックします
3. 招待したいユーザーの GitHub ユーザー名またはメールアドレスを入力します
4. ロール（Owner または Member）を選択します
5. 「**Send invitation**」をクリックします

招待されたユーザーには通知が届き、承認すると Organization に参加できます。

スクリーンショット: メンバー招待画面

16.5. Teams（チーム）

Organization の真価は、Teams 機能にあると言っても過言ではありません。Teams を使うことで、メンバーを論理的なグループに分け、グループ単位でリポジトリへの権限を設定できます。

16.5.1. なぜ Teams が必要なのか

10 人の Organization に 5 つのリポジトリがあるとします。Teams を使わない場合、各メンバーの各リポジトリへの権限を個別に設定する必要があり、最大で 50 個の権限設定が必要です。メンバーが増えるたびにすべてのリポジトリへの権限を個別に設定しなければなりません。

Teams を使えば、「frontend チームには web-app と design-system リポジトリの Write 権限」のように設定し、新しいフロントエンドエンジニアが入社したら、frontend チームに追加するだけで適切な権限が自動的に付与されます。

16.5.2. チームの作成

チームの作成は Web UI から行います。

1. Organization のページで **Teams** タブを開きます
2. 「**New team**」ボタンをクリックします
3. チーム名を入力します（例：`frontend`、`backend`、`devops`）

4. 説明文を入力します（任意ですが、チームの役割を明記すると分かりやすい）
5. 可視性を設定します。**Visible** は Organization の全メンバーに公開、**Secret** は Team メンバーと Owner のみに表示されます
6. メンバーを追加します

スクリーンショット: チーム作成画面

16.5.3. チームの権限レベル

Team にリポジトリを関連付ける際、5 段階の権限レベルから選択できます。権限は低い方から順に、Read、Triage、Write、Maintain、Admin です。上位の権限は下位の権限をすべて含みます。

Read（読み取り）は、リポジトリの閲覧、Issue/PR の閲覧、clone が可能です。コードを読む必要はあるが、変更は加えないメンバー（PM やデザイナーなど）に適しています。

Triage（トリアージ）は、Read の権限に加えて、Issue やプルリクエストの管理（ラベル付け、アサイン、クローズなど）が可能です。プロジェクトマネージャーや QA エンジニアに適しています。

Write（書き込み）は、リポジトリへの push、ブランチの作成・削除、PR の作成が可能です。日常的にコードを書く開発者に最も一般的な権限です。

Maintain（メンテナンス）は、Write の権限に加えて、ブランチ保護ルールの管理やリポジトリの一部設定の変更が可能です。テックリードやシニアエンジニアに適しています。

Admin（管理者）は、リポジトリに対する完全な管理権限です。リポジトリの削除、Webhook の設定、アクセス権の管理などが可能です。

操作	Read	Triage	Write	Maintain	Admin
コード閲覧・clone	o	o	o	o	o
Issue/PR の管理	-	o	o	o	o
Push・ブランチ操作	-	-	o	o	o
ブランチ保護の管理	-	-	-	o	o
リポジトリ設定変更	-	-	-	-	o

16.5.4. ネストされたチーム

Teams には**親子関係（ネスト）**を設定できます。これにより、組織の階層構造を Teams の構造に反映できます。

```
Engineering（親チーム）
 ├── Frontend（子チーム）
 ├── Backend（子チーム）
 └── DevOps（子チーム）
```

子チームは**親チームの権限を自動的に継承**します。たとえば、Engineering チームに docs リポジトリへの Read 権限を付与すると、Frontend、Backend、DevOps の各子チームのメンバーも自動的に docs リポジトリを閲覧できるようになります。

この仕組みを使えば、全エンジニアに共通する基本権限は親チームで設定し、チーム固有の権限は子チームで追加するという効率的な管理が可能です。

子チームを作成するには、チーム作成時に「Parent team」フィールドで親チームを選択します。

16.6. CODEOWNERS

16.6.1. ファイルやディレクトリの担当者を定義する仕組み

開発チームが大きくなると、「このファイルの変更は誰にレビューしてもらうべきか」という判断が難しくなります。特に、フロントエンドのコード変更をバックエンドエンジニアだけがレビューしてしまったり、重要な設定ファイルの変更がレビューなしでマージされてしまったりすると、問題が発生するリスクがあります。

CODEOWNERS ファイルは、リポジトリ内のファイルやディレクトリごとに「担当者（オーナー）」を定義する仕組みです。CODEOWNERS ファイルに記載されたパターンに一致するファイルが変更された Pull Request が作成されると、対応する担当者が**自動的にレビュアーとして追加**されます。

CODEOWNERS ファイルは、リポジトリのルートディレクトリ、`docs/` ディレクトリ、または `.github/` ディレクトリに配置できます。一般的には `.github/CODEOWNERS` に置かれることが多いです。

16.6.2. CODEOWNERS ファイルの書き方

CODEOWNERS ファイルの各行は、ファイルパターンとオーナー（ユーザーまたは Team）の組み合わせで構成されます。パターンの記法は `.gitignore` と同じ glob 構文を使います。

`.github/CODEOWNERS`：

```
# デフォルトのオーナー（他のルールに一致しないすべてのファイル）
* @my-org/engineering

# フロントエンドのコード
/src/frontend/          @my-org/frontend
*.tsx                   @my-org/frontend
*.css                   @my-org/frontend

# バックエンドのコード
/src/api/               @my-org/backend
*.py                    @my-org/backend
```

```
# データベース関連（特に慎重なレビューが必要）
/migrations/            @my-org/backend @my-org/devops

# ドキュメント
/docs/                  @my-org/docs-team

# CI/CD設定とインフラ
/.github/               @my-org/devops
Dockerfile              @my-org/devops
docker-compose.yml      @my-org/devops

# セキュリティに関わるファイル（Ownerレベルのレビューが必要）
/src/auth/              @security-lead
```

いくつかのポイントを補足します。

複数のオーナーを指定できます。`/migrations/ @my-org/backend @my-org/devops` のように書くと、migration ファイルが変更された場合、backend チームと devops チームの両方がレビュアーに追加されます。

後の行が優先されます。同じファイルに複数のパターンが一致する場合、ファイルの後の方に書かれたルールが優先されます。たとえば、最初の行の `*` は全ファイルに一致しますが、`/src/frontend/` 配下のファイルは後の行のルールで上書きされます。

個人と Team の両方を指定できます。`@username` で個人ユーザーを、`@org-name/team-name` で Team を指定します。

16.6.3. ブランチ保護ルールとの連携

CODEOWNERS ファイルの真価は、ブランチ保護ルールと組み合わせたときに発揮されます。ブランチ保護の設定で「Require review from Code Owners」を有効にすると、CODEOWNERS に定義されたオーナーの承認なしには Pull Request をマージできなくなります。これにより、重要なファイルの変更が必ず適切な担当者のレビューを経ることが保証されます。

16.7. まとめ

この章では、GitHub の Organization 機能について学びました。重要なポイントを振り返りましょう。

- **Organization** は、チームや企業でリポジトリを共有管理するためのアカウントであり、個人アカウントとは異なる特徴を持つ
- Organization のメンバーには **Owner**、**Member**、**Outside Collaborator**、**Billing Manager** の 4 つのロールがあり、それぞれ異なる権限を持つ
- **Teams** を使ってメンバーをグループ化し、Team 単位でリポジトリへのアクセス権限（Read〜Admin）を効率的に管理できる

- Teams は**ネスト（親子関係）**が可能で、子チームは親チームの権限を継承する
- **CODEOWNERS** ファイルを使うことで、ファイルやディレクトリの担当者を定義し、Pull Request 作成時に自動的にレビュアーを割り当てることができる

Organization とチーム管理は、プロジェクトの規模が大きくなるにつれてその重要性が増してきます。最初は少人数でも、将来のチーム拡大を見据えて早い段階から Organization を導入しておくと、後々の管理がスムーズになります。

コードレビュー — 品質を高め合う文化

17.1. この章で学ぶこと

ソフトウェア開発において、コードを書くことと同じくらい重要なプロセスがあります。それが**コードレビュー**です。コードレビューとは、あるメンバーが書いたコードを別のメンバーが読み、フィードバックを返す活動のことです。

コードレビューは単なるバグチェックの手段ではありません。チーム全体の技術力を向上させ、知識を共有し、コードベースの一貫性を保つための文化的なプラクティスです。世界中の優れたソフトウェアチームが、コードレビューを開発プロセスの中核に位置づけています。

この章を読み終えると、以下のことができるようになります。

- コードレビューの目的と価値を深く理解する
- GitHub の Pull Request 上でレビューを実施する具体的な手順を身に付ける
- ブランチ保護ルールを設定して、レビューを開発プロセスに組み込む方法を知る
- レビュアーとしての効果的なフィードバックの書き方を習得する
- PR 作成者として、レビューしやすい Pull Request を作成するコツを学ぶ

17.2. コードレビューとは

17.2.1. なぜレビューするのか

コードレビューの目的は多岐にわたります。ひとつずつ見ていきましょう。

バグの早期発見は、コードレビューの最も分かりやすい効果です。書いた本人は自分のロジックに没頭しているため、見落としやすいミスがあります。別の視点を持つ人が読むことで、null チェックの漏れ、境界値の処理ミス、セキュリティの脆弱性などを本番環境に入る前に発見できます。本番環境でバグが見つかった場合の修正コストは、開発段階で見つかった場合の数倍から数十倍になると言われています。

知識の共有も重要な目的です。チーム開発では、特定の機能を特定のメンバーだけが理解している状態（「バス係数が 1」と呼ばれます）はリスクです。コードレビューを通じて、チームの全員がコードベースの異なる部分を理解する機会が生まれます。レビュアーは他のメンバーが書いたコードを読むことで新しい技法を学び、レビューを受ける側も別の視点からのフィードバックによって成長できます。

コード品質の向上は、レビューの継続的な効果です。自分のコードが他の人に読まれると分かっていると、自然とより読みやすく、より丁寧なコードを書くようになります。また、レビューを通じてチームの「良いコードとは何か」という基準が共有され、時間とともにコードベース全体の品質が向上していきます。

一貫性の維持も見逃せません。大きなプロジェクトでは、複数のメンバーがそれぞれ異なるスタイルでコードを書くと、コードベースの可読性が低下します。レビューを通じて命名規則、エラーハンドリングのパターン、ディレクトリ構成などの一貫性を保つことができます。

17.2.2. コードレビューは「検査」ではなく「対話」

コードレビューを効果的に行うためには、マインドセットが大切です。レビューは上司が部下のコードを「検査」するものではなく、チームメンバー同士が「より良いコードにするために対話する」プロセスです。経験の浅いメンバーが書いたコードでも、ベテランが書いたコードでも、レビューを受けることには等しく価値があります。

17.3. レビューの進め方

GitHub では、Pull Request の画面上でコードレビューの一連の作業を行うことができます。ここでは、レビュアーとしての具体的な操作手順を解説します。

17.3.1. Files changed タブ

Pull Request のページには複数のタブがありますが、レビューの中心となるのは「**Files changed**」タブです。このタブを開くと、Pull Request に含まれるすべてのファイル変更が差分（diff）形式で表示されます。

スクリーンショット: *Files changed* タブの差分表示

差分表示では、**緑色の背景**が追加された行、**赤色の背景**が削除された行を示しています。行が変更された場合は、変更前の行が赤色で、変更後の行が緑色で表示されます。

大きな Pull Request の場合、すべてのファイルを一度に見るのは大変です。左側のファイルツリーを使って、特定のファイルにジャンプできます。また、各ファイルの横にある「Viewed」チェックボックスを使って、レビュー済みのファイルをマークしておくと、後でどこまで確認したかが分かりやすくなります。

17.3.2. コメントの追加方法

差分表示で気になる箇所を見つけたら、その行にコメントを追加できます。コメントを追加したい行の行番号にカーソルを合わせると、青い「+」アイコンが表示されます。これをクリックするとコメント入力欄が開きます。

複数行にまたがるコメントを書きたい場合は、開始行の「+」アイコンをクリックしたまま終了行までドラッグします。これにより、コードの範囲を指定してコメントできます。

17.3.3. コメントの種類

GitHub のレビューコメントには、いくつかの種類があります。

通常のコメントは、質問、指摘、提案などを自由に書けます。

```
ここのロジックですが、userがnullの場合にNullPointerExceptionが
```

```
発生する可能性がありそうです。nullチェックを追加した方が安全ではな
  いでしょうか。
```

suggestion（提案）は、GitHub の強力な機能のひとつです。具体的なコード修正案をコメントに含めることができ、PR 作成者はワンクリックでその修正を適用できます。コメント入力欄のツールバーにある提案ボタンをクリックするか、以下のように手動で記述します。

```
nullチェックを追加することで安全になります。

```suggestion
if (user != null) {
    return user.name;
}
return "Unknown";
```
```

PR 作成者の画面では、この suggestion に対して「Apply suggestion」ボタンが表示され、クリックするだけでコードが修正されます。小さな修正であれば、PR 作成者がわざわざローカルに戻って修正する手間が省けるため、非常に効率的です。

質問コメントは、コードの意図や設計判断について確認するためのものです。

```
この変数名の `ctx` は何の略でしょうか？
ドメイン固有の用語であれば、もう少し明示的な名前にした方が
新しいメンバーにも分かりやすいかもしれません。
```

17.3.4. レビュー結果の送信

個々のコメントを書き終えたら、レビュー全体の結論を送信します。画面右上の「**Review changes**」ボタン（または「**Finish your review**」ボタン）をクリックすると、3 つの選択肢が表示されます。

Comment（コメント）は、承認でも拒否でもない中立的なフィードバックです。「いくつか質問がありますが、全体的には良さそうです」のような場合に使います。

Approve（承認）は、変更を承認する意思表示です。「この Pull Request はマージして問題ない」という判断を示します。ブランチ保護ルールで必須レビュー数が設定されている場合、必要な数の Approve が集まるまでマージできません。

Request changes（変更を要求）は、マージ前に修正が必要であるという意思表示です。セキュリティ上の問題や、明らかなバグ、設計上の大きな懸念がある場合に使います。Request changes が付いた Pull Request は、そのレビュアーが Approve に変更するか、レビューを dismiss するまでマージがブロックされます（ブランチ保護の設定による）。

選択肢	意味	使いどころ
Comment	フィードバックのみ	軽微な指摘や質問のみの場合
Approve	承認	マージしてよいと判断した場合
Request changes	修正要求	マージ前に修正が必要な場合

スクリーンショット: *Review changes* ボタンと *3* つの選択肢

17.4. ブランチ保護ルール

コードレビューの文化をチームに根付かせるためには、仕組みとしてレビューを必須にすることが効果的です。GitHub の**ブランチ保護ルール（Branch Protection Rules）**を使うと、main ブランチへの直接 push を禁止し、Pull Request を経由したレビュー付きのマージを強制できます。

17.4.1. 設定方法

ブランチ保護ルールは、リポジトリの設定画面から構成します。

1. リポジトリのページで **Settings** タブを開きます
2. 左のメニューから **Branches** を選択します
3. 「**Add branch protection rule**」（または「Add branch ruleset」）をクリックします
4. 「Branch name pattern」フィールドに保護したいブランチ名を入力します（例：`main`）

スクリーンショット: ブランチ保護ルールの設定画面

17.4.2. 主な設定項目の解説

ブランチ保護ルールには多くの設定項目がありますが、ここでは特に重要なものを一つずつ解説します。

Require a pull request before merging（マージ前に Pull Request を必須にする）は、最も基本的な保護ルールです。この設定を有効にすると、保護されたブランチへの直接 push ができなくなり、必ず Pull Request を経由する必要があります。これにより、すべての変更がレビュー可能な状態になります。

Require approvals（承認を必須にする）は、Pull Request のマージに必要な承認レビューの数を指定します。1 人のレビュアーの承認を必須とするのが最も一般的ですが、セキュリティが重要なプロジェクトでは 2 人以上を求める場合もあります。承認数は多いほど安全ですが、開発スピードとのバランスを考える必要があります。小〜中規模のチームでは 1〜2 人が適切でしょう。

Dismiss stale pull request approvals when new commits are pushed（新しい push 後に承認をリセットする）は、レビュー承認後に PR 作成者が新しいコミットをプッシュした場合、以

前の承認を無効にする設定です。この設定がないと、承認を得た後にレビューされていないコードを追加してマージできてしまいます。セキュリティの観点から、この設定は有効にすることを強く推奨します。

Require review from Code Owners（Code Owners のレビューを必須にする）は、前章で学んだ CODEOWNERS ファイルと連携する設定です。変更されたファイルの Code Owner として指定されたメンバーの承認がないとマージできなくなります。

Require status checks to pass before merging（マージ前にステータスチェックの成功を必須にする）は、CI/CD パイプライン（GitHub Actions など）の結果をマージ条件にする設定です。テストが失敗した状態のコードがマージされることを防ぎます。第 12 章で学んだ GitHub Actions と組み合わせると特に効果的です。

Require branches to be up to date before merging（マージ前にブランチの最新化を必須にする）は、Pull Request のブランチが main ブランチの最新の状態と統合されていることを要求する設定です。古い main ブランチに基づいたコードがマージされて問題が発生するのを防ぎます。

Include administrators（管理者にもルールを適用する）は、リポジトリの管理者であっても保護ルールの例外にしない設定です。「管理者だから直接 push してもいい」という例外を許すと、ルールの意味が薄れてしまいます。チーム全体の規律を保つために、この設定を有効にすることを推奨します。

設定項目	説明	推奨
Require a pull request	PR 経由のマージを必須	有効
Require approvals	承認レビューの数	1〜2 人
Dismiss stale approvals	新 push 後に承認をリセット	有効
Require Code Owner reviews	Code Owner のレビュー必須	チーム規模による
Require status checks	CI 成功を必須	有効
Require up to date branch	最新 main との統合必須	有効
Include administrators	管理者にも適用	有効

17.4.3. CLI での設定

ブランチ保護ルールは、GitHub API を使って CLI から設定することも可能です。

```
# CLIでブランチ保護を設定する例
gh api repos/{owner}/{repo}/branches/main/protection -X PUT \
  -f "required_pull_request_reviews[required_approving_review_count
     ]=1" \
  -F "required_pull_request_reviews[dismiss_stale_reviews]=true" \
  -F "enforce_admins=true" \
  -F "required_status_checks=null" \
  -F "restrictions=null"
```

17.5. レビュアーとしてのベストプラクティス

コードレビューの質は、レビュアーの姿勢と技術に大きく左右されます。ここでは、効果的なレビュアーになるためのベストプラクティスを紹介します。

17.5.1. 建設的なフィードバックを心がける

レビューのコメントは、批判ではなく改善提案として書きましょう。「このコードはダメだ」ではなく「こうすると可読性が上がるかもしれません」のように、具体的な改善の方向性を示すことが大切です。

人はコードに自分のアイデンティティを投影しがちです。「あなたのコードが悪い」は「あなたが悪い」と受け取られることがあります。主語を「コード」にして、「この部分」「このロジック」のように対象を限定すると、フィードバックが受け入れられやすくなります。

17.5.2. コメントのプレフィックスを活用する

レビューコメントの重要度が分からないと、PR 作成者はすべてのコメントに同じ重みで対応しようとして疲弊してしまいます。以下のようなプレフィックスを使って、コメントの性質を明示しましょう。

```
[Must] ここにSQLインジェクションの脆弱性があります。
ユーザー入力をそのままクエリに含めるのは危険です。
プレースホルダーを使ったパラメータ化クエリに変更してください。

[Nit] 変数名は camelCase に統一した方がコード全体の一貫性が保てそう
   です。
ここだけ snake_case になっています。

[Question] この実装でMapではなくListを選んだ理由を教えてください。
キーによる検索が多い場面ではMapの方が効率的かもしれないと感じまし
   た。

[Praise] このエラーハンドリングのパターン、とても分かりやすいです！
今後のコードでも参考にしたいと思います。
```

Must（必須）は、マージ前に必ず修正が必要な問題です。セキュリティの脆弱性、データ損失のリスク、明確なバグなどがこれに当たります。

Nit（些細な指摘）は、修正が望ましいが、必須ではない軽微な問題です。命名規則のずれ、コードスタイルの微調整、より良い書き方の提案などです。PR 作成者は Nit を後回しにしたり、別の PR で対応したりできます。

Question（質問）は、コードの意図や設計判断について確認するための質問です。レビュアー自身の理解を深めるためのものであり、必ずしも修正を求めているわけではありません。

Praise（称賛）は、良いコードを見つけたときに積極的に書きましょう。問題点だけを指摘するレビューは、受ける側のモチベーションを下げてしまいます。良い点を認めることで、チーム内にポジティブなレビュー文化が育ちます。

17.5.3. 迅速にレビューする

Pull Request が長時間レビューされずに放置されると、開発の流れが停滞します。理想的には、レビューリクエストから 24 時間以内に最初のフィードバックを返すことを目標にしましょう。すべてを詳細に見る時間がない場合でも、「今日中に確認します」という一言を返すだけで、PR 作成者の不安が和らぎます。

17.5.4. 全体像を把握してからレビューする

行単位でコードを読む前に、まず Pull Request の説明文を読み、変更の全体像を把握しましょう。何を目的とした変更なのか、どのような設計判断があるのかを理解した上でコードを読むと、的確なフィードバックが書けます。

17.6. PR 作成者としてのベストプラクティス

レビューの質は、レビュアーだけでなく PR 作成者の工夫にも大きく依存します。レビューしやすい Pull Request を作ることは、PR 作成者の責任であり、チーム全体の生産性を高める重要なスキルです。

17.6.1. 小さな Pull Request を作る

Pull Request のサイズは、レビューの質に直結します。100 行の変更であれば、レビュアーは一つひとつの行を注意深く確認できます。しかし、1000 行を超える Pull Request では、集中力が続かず、重要な問題を見逃す確率が高くなります。

目安として、**変更は 400 行以内に収める**ことを心がけましょう。もし大きな機能を開発する場合は、複数の Pull Request に分割することを検討してください。たとえば、データベースのスキーマ変更、バックエンドの API 実装、フロントエンドの UI 実装をそれぞれ別の Pull Request として送ることで、各 PR のレビューが容易になります。

17.6.2. セルフレビューを行う

Pull Request を提出する前に、自分自身で Files changed タブを確認しましょう。意外なほど多くの問題が、この段階で見つかります。デバッグ用の print 文の消し忘れ、コメントアウトしたままのコード、不要なファイルの追加など、セルフレビューで発見できる問題は少なくありません。

セルフレビューのもうひとつの効果は、レビュアーの負担を減らすことです。明らかな問題がない状態でレビューに出すことで、レビュアーはより本質的な設計やロジックの議論に集中できます。

17.6.3. テストを含める

Pull Request にテストコードを含めることは、レビュアーに大きな安心感を与えます。テストがあれば、レビュアーは「この変更がどのような動作を期待しているのか」をテストコードから読み取ることができます。また、CI でテストが通っていることを確認できれば、基本的な動作に問題がないことが保証されます。

新しい機能を追加する PR には対応するテストを、バグ修正の PR にはそのバグを再現するテストを含めましょう。テストなしの Pull Request は、レビュアーに「本当にこの変更は正しく動作するのだろうか」という不安を抱かせます。

17.6.4. 説明文を丁寧に書く

Pull Request の説明文（description）は、レビュアーへの手紙のようなものです。何を変更したのか、なぜその変更が必要なのか、どのようにテストしたのかを明確に書きましょう。関連する Issue 番号があれば `Closes #42` のように記載して紐付けます。スクリーンショットや GIF アニメーションで変更の効果を視覚的に示すのも効果的です。

17.6.5. コンテキストを共有する

変更の背景にある設計上の判断や、検討した代替案があれば、PR の説明文やコメントで共有しましょう。「A ではなく B を選んだ理由は〜」という情報があると、レビュアーは同じ議論を繰り返さずに済みます。

17.7. まとめ

この章では、コードレビューの目的、GitHub での具体的な操作方法、そしてレビュアーと PR 作成者それぞれのベストプラクティスについて学びました。重要なポイントを振り返りましょう。

- コードレビューは、**バグの早期発見、知識の共有、品質の向上、一貫性の維持**のために行う
- GitHub では **Files changed タブ**で差分を確認し、行コメントや suggestion 機能を使ってフィードバックを提供する
- レビューの結論として **Comment（コメント）**、**Approve（承認）**、**Request changes（変更要求）**の 3 種類を選択できる
- **ブランチ保護ルール**を設定することで、レビューを開発プロセスに組み込み、main ブランチの品質を保つことができる
- レビュアーは**建設的なフィードバック**を心がけ、コメントには **Must/Nit/Question/Praise** などのプレフィックスで重要度を明示する
- PR 作成者は**小さな PR、セルフレビュー、テストの付与、丁寧な説明文**を心がけることで、レビューの質と速度を向上させる

コードレビューは技術的なプラクティスであると同時に、チームのコミュニケーションの場でもあります。互いを尊重し、学び合う姿勢で取り組むことで、チーム全体の成長につながります。

Projects — プロジェクトの進捗を可視化する

18.1. この章で学ぶこと

ソフトウェア開発では、コードを書くことと同じくらい「何を、いつ、誰が作るか」を管理することが重要です。タスクの優先順位、進捗状況、期限、担当者といった情報が整理されていないと、チームは迷走してしまいます。

これまで多くのチームが、Trello、Jira、Asana といった外部のプロジェクト管理ツールを使ってきました。しかし、外部ツールを使うと、GitHub の Issue や Pull Request との連携が煩雑になりがちです。「Jira のチケットを更新して、GitHub の Issue も更新して...」という二重管理は、開発者の負担を増やします。

GitHub Projects は、この問題を解決するために GitHub が提供するプロジェクト管理機能です。Issue や Pull Request と完全に統合されているため、開発の流れから離れることなくタスクの管理ができます。

この章を読み終えると、以下のことができるようになります。

- GitHub Projects の位置づけと特徴を理解する
- Board、Table、Roadmap の 3 つのビューを使い分けられる
- カスタムフィールドを定義してプロジェクトに合った管理体制を構築できる
- ワークフロー自動化を設定して手作業を減らせる
- Organization レベルのプロジェクトで複数リポジトリを横断的に管理できる

18.2. GitHub Projects とは

18.2.1. プロジェクト管理ツールとしての位置づけ

GitHub Projects は、GitHub に内蔵されたプロジェクト管理ツールです。以前は「Projects（Classic）」と呼ばれるシンプルなカンバンボード機能がありましたが、現在の「Projects」はそれを大幅に進化させた新しいバージョンです。

GitHub Projects の最大の特徴は、**Issue や Pull Request とのネイティブな統合**です。Project のアイテムは、既存の Issue や Pull Request そのものです。Issue のステータスを変更すれば Project のボードに反映され、Pull Request がマージされれば自動的にタスクが完了になる、といった連携が自然に行えます。

もうひとつの重要な特徴は、**柔軟なビューとカスタマイズ性**です。同じデータをカンバンボード、スプレッドシート、タイムラインなど複数の形式で表示でき、プロジェクトの状況をさまざまな角度から把握できます。

GitHub Projects を使うべき場面と、外部ツールを検討すべき場面を整理しておきましょう。GitHub でコードを管理しているチームで、Issue ベースのタスクをシンプルに追跡したい場合は、GitHub Projects が最適です。一方、大規模なエンタープライズで高度なレポーティングやリソース管理が必要な場合は、Jira のような専門ツールの方が適しているかもしれません。

18.3. Projectの作成

18.3.1. 作成手順

Projectの作成は、GitHubのWebインターフェースから行います。

1. GitHubの右上にあるプロフィールアイコンをクリックし、「**Your projects**」を選択します
2. 「**New project**」ボタンをクリックします
3. テンプレートを選択します。いくつかのプリセットテンプレートが用意されていますが、「**Board**」「**Table**」「**Roadmap**」から選ぶのが基本です。白紙から始めたい場合は空のテンプレートを選びます
4. プロジェクト名を入力し、「**Create project**」をクリックします

スクリーンショット: *Project*の新規作成画面でテンプレートを選択する様子

Projectが作成されたら、既存のIssueやPull Requestをアイテムとして追加できます。CLIからも操作できます。

```
# Projectに Issueを追加
gh project item-add PROJECT_NUMBER --owner USERNAME \
  --url https://github.com/owner/repo/issues/42
```

ボード上で「+」ボタンをクリックすると、**ドラフトアイテム**を直接作成することもできます。ドラフトアイテムはまだIssueになっていない簡易的なメモのようなもので、後からIssueに変換できます。アイデアを素早くメモしておきたいときに便利です。

18.3.2. 3つのビュー

GitHub Projectsでは、同じデータを異なるビューで表示できます。それぞれのビューには得意とする場面があり、プロジェクトの状況に応じて使い分けることが重要です。

18.3.2.1. Board（ボード）ビュー

Boardビューは、いわゆる**カンバンボード**です。アイテムがカード形式で表示され、ステータスごとの列（カラム）に分類されます。典型的な列の構成は「Todo」「In Progress」「Done」の3列ですが、プロジェクトに応じて自由にカスタマイズできます。

スクリーンショット: *Board*ビューのカンバンボード

Boardビューの利点は、**進捗状況が一目で分かる**ことです。「In Progress」の列にカードが溜まりすぎていれば、チームが多くのタスクを同時に抱えすぎていることが分かります。カードをドラッ

グ&ドロップで別の列に移動でき、直感的な操作が可能です。

Board ビューは、日々の開発作業の管理、スプリントの進捗確認、チームのデイリースタンドアップミーティングでの状況共有などに適しています。

18.3.2.2. Table（テーブル）ビュー

Table ビューは、**スプレッドシートのような表形式**でアイテムを表示します。各列にカスタムフィールド（後述）を配置でき、データの並べ替えやフィルタリングが自在にできます。

スクリーンショット: *Table* ビューのスプレッドシート風表示

Table ビューの利点は、**大量のアイテムを効率的に一覧・編集できる**ことです。優先度、見積もりポイント、期限、担当者など、複数の情報を同時に確認・編集できます。また、列ヘッダーをクリックして並べ替えたり、フィルターを適用して特定の条件のアイテムだけを表示したりできます。

Table ビューは、バックログの管理、スプリントプランニング、プロジェクト全体のタスク一覧の俯瞰などに適しています。

18.3.2.3. Roadmap（ロードマップ）ビュー

Roadmap ビューは、**タイムライン形式（ガントチャート風）**でアイテムを時系列に表示します。このビューを使うには、アイテムに日付フィールド（開始日や期限）が設定されている必要があります。

スクリーンショット: *Roadmap* ビューのタイムライン表示

Roadmap ビューの利点は、**時間軸に沿った計画と進捗が視覚的に分かる**ことです。各タスクがいつ開始され、いつ完了予定かをバーの長さで表現するため、スケジュールの重なりや遅延が一目で把握できます。

Roadmap ビューは、中長期的なプロジェクト計画の共有、マイルストーンの管理、ステークホルダーへの報告などに適しています。

18.3.3. 複数のビューを組み合わせる

ひとつの Project に対して、複数のビューを作成できます。たとえば、「開発チーム用の Board ビュー」「プロジェクトマネージャー用の Table ビュー」「経営層向けの Roadmap ビュー」を同じ Project に用意しておけば、同じデータを異なる視点から確認できます。

ビューの追加は、Project のタブの右端にある「+」ボタンから行えます。各ビューには独立したフィルターやソート設定を保存できるため、用途に応じた表示をいつでも切り替えられます。

18.4. カスタムフィールド

GitHub Projects では、プロジェクトの管理に必要な情報を**カスタムフィールド**として定義できます。デフォルトで用意されている「Title」「Assignees」「Status」に加えて、プロジェクト固有の情報を追加することで、より効果的な管理が可能になります。

カスタムフィールドの追加は、Table ビューの列ヘッダーの右端にある「+」ボタンから行います。以下の型が利用できます。

18.4.1. Text（テキスト）

自由記述のテキストフィールドです。メモ、関連するドキュメントの URL、設計上の注意点など、定型化しにくい情報を記録するのに適しています。

使用例：参照ドキュメントの URL、技術的な補足メモ

18.4.2. Number（数値）

数値を入力するフィールドです。ストーリーポイント（タスクの相対的な作業量を表す指標）、見積もり時間、優先度のスコアなどに使います。Table ビューでは、Number フィールドの合計値を表示することもでき、スプリント全体の作業量を把握するのに役立ちます。

使用例：ストーリーポイント（1, 2, 3, 5, 8, 13）、見積もり工数（時間）

18.4.3. Date（日付）

日付を入力するフィールドです。タスクの開始日、期限（デッドライン）、実際の完了日などを記録します。Roadmap ビューを使う場合、Date フィールドは必須です。開始日と終了日の 2 つの Date フィールドを設定すると、Roadmap ビューでタスクの期間がバーとして表示されます。

使用例：開始予定日、期限、実際の完了日

18.4.4. Single select（単一選択）

あらかじめ定義した選択肢の中から 1 つを選ぶフィールドです。優先度（High / Medium / Low）、カテゴリ（Bug / Feature / Improvement）、担当チーム（Frontend / Backend / DevOps）など、分類に使うフィールドに最適です。各選択肢には色を設定できるため、視覚的にも分かりやすい管理が可能です。

使用例：

- 優先度：`Critical`（赤）、`High`（オレンジ）、`Medium`（黄）、`Low`（青）
- カテゴリ：`Bug`（赤）、`Feature`（緑）、`Documentation`（紫）

18.4.5. Iteration（イテレーション）

スプリントやイテレーションを管理するための特別なフィールドです。開始日と期間（通常 1〜4 週間）を設定すると、自動的にイテレーションの区切りが生成されます。アジャイル開発で「Sprint 1」「Sprint 2」のようにタスクを区切って管理する場合に便利です。

Iteration フィールドを使うと、Board ビューでイテレーションごとにグループ化して表示したり、Table ビューで特定のイテレーションのタスクだけをフィルタリングしたりできます。

使用例：2 週間スプリント（Sprint 1: 4/1〜4/14、Sprint 2: 4/15〜4/28）

スクリーンショット: カスタムフィールドの設定画面

18.4.6. フィールド設計のヒント

カスタムフィールドは多すぎるとかえって管理の負担が増えます。最初は必要最小限のフィールドから始めて、チームの運用を見ながら徐々に追加していくことをお勧めします。以下は、小〜中規模のプロジェクトにおける基本的なフィールド構成の例です。

フィールド名	型	用途
Status	Single select	Todo / In Progress / In Review / Done
Priority	Single select	High / Medium / Low
Sprint	Iteration	2 週間ごとのスプリント
Story Points	Number	タスクの相対的な作業量
Due Date	Date	タスクの期限

18.5. ワークフロー自動化

プロジェクト管理で最も面倒なのは、ステータスの手動更新です。Issue がクローズされたら「Done」に移動する、Pull Request がマージされたらタスクを完了にするといった操作を毎回手動で行うのは、忘れがちですし時間の無駄です。

GitHub Projects の**ワークフロー自動化**機能を使えば、特定のイベントが発生したときにアイテムのステータスを自動的に変更できます。

18.5.1. 設定方法

ワークフローの設定は、Project の設定画面から行います。

1. Project のページを開きます
2. 右上の「**...**」メニュー（または設定アイコン）をクリックし、「**Workflows**」を選択します
3. 利用可能なワークフローの一覧が表示されます
4. 有効にしたいワークフローのトグルをオンにし、条件と動作を設定します

スクリーンショット: ワークフロー設定画面

18.5.2. 主なワークフロー

Item added to project（アイテム追加時）は、Issue や Pull Request が Project に追加されたときに発動するワークフローです。新しいアイテムが追加されたら自動的にステータスを「Todo」に設定する、といった使い方が一般的です。これにより、追加されたアイテムがステータスなしの状態で放置されることを防げます。

Item closed（アイテムクローズ時）は、Issue がクローズされたときに Project のステータスを自動更新するワークフローです。たとえば、Issue がクローズされたら自動的にステータスを「Done」に変更する設定にしておけば、手動でカードを移動する手間が省けます。

Pull request merged（PR マージ時）は、Pull Request がマージされたときに発動するワークフローです。マージされたらステータスを「Done」に変更する設定が典型的です。`Closes #42` のように PR と Issue が紐付いている場合、PR のマージによって Issue もクローズされ、両方が「Done」になります。

Item reopened（アイテム再オープン時）は、一度クローズされた Issue が再オープンされたときに、ステータスを「In Progress」や「Todo」に戻すワークフローです。

これらのワークフローを適切に設定しておくと、チームメンバーは日常的に Issue のオープン・クローズや Pull Request のマージだけに集中すれば、Project のボードが自動的に最新の状態に保たれます。

18.5.3. より高度な自動化

組み込みのワークフロー以外にも、GitHub Actions を使ってさらに高度な自動化を実現できます。たとえば、特定のラベルが付けられた Issue を自動的に Project に追加する、担当者がアサインされたら自動的にステータスを「In Progress」に変更する、といったカスタムワークフローを構築できます。第 12 章で学んだ GitHub Actions の知識が、ここでも活きてきます。

18.6. フィルタリングとソート

Project に多くのアイテムが登録されてくると、必要な情報を素早く見つけるためのフィルタリングとソートが重要になります。GitHub Projects では、ビューごとに独立したフィルターとソート条件を設定・保存できます。

18.6.1. フィルタの記法

フィルターバーに以下のような構文で条件を入力します。

```
# ステータスが「Todo」のアイテムだけ表示
status:Todo
```

```
# 自分にアサインされたアイテムだけ表示
assignee:@me

# 特定のラベルが付いたアイテムだけ表示
label:bug

# 優先度が「High」のアイテムだけ表示
priority:High

# 複数の条件を組み合わせる（AND条件）
status:Todo,In Progress assignee:@me label:bug

# 否定条件（特定のステータスを除外）
-status:Done
```

複数の値をカンマで区切ると「OR 条件」として扱われます。`status:Todo,In Progress` は「Todo または In Progress のアイテム」を意味します。スペースで区切った複数のフィルターは「AND 条件」として扱われます。

18.6.2. ソート

Table ビューでは、列ヘッダーをクリックすることでその列の値に基づいてソートできます。昇順・降順を切り替えることも可能です。複数の列でソートしたい場合は、ビューの設定から詳細なソート条件を指定できます。

たとえば、「優先度の高い順に並べた上で、同じ優先度のアイテムは期限の早い順に並べる」といった複合的なソートが可能です。

18.6.3. ビューの保存

フィルターやソートの設定は、ビューごとに保存できます。日常的に使うフィルター条件は、専用のビューとして保存しておくと便利です。たとえば：

- 「マイタスク」ビュー：`assignee:@me -status:Done` でフィルタリング
- 「今週の優先タスク」ビュー：`status:Todo,In Progress priority:High` でフィルタリング
- 「バグ一覧」ビュー：`label:bug -status:Done` でフィルタリング

これらのビューをタブとして並べておけば、ワンクリックで見たい情報に切り替えられます。

18.6.4. グループ化

Board ビューだけでなく、Table ビューでもアイテムをグループ化して表示できます。たとえば、担当者ごとにグループ化すれば「各メンバーがどれだけのタスクを抱えているか」が一目で分かります。スプリントごとにグループ化すれば「各スプリントにどれだけのタスクが割り当てられているか」を確認できます。

18.7. Organization レベルのプロジェクト

18.7.1. 複数リポジトリにまたがるプロジェクト管理

実際の開発では、ひとつのプロダクトが複数のリポジトリにまたがることが珍しくありません。たとえば、フロントエンドのリポジトリ、バックエンド API のリポジトリ、インフラ設定のリポジトリが分かれているケースです。それぞれのリポジトリに Project を作成すると、プロジェクト全体の進捗を把握するのが困難になります。

Organization レベルのプロジェクトは、この問題を解決します。Organization に紐付いた Project を作成すれば、その Organization 配下の複数のリポジトリの Issue や Pull Request をひとつのボードで一元管理できます。

```
Organization Project: "Q1 プロダクトリリース"
 ├── frontend-app リポジトリの Issue
 │    ├── #12 ログイン画面のリデザイン
 │    └── #15 ダークモード対応
 ├── backend-api リポジトリの Issue
 │    ├── #8  認証APIの実装
 │    └── #11 パフォーマンス改善
 └── infrastructure リポジトリの Issue
      ├── #3  本番環境のスケーリング設定
      └── #5  監視アラートの追加
```

18.7.2. Organization プロジェクトの作成

Organization レベルのプロジェクトを作成するには、Organization のページから操作します。

1. Organization のページを開きます
2. **Projects** タブをクリックします
3. 「**New project**」をクリックし、通常の Project 作成と同じ手順で作成します

この Project には、Organization が所有する任意のリポジトリの Issue や Pull Request を追加できます。あるリポジトリの Issue 画面から直接 Project に追加することも、Project の画面から URL を指定して追加することもできます。

18.7.3. 実践的な活用パターン

Organization プロジェクトの活用パターンをいくつか紹介します。

プロダクト単位のプロジェクトは、ひとつのプロダクトに関わるすべてのリポジトリのタスクを集約します。プロダクトマネージャーがプロダクト全体の進捗を把握するのに適しています。

四半期目標のプロジェクトは、Q1、Q2 のように四半期ごとのプロジェクトを作成し、その期間に達成すべきタスクを管理します。期間が終わったらプロジェクトをアーカイブし、新しい四半期のプロジェクトを作成します。

チーム横断のプロジェクトは、複数のチームが協力して進めるプロジェクト（たとえば、大規模なリファクタリングやセキュリティ対応）を管理するのに適しています。各チームのリポジトリから Issue を集め、全体の進捗を可視化します。

18.8. まとめ

この章では、GitHub Projects を使ったプロジェクト管理の方法について学びました。重要なポイントを振り返りましょう。

- **GitHub Projects** は、Issue や Pull Request と統合されたプロジェクト管理ツールであり、外部ツールとの二重管理を解消できる
- **Board（カンバン）、Table（表形式）、Roadmap（タイムライン）**の 3 つのビューを目的に応じて使い分ける
- **カスタムフィールド**（Text、Number、Date、Single select、Iteration）を定義して、プロジェクト固有の情報を管理する
- **ワークフロー自動化**を設定すると、Issue のクローズ時や PR のマージ時にステータスが自動更新され、手動管理の手間が省ける
- フィルタリング、ソート、グループ化を活用して、大量のアイテムから必要な情報を素早く見つける
- **Organization レベルのプロジェクト**を使えば、複数リポジトリの Issue を 1 つのボードで横断的に管理できる

GitHub Projects は、小さなプロジェクトではシンプルなカンバンボードとして、大きなプロジェクトではカスタムフィールドやワークフロー自動化を駆使した高度な管理ツールとして、柔軟に活用できます。まずはシンプルな Board ビューから始めて、チームの運用に合わせて徐々にカスタマイズしていくことをお勧めします。

ファイルの扱い方 — Raw URL・画像・LFS

19.1. この章で学ぶこと

GitHub は単なる「コードの保管庫」ではありません。リポジトリの中にあるファイルは、それぞれ固有の URL を持ち、特定の行を指し示したり、過去のある時点のスナップショットを永久に参照したりできます。さらに、画像や大容量ファイルの管理、別のリポジトリを取り込むサブモジュールなど、ファイルにまつわるさまざまな仕組みが用意されています。

この章では以下のことを学びます。

- GitHub 上のファイル URL の構造と、blob・tree・raw・blame・commits の違い
- 特定の行へのリンクと、パーマリンク（永続リンク）の作り方
- Raw URL を使った生データの取得と、プログラムからの読み込み
- 画像をリポジトリに保存・表示する方法
- Git LFS（Large File Storage）で大容量ファイルを効率的に管理する方法
- サブモジュールでリポジトリの中に別のリポジトリを含める方法
- GitHub 上でのファイル操作と便利なキーボードショートカット

19.2. GitHub上のファイルURL構造 — blob/tree/raw/blamitsの違いを詳しく

GitHub 上でファイルやフォルダを閲覧するとき、ブラウザのアドレスバーに表示される URL には一定の規則があります。この規則を理解しておくと、他の人にファイルの場所を伝えたり、自動化スクリプトからファイルを取得したりする際に非常に役立ちます。

GitHub のファイル URL は、以下の構造になっています。

```
https://github.com/{owner}/{repo}/{表示モード}/{ブランチまたはコ
    ミット}/{ファイルパス}
```

ここで「表示モード」にあたる部分が、ファイルをどのような形で見るかを決定します。主な表示モードは 5 つあります。

表示モード	意味	URL の例
`blob`	ファイルの中身をシンタックスハイライト付きで表示	`/blob/main/src/app.py`
`tree`	ディレクトリの中身（ファイル一覧）を表示	`/tree/main/src/`
`raw`	ファイルの生データをそのまま返す（HTML の装飾なし）	`/raw/main/data.csv`
`blame`	ファイルの各行を「誰がいつ変更したか」と共に表示	`/blame/main/app.py`
`commits`	そのファイルに関するコミット履歴だけを表示	`/commits/main/app.py`

スクリーンショット: *blob* 表示と *tree* 表示の比較

blob は最もよく使う表示モードです。リポジトリのファイル一覧からファイル名をクリックしたとき、自動的にこのモードで開かれます。コードにはシンタックスハイライトが適用され、行番号も表示されるため、読みやすい形で内容を確認できます。

tree はディレクトリを表示するモードです。リポジトリのトップページや、フォルダ名をクリックしたときに使われます。blob がファイル単体を表示するのに対し、tree はフォルダの中身の一覧を表示します。この名前は Git の内部データ構造に由来しています。Git では、ディレクトリを「tree オブジェクト」、ファイルを「blob オブジェクト」として管理しているのです。

raw はファイルの生データをそのまま返します。HTML の装飾やヘッダー・フッターは一切なく、ファイルの中身だけが返されます。このモードは次のセクションで詳しく扱います。

blame は「git blame」コマンドの Web 版です。ファイルの各行について、「その行を最後に変更したのは誰か」「どのコミットで変更されたか」を表示します。バグの原因を追跡するときや、ある行がなぜそのように書かれているかを調べるときに重宝します。「blame（非難する）」という名前が少し物騒ですが、実際には「責任の所在を明らかにする」という意味合いで使われています。

commits はそのファイルに関係するコミットだけを時系列で表示します。リポジトリ全体のコミット履歴ではなく、特定のファイルがどのように変遷してきたかを追跡できるので、ファイルの変更履歴を調べるときに便利です。

19.3. 特定行へのリンクとパーマリンク

19.3.1. 行番号リンク

コードレビューや Issue の議論で「このファイルの 42 行目を見てほしい」と伝えたいことがあります。そんなとき、GitHub 上のファイル表示画面で行番号をクリックすると、URL の末尾に `#L42` が追加され、その行がハイライトされます。

```
# 42行目へのリンク
https://github.com/owner/repo/blob/main/src/app.py#L42

# 42行目から50行目までの範囲をハイライト
https://github.com/owner/repo/blob/main/src/app.py#L42-L50
```

範囲指定は、最初の行番号をクリックした後、Shift キーを押しながら最後の行番号をクリックすることで設定できます。この URL をコピーして Issue や PR のコメントに貼り付ければ、他のメンバーに正確な場所を伝えることができます。

スクリーンショット: 行番号をクリックしてハイライトされた状態

19.3.2. パーマリンク — なぜ永続リンクが重要なのか

ここで一つ、重要な注意点があります。ブランチ名を含むURL（例えば `/blob/main/src/app.py#L42`）は、ブランチの先端を指しています。つまり、誰かがそのファイルを編集してコミットすると、同じURLで表示される内容が変わってしまいます。42行目にあったコードが別の行に移動したり、まったく別の内容になっていたりする可能性があるのです。

これは、本の「最新版の42ページ」を参照するようなものです。改訂されれば42ページの内容は変わります。一方、「初版第3刷の42ページ」と言えば、いつ誰が参照しても同じ内容を指します。

Gitの世界では、この「初版第3刷」にあたるのがコミットハッシュです。コミットハッシュを含むURLは**パーマリンク（永続リンク）**と呼ばれ、未来永劫同じ内容を指し続けます。

```
# ブランチ指定（内容が変わりうる）
https://github.com/owner/repo/blob/main/src/app.py#L42

# コミットハッシュ指定（パーマリンク --- 永久に同じ内容）
https://github.com/owner/repo/blob/a1b2c3d4e5f6/src/app.py#L42
```

パーマリンクを手動で作るのは面倒ですが、GitHubには便利なショートカットがあります。ファイル表示画面で **`y` キー** を押すと、URLのブランチ名部分がその時点のコミットハッシュに自動的に置き換えられます。これだけでパーマリンクの完成です。

IssueやPRのコメント、ドキュメント、ブログ記事などでコードの特定箇所を参照するときは、必ずパーマリンクを使いましょう。ブランチ名を含むURLを貼ると、半年後にリンクをクリックした人が見る内容が、あなたが意図したものとまったく違っている可能性があります。

19.4. Raw URL — 生データの取得

19.4.1. Raw URLとは

GitHubの通常のファイル表示（blob）は、HTMLページの中にコードが埋め込まれた形で表示されます。人間が読むには見やすいですが、プログラムからファイルの中身を取得したい場合には不便です。そこで登場するのが**Raw URL**です。

Raw URLはファイルの中身だけをそのまま返します。HTMLの装飾は一切ありません。CSVファイルならCSVデータそのもの、Pythonスクリプトならソースコードそのものが返されます。

Raw URLのドメインは `raw.githubusercontent.com` であり、通常のGitHubのURLとは異なります。

```
# 通常の表示URL（HTMLページとして表示される）
https://github.com/owner/repo/blob/main/data.csv

# Raw URL（ファイルの中身だけが返される）
https://raw.githubusercontent.com/owner/repo/main/data.csv
```

19.4.2. curlでのダウンロード

Raw URLを使えば、コマンドラインからファイルを直接ダウンロードできます。

```
# ファイルをダウンロード
curl -O https://raw.githubusercontent.com/owner/repo/main/install.
   sh

# .gitignoreテンプレートをダウンロード
curl -o .gitignore https://raw.githubusercontent.com/github/
   gitignore/main/Python.gitignore

# シェルスクリプトを直接ダウンロードして実行（信頼できるソースに限
   る）
curl -fsSL https://raw.githubusercontent.com/owner/repo/main/setup.
   sh | bash
```

最後の例（ダウンロードして即実行）は便利ですが、信頼できるソースからのスクリプトに限って使うようにしてください。内容を確認せずに実行するのはセキュリティ上のリスクがあります。

19.4.3. プログラムからの読み込み

Raw URLはプログラムから直接データを読み込むのにも使えます。例えば、Pythonのpandasライブラリでは、URLを直接指定してCSVファイルを読み込めます。

```
import pandas as pd

# GitHubに置いたCSVデータを直接読み込む
url = "https://raw.githubusercontent.com/owner/repo/main/data.csv"
df = pd.read_csv(url)
print(df.head())
```

この方法は、データ分析のチュートリアルやサンプルコードでよく使われます。読者がデータをダウンロードする手間なく、コードをそのまま実行できるからです。ただし、大量のデータを繰り返し取得する場合は、ローカルにダウンロードしてから読み込む方が効率的です。

19.5. 画像の保存と表示

GitHubのREADMEやドキュメントに画像を表示する方法は、主に2つあります。それぞれに利点と欠点があるので、用途に応じて使い分けましょう。

19.5.1. 方法1: リポジトリに画像ファイルを保存する

最も正統派の方法は、リポジトリ内に画像ファイルを含めることです。プロジェクトの一部として画像をバージョン管理できるため、画像がなくなる心配がありません。

```
my-repo/
├── docs/
│   ├── images/          ← 画像用のフォルダを作る
│   │   ├── screenshot1.png
│   │   ├── architecture.svg
│   │   └── demo.gif
│   └── guide.md
└── README.md
```

Markdown からは相対パスで参照します。

```
<!-- README から画像を参照する場合 -->
![スクリーンショット](docs/images/screenshot1.png)

<!-- docs/guide.md から同じフォルダの画像を参照する場合 -->
![アーキテクチャ図](images/architecture.svg)
```

SVG 形式の画像はファイルサイズが小さく、拡大してもきれいに表示されるため、図やダイアグラムには SVG がおすすめです。ただし、スクリーンショットのような写真的な画像には PNG や JPEG が適しています。

この方法の注意点として、画像ファイルはバイナリデータであり、変更するたびにリポジトリのサイズが増えていくことがあります。大量の高解像度画像を含める場合は、後述する Git LFS の利用を検討してください。

19.5.2. 方法2: Issue にドラッグアンドドロップして CDN を利用する

もう一つの便利な方法は、GitHub 自身の CDN（コンテンツ配信ネットワーク）を活用するやり方です。Issue や Pull Request のコメント欄に画像をドラッグアンドドロップすると、GitHub が自動的に画像をアップロードし、URL を生成してくれます。

スクリーンショット: *Issue* のコメント欄に画像をドラッグアンドドロップする操作

生成される URL は以下のような形式です。

```
https://github.com/user-attachments/assets/xxxxxxxx-xxxx-xxxx-xxxx-
```

```
    xxxxxxxxxxxx
```

このURLをREADMEや他のMarkdownファイルで使えば、リポジトリのサイズを増やさずに画像を表示できます。

```
![デモ画面](https://github.com/user-attachments/assets/xxxxxxxx-
    xxxx-xxxx-xxxx-xxxxxxxxxxxx)
```

ただし、この方法で生成されたURLがいつまで有効であるかはGitHubの裁量に依存します。長期的に重要な画像は、リポジトリに含める方法の方が確実です。

19.5.3. HTMLタグでのサイズ指定

Markdownの画像記法（`![alt](url)`）ではサイズを指定できません。画像が大きすぎる場合は、HTMLの `<img>` タグを使ってサイズを制御しましょう。

```
<!-- 幅を500ピクセルに指定（高さは自動調整） -->
<img src="docs/images/screenshot.png" width="500">

<!-- 幅と高さを両方指定 -->
<img src="docs/images/screenshot.png" width="400" height="300">

<!-- 中央寄せにする場合 -->
<p align="center">
  <img src="docs/images/logo.png" width="200">
</p>
```

GitHubのMarkdownレンダラーはHTMLタグの一部をサポートしています。`<img>` タグは問題なく使えるので、サイズの調整が必要な場合は積極的に活用しましょう。

19.6. Git LFS — 大容量ファイルの管理

19.6.1. なぜ大きなファイルが問題になるのか

Gitは本来、テキストファイル（ソースコード）を管理するために設計されたツールです。テキストファイルであれば、差分（diff）を効率的に計算・保存できるため、ファイルが何千回変更されてもリポジトリのサイズはそれほど大きくなりません。

しかし、バイナリファイル（画像、動画、データセット、コンパイル済みバイナリなど）は事情が違います。Gitはバイナリファイルの差分を効率的に計算できないため、ファイルが変更されるたびにファイル全体のコピーが履歴に追加されます。例えば、100MBの画像ファイルを10回更新すると、リポジトリの履歴には約1GBのデータが蓄積されることになります。

リポジトリが肥大化すると、`git clone` に時間がかかり、ディスク容量を圧迫し、チーム全体の生産性が低下します。GitHub にはリポジトリサイズの推奨上限（5GB 程度）もあります。

19.6.2. LFS の仕組み — ポインタファイル

Git LFS（Large File Storage）はこの問題を解決するための拡張機能です。仕組みはシンプルです。大きなファイルの実体をリポジトリの外（LFS 専用のストレージサーバー）に保管し、リポジトリには「ポインタ」だけを保存します。

ポインタファイルは数百バイト程度の小さなテキストファイルで、以下のような内容です。

```
version https://git-lfs.github.com/spec/v1
oid sha256:4
   cac19622fc3ada9c0fdeadb33f88f367b541f38b89de0ee093e6d0e2e13df38
size 104857600
```

`git clone` するとポインタファイルだけがダウンロードされるため高速です。ファイルの実体は、実際に必要になったとき（チェックアウトや diff の際）に自動的に LFS サーバーからダウンロードされます。

19.6.3. セットアップ手順

```
# 1. Git LFSのインストール
brew install git-lfs       # macOS (Homebrew)
apt install git-lfs         # Ubuntu/Debian

# 2. Git LFSの初期化（ユーザーごとに一度だけ実行）
git lfs install

# 3. 追跡するファイルパターンを指定
git lfs track "*.psd"            # Photoshopファイル
git lfs track "*.zip"            # ZIPアーカイブ
git lfs track "*.mp4"            # 動画ファイル
git lfs track "data/*.csv"       # dataフォルダ内のCSV

# 4. 自動生成された .gitattributes を確認
cat .gitattributes
# *.psd filter=lfs diff=lfs merge=lfs -text
# *.zip filter=lfs diff=lfs merge=lfs -text

# 5. .gitattributes をコミット（これが重要！）
git add .gitattributes
git commit -m "Configure Git LFS tracking"
```

`.gitattributes` ファイルは他のチームメンバーと共有されるため、必ずコミットしてください。これにより、リポジトリをクローンした全員が同じ LFS 設定を使えるようになります。

設定後は、通常通り `git add`、`git commit`、`git push` するだけです。LFS が追跡対象に指定されたファイルは、自動的に LFS ストレージにアップロードされます。

```
git add large-dataset.zip
git commit -m "Add training dataset"
git push
```

19.6.4. 容量制限

GitHub 上の LFS ストレージには容量制限があります。

プラン	ストレージ	帯域幅（月あたり）
Free	1 GB	1 GB
Pro	1 GB	1 GB
追加データパック	+50 GB ストレージ / +50 GB 帯域（$5/月）	

無料プランでも 1GB までは使えますが、それ以上のデータを扱う場合はデータパックの購入が必要です。大規模なデータセットを扱う研究プロジェクトなどでは、別のストレージサービス（S3 や GCS など）を併用することも検討しましょう。

19.7. サブモジュール — リポジトリの中に別リポジトリを含める

19.7.1. サブモジュールとは

開発を進めていると、「複数のプロジェクトで共通して使うライブラリ」を別リポジトリとして管理したい場面が出てきます。このとき、共通ライブラリのコードをコピーして各プロジェクトに含めると、更新の反映が面倒になります。かといって、パッケージマネージャー経由で配布するほど大げさなものでもない場合もあるでしょう。

サブモジュール（submodule）は、あるリポジトリの中に別のリポジトリを「入れ子」にする仕組みです。親リポジトリは、子リポジトリの特定のコミットへの参照を保持します。

```
my-project/              ← 親リポジトリ
 ├── src/
 ├── lib/
 │    └── shared-utils/  ← サブモジュール（別リポジトリ）
 ├── .gitmodules         ← サブモジュールの設定ファイル
```

```
└── README.md
```

19.7.2. サブモジュールの追加

```
# サブモジュールを追加（lib/shared-utils/ に配置）
git submodule add git@github.com:owner/shared-utils.git lib/shared-
   utils

# .gitmodules ファイルが自動的に作成される
cat .gitmodules
# [submodule "lib/shared-utils"]
#     path = lib/shared-utils
#     url = git@github.com:owner/shared-utils.git

# 設定をコミット
git add .gitmodules lib/shared-utils
git commit -m "Add shared-utils as submodule"
```

19.7.3. サブモジュール付きリポジトリのクローン

サブモジュールを含むリポジトリをクローンする際は、`--recursive` オプションを付ける必要があります。このオプションを忘れると、サブモジュールのディレクトリは空のままになります。

```
# クローンと同時にサブモジュールも取得（推奨）
git clone --recursive git@github.com:owner/my-project.git

# すでにクローン済みの場合は、後からサブモジュールを取得
cd my-project
git submodule init
git submodule update
```

19.7.4. サブモジュールの更新

サブモジュールの元リポジトリに新しいコミットが追加された場合、親リポジトリ側で明示的に更新する必要があります。

```
# サブモジュールを最新のコミットに更新
cd lib/shared-utils
git pull origin main
```

```
cd ../..

# 親リポジトリでサブモジュールの更新をコミット
git add lib/shared-utils
git commit -m "Update shared-utils submodule"

# または、一括で全サブモジュールを更新
git submodule update --remote --merge
```

サブモジュールは強力な仕組みですが、チームメンバーがサブモジュールの概念を理解していないと混乱の原因になることもあります。使用する際は README にサブモジュールの存在と、`--recursive` でのクローンが必要である旨を記載しておくとよいでしょう。

19.8. GitHub 上でのファイル操作

Git コマンドやエディタを使わずに、GitHub の Web 画面上でもファイルの作成・編集ができます。ちょっとしたタイポの修正やドキュメントの更新には、Web 上で直接編集する方が手軽です。

19.8.1. ファイルの作成

1. リポジトリのページを開く
2. 「**Add file**」ボタンをクリックし、「**Create new file**」を選択
3. ファイル名の入力欄にパスを入力（スラッシュを含めるとフォルダが自動作成される）
4. エディタにファイルの内容を入力
5. 「**Commit changes**」をクリック

スクリーンショット: *GitHub 上でのファイル作成画面*

例えば、ファイル名の入力欄に `docs/guide/getting-started.md` と入力すると、`docs/guide/` というフォルダ構造が自動的に作成され、その中に `getting-started.md` が作られます。

19.8.2. ファイルの編集

既存のファイルを表示した状態で、鉛筆アイコン（Edit）をクリックすると、その場で編集できます。編集後は通常のコミットと同様に、コミットメッセージを入力して保存します。直接 main ブランチにコミットすることも、新しいブランチを作って Pull Request を開くこともできます。

19.8.3. キーボードショートカットによる検索

GitHub 上でファイルを探すとき、マウスでフォルダを一つずつ開いていくのは非効率です。以下のキーボードショートカットを覚えておくと、素早くファイルを見つけられます。

キー	機能
`t`	ファイルファインダーを開く（ファイル名のファジー検索）
`s` または `/`	リポジトリ内の検索バーにフォーカス

`t` **キー**は特に便利です。リポジトリのどのページにいても `t` を押すと、ファイル名を部分一致で検索できるファインダーが開きます。例えば「conf」と入力すれば、`config.yml`、`.prettierrc.conf`、`settings/configuration.py` などが候補に表示されます。大規模なリポジトリでファイルを探すときに重宝するショートカットです。

スクリーンショット: *t* キーで開くファイルファインダー

19.9. まとめ

この章では、GitHub 上でのファイルの扱い方を幅広く学びました。

- **URL 構造**: blob（ファイル表示）、tree（ディレクトリ表示）、raw（生データ）、blame（行ごとの変更者）、commits（ファイルの変更履歴）の 5 つの表示モードがある
- **行番号リンク**: `#L42` や `#L42-L50` で特定行を指し示せる
- **パーマリンク**: `y` キーでブランチ指定 URL をコミットハッシュ指定に変換でき、内容が変わらない永続リンクを作れる
- **Raw URL**: `raw.githubusercontent.com` から生データを取得でき、curl やプログラムから直接読み込める
- **画像**: リポジトリに保存する方法と Issue に D&D して CDN を使う方法がある。HTML の `<img>` タグでサイズ指定が可能
- **Git LFS**: 大容量のバイナリファイルはポインタだけを Git に保存し、実体は別サーバーに保管する仕組み
- **サブモジュール**: リポジトリの中に別リポジトリを入れ子にでき、`--recursive` オプションでクローンする
- **Web 上のファイル操作**: ブラウザ上でファイルの作成・編集ができ、`t` キーでファイルを素早く検索できる

READMEとドキュメント — プロジェクトの顔を作る

20.1. この章で学ぶこと

あなたがGitHubで面白そうなリポジトリを見つけたとき、最初に目にするのは何でしょうか。ファイル一覧の下に表示される、プロジェクトの説明文です。これが**README**と呼ばれるファイルの中身です。READMEはプロジェクトの「顔」であり、「玄関」であり、「名刺」でもあります。どんなに優れたコードでも、READMEがなければ誰にも使ってもらえません。

この章では以下のことを学びます。

- READMEの歴史と役割、GitHubにおける表示の仕組み
- 良いREADMEの書き方とテンプレート
- GitHub Flavored Markdown（GFM）の記法を網羅的に
- 用途別のREADMEテンプレート（CLI、Web、ライブラリ、研究）
- README作成のコツ — 最初の3秒、バッジ、スクリーンショット
- Wiki、LICENSE、CONTRIBUTING、CODE_OF_CONDUCTなどの関連ファイル
- オープンソースライセンスの選び方

20.2. READMEとは何か

20.2.1. 名前の由来 — 1970年代からの伝統

READMEという名前は文字通り**"Read Me"（私を読んで）**という意味です。この慣習は1970年代のソフトウェア配布にまで遡ります。当時、ソフトウェアはテープやフロッピーディスクで配布されており、パッケージを開いた人が最初に読むべき説明書として、大文字で `README` と名付けたテキストファイルが同梱されていました。大文字にする理由は単純で、ファイル一覧がアルファベット順に表示されたとき、大文字のファイル名は小文字のファイル名よりも先に表示される（ASCIIコード順）ため、目立つからです。

この伝統は半世紀を経た現在でも生き続けています。GitHubをはじめとするほぼ全てのコードホスティングサービスは、READMEファイルを特別扱いし、自動的にレンダリングして表示します。

20.2.2. GitHubでの役割 — 自動表示の仕組み

GitHubでは、リポジトリのルートディレクトリに `README.md` ファイルがあると、ファイル一覧の下にその内容がHTMLに変換されて自動表示されます。つまり、リポジトリにアクセスした人が**スクロールせずに**最初に読める情報になります。

```
github.com/owner/repo にアクセスすると...
```

```
┌──────────────
```

場所	表示されるREADME
リポジトリのトップページ	ルートの `README.md`
サブディレクトリ	そのフォルダ内の `README.md`（あれば）
ユーザープロフィール	`{username}/{username}` リポジトリの `README.md`
Organization プロフィール	`.github` リポジトリの `profile/README.md`

20.3. READMEの書き方

20.3.1. 最低限のREADME

READMEを書くのに完璧を目指す必要はありません。まずは最低限の情報を書くところから始めましょう。以下の3つの情報があれば、「READMEがない」状態とは雲泥の差です。

```
# プロジェクト名

何をするプロジェクトかの1-2文の説明。

## 使い方

```bash
pip install my-package
my-command --input data.csv
```

ライセンス

MIT
```

たった10行ほどですが、「何のプロジェクトか」「どうやって使うか」「ライセンスは何か」という最も基本的な情報が伝わります。

20.3.2. 標準的なREADMEの構成

プロジェクトが成長してきたら、以下のようなセクション構成を目指しましょう。この構成はオープンソースプロジェクトで広く採用されている「型」です。

```
# プロジェクト名

![Build Status](https://img.shields.io/github/actions/workflow/
   status/owner/repo/ci.yml)
![License](https://img.shields.io/github/license/owner/repo)
```

```
プロジェクトの概要を1-2文で。何を解決するのか、何が特徴か。

## 特徴

- 機能1: 簡潔な説明
- 機能2: 簡潔な説明
- 機能3: 簡潔な説明

## インストール

```bash
pip install my-package
```

使い方

```python
from my_package import MyClass

obj = MyClass()
result = obj.do_something()
print(result)
```

設定

環境変数	説明	デフォルト
`API_KEY`	APIキー	なし（必須）
`DEBUG`	デバッグモード	`false`

開発

```bash
git clone https://github.com/owner/repo.git
cd repo
pip install -e ".[dev]"
pytest
```

貢献
```

```
[CONTRIBUTING.md](CONTRIBUTING.md) を参照してください。

## ライセンス

MIT License - 詳細は [LICENSE](LICENSE) を参照。
```

この構成のポイントは、読者が上から順に読んでいくと、「概要 → 導入 → 使い方 → 詳細設定 → 開発参加」という自然な流れになっていることです。

20.4. Markdown記法

GitHub Flavored Markdown（GFM）は、標準的なMarkdownにGitHub独自の拡張を加えたものです。READMEだけでなく、Issue、Pull Request、Wikiなど、GitHub上のあらゆるテキスト入力欄で使えます。ここでは主要な記法をすべて紹介します。

20.4.1. 見出し

`#` の数で見出しレベルを指定します。READMEでは `#`（h1）をプロジェクト名に使い、セクションには `##`（h2）以下を使うのが一般的です。

```
# 見出し1（プロジェクト名に使う）
## 見出し2（主要セクションに使う）
### 見出し3（サブセクションに使う）
#### 見出し4
```

20.4.2. 太字・斜体・取り消し線

```
**太字（ボールド）** は重要な語句に使う
*斜体（イタリック）* は英語の用語や強調に使う
~~取り消し線~~ は古い情報の訂正に使う
`インラインコード` はコマンド名やファイル名に使う
```

20.4.3. リスト

```
- 箇条書きリスト
- 2つ目の項目
  - インデントでネストできる
  - さらにネスト
```

```
1. 番号付きリスト
2. 順序が重要なときに使う
3. 手順の説明に適している
```

20.4.4. リンクと画像

```
[リンクテキスト](https://example.com)
[別のリポジトリ](../other-repo)

![画像の代替テキスト](docs/images/screenshot.png)
```

20.4.5. テーブル

テーブルはデータの比較や設定項目の一覧に便利です。コロン（`:`）の位置で列の揃え方を指定できます。

```
| 左寄せ | 中央寄せ | 右寄せ |
|:-------|:------:|-------:|
| データ | データ  | データ |
| 左に揃う | 中央に揃う | 右に揃う |
```

20.4.6. コードブロック

コードブロックはバッククォート 3 つで囲みます。言語名を指定すると、シンタックスハイライトが適用されます。

```
```python
def hello(name):
    print(f"Hello, {name}!")
```

```bash
npm install express
node server.js
```

```diff
- 削除された行（赤く表示される）
+ 追加された行（緑に表示される）
```
```

```
```

`diff` 言語を指定すると、`-` で始まる行が赤、`+` で始まる行が緑で表示されます。変更点を説明するときに便利な記法です。

20.4.7. タスクリスト

```
- [x] 完了したタスク
- [ ] 未完了のタスク
- [ ] もう一つの未完了タスク
```

Issue や PR の本文に書くと、チェックボックスとして表示され、クリックで完了/未完了を切り替えられます。進捗管理に非常に便利です。

20.4.8. 折りたたみ（details/summary）

長い内容を折りたたんで、必要なときだけ展開できるようにする記法です。HTML タグを使います。

```
<details>
<summary>クリックで展開 --- 詳細なログ出力</summary>

ここに折りたたまれた内容を書く。
コードブロックやリストなども使える。

```
長いログ出力
...
```

</details>
```

スクリーンショット: 折りたたみの展開前と展開後

20.4.9. 注意・警告（Alerts）

GitHub 独自の拡張で、注意事項や警告を目立つ形で表示できます。

```
> [!NOTE]
> 補足情報。知っておくと便利だが必須ではない。
```

```
> [!TIP]
> ヒント。効率的な使い方のアドバイス。

> [!IMPORTANT]
> 重要。必ず理解しておくべき情報。

> [!WARNING]
> 警告。注意しないと問題が起きる可能性がある。

> [!CAUTION]
> 危険。データ損失やセキュリティリスクにつながる操作。
```

20.4.10. Mermaid図

GitHubはMermaid記法をサポートしており、コードブロック内にテキストで書いた図をそのままレンダリングしてくれます。フローチャートやシーケンス図を画像として用意する必要がありません。

```
```mermaid
graph LR
    A[開発] --> B[テスト]
    B --> C[レビュー]
    C --> D[デプロイ]
```

```mermaid
sequenceDiagram
    Client->>Server: リクエスト
    Server->>Database: クエリ
    Database-->>Server: 結果
    Server-->>Client: レスポンス
```
```

20.4.11. 数式（LaTeX）

GitHubはLaTeX形式の数式レンダリングもサポートしています。

```
インライン数式: $E = mc^2$

ブロック数式:
```

```
$$
\frac{n!}{k!(n-k)!} = \binom{n}{k}
$$
```

研究プロジェクトのREADMEや、アルゴリズムの説明に数式を使いたい場合に便利です。

20.5. ジャンル別READMEテンプレート

プロジェクトの種類によって、READMEに書くべき情報は異なります。ここでは4つの代表的なジャンルについて、それぞれのテンプレートを紹介します。

20.5.1. CLIツール

CLIツールのREADMEでは、インストール方法と具体的なコマンド例が最も重要です。ユーザーは「すぐに使いたい」ので、コピーアンドペーストで動く例を最初に示しましょう。

```
# my-cli-tool

CSVファイルを各種フォーマットに変換するコマンドラインツール。

## インストール

```bash
pip install my-cli-tool
```

使い方

```bash
# 基本的な使い方
my-cli input.csv --output result.json

# オプション一覧
my-cli --help
```

オプション

オプション	短縮形	説明	デフォルト
`--output`	`-o`	出力ファイル	stdout
`--format`	`-f`	出力形式（json/yaml/xml）	json
`--verbose`	`-v`	詳細表示	false
```

```
## 使用例

```bash
# CSVをJSONに変換
my-cli data.csv -o data.json

# パイプと組み合わせる
cat data.csv | my-cli -f yaml > data.yml
```
```

20.5.2. Webアプリ / API

Webアプリではデモへのリンク、技術スタック、環境変数の一覧が重要です。

```
# My Web App

チームのタスク管理を効率化するWebアプリケーション。

## デモ

https://my-app.example.com

## 技術スタック

- **Frontend**: React, TypeScript, Tailwind CSS
- **Backend**: Node.js, Express
- **Database**: PostgreSQL
- **Infrastructure**: Docker, Vercel

## セットアップ

```bash
git clone https://github.com/owner/repo.git
cd repo
npm install
cp .env.example .env   # 環境変数を設定
npm run dev
```

環境変数
```

```
| 変数名 | 説明 | 必須 |
|--------|------|------|
| `DATABASE_URL` | DB接続文字列 | Yes |
| `API_KEY` | 外部API用キー | Yes |
| `PORT` | サーバーポート | No (default: 3000) |

## APIエンドポイント

| メソッド | パス | 説明 |
|---------|------|------|
| GET | `/api/users` | ユーザー一覧 |
| POST | `/api/users` | ユーザー作成 |
| GET | `/api/users/:id` | ユーザー詳細 |
```

20.5.3. ライブラリ / パッケージ

ライブラリのREADMEでは、APIリファレンス（主要なクラスや関数の使い方）が重要です。

```
# my-library

データ変換を簡潔に書けるPythonライブラリ。

![PyPI](https://img.shields.io/pypi/v/my-library)
![License](https://img.shields.io/github/license/owner/repo)

## インストール

```bash
pip install my-library
```

クイックスタート

```python
from my_library import Transformer

t = Transformer(format="json")
result = t.convert(data)
print(result)
```

APIリファレンス
```

```
### `Transformer(format)`

| パラメータ | 型 | 説明 |
|-----------|-----|------|
| `format` | str | 出力形式（"json", "yaml", "xml"） |

### `Transformer.convert(data)`

入力データを指定形式に変換して返す。

| パラメータ | 型 | 説明 |
|-----------|-----|------|
| `data` | list / dict | 入力データ |
| **戻り値** | str | 変換結果の文字列 |
```

20.5.4. 研究プロジェクト

研究プロジェクトのREADMEでは、論文へのリンク、実験の再現手順、結果の表、引用情報（BibTeX）が重要です。

```
# ShortTitle

> Full Paper Title
> Authors (Year). *Journal Name*.

[論文](https://doi.org/xxx) | [プロジェクトページ](https://example.
   com)

## 概要

この研究では○○を○○する手法を提案した。

## 環境構築

```bash
conda create -n myenv python=3.11
conda activate myenv
pip install -r requirements.txt
```

データの準備
```

```
1. [こちら](https://example.com/data)からデータをダウンロード
2. `data/` ディレクトリに配置

## 実験の再現

```bash
python train.py --config configs/experiment1.yaml
python evaluate.py --checkpoint outputs/best_model.pt
```

結果

手法	Accuracy	F1 Score
Baseline	85.2	83.1
Ours	**91.7**	**90.3**

引用

```bibtex
@article{author2024title,
  title={Full Paper Title},
  author={Author, A. and Author, B.},
  journal={Journal Name},
  year={2024}
}
```
```

20.6. README作成のコツ

20.6.1. 最初の3秒で勝負が決まる

人がWebページに滞在するかどうかは、最初の数秒で決まると言われています。READMEも同じです。リポジトリを訪れた人は、最初の3秒で「自分にとって有用なプロジェクトかどうか」を判断します。

プロジェクト名の直後に、1-2文の明確な説明を書きましょう。「何を」「なぜ」「どう」を簡潔に伝えます。

```
# fast-csv

Node.js向けの高速CSVパーサー。標準のcsv-parseより5倍速い。
```

20.6.2. コピーアンドペーストで動くコードを載せる

使い方のセクションに載せるコード例は、読者がそのままコピーして実行できるものにしましょう。「ここを適宜変更してください」という箇所が多いと、試す前に心が折れてしまいます。

20.6.3. スクリーンショットや GIF を使う

UI のあるプロジェクトでは、文章よりもスクリーンショット 1 枚の方が雄弁です。操作の流れを見せたい場合は GIF アニメーションが効果的です。

GIF 作成ツールの例:

- **macOS**: Gifski、Kap
- **Windows**: ScreenToGif
- **CLI**: `ffmpeg -i video.mp4 -vf "fps=10,scale=800:-1" demo.gif`

20.6.4. 目次を付ける（長い README の場合）

README が長くなったら、冒頭に目次を付けると読者が必要なセクションに直接ジャンプできます。

```
## 目次

- [インストール](#インストール)
- [使い方](#使い方)
- [設定](#設定)
- [貢献](#貢献)
```

GitHub は Markdown の見出しに対して自動的にアンカー ID を生成するため、日本語の見出しでもリンクが機能します。

20.7. バッジの付け方

20.7.1. shields.io — 動的バッジ

README の冒頭にバッジを並べると、プロジェクトの状態が一目で分かり、プロフェッショナルな印象を与えます。**shields.io** は、GitHub のリポジトリ情報や CI/CD の状態などを元に動的なバッジ画像を生成してくれるサービスです。

```
<!-- ビルド状態 -->
![Build](https://img.shields.io/github/actions/workflow/status/
```

```
   owner/repo/ci.yml?branch=main)

<!-- ライセンス -->
![License](https://img.shields.io/github/license/owner/repo)

<!-- 最新リリースバージョン -->
![Version](https://img.shields.io/github/v/release/owner/repo)

<!-- 主要言語 -->
![Language](https://img.shields.io/github/languages/top/owner/repo)

<!-- スター数 -->
![Stars](https://img.shields.io/github/stars/owner/repo)

<!-- パッケージレジストリのバージョン -->
![npm](https://img.shields.io/npm/v/package-name)
![PyPI](https://img.shields.io/pypi/v/package-name)

<!-- カスタムバッジ（色とメッセージを自由に設定） -->
![Custom](https://img.shields.io/badge/label-message-blue)
```

バッジの URL は `https://img.shields.io/` で始まり、その後に種類やパラメータを指定します。shields.io の Web サイトにアクセスすると、対話的にバッジを作成できます。

20.7.2. 技術スタックバッジ

プロジェクトで使っている技術をバッジで表示すると、一目で技術スタックが分かります。`style=for-the-badge` を指定すると、大きめの角丸バッジになります。

```
![Python](https://img.shields.io/badge/Python-3776AB?style=for-the-
   badge&logo=python&logoColor=white)
![React](https://img.shields.io/badge/React-61DAFB?style=for-the-
   badge&logo=react&logoColor=black)
![Docker](https://img.shields.io/badge/Docker-2496ED?style=for-the-
   badge&logo=docker&logoColor=white)
![TypeScript](https://img.shields.io/badge/TypeScript-3178C6?style=
   for-the-badge&logo=typescript&logoColor=white)
```

ロゴ名は *Simple Icons* で検索できます。色は各ブランドの公式カラーを使うのが一般的です。

20.8. Wiki

README はプロジェクトの「入口」ですが、詳細なドキュメントをすべて README に書くと長くなりすぎます。そんなときは **Wiki** を活用しましょう。GitHub の Wiki は、リポジトリに付属するドキュメント用のスペースで、複数ページのドキュメントを整理して書けます。

20.8.1. 有効化

Wiki はリポジトリの設定から有効化します。

1. リポジトリページ → **Settings**
2. 左メニュー → **General**
3. **Features** セクションの **Wikis** にチェックを入れる

スクリーンショット: *Wiki* 有効化の設定画面

20.8.2. クローンと編集

Wiki は Git リポジトリとしても扱えます。ブラウザ上で編集することもできますが、大量の編集をする場合はローカルにクローンして作業する方が効率的です。

```
# Wikiリポジトリをクローン
git clone https://github.com/owner/repo.wiki.git

# 編集作業
cd repo.wiki
# Markdownファイルを作成・編集

# 変更をプッシュ
git add .
git commit -m "Update documentation"
git push
```

Wiki の URL は通常のリポジトリ URL の末尾に `.wiki` を付けたものです。Wiki 内のページは Markdown ファイルとして管理されるため、Git の機能（履歴、差分、ブランチなど）をすべて活用できます。

20.9. よく使うファイル

GitHub のリポジトリには、README 以外にもいくつかの「お約束」のファイルがあります。これらのファイルを適切に配置すると、GitHub が自動的に認識し、適切な場所にリンクを表示して

くれます。

ファイル	配置場所	用途
`README.md`	ルート	プロジェクトの説明
`LICENSE`	ルート	ライセンス（GitHub がバッジで表示）
`CONTRIBUTING.md`	ルートまたは `.github/`	貢献ガイドライン（Issue/PR 作成時にリンク表示）
`CODE_OF_CONDUCT.md`	ルートまたは `.github/`	行動規範
`CHANGELOG.md`	ルート	バージョンごとの変更履歴
`SECURITY.md`	ルートまたは `.github/`	脆弱性報告の方法
`.github/FUNDING.yml`	`.github/`	スポンサーボタンの設定

CONTRIBUTING.md を配置すると、新しい Issue や Pull Request を作成するときに「First time contributing? Read the contributing guidelines」というリンクが GitHub によって自動的に表示されます。コントリビューターに対して、コーディング規約や PR の出し方を事前に伝えることができます。

CODE_OF_CONDUCT.md は行動規範です。コミュニティの参加者に対して、敬意を持ったコミュニケーションを求めるルールを定めます。GitHub はテンプレートも用意しています（リポジトリの「Insights」タブ →「Community Standards」から追加可能）。

CHANGELOG.md は「Keep a Changelog」（https://keepachangelog.com/）の形式で書くのが一般的です。バージョンごとに「Added（追加）」「Changed（変更）」「Fixed（修正）」「Removed（削除）」をまとめます。

SECURITY.md はセキュリティ上の脆弱性を発見した人が、どこに報告すればよいかを案内するファイルです。公開の Issue ではなく、非公開の連絡手段（メールなど）を案内するのが一般的です。詳しくは第 22 章で扱います。

.github/FUNDING.yml を配置すると、リポジトリのトップページに「Sponsor」ボタンが表示されます。GitHub Sponsors、Open Collective、Patreon などのリンクを設定できます。

20.10. ライセンスの選び方

オープンソースプロジェクトにとって、ライセンスは非常に重要です。ライセンスがないリポジトリは、たとえ公開されていても法的には「すべての権利が著者に留保されている」状態であり、他の人が安心してコードを使うことができません。

以下に代表的なオープンソースライセンスを比較します。

20.10.1. 判断基準

とにかく自由に使ってほしい場合: **MIT License** が最も一般的です。短くてシンプルで、「著作権表示を残してくれれば何に使ってもいいですよ」というライセンスです。迷ったら MIT を選んでおけば間違いありません。

ライセンス	商用利用	改変	配布	ソース公開義務	特徴
MIT	可	可	可	なし	最もシンプル。制約が最も少ない
Apache 2.0	可	可	可	なし	特許権の明示的許諾を含む
GPL v3	可	可	可	あり	派生物も同じ GPL で公開する義務
BSD 2-Clause	可	可	可	なし	MIT とほぼ同等。歴史的に使われている
Unlicense	可	可	可	なし	パブリックドメインに近い。制約ゼロ

特許のリスクを考慮したい場合: **Apache License 2.0** は特許権の許諾条項を含んでいるため、企業での利用時に特許訴訟のリスクを軽減できます。大規模なプロジェクトや企業主導のプロジェクトで採用されることが多いライセンスです。

派生物もオープンソースであり続けてほしい場合: **GPL v3** は、コードを使って作られた派生物（改変版やそれを組み込んだソフトウェア）も同じ GPL で公開する義務を課します。このため、企業がコードを取り込んでプロプライエタリなソフトウェアに使うことを防げます。Linux カーネルや GNU ソフトウェアがこのライセンスを採用しています。

完全に自由にしたい場合: **Unlicense** はパブリックドメインに相当するもので、著作権表示すら不要です。ただし、国によってはパブリックドメインの概念が法的に認められていないため、Unlicense はその問題を回避するための法的文書を含んでいます。

GitHub では、リポジトリ作成時にライセンスを選択するオプションがあります。後からでも「Add file」→「Create new file」でファイル名に `LICENSE` と入力すると、テンプレートを選択できる UI が表示されます。

スクリーンショット: GitHub のライセンス選択 UI

20.11. まとめ

この章では、README とプロジェクトドキュメントについて包括的に学びました。

- **README** は 1970 年代から続く伝統的なファイルで、GitHub ではリポジトリのトップページに自動表示される「プロジェクトの顔」
- **最低限の README** でも「プロジェクト名」「使い方」「ライセンス」の 3 つがあれば十分に有用
- **Markdown 記法** は見出し、リスト、テーブル、コードブロックに加え、タスクリスト、折りたたみ、注意/警告、Mermaid 図、数式まで対応
- **ジャンル別テンプレート** を参考に、CLI ツール、Web アプリ、ライブラリ、研究プロジェクトそれぞれに適した構成で README を書く
- **バッジ** は shields.io を使って動的に生成でき、プロジェクトの信頼感を高める
- **Wiki** は README に収まらない詳細ドキュメントを整理するためのスペース

- **LICENSE、CONTRIBUTING、CODE_OF_CONDUCT、SECURITY** などの定番ファイルは GitHub が自動認識して適切にリンクしてくれる
- **ライセンス選び** は迷ったら MIT。特許が気になるなら Apache 2.0、コピーレフトを求めるなら GPL v3

カスタマイズ — GitHubを自分好みに整える

21.1. この章で学ぶこと

道具は使いやすいように調整してこそ、その真価を発揮します。大工が鉋（かんな）の刃を研ぎ、料理人が包丁の柄を自分の手に馴染ませるように、開発者も Git や GitHub を自分の作業スタイルに合わせてカスタマイズすべきです。

この章では、GitHub と Git をより快適に使うためのカスタマイズ方法を学びます。

- GitHub のテーマ設定（ダークモード/ライトモード）
- キーボードショートカットの活用
- GitHub CLI（gh）のエイリアス設定
- Git コマンドのエイリアスと `.gitconfig` の書き方
- 通知の最適化 — 情報の洪水に飲まれないために
- GitHub Mobile アプリの活用
- Saved Replies（保存済みの返信）による効率化

21.2. テーマ設定 — ダークモード/ライトモード

長時間コードを読む開発者にとって、画面の明るさは目の疲れに直結します。GitHub では複数のテーマが用意されており、好みや環境に合わせて選択できます。

設定方法は以下の通りです。

1. 右上のプロフィールアイコンをクリック
2. **Settings** を選択
3. 左メニューの **Appearance** をクリック
4. 好みのテーマを選択

スクリーンショット: テーマ設定画面（*Appearance*）

選択できるテーマは以下の通りです。

「System に同期」を選ぶと、macOS や Windows のシステム設定でダークモード/ライトモードを切り替えたとき、GitHub のテーマも自動的に切り替わります。日中はライトモード、夜間はダークモードを使い分けたい場合に便利です。

テーマ	特徴
Light default	標準的な白背景。明るい環境での作業に適している
Light high contrast	白背景だがコントラストが高い。視認性を重視
Dark default	暗い背景。夜間の作業や暗い部屋での使用に適している
Dark high contrast	暗い背景でコントラストが高い
Dark dimmed	やや明るめのダークテーマ。目に優しい
System に同期	OS のダークモード設定に自動追従

21.3. キーボードショートカット

GitHub の Web 画面には多数のキーボードショートカットが用意されています。マウス操作を減らすことで、ページ間の移動やファイルの検索が格段に速くなります。まず覚えるべきは `?` **キー** です。どのページにいても `?` を押すと、そのページで使えるショートカットの一覧が表示されます。

スクリーンショット: ?キーで表示されるショートカット一覧

21.3.1. グローバルショートカット（どのページでも使える）

ショートカット	機能
`?`	ショートカット一覧を表示
`s` または `/`	検索バーにフォーカスを移動
`g` → `n`	通知ページに移動
`g` → `d`	ダッシュボード（ホーム画面）に移動

`g` → `n` のように 2 つのキーを続けて押す操作は「コードシーケンス」と呼ばれます。最初の `g` を押した後、一定時間内に 2 つ目のキーを押す必要があります。`g` は ”go to”（移動する）の略と考えると覚えやすいでしょう。

21.3.2. リポジトリ内ショートカット

リポジトリのページでは、各タブへの素早い移動が可能です。

21.3.3. ファイル表示時のショートカット

21.3.4. Pull Request と Issue のショートカット

特に `Cmd/Ctrl + Shift + p` は、コメントを書いている途中で Markdown がどのように表示されるかを確認するのに便利です。

ショートカット	機能
t	ファイルファインダー（ファジー検索）を開く
w	ブランチ切り替えメニューを開く
g → c	Code タブに移動
g → i	Issues タブに移動
g → p	Pull requests タブに移動
g → a	Actions タブに移動
g → b	Projects タブに移動
g → w	Wiki タブに移動

ショートカット	機能
y	パーマリンク（コミットハッシュ指定 URL）に変換
l	行番号を入力してジャンプ
b	blame ビュー（各行の変更者表示）に切り替え
e	ファイルを編集モードで開く

21.4. GitHub CLI (gh) のカスタマイズ — エイリアスの設定と活用例

GitHub CLI（`gh`）は第 4 章で紹介した通り、ターミナルから GitHub の操作ができるツールです。よく使うコマンドにはエイリアス（短縮名）を設定しておくと、タイプ量が減って作業が効率化されます。

21.4.1. エイリアスの設定

```
# 基本的なエイリアス
gh alias set prc 'pr create'            # PR作成
gh alias set prv 'pr view --web'        # PRをブラウザで開く
gh alias set prl 'pr list'              # PR一覧
gh alias set il 'issue list'             # Issue一覧
gh alias set ic 'issue create'           # Issue作成

# 自分に関連するものだけ表示するエイリアス
gh alias set my-prs 'pr list --author @me'
gh alias set my-issues 'issue list --assignee @me'

# 現在のブランチのPRをチェックアウト
```

ショートカット	機能
`r`	選択テキストを引用して返信
`Cmd/Ctrl + Enter`	コメントを送信
`Cmd/Ctrl + Shift + p`	Markdown プレビューの切り替え

```
gh alias set prco 'pr checkout'

# 設定済みのエイリアス一覧を確認
gh alias list
```

設定したエイリアスは以下のように使います。

```
# gh pr create の代わりに
gh prc --title "Add login feature"

# 自分が出したPRの一覧
gh my-prs

# 自分にアサインされたIssue一覧
gh my-issues
```

21.4.2. gh の設定

エイリアス以外にも、`gh config` コマンドでデフォルトの挙動をカスタマイズできます。

```
# デフォルトエディタをVS Codeに設定
gh config set editor "code --wait"

# デフォルトブラウザを指定
gh config set browser "firefox"

# 対話的プロンプトを有効化（確認ダイアログを表示する）
gh config set prompt enabled

# 現在の設定を一覧表示
gh config list
```

21.5. Gitのカスタマイズ — エイリアスと.gitconfig

Git 自体にもエイリアス機能があります。`git status` を毎日何十回も打つ開発者にとって、`git st` と短縮できるだけでも積み重なると大きな時間の節約になります。

```
# 基本コマンドの短縮
git config --global alias.st status
git config --global alias.co checkout
git config --global alias.br branch
git config --global alias.ci commit
git config --global alias.sw switch

# ログ表示（見やすいフォーマット）
git config --global alias.lg "log --oneline --graph --decorate --
   all"
git config --global alias.ll "log --oneline -20"

# 差分の確認
git config --global alias.df diff
git config --global alias.ds "diff --staged"

# 直前のコミットにファイルを追加（メッセージは変えない）
git config --global alias.amend "commit --amend --no-edit"

# マージ済みブランチの一括削除
git config --global alias.cleanup "!git branch --merged main | grep
    -v main | xargs -n 1 git branch -d"
```

設定したエイリアスは即座に使えます。

```
git st          # git status と同じ
git co main     # git checkout main と同じ
git lg          # きれいなグラフ付きログ
git ds          # ステージ済みの差分を確認
git amend       # 直前のコミットを修正
```

`git lg` は特におすすめのエイリアスです。ブランチの分岐と合流がグラフで視覚的に表示されるため、リポジトリの状態を直感的に把握できます。

21.5.2. .gitconfig の直接編集

エイリアスを一つずつ `git config` コマンドで設定するのが面倒な場合は、設定ファイルを直接編集することもできます。Git のグローバル設定は `~/.gitconfig`（ホームディレクトリ直下の `.gitconfig`）に保存されています。

```
[user]
    name = Your Name
    email = your@email.com

[core]
    editor = code --wait
    autocrlf = input

[init]
    defaultBranch = main

[push]
    default = current

[pull]
    rebase = true

[alias]
    st = status
    co = checkout
    br = branch
    ci = commit
    sw = switch
    lg = log --oneline --graph --decorate --all
    ll = log --oneline -20
    df = diff
    ds = diff --staged
    amend = commit --amend --no-edit

[color]
    ui = auto

[diff]
    tool = vscode

[difftool "vscode"]
    cmd = code --wait --diff $LOCAL $REMOTE
```

```
[merge]
    tool = vscode

[mergetool "vscode"]
    cmd = code --wait $MERGED
```

いくつかの設定項目を補足します。

- `push.default = current`: `git push` の際にブランチ名を省略すると、現在のブランチと同名のリモートブランチにプッシュする設定です。ブランチ名を毎回指定する手間が省けます。
- `pull.rebase = true`: `git pull` の際にマージコミットを作らず、リベースで履歴を直線的に保つ設定です。履歴をきれいに保ちたい場合に便利です。
- `init.defaultBranch = main`: `git init` で新しいリポジトリを作るときのデフォルトブランチ名を `main` にする設定です。
- `color.ui = auto`: ターミナルが対応していれば、diff や status の出力に色を付ける設定です。

21.6. 通知のカスタマイズ

GitHub の通知は、設定を適切に行わないと大量のメールやアラートに埋もれてしまいます。「本当に重要な通知だけ受け取る」ように最適化しましょう。

21.6.1. 通知設定の場所

Settings → **Notifications** で、通知の受け取り方を細かく設定できます。

```
Participating（参加中の会話）:
  Web通知:  有効にする
  メール:   有効にする
  → 自分が関わるIssue/PRの更新は確実に受け取りたい

Watching（ウォッチ中のリポジトリ）:
  Web通知:  有効にする
  メール:   無効にする（ノイズが多くなるため）
  → WebのNotificationsページで確認すれば十分

GitHub Actions:
  失敗時のみ通知にする
  → 成功したワークフローの通知は不要
```

スクリーンショット: 通知設定画面

21.6.2. リポジトリごとの通知設定

リポジトリのページ上部にある「**Watch**」ボタンで、リポジトリ単位の通知レベルを設定できます。

設定	通知対象
Participating and @mentions	自分が参加中の会話と、@メンションされた場合のみ
All Activity	リポジトリ内のすべての活動（Issue 作成、PR 作成、コメントなど）
Ignore	このリポジトリからの通知を完全に無視
Custom	Issue、PR、Release、Discussion などから受け取りたいものを選択

おすすめの設定方針は以下の通りです。

- **自分のリポジトリ**: All Activity（すべてを把握したい）
- **チームのリポジトリ**: Participating and @mentions（関係するものだけ）
- **参考にしている OSS リポジトリ**: Custom → Releases only（新バージョンだけ知りたい）
- **もう関わらないリポジトリ**: Ignore

通知が溜まりすぎたときは、GitHub の通知ページ（https://github.com/notifications）で「Mark all as read」を使って一括既読にできます。未読の通知を溜め込んでも何も良いことはありません。定期的にトリアージ（仕分け）する習慣をつけましょう。

21.7. GitHub Mobile

GitHub Mobile は、iOS/Android に対応した公式アプリです。外出先でもスマートフォンから GitHub の主要な操作を行えます。

GitHub Mobile でできる主な操作は以下の通りです。

- **Issue / PR の確認とコメント**: 通勤中や移動中に Issue の内容を確認し、コメントを返すことができます
- **コードの閲覧**: ファイルの中身を読むことができます（ただし編集は Web 版の方が快適です）
- **通知の管理**: プッシュ通知でリアルタイムに更新を受け取り、既読/未読の管理ができます
- **マージの承認**: Pull Request のレビューを行い、Approve やマージができます
- **Discussions への参加**: ディスカッションの閲覧と投稿

特に便利なのは通知機能です。重要な Pull Request へのコメントや、CI の失敗通知をリアルタイムで受け取れるため、すぐに対応が必要な問題を見逃しにくくなります。

アプリのインストールは、App Store（iOS）または Google Play Store（Android）から「GitHub」を検索してください。ログインには Web ブラウザ経由の認証が使われ、2FA にも対応しています。

21.8. Saved Replies

Issue や Pull Request のレビューでは、同じような内容のコメントを繰り返し書くことがあります。例えば、「LGTM（Looks Good To Me）」や「テストを追加してください」といったメッセージです。こうした定型文を毎回手入力するのは非効率です。

Saved Replies（保存済みの返信）は、よく使うコメントをテンプレートとして保存しておき、ワンクリックで挿入できる機能です。

21.8.1. 設定方法

1. **Settings** → **Saved replies** を開く
2. テンプレートのタイトルと本文を入力して保存

21.8.2. 便利なテンプレート例

```
タイトル: LGTM
本文:
LGTM! Thank you for the contribution.

タイトル: Needs tests
本文:
Could you add tests for the new functionality?
We want to make sure this doesn't regress in the future.

タイトル: Duplicate
本文:
This appears to be a duplicate of #___. Closing in favor of the
   original issue.
Please add any additional context to the original issue.

タイトル: Stale
本文:
This issue has been inactive for a while. If the problem still
   exists,
please provide updated information. Otherwise, this will be closed.
```

Saved Replies を使うには、Issue や PR のコメント欄の右上にあるメニューアイコンから選択するか、`Ctrl + .`（macOS では `Cmd + .`）のショートカットで挿入できます。

チーム全体で同じ Saved Replies を使うことはできません（ユーザーごとの設定です）が、チーム内で「こういうテンプレートを使おう」というガイドラインを共有しておくと、コミュニケーションの一貫性が保たれます。

21.9. まとめ

この章では、GitHub と Git を自分好みにカスタマイズする方法を学びました。

- **テーマ設定**: ダークモード、ライトモード、システム同期などから選べる。Appearance 設定で変更
- **キーボードショートカット**: `?` キーで一覧表示。`t`（ファイル検索）、`g` → `i`（Issues 移動）、`y`（パーマリンク変換）などが特に便利
- **GitHub CLI エイリアス**: `gh alias set` で頻用コマンドを短縮。`gh prc` で PR 作成、`gh my-prs` で自分の PR 一覧など
- **Git エイリアス**: `git config --global alias.st status` のように設定。`~/.gitconfig` を直接編集すると一括設定が楽
- **通知のカスタマイズ**: Participating は Web+メール、Watching は Web のみ、Actions は失敗時のみが推奨設定。リポジトリごとの Watch 設定で粒度を調整
- **GitHub Mobile**: iOS/Android 対応。通知のリアルタイム受信と Issue/PR へのコメントが外出先でも可能
- **Saved Replies**: 定型コメントをテンプレート化。レビューの効率とコミュニケーションの一貫性が向上

カスタマイズに「正解」はありません。自分の作業スタイルに合わせて少しずつ調整し、快適な開発環境を作り上げていきましょう。

セキュリティ — リポジトリを安全に保つ

22.1. この章で学ぶこと

ソフトウェア開発において、セキュリティは「後から追加する機能」ではなく、最初から意識すべき基本姿勢です。特に GitHub のようなオンラインプラットフォームを使う場合、一度の不注意がパスワードや API キーの漏洩につながり、深刻な被害を引き起こす可能性があります。

この章では、以下の内容を学びます。

- 絶対にリポジトリにコミットしてはならない情報の種類と、なぜそれが危険なのか
- 万が一、秘密情報をコミットしてしまった場合の緊急対応手順
- Dependabot による依存パッケージの脆弱性自動検出と更新
- Secret Scanning によるリポジトリ内の秘密情報の自動スキャン
- SECURITY.md によるセキュリティポリシーの明文化
- 日常的に実践すべきセキュリティのベストプラクティス 7 箇条

セキュリティは地味な作業に見えるかもしれませんが、「事故が起きてから対処する」のと「事前に防ぐ」のとでは、必要な労力も被害の規模も桁違いです。この章を通じて、安全な開発習慣を身につけましょう。

22.2. 絶対にコミットしてはいけないもの

22.2.1. なぜ秘密情報のコミットが危険なのか

GitHub のリポジトリ、特に Public リポジトリにコミットされた情報は、世界中の誰でも閲覧できます。しかし、問題はそれだけではありません。**Git の性質上、一度コミットした情報は、たとえ後から削除しても履歴に残り続ける**のです。

たとえば、`.env` ファイルに書かれた API キーを誤ってコミットしてしまったとします。気づいて慌ててファイルを削除し、新しいコミットを作ったとしても、古いコミットの中には API キーがそのまま保存されています。`git log` で過去のコミットを辿り、`git show` でその時点のファイル内容を表示すれば、削除したはずの API キーを誰でも復元できてしまうのです。

実際に、GitHub の Public リポジトリを自動的にスキャンして、漏洩した認証情報を収集する悪意あるボットが常時稼働していることが知られています。API キーがコミットされてから数分以内に悪用されたという報告もあります。

22.2.2. コミットしてはならない情報の一覧

以下に挙げるような情報は、いかなる場合もリポジトリにコミットしてはなりません。

これらの情報を安全に管理するためには、`.env` ファイルに秘密情報を記述し、`.gitignore` でそのファイルを Git の追跡対象から除外するのが基本的なアプローチです。CI/CD 環境では、第 12 章で学んだ GitHub Secrets を使って秘密情報を安全に渡します。

カテゴリ	具体例	ファイル名の例
環境変数ファイル	データベース URL、API キー、シークレットキー	`.env`, `.env.local`,
秘密鍵	SSH 秘密鍵、TLS 証明書の秘密鍵	`id_rsa`, `id_ed25519`
認証情報	パスワード、トークン、アクセスキー	`credentials.json`, s
クラウド認証	AWS/GCP/Azure の認証情報	`aws-credentials`, se
データベース接続情報	ホスト名、ユーザー名、パスワードが含まれる接続文字列	`database.yml` （パス

代わりに、`.env.example` というファイルを用意して、秘密情報の「形式」だけを示しておくと、新しい開発メンバーが「どんな環境変数を設定すべきか」を理解できます。

```
# .env.example（これはコミットしてOK）
DATABASE_URL=postgres://user:password@localhost:5432/mydb
API_KEY=your-api-key-here
SECRET_KEY=generate-a-random-string
```

```
# .gitignore に追加
.env
.env.local
.env.production
*.pem
*.key
```

22.3. 万が一コミットしてしまったときの対処

22.3.1. 最優先事項：秘密情報を無効化する

秘密情報のコミットに気づいたとき、最も重要なのは**そのシークレットを直ちに無効化する**ことです。リポジトリの履歴からファイルを削除するよりも先に、漏洩した可能性のあるキーやパスワードを使えない状態にしなければなりません。

具体的には以下の手順で対応します。

1. **漏洩した秘密情報を即座に無効化する**：該当するサービス（AWS、Google Cloud、Slack、データベース等）の管理画面にアクセスし、漏洩したキーやトークンを失効させます

2. **新しい秘密情報を発行する**：新しい API キーやパスワードを生成し、`.env` ファイルなどの安全な場所に設定し直します

3. **リポジトリの履歴から秘密情報を完全に削除する**

22.3.2. 履歴からの完全削除

ファイルを `git rm` で削除してコミットしても、過去のコミットの中に秘密情報は残ったままです。履歴から完全に削除するには、特別なツールを使う必要があります。

方法 1：BFG Repo-Cleaner（推奨）

BFG Repo-Cleaner は、Git リポジトリの履歴を高速に書き換えるためのツールです。`git filter-branch` よりも 10〜720 倍速く、使い方もシンプルです。

```
# BFGのインストール（macOS）
brew install bfg

# 特定のファイルをすべての履歴から削除
bfg --delete-files .env

# 特定の文字列（例：APIキー）を履歴からすべて置換
bfg --replace-text passwords.txt
# passwords.txt には置換対象の文字列を1行ずつ記載

# 履歴の再構築
git reflog expire --expire=now --all
git gc --prune=now --aggressive

# リモートに強制プッシュ（この操作は破壊的なため慎重に）
git push --force
```

方法 2：git filter-branch

BFG が使えない場合は、Git 標準の `filter-branch` コマンドを使います。

```
git filter-branch --force --index-filter \
  'git rm --cached --ignore-unmatch .env' \
  --prune-empty --tag-name-filter cat -- --all

git reflog expire --expire=now --all
git gc --prune=now --aggressive
git push --force
```

いずれの方法も**リポジトリの履歴を書き換える破壊的な操作**です。チームで作業している場合は、全メンバーにリポジトリを改めてクローンし直してもらう必要があります。だからこそ、「最初からコミットしない」ことが最善の対策なのです。

22.4. Dependabot — 依存パッケージの脆弱性を自動検出する

22.4.1. なぜ依存パッケージの管理が重要なのか

現代のソフトウェアは、数十から数百のオープンソースライブラリ（依存パッケージ）の上に成り立っています。あなたのプロジェクトが直接使っているライブラリだけでなく、そのライブラリがさらに依存しているライブラリまで含めると、依存関係は非常に深くなります。

これらのライブラリのいずれかに脆弱性が発見された場合、あなたのプロジェクトも影響を受ける可能性があります。しかし、何百もの依存パッケージの脆弱性情報を手動で追跡するのは現実的ではありません。ここで Dependabot が力を発揮します。

22.4.2. Dependabot とは

Dependabot は、GitHub が提供する依存パッケージの自動監視・更新サービスです。Dependabot は以下の 3 つの機能を持っています。

Dependency graph（依存グラフ）は、リポジトリが依存しているパッケージの一覧を自動的に解析して可視化します。`package.json`（Node.js）、`requirements.txt`（Python）、`Gemfile`（Ruby）などのファイルから依存関係を読み取ります。

Dependabot alerts（セキュリティアラート）は、依存パッケージに既知の脆弱性が発見された場合に自動的に通知してくれます。GitHub の脆弱性データベース（GitHub Advisory Database）と照合して、影響のあるパッケージを特定します。

Dependabot security updates（セキュリティ更新）は、脆弱性が発見されたパッケージを安全なバージョンに更新する Pull Request を自動的に作成してくれます。開発者はその PR をレビューしてマージするだけで、脆弱性への対応が完了します。

22.4.3. 有効化の手順

1. リポジトリの **Settings** タブを開きます
2. 左メニューの **Code security and analysis** を選択します
3. 以下の 3 つを順番に有効化します
   - **Dependency graph**: 有効
   - **Dependabot alerts**: 有効
   - **Dependabot security updates**: 有効

スクリーンショット: *Code security and analysis* 設定画面で *Dependabot* の各機能を有効化する

22.4.4. 設定ファイルによるカスタマイズ

Dependabot の動作をより細かく制御するには、`.github/dependabot.yml` ファイルをリポジトリに配置します。

```
version: 2
updates:
  # Pythonの依存パッケージを毎週チェック
  - package-ecosystem: "pip"
    directory: "/"
    schedule:
      interval: "weekly"
      day: "monday"
      time: "09:00"
      timezone: "Asia/Tokyo"
    labels:
      - "dependencies"
      - "python"
    reviewers:
      - "username"
    commit-message:
      prefix: "chore(deps)"

  # Node.jsの依存パッケージ
  - package-ecosystem: "npm"
    directory: "/"
    schedule:
      interval: "weekly"

  # GitHub Actionsのアクション
  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
```

`schedule` セクションでは、Dependabot がチェックを行う頻度を `daily`（毎日）、`weekly`（毎週）、`monthly`（毎月）から選択できます。プロジェクトの重要度に応じて適切な頻度を設定しましょう。`labels` や `reviewers` を指定しておくと、自動生成される PR が整理しやすくなります。

22.5. Secret Scanning — リポジトリ内の秘密情報を自動検出する

22.5.1. Secret Scanningとは

Secret Scanning は、リポジトリにコミットされたコードの中に、API キーやトークンなどの秘密情報が含まれていないかを自動的にスキャンする機能です。22.2 節で「コミットしてはいけない」と書きましたが、人間はミスをするものです。Secret Scanning は、そのミスが被害に発展する前に検出してくれるセーフティネットです。

22.5.2. 検出できる秘密情報の種類

Secret Scanning が検出できるパターンは、GitHub のパートナー企業と連携して随時追加されています。代表的なものを挙げます。

- **GitHub** — Personal Access Token、OAuth Token
- **AWS** — Access Key ID、Secret Access Key
- **Google Cloud** — API Key、Service Account Key
- **Slack** — Bot Token、Webhook URL
- **Stripe** — API Key（テスト用/本番用）
- **npm** — Access Token
- **Azure** — Storage Account Key、Service Principal
- その他、100 種類以上のパターンに対応

22.5.3. 有効化と動作

Secret Scanning は、リポジトリの Settings → **Code security and analysis** → **Secret scanning** で有効化します。パブリックリポジトリでは一部の機能が自動的に有効になっています。

秘密情報が検出されると、リポジトリの管理者にアラートが送信されます。GitHub のパートナー企業（AWS、Google 等）に対しては、検出された時点で自動的にパートナー企業にも通知が行き、そのトークンが自動的に無効化されることもあります。

22.5.4. Push protection

Secret Scanning には「**Push protection**」という強力な機能もあります。これを有効にすると、秘密情報が含まれるコミットを**プッシュする前にブロック**してくれます。つまり、秘密情報がリポジトリに入り込む前の段階で防御できるのです。

Settings → Code security and analysis → Secret scanning → **Push protection** を有効にすることで利用できます。

22.6. セキュリティポリシー — SECURITY.mdの作成

22.6.1. なぜセキュリティポリシーが必要か

あなたのプロジェクトを使っている人が脆弱性を発見した場合、どこに報告すればよいのでしょうか。公開のIssueに書かれてしまうと、修正が行われる前に悪意ある第三者に知られてしまう可能性があります。

`SECURITY.md` ファイルは、脆弱性を発見した人に対して「どのように報告してほしいか」を伝えるための文書です。リポジトリのルートまたは `.github/` ディレクトリに配置すると、GitHubのUIに「Security policy」として表示されます。

22.6.2. SECURITY.mdの書き方

```
# Security Policy

## サポート対象バージョン

| バージョン | サポート状況 |
| --------- | ---------- |
| 2.x       | サポート中   |
| 1.x       | サポート終了 |

## 脆弱性の報告方法

脆弱性を発見された場合は、以下の方法でご報告ください。

**公開のIssueでの報告はお控えください。**
脆弱性の詳細が公開されると、修正が完了する前に悪用されるリスクがあ
   ります。

### 報告先

- メール: security@example.com
- GitHub Security Advisories: リポジトリの「Security」タブから
  「Report a vulnerability」をクリック

### 報告に含めてほしい情報

- 脆弱性の概要
- 再現手順
- 影響範囲
```

```
- 可能であれば修正案

### 対応の流れ

1. 報告を受領後、48時間以内に確認のご連絡をします
2. 脆弱性の検証と影響評価を行います
3. 修正パッチを作成します
4. 修正をリリースし、報告者にクレジットを記載します
```

GitHub には「Security Advisories」という機能もあり、非公開で脆弱性の報告・議論・修正を行い、準備が整ってから公開するというワークフローをサポートしています。

22.7. セキュリティのベストプラクティス7箇条

日々の開発で実践すべきセキュリティ対策を 7 つにまとめました。

22.7.1. 第1条：二段階認証（2FA）を必ず有効にする

第 3 章で解説した二段階認証は、アカウントセキュリティの最も基本的かつ効果的な対策です。パスワードが漏洩しても、2FA が有効であれば不正ログインを防げます。GitHub では 2023 年以降 2FA が必須化されていますが、認証アプリの設定とリカバリーコードの保管を確実に行いましょう。

22.7.2. 第2条：.gitignoreを徹底する

`.env` ファイルや認証情報ファイルは、プロジェクトの最初の段階で `.gitignore` に追加しておきましょう。「後で追加すればいい」と思っていると、つい忘れてコミットしてしまいます。第 5 章で紹介した言語別の `.gitignore` テンプレートを活用するのが手軽です。

22.7.3. 第3条：GitHub Secretsで秘密情報を管理する

CI/CD パイプラインで API キーやパスワードが必要な場合は、コードに直接書くのではなく、GitHub Secrets に保存して `${{ secrets.API_KEY }}` の形式で参照します。Secrets の値はログに出力されないよう保護されています。

22.7.4. 第4条：最小権限の原則を守る

Personal Access Token（PAT）を作成する際は、必要最小限のスコープ（権限）だけを付与してください。「全部チェックしておけば楽だから」という考えは、トークンが漏洩した際の被害を最大化します。有効期限も設定し、定期的にトークンを更新する習慣をつけましょう。

22.7.5. 第5条：Dependabotのアラートに迅速に対応する

Dependabotから脆弱性のアラートが届いたら、放置せずに対応しましょう。特に「Critical」や「High」の深刻度のアラートは、攻撃者に悪用される可能性が高いため、速やかにPRをマージしてパッケージを更新してください。

22.7.6. 第6条：ブランチ保護ルールを設定する

mainブランチへの直接pushを禁止し、Pull Requestとコードレビューを必須にすることで、意図しない変更や秘密情報の混入を防ぎます。第17章で解説したブランチ保護ルールを設定しましょう。

22.7.7. 第7条：定期的にアクセス権限を見直す

プロジェクトを離れたメンバーのCollaborator権限が残っていないか、不要になったGitHub Appがインストールされたままになっていないか、PATの有効期限が切れていないか——こうした点を定期的に確認する習慣をつけましょう。Settings → Collaborators、Settings → Applications、Settings → Developer settingsを定期的にチェックします。

22.8. まとめ

本章では、GitHubリポジトリを安全に保つためのセキュリティ対策について学びました。

- **秘密情報は絶対にコミットしない**。一度コミットすると履歴に残り続け、削除は非常に困難。`.gitignore`による事前防止が最善策
- 万が一コミットしてしまった場合は、まず**秘密情報を無効化**し、BFG Repo-Cleanerなどで**履歴から完全削除**する
- **Dependabot**は依存パッケージの脆弱性を自動検出し、更新PRを作成してくれる
- **Secret Scanning**はリポジトリ内の秘密情報を自動スキャンし、Push protectionでコミット前にブロックもできる
- **SECURITY.md**でセキュリティポリシーを明文化し、脆弱性の報告先と対応フローを案内する
- **7つのベストプラクティス**（2FA、.gitignore、Secrets、最小権限、Dependabot対応、ブランチ保護、権限見直し）を日常的に実践する

セキュリティは一度設定すれば終わりではなく、継続的に注意を払うべきものです。この章で学んだ対策を習慣化することで、安心して開発に集中できる環境を維持しましょう。

外部連携 — 開発ツールとつなげる

23.1. この章で学ぶこと

GitHub は単体でも非常に強力なプラットフォームですが、その真の力は他のツールやサービスと連携したときに発揮されます。現代のソフトウェア開発では、コードエディタ、チャットツール、CI/CD サービス、プロジェクト管理ツールなど、数多くのツールが日常的に使われています。これらのツールと GitHub をつなげることで、開発のワークフロー全体がシームレスにつながり、生産性が飛躍的に向上します。

この章では、以下の内容を学びます。

- VS Code との連携によって、エディタを離れずに Git 操作や PR レビューを行う方法
- Claude Code との連携によって、AI の支援を受けながら GitHub 上の開発を加速する方法
- GitHub Apps や OAuth Apps を使って、外部サービスをリポジトリに統合する方法
- Slack との連携によって、チームの通知をリアルタイムに受け取る方法
- Webhook の仕組みを理解し、独自の自動化を構築する方法
- GitHub API を使ってプログラムから GitHub を操作する方法
- Gist を使ってコードスニペットを手軽に共有する方法

開発ツールを「点」として個別に使うのではなく、GitHub を中心として「線」でつなげることで、チーム全体の開発体験が大きく変わります。それでは、最も身近なツールである VS Code との連携から見ていきましょう。

23.2. VS Code との連携

23.2.1. なぜ VS Code と GitHub の連携が重要なのか

Visual Studio Code（VS Code）は、Microsoft が開発した無料のコードエディタであり、現在世界で最も広く使われている開発環境のひとつです。GitHub もまた Microsoft 傘下のサービスであるため、両者の連携は非常に密接に設計されています。コードを書いている最中にブラウザに切り替えて PR を確認したり、ターミナルに移動して Git コマンドを打ったりする必要がなく、VS Code の中だけで開発からレビュー、コミット、プッシュまでの一連の作業を完結させることができます。

23.2.2. 拡張機能一覧と用途

VS Code の強みのひとつは、拡張機能（Extension）によって機能を自由に追加できることです。GitHub 関連の拡張機能は数多く存在しますが、特に重要なものを以下に紹介します。

拡張機能のインストールは、VS Code の左サイドバーにある四角いアイコン（Extensions）をクリックし、検索バーに拡張機能名を入力して「Install」ボタンを押すだけです。

拡張機能	用途
GitHub Pull Requests and Issues	VS Code 内で PR の作成・レビュー・マージ、Issue の閲覧・作成が
GitLens	各行の最終変更者・日時を表示し、コミット履歴やブランチ比較を
GitHub Copilot	AI によるコード補完。コメントや文脈からコードを自動生成する
GitHub Copilot Chat	AI との対話形式でコーディングの相談やコード生成ができる
GitHub Actions	ワークフローの状態確認や管理を VS Code 内で行える
Git Graph	ブランチの分岐・マージの履歴をグラフとして視覚的に表示する

23.2.3. ソース管理パネルの使い方

VS Code には、拡張機能を一切追加しなくても最初から Git 連携機能が組み込まれています。左サイドバーの分岐アイコン、またはキーボードショートカット `Cmd+Shift+G`（macOS）/ `Ctrl+Shift+G`（Windows/Linux）を押すと、**ソース管理パネル**が表示されます。

スクリーンショット: *VS Code* のソース管理パネル全体像

ソース管理パネルでは、以下の操作を GUI で行うことができます。

- **変更ファイルの確認**: 変更・追加・削除されたファイルが一覧表示される。ファイルをクリックすると左右分割の差分表示（diff）が開き、何がどう変わったかを視覚的に確認できる
- **ステージング**: 各ファイルの横にある `+` ボタンをクリックすることで、`git add` に相当する操作ができる。個別のファイルだけをステージすることも、全ファイルをまとめてステージすることもできる
- **コミット**: パネル上部のテキストボックスにコミットメッセージを入力し、チェックマークボタン（または Cmd+Enter）を押すとコミットが作成される
- **Push / Pull / Sync**: パネル下部やステータスバーの同期ボタンで、リモートリポジトリとの同期が行える

また、ステータスバー（画面最下部）にはブランチ名が表示されており、これをクリックすることでブランチの切り替えや新規作成ができます。ブランチ名の横に表示される矢印と数字（例: ↑1 ↓2）は、ローカルとリモートの差分を示しており、ローカルに未プッシュのコミットが 1 つ、リモートに未プルのコミットが 2 つあることを意味します。

23.2.4. インライン blame

GitLens 拡張機能をインストールすると、エディタ内の各行の右端に、その行を最後に変更した人の名前・日時・コミットメッセージが薄いグレーの文字で表示されるようになります。これを**インライン blame** と呼びます。

たとえば、あるコードの意図がわからないとき、インライン blame を見れば「誰が」「いつ」「なぜ」その行を変更したかがすぐにわかります。表示されている情報をクリックすると、そのコミットの詳細（変更の全体像、コミットメッセージの全文など）を確認できます。これはコードレビューや

バグ調査のときに非常に役立つ機能です。

23.2.5. github.dev — ブラウザ版VS Code

GitHub リポジトリのページを開いているとき、キーボードの `.`（ピリオド）キーを押すと、ブラウザ上で VS Code が起動します。これが **github.dev** です。URL で表現すると、以下のように `github.com` を `github.dev` に置き換えるだけで同じことが起こります。

```
https://github.com/owner/repo → https://github.dev/owner/repo
```

github.dev はブラウザ上で動作するため、何もインストールする必要がありません。軽い編集やコードレビューに非常に便利です。ただし、ターミナルは使えず、ビルドやテストの実行はできません。本格的な開発には Codespaces（クラウド上の完全な開発環境）が適していますが、ちょっとした typo の修正やドキュメントの編集には github.dev で十分です。

スクリーンショット: *github.dev* の画面（ブラウザ内に *VS Code* が表示されている様子）

23.3. Claude Codeとの連携

23.3.1. Claude Codeとは何か

Claude Code は、Anthropic 社が開発した AI コーディングアシスタントです。ターミナルや VS Code の中で動作し、リポジトリのコードを理解した上で、コードの読み書き、テストの実行、コミットの作成、PR の作成といった開発作業を対話的に支援してくれます。GitHub のリポジトリと組み合わせて使うことで、開発の生産性を大きく向上させることができます。

23.3.2. インストールと認証

Claude Code は npm パッケージとして配布されています。Node.js がインストールされている環境であれば、以下のコマンドで簡単にインストールできます。

```
# Claude Codeのインストール
npm install -g @anthropic-ai/claude-code

# Claude Codeを起動（初回は認証画面がブラウザで開く）
claude
```

初回起動時にはブラウザが開き、Anthropic アカウントでの認証が求められます。認証が完了すると、ターミナル上で Claude Code との対話が始まります。

23.3.3. GitHub リポジトリでの使用方法

Claude Code は、起動したディレクトリのコンテキスト（ファイル構成、コードの内容、Git 履歴など）を理解します。そのため、GitHub リポジトリのローカルクローン内で Claude Code を起動すると、そのプロジェクトに特化した支援を受けることができます。

```
# Gitリポジトリに移動してClaude Codeを起動
cd my-project
claude

# リポジトリのコンテキストを理解した上でタスクを依頼できる
claude "このリポジトリのREADMEを改善して"
claude "テストカバレッジを向上させて"
claude "このバグを調査して修正して"
```

Claude Code はリポジトリ内のファイルを読み、必要に応じてファイルを作成・編集し、コマンドを実行することができます。たとえば「テストを書いて」と依頼すると、既存のコードを分析し、適切なテストファイルを生成し、テストが通ることを確認するところまで自動的に行ってくれます。

23.3.4. Git 操作の対話的な依頼

Claude Code の大きな特徴は、Git 操作を自然言語で依頼できることです。コミットメッセージを考えたり、複雑な Git コマンドを覚えたりする必要がありません。

```
# Claude Codeにコミットを依頼
> この変更をコミットして

# PRの作成を依頼
> この変更でPRを作って、説明文も書いて

# ブランチ操作を依頼
> 新しいブランチを作ってこの機能を実装して
```

Claude Code は変更の内容を分析して適切なコミットメッセージを自動生成し、PR の説明文もコードの変更内容に基づいて作成します。特に PR の説明文は、変更の「何を」「なぜ」行ったかを的確にまとめてくれるため、レビュアーにとっても理解しやすいものになります。

また、リポジトリに `CLAUDE.md` というファイルを配置しておくと、Claude Code がそのファイルの指示に従って動作します。プロジェクト固有のコーディング規約やテスト方針を記述しておけば、それに準拠したコードを生成してくれます。

23.3.5. VS Code拡張としての使用

Claude CodeはCLI（コマンドライン）だけでなく、VS Codeの拡張機能としても使用できます。VS Code内のサイドパネルにClaude Codeのチャットインターフェースが表示され、コードを編集しながらAIに相談したり、作業を依頼したりすることができます。

スクリーンショット: *VS Code内のClaude Codeパネル*

エディタで開いているファイルのコンテキストを自動的に理解するため、「この関数のバグを直して」「この部分のリファクタリングを提案して」といった依頼がスムーズに行えます。

23.4. GitHub Apps / OAuth Apps

23.4.1. GitHub Marketplaceの概要

GitHubには**Marketplace**という仕組みがあり、サードパーティの開発者が作ったアプリケーション（GitHub App）を簡単にリポジトリにインストールして使うことができます。Marketplaceは以下のURLからアクセスできます。

```
https://github.com/marketplace
```

Marketplaceには無料のアプリから有料のアプリまで、数千のアプリが登録されています。スマートフォンのApp StoreやGoogle Playのように、カテゴリ別に探すことができ、レビューや評価も参考にできます。

23.4.2. カテゴリ別の代表的なアプリ

GitHub Marketplaceのアプリは、おおまかに以下のカテゴリに分類できます。それぞれの代表的なアプリを見てみましょう。

カテゴリ	代表的なアプリ	用途
CI/CD	CircleCI, Travis CI, Jenkins	コードの自動テスト・自動デプロイ
コード品質	Codecov, SonarCloud, Code Climate	テストカバレッジの計測やコード品質の分析
プロジェクト管理	Jira, Linear, ZenHub	タスク管理やスプリント計画
セキュリティ	Snyk, Socket, Dependabot	依存ライブラリの脆弱性スキャン
デプロイ	Vercel, Netlify, Railway	Webサイトやアプリの自動デプロイ
チャット連携	Slack, Discord, Microsoft Teams	通知やコマンド連携

たとえばVercelをインストールすると、PRを作成するたびにプレビュー環境が自動的に作成され、PRのコメントにプレビューURLが投稿されます。レビュアーはそのURLにアクセスするだけで、変更後のWebサイトを実際に確認できるのです。

23.4.3. インストール方法

GitHub App のインストール手順は以下のとおりです。

1. GitHub Marketplace（https://github.com/marketplace）でアプリを検索する
2. アプリの詳細ページを開き、「Install」または「Set up a plan」ボタンをクリックする
3. アプリが要求する権限（リポジトリの読み取り、Issue の書き込みなど）を確認し、許可する
4. アプリのインストール先を選択する。すべてのリポジトリに適用するか、特定のリポジトリだけに限定するかを選べる
5. 必要に応じて、アプリ側の設定画面で追加の設定を行う

権限の確認は非常に重要です。アプリがどのデータにアクセスし、何を変更できるのかを必ず確認してからインストールしましょう。不必要に広い権限を要求するアプリには注意が必要です。

23.4.4. 管理方法

インストール済みのアプリは、以下の場所から管理できます。

- **個人アカウント**: Settings → Applications → Installed GitHub Apps
- **Organization**: Organization Settings → Installed GitHub Apps

スクリーンショット: インストール済みアプリの管理画面

この画面から、アプリの権限変更、対象リポジトリの変更、アプリのアンインストールが行えます。使わなくなったアプリは、セキュリティの観点からもこまめにアンインストールすることをおすすめします。

23.5. Slack 連携

23.5.1. なぜ Slack 連携が必要なのか

開発チームの多くが日常的なコミュニケーションに Slack を使っています。GitHub で PR が作られたり、Issue にコメントが付いたりしたとき、毎回 GitHub の Web サイトを確認しに行くのは非効率です。Slack 連携を設定すると、GitHub で起きたイベントがリアルタイムで Slack チャンネルに通知されるため、チームメンバーは普段使っている Slack の画面から離れることなく、開発の状況を把握できます。

23.5.2. GitHub for Slack の設定

GitHub と Slack を連携するには、**GitHub for Slack** というアプリを使います。Slack ワークスペースにこのアプリをインストールした後、通知を受け取りたいチャンネルで以下のコマンドを実行します。

```
/github subscribe owner/repo
```

この1つのコマンドを実行するだけで、指定したリポジトリのIssue、Pull Request、デプロイ、リリースに関する通知がそのチャンネルに届くようになります。

23.5.3. 通知カスタマイズ

デフォルトの通知ではうるさすぎる場合や、逆に特定のイベントだけを受け取りたい場合は、通知の種類を細かくカスタマイズできます。

```
# 特定のイベントだけを購読する
/github subscribe owner/repo issues pulls deployments releases

# 特定のイベントの購読を解除する
/github unsubscribe owner/repo issues

# PRのレビュー通知も受け取る
/github subscribe owner/repo reviews

# コミット通知を受け取る
/github subscribe owner/repo commits

# 現在の購読状態を確認する
/github subscribe list
```

チーム開発では、たとえば `#frontend` チャンネルにはフロントエンドのリポジトリを、`#backend` チャンネルにはバックエンドのリポジトリを購読させるといった使い分けが効果的です。通知が多すぎるとノイズになりますので、チームで話し合って適切な粒度を決めましょう。

23.6. Webhook

23.6.1. 仕組みの解説

Webhookは、GitHubで特定のイベントが発生したときに、あらかじめ指定した外部のURL（エンドポイント）にHTTPリクエスト（POSTリクエスト）を自動送信する仕組みです。前節で紹介したSlack連携も、内部的にはWebhookの仕組みを利用しています。

たとえるなら、Webhookは「電話の転送設定」のようなものです。GitHubで何かイベント（電話がかかってきた）が起きたとき、事前に登録しておいた番号（URL）に自動的に転送されます。転送先のサーバーは、受け取った情報をもとに好きな処理（通知を送る、ビルドを開始する、データベースを更新するなど）を行うことができます。

具体的な流れは以下のとおりです。

1. 開発者が GitHub にコードをプッシュする（イベント発生）
2. GitHub が事前に登録された URL に HTTP POST リクエストを送信する
3. リクエストのボディには、イベントの詳細情報（誰が、いつ、何をしたか）が JSON 形式で含まれる
4. 受信側のサーバーがリクエストを処理し、必要なアクションを実行する

23.6.2. 設定方法

Webhook の設定は、リポジトリの Settings → **Webhooks** →「Add webhook」から行います。

設定項目	説明
Payload URL	リクエストの送信先 URL。HTTPS である必要がある
Content type	リクエストボディの形式。通常は `application/json` を選択する
Secret	署名検証用の秘密鍵。受信側がリクエストの正当性を検証するために使う
Events	どのイベントで Webhook を発火させるか。個別に選択できる

スクリーンショット：*Webhook* 設定画面

Secret の設定は非常に重要です。Secret を設定すると、GitHub はリクエストのヘッダーに `X-Hub-Signature-256` というフィールドを追加します。受信側のサーバーはこの署名を検証することで、リクエストが本当に GitHub から送られたものであることを確認できます。Secret を設定しないと、悪意のある第三者が Webhook URL にリクエストを送りつける攻撃（リプレイ攻撃）に対して脆弱になります。

23.6.3. よく使うイベント

Webhook で使用できるイベントは数十種類ありますが、実際によく使われるのは以下のイベントです。

イベント	発生タイミング	用途の例
`push`	コードがプッシュされた	自動ビルド・デプロイのトリガー
`pull_request`	PR が作成・更新・マージされた	コードレビュー通知、プレビュー環境の構築
`issues`	Issue が作成・更新・クローズされた	タスク管理ツールとの同期
`release`	リリースが公開された	配布パッケージの作成、告知の自動投稿
`workflow_run`	GitHub Actions ワークフローが完了した	後続の処理のトリガー
`create`	ブランチやタグが作成された	環境の自動構築

なお、GitHub Actions が登場してからは、以前 Webhook で実現していた自動化の多くを Actions 内で完結できるようになりました。外部サーバーを自前で用意する必要がない分、Actions のほうが手軽です。ただし、GitHub 外部のシステムと連携する場合や、GitHub の外にあるサーバーで処理を実行したい場合には、Webhook が依然として有用です。

23.7. GitHub API

23.7.1. REST API と GraphQL API の概要

GitHub は、プログラムから GitHub の機能を操作するための API を 2 種類提供しています。

REST API は、HTTP の GET/POST/PATCH/DELETE リクエストを使って、リソース（リポジトリ、Issue、PR など）を操作する伝統的な API 形式です。URL のパスでリソースを指定し、HTTP メソッドで操作を指定します。直感的でわかりやすいのが特徴です。

GraphQL API は、クライアント側が「どのデータが欲しいか」を細かく指定できるクエリ言語ベースの API です。REST API では 1 つのリクエストで 1 つのリソースしか取得できませんが、GraphQL API では 1 回のリクエストで必要なデータだけをまとめて取得できます。複雑なデータ取得を効率的に行いたい場合に適しています。

23.7.2. gh api コマンドでの使い方

GitHub CLI（`gh`）を使うと、認証の処理を自動的に行ってくれるため、API の呼び出しが非常に簡単になります。`curl` コマンドでトークンを手動で設定する必要がありません。

```
# リポジトリの情報を取得する
gh api repos/owner/repo

# リポジトリのスター数だけを取得する（jqフィルタ）
gh api repos/owner/repo --jq '.stargazers_count'

# Issue一覧のタイトルを取得する
gh api repos/owner/repo/issues --jq '.[].title'

# PRのコメント一覧を取得する
gh api repos/owner/repo/pulls/42/comments

# 新しいIssueを作成する（POSTリクエスト）
gh api repos/owner/repo/issues -f title="バグ報告" -f body="詳細な
   説明"
```

`--jq` オプションは、返却される JSON データから必要な部分だけを抽出するフィルタです。JSON の全体を見る必要がなく、必要なフィールドだけをピンポイントで取り出せます。

`curl` コマンドを使う場合は、個人アクセストークン（PAT）を手動で指定する必要があります。

```
curl -H "Authorization: token YOUR_TOKEN" \
  https://api.github.com/repos/owner/repo
```

23.7.3. GraphQL APIの使い方

GraphQL APIは `gh api graphql` コマンドで呼び出します。REST APIとは異なり、クエリ文字列でどのデータが欲しいかを明示的に指定します。

```
# リポジトリのオープンなIssueを5件取得する
gh api graphql -f query='
  query {
    repository(owner: "owner", name: "repo") {
      issues(first: 5, states: OPEN) {
        nodes {
          title
          url
          createdAt
          author {
            login
          }
        }
      }
    }
  }
'
```

GraphQL APIの利点は、1回のリクエストで複数のリソースをまとめて取得できることです。たとえば、リポジトリの情報とIssueとPRを同時に取得することも可能です。REST APIでは3回のリクエストが必要になる処理を、GraphQLなら1回で済ませられます。

23.7.4. 実用例

APIの実用的な使い方をいくつか紹介します。

```
# 自分がアサインされたIssueを全リポジトリから一覧表示する
gh api "/issues" --jq '.[].title'

# リポジトリのコントリビューター一覧を取得する
gh api repos/owner/repo/contributors --jq '.[].login'
```

```
# 特定のラベルが付いたIssueだけを取得する
gh api "repos/owner/repo/issues?labels=bug" --jq '.[].title'

# リリースの最新バージョンを取得する
gh api repos/owner/repo/releases/latest --jq '.tag_name'
```

これらのコマンドをシェルスクリプトに組み込むことで、日次レポートの自動生成、古い Issue の自動クローズ、リリースノートの自動作成といった自動化が実現できます。

23.8. Gist

23.8.1. コードスニペット共有の仕組み

Gist は、GitHub が提供する「ちょっとしたコードやテキストを手軽に共有する」サービスです。リポジトリを作るほどではない小さなコードの断片（スニペット）、設定ファイルのサンプル、メモなどを素早く共有するのに適しています。

Gist には 2 種類あります。**Public Gist** は誰でも閲覧でき、検索にもヒットします。**Secret Gist** は URL を知っている人だけが閲覧でき、検索にはヒットしませんが、URL さえ知っていれば誰でもアクセスできるため、完全な非公開ではない点に注意が必要です。

Gist の URL は `https://gist.github.com/ユーザー名/Gist_ID` という形式になります。

23.8.2. 作成・一覧・表示のコマンド

GitHub CLI（`gh`）を使うと、ターミナルから Gist の操作が行えます。

```
# ファイルからGistを作成する（公開）
gh gist create script.py --public --desc "便利なユーティリティ関数"

# 複数ファイルをまとめて1つのGistにする
gh gist create file1.py file2.py config.yaml

# 標準入力からGistを作成する（パイプで渡す）
echo "print('Hello, World!')" | gh gist create --filename hello.py

# 自分のGist一覧を表示する
gh gist list

# 特定のGistの内容を表示する
gh gist view GIST_ID

# Gistをブラウザで開く
gh gist view GIST_ID --web
```

```
# 既存のGistを編集する
gh gist edit GIST_ID
```

Gist はバージョン管理もされており、編集履歴を確認することもできます。内部的には Git リポジトリとして管理されているため、`git clone` でローカルにクローンして編集し、`git push` で更新することも可能です。

23.9. まとめ

この章では、GitHub と外部ツールの連携について幅広く学びました。

- **VS Code** との連携では、ソース管理パネルや GitLens のインライン blame を使って、エディタ内で Git 操作やコードの履歴確認ができることを学びました。github.dev を使えば、ブラウザ上でも同様の編集体験が得られます。
- **Claude Code** との連携では、AI の支援を受けながらコミットや PR 作成を自然言語で依頼でき、開発の生産性を向上させられることを確認しました。
- **GitHub Apps** では、Marketplace を通じて CI/CD、コード品質、セキュリティなどの外部サービスをリポジトリに統合できることを学びました。
- **Slack 連携**では、`/github subscribe` コマンドひとつでリアルタイム通知を設定できることを確認しました。
- **Webhook** では、GitHub のイベントを外部 URL に転送する仕組みを理解し、独自の自動化を構築する方法を学びました。
- **GitHub API** では、REST API と GraphQL API の 2 種類があり、`gh api` コマンドで手軽に呼び出せることを学びました。
- **Gist** では、ちょっとしたコードスニペットを素早く共有できる仕組みを確認しました。

GitHub を中心として開発ツールをつなげることで、チーム全体のワークフローがスムーズになります。次の章では、GitHub を使っていて遭遇するさまざまなトラブルとその解決法を見ていきましょう。

トラブルシューティング — よくある問題の処方箋

24.1. この章で学ぶこと

Git や GitHub を使っていると、さまざまなエラーメッセージや予期しない挙動に遭遇することがあります。特に初心者のうちは、ターミナルに赤い文字で表示されるエラーを見ると不安になるかもしれません。しかし、実際には Git 関連のトラブルの大半はパターンが決まっており、原因さえわかれば落ち着いて対処できるものばかりです。

この章では、Git や GitHub を使う際によく遭遇するエラーとトラブルを、カテゴリ別に整理して解説します。それぞれのトラブルについて、**なぜそのエラーが発生するのか（原因）**と**どうすれば解決できるのか（対処法）**を丁寧に説明します。

- 認証エラー（SSH 鍵やトークンに関する問題）
- push/pull エラー（リモートリポジトリとの同期に関する問題）
- ブランチ関連の問題（ブランチ切り替えや取得に関する問題）
- コミット関連の問題（間違ったコミットの修正に関する問題）
- .gitignore が効かない問題
- ネットワーク関連の問題（接続やパフォーマンスに関する問題）
- その他のよくある問題

この章は「困ったときに引く辞書」として使えるように構成しています。目次からエラーメッセージを探して、該当する節に飛んでください。

24.2. 認証エラー

認証エラーは、GitHub への接続がうまくいかないときに発生します。GitHub との接続方式には SSH 接続と HTTPS 接続の 2 種類があり、それぞれ異なる認証の仕組みを使うため、エラーの種類も異なります。

24.2.1. Permission denied (publickey) の原因と解決法

```
git@github.com: Permission denied (publickey).
fatal: Could not read from remote repository.
```

このエラーは、SSH 接続を使って GitHub にアクセスしようとしたとき、正しい SSH 鍵が見つからなかった場合に発生します。SSH 接続では、ローカルマシンに保存された秘密鍵と GitHub に登録された公開鍵のペアを使って認証を行います。この「鍵のペア」がどちらか一方でも欠けていたり、正しく設定されていなかったりすると、このエラーが表示されます。

主な原因は 3 つです。(1) SSH 鍵がそもそも作成されていない、(2) SSH 鍵は作成されているが GitHub に公開鍵を登録していない、(3) SSH エージェントに秘密鍵が追加されていない。

以下の手順で問題を診断し、解決しましょう。

```
# ステップ1: SSH鍵が存在するか確認する
ls -la ~/.ssh/

# id_ed25519 や id_rsa といったファイルがあれば鍵は存在する
# なければステップ2へ

# ステップ2: SSH鍵を新規作成する（鍵がない場合）
ssh-keygen -t ed25519 -C "your-email@example.com"
# パスフレーズの設定を求められる。空でもよいがセキュリティのため設
  定推奨

# ステップ3: SSHエージェントに秘密鍵を追加する
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519

# ステップ4: 公開鍵の内容を表示してコピーする
cat ~/.ssh/id_ed25519.pub

# ステップ5: GitHubに公開鍵を登録する
# GitHub → Settings → SSH and GPG keys → New SSH key
# 表示された公開鍵の全文を貼り付けて保存する

# ステップ6: 接続テストを行う
ssh -T git@github.com
# "Hi username! You've successfully authenticated" と表示されれば成
  功
```

もし複数の SSH 鍵を使い分けている場合（仕事用と個人用など）は、`~/.ssh/config` ファイルでホストごとに使用する鍵を指定する必要があります。

24.2.2. Invalid username or password の原因と解決法

```
remote: Invalid username or password.
fatal: Authentication failed for 'https://github.com/...'
```

このエラーは、HTTPS 接続で GitHub にアクセスしようとしたときに発生します。最も多い原因は、**GitHub のパスワード認証が 2021 年 8 月に廃止された**ことを知らずに、パスワードでログインしようとしているケースです。

現在、HTTPS 接続で GitHub にアクセスするには、パスワードの代わりに**個人アクセストークン**

（**Personal Access Token: PAT**）を使う必要があります。ただし、トークンを手動で管理するのは面倒なので、GitHub CLI（`gh`）を使って認証するのが最も簡単で推奨される方法です。

```
# GitHub CLIで認証をセットアップする（最も簡単な方法）
gh auth login
# プロンプトに従って、ブラウザで認証を完了する

# 認証状態を確認する
gh auth status
```

`gh auth login` を実行すると、対話的に接続方式（HTTPS/SSH）や認証方法を選択できます。認証が完了すると、以降の `git push` や `git pull` では自動的にトークンが使用されるため、パスワードの入力を求められることはなくなります。

もし何らかの理由で PAT を手動で作成する必要がある場合は、GitHub → Settings → Developer settings → Personal access tokens から作成できます。トークンには必要最小限のスコープ（権限）だけを付与し、有効期限を設定することがセキュリティ上重要です。

24.2.3. 403 Forbidden の原因と解決法

`403 Forbidden` は、認証自体は成功しているものの、実行しようとした操作に対する権限が不足している場合に発生します。

最も多い原因は、使用している PAT のスコープに `repo` が含まれていないことです。PAT を作成するとき、どの操作を許可するかをスコープで指定しますが、リポジトリへの push/pull には `repo` スコープが必要です。GitHub → Settings → Developer settings → Personal access tokens で、トークンのスコープを確認・更新してください。

また、Organization（組織）のリポジトリにアクセスする場合は、Organization の管理者がそのトークンの使用を承認している必要がある場合もあります。

24.3. push/pullエラー

push/pull エラーは、ローカルリポジトリとリモートリポジトリの間でコミット履歴に不整合が生じたときに発生します。チーム開発で複数人が同じリポジトリを操作しているときに起きやすいトラブルです。

24.3.1. rejected non-fast-forward の原因と解決法

```
! [rejected]        main -> main (non-fast-forward)
error: failed to push some refs to 'github.com:owner/repo.git'
hint: Updates were rejected because the remote contains work that
   you do not
```

```
hint: have locally.
```

このエラーは、`git push` しようとしたときに最もよく遭遇するエラーのひとつです。原因は明快で、**リモートリポジトリに、自分のローカルにはないコミットが存在している**ということです。

たとえるなら、あなたが本に新しいページを書いて出版社に送ろうとしたら、「他の著者が先に新しいページを追加したので、まずそれを読んでから送り直してください」と言われたようなものです。

解決方法は、まずリモートの変更を取り込んでから、改めてプッシュすることです。

```
# 方法1: pull --rebase で取り込む（推奨）
# リモートの変更を取り込んだ上に、自分のコミットを積み直す
git pull --rebase
git push

# 方法2: 先にfetchして確認してから取り込む（慎重にやりたい場合）
git fetch origin
git log HEAD..origin/main  # リモートにある自分にないコミットを確認
git rebase origin/main     # 自分のコミットをリモートの上に載せ替え
   る
git push
```

`git pull --rebase` と通常の `git pull`（マージ）の違いは、履歴の見た目です。`--rebase` を使うとコミット履歴が直線的になり、不要なマージコミットが作られません。チーム開発では `--rebase` を使うことが多いですが、どちらを使うかはチームのルール次第です。

注意: `git push --force` で強制的にプッシュすることも技術的には可能ですが、これはリモートの履歴を上書きする危険な操作です。他のチームメンバーの作業を消してしまう可能性があるため、特別な理由がない限り使わないでください。

24.3.2. refusing to merge unrelated histories の原因と解決法

```
fatal: refusing to merge unrelated histories
```

このエラーは、共通の祖先コミットを持たない 2 つの履歴をマージしようとしたときに発生します。最もよくあるシナリオは、GitHub で README 付きのリポジトリを作成した後に、ローカルで別途 `git init` して作業を始め、それをリモートにプッシュしようとした場合です。

GitHub で作成されたリポジトリ（README の初期コミットを含む）と、ローカルで `git init` したリポジトリは、完全に別の履歴を持つため、Git は安全のためにマージを拒否します。

```
# --allow-unrelated-histories オプションで明示的に許可する
git pull origin main --allow-unrelated-histories

# コンフリクトが発生する場合は解決してコミットする
```

このエラーを根本的に避けるには、GitHub でリポジトリを作成するときに README を追加せず（空のリポジトリとして作成し）、ローカルの既存リポジトリをそのままプッシュするのが最もスムーズです。

24.3.3. Your branch is behind

```
Your branch is behind 'origin/main' by 3 commits, and can be fast-
   forwarded.
```

このメッセージはエラーではなく、「リモートリポジトリに新しいコミットがあり、ローカルはそれを取り込んでいない」という状態を知らせる情報メッセージです。他のチームメンバーがプッシュした変更がリモートにあるとき表示されます。

```
# リモートの変更を取り込む
git pull

# リベースで取り込む（コミット履歴を直線的に保ちたい場合）
git pull --rebase
```

この状態でローカルに未コミットの変更がない場合は、`git pull` するだけで解決します。

24.4. ブランチ関連の問題

24.4.1. local changes would be overwritten（未コミット変更時のブランチ切替）

```
error: Your local changes to the following files would be
   overwritten by checkout:
    src/app.py
Please commit your changes or stash them before you switch branches
   .
```

このエラーは、ファイルを編集したもののまだコミットしていない状態で、別のブランチに切り替えようとしたときに発生します。Git は、未コミットの変更がブランチ切り替えによって失われる

ことを防ぐために、このエラーを出して切り替えを拒否します。

対処法は2つあります。

```
# 方法1: 変更を一時退避する（stash）
# 今の変更を「棚に上げて」から、ブランチを切り替える
git stash
git switch other-branch

# 作業が終わったら元のブランチに戻り、退避した変更を復元する
git switch original-branch
git stash pop

# 方法2: 変更をコミットしてからブランチを切り替える
# 作業途中でも仮のコミットとして保存する
git add .
git commit -m "WIP: 作業途中の保存"
git switch other-branch
```

`git stash` は「一時的な引き出し」のようなものです。今の変更をいったん退避させておき、後で元に戻すことができます。`stash pop` で最後に退避した変更が復元されます。退避した内容の一覧は `git stash list` で確認できます。

24.4.2. branch not found（リモートブランチの取得）

```
error: pathspec 'feature-x' did not match any file(s) known to git
```

このエラーは、存在しないブランチに切り替えようとしたときに発生します。チームメンバーが作成したリモートブランチに切り替えたい場合、まずローカルにリモートの情報を取得する必要があります。

```
# ステップ1: リモートの最新情報を取得する
git fetch origin

# ステップ2: リモートブランチの一覧を確認する
git branch -r

# ステップ3: リモートブランチを元にローカルブランチを作成して切り替
  える
git switch -c feature-x origin/feature-x

# または単純に（Gitが自動的にリモートブランチを追跡する）
```

```
git switch feature-x
```

`git fetch` はリモートの情報を取得するだけで、ローカルのファイルは一切変更しません。安全に実行できるコマンドなので、「なんかおかしいな」と思ったらまず `git fetch` を実行する癖をつけるとよいでしょう。

24.5. コミット関連の問題

コミットに関する問題は、「間違えた」「取り消したい」というケースがほとんどです。重要なのは、**まだプッシュしていないコミットは比較的安全に修正できる**が、**既にプッシュしたコミットの修正は慎重に行う必要がある**ということです。

24.5.1. 間違ったブランチにコミットした場合

作業に集中していると、`main` ブランチに直接コミットしてしまうことがあります。本来は `feature` ブランチに対してコミットすべきだったのに、ブランチを切り替え忘れていたケースです。

```
# ステップ1: 直前のコミットを取り消す（変更内容はそのまま残る）
git reset --soft HEAD~1

# ステップ2: 変更を一時退避する
git stash

# ステップ3: 正しいブランチに切り替える（なければ作成する）
git switch -c feature-branch
# または既存のブランチに切り替える場合
# git switch feature-branch

# ステップ4: 退避した変更を復元する
git stash pop

# ステップ5: 正しいブランチで改めてコミットする
git add .
git commit -m "適切なコミットメッセージ"
```

`git reset --soft HEAD~1` は、直前のコミットを「なかったこと」にしますが、コミットされていたファイルの変更内容はステージングエリアに残ります。つまり、コミットだけが消えて、ファイルの中身はそのままです。`--soft` と `--hard` の違いは非常に重要で、`--hard` を使うとファイルの変更内容も完全に消えてしまいます。

24.5.2. コミットメッセージを間違えた場合

コミットした直後に、コミットメッセージにtypoがあったり、内容が不適切だったりすることに気づくことがあります。

```
# まだプッシュしていない場合: --amend で直前のコミットメッセージを
  修正できる
git commit --amend -m "正しいコミットメッセージ"
```

`--amend` は直前のコミットを「修正」するコマンドです。コミットメッセージだけでなく、コミットに含めるファイルも変更できます。たとえば、コミットし忘れたファイルを後から追加する場合にも使えます。

```
# コミットし忘れたファイルを追加する
git add forgotten-file.py
git commit --amend --no-edit  # メッセージはそのままで、ファイルだ
  け追加
```

注意: 既にプッシュ済みのコミットに `--amend` を使うと、履歴が書き換わるため、`git push --force` が必要になります。これは他のチームメンバーに影響を与える可能性があるため、十分注意してください。

24.5.3. 大きなファイルをコミットしてしまった場合

誤って数百MBの動画ファイルやデータファイルをコミットしてしまうことがあります。GitHubでは100MBを超えるファイルのプッシュが拒否されるため、このようなファイルを含むコミットはプッシュできません。

```
# 直前のコミットの場合: コミットを取り消して .gitignore を設定する
git reset --soft HEAD~1
echo "large-file.zip" >> .gitignore
git add .gitignore
git rm --cached large-file.zip   # ステージングから大きなファイルを
  除外
git add .
git commit -m "コミットし直し（大きなファイルを除外）"
```

過去のコミット（何個も前のコミット）に大きなファイルが含まれている場合は、`git filter-branch` や **BFG Repo-Cleaner** というツールを使って履歴からファイルを完全に除去する必要があります。

```
# BFG Repo-Cleaner を使う方法
```

```
brew install bfg
bfg --strip-blobs-bigger-than 100M
git reflog expire --expire=now --all
git gc --prune=now --aggressive
```

BFG はリポジトリの全履歴をスキャンし、指定サイズ以上のファイルを過去のコミットからも除去します。ただし、これは履歴を書き換える操作であるため、チームメンバー全員に事前に周知し、操作後は全員がリポジトリを再クローンする必要があります。

24.6. .gitignore が効かない問題

24.6.1. 既に追跡されたファイルの対処

`.gitignore` ファイルに特定のファイルやディレクトリのパターンを追加したのに、Git が依然としてそのファイルの変更を追跡し続けている——これは非常によくある問題です。

原因は明確です。`.gitignore` は**まだ Git に追跡されていない（untracked な）ファイル**を無視するための仕組みであり、**既に Git に追跡されている（tracked な）ファイル**には効果がありません。つまり、一度でも `git add` してコミットしたファイルは、後から `.gitignore` に追加しても無視されないのです。

解決するには、まず Git の追跡を明示的に解除する必要があります。

```
# ファイルの追跡を解除する（ファイル自体は削除されない）
git rm --cached ファイル名

# ディレクトリの場合は -r オプションを付ける
git rm -r --cached ディレクトリ名/

# .gitignore にパターンが追加されていることを確認する
cat .gitignore

# 変更をコミットする
git add .gitignore
git commit -m "追跡対象から除外し、.gitignore を更新"
```

`git rm --cached` の `--cached` オプションが重要です。このオプションを付けないと、ファイルがディスクから物理的に削除されてしまいます。`--cached` を付けることで、「Git の追跡からは外すが、ファイル自体は手元に残す」という動作になります。

よくある具体例としては、`.env` ファイル（環境変数や秘密情報を含むファイル）を最初に誤ってコミットしてしまい、後から `.gitignore` に追加しても追跡が止まらないケースがあります。こ

のような場合、上記の手順で追跡を解除した後、GitHub の履歴にも秘密情報が残っているため、トークンやパスワードをすべて無効化して新しいものに更新することも忘れないでください。

24.7. ネットワーク関連

24.7.1. Connection timed out（ポート 443 の使用）

```
ssh: connect to host github.com port 22: Connection timed out
fatal: Could not read from remote repository.
```

SSH 接続で GitHub にアクセスしようとしたとき、接続がタイムアウトすることがあります。原因の多くは、ファイアウォールやプロキシが SSH のデフォルトポート（22 番）をブロックしているためです。企業ネットワークや公共 Wi-Fi でよく発生します。

解決策は、SSH 接続をポート 443（HTTPS 用のポート。通常はブロックされない）経由で行うように設定することです。

```
# ~/.ssh/config に以下の設定を追加する
cat >> ~/.ssh/config << 'EOF'
Host github.com
  Hostname ssh.github.com
  Port 443
  User git
EOF

# 接続テスト
ssh -T git@github.com
```

GitHub は `ssh.github.com` のポート 443 でも SSH 接続を受け付けてくれるため、この設定を行うことでファイアウォールを回避できます。この設定は一度行えば永続的に有効です。

24.7.2. クローンが遅い（shallow clone）

大きなリポジトリ（長い歴史を持つリポジトリや多くのファイルを含むリポジトリ）をクローンすると、非常に時間がかかることがあります。Git のクローンは全ての履歴（全てのコミット、全てのブランチ）をダウンロードするため、歴史が長いリポジトリほどデータ量が大きくなります。

これを解決する方法が **shallow clone（浅いクローン）**です。

```
# 最新のコミットだけをクローンする（最も高速）
git clone --depth 1 git@github.com:owner/repo.git

# 特定のブランチだけをクローンする
```

```
git clone --single-branch -b main git@github.com:owner/repo.git

# 両方を組み合わせる（最も高速かつ最小のデータ量）
git clone --depth 1 --single-branch -b main git@github.com:owner/
   repo.git
```

shallow cloneで取得したリポジトリは、後から全ての履歴を取得することもできます。

```
# 全ての履歴を後から取得する
git fetch --unshallow
```

24.8. その他の問題

24.8.1. detached HEAD

```
You are in 'detached HEAD' state. You can look around, make
   experimental
changes and commit them, and you can discard any commits you make
   in this
state without impacting any branches by switching back to a branch.
```

detached HEADは、ブランチではなく特定のコミットを直接チェックアウトした状態です。通常、HEADは現在のブランチの先頭を指しますが、detached HEADの状態ではHEADが特定のコミットを直接指しています。

この状態自体は危険ではありませんが、この状態で新しいコミットを作成すると、どのブランチにも属さない「浮いた」コミットになってしまいます。ブランチに戻ると、そのコミットはGCの対象となり、最終的に削除される可能性があります。

```
# 単純にブランチに戻りたい場合
git switch main

# 現在の状態を新しいブランチとして保存したい場合
git switch -c new-branch-name
```

detached HEAD状態になる典型的な原因は、`git checkout <コミットハッシュ>`で過去のコミットを直接チェックアウトした場合です。過去のコードを確認したいだけなら問題ありませんが、確認が終わったら必ずブランチに戻りましょう。

24.8.2. LF/CRLFの改行コード問題

```
warning: LF will be replaced by CRLF in file.txt
```

この警告は、ファイルの改行コードに関する問題です。改行コードにはLF（Line Feed、`\n`、Unix/macOS 標準）と CRLF（Carriage Return + Line Feed、`\r\n`、Windows 標準）の 2 種類があり、異なる OS 間で共同作業をすると改行コードの不一致が発生することがあります。

改行コードの不一致は、差分表示でファイル全体が「変更された」と表示されるなどの問題を引き起こします。

```
# macOS / Linux での推奨設定
# コミット時にCRLFをLFに変換、チェックアウト時はそのまま
git config --global core.autocrlf input

# Windows での推奨設定
# コミット時にCRLFをLFに変換、チェックアウト時にLFをCRLFに変換
git config --global core.autocrlf true
```

プロジェクト全体で改行コードを統一するには、`.gitattributes` ファイルをリポジトリに配置するのが確実です。

```
# すべてのテキストファイルの改行をLFに統一する
* text=auto eol=lf

# Windows用バッチファイルはCRLFを維持する
*.bat text eol=crlf
```

24.8.3. not a git repository

```
fatal: not a git repository (or any of the parent directories): .
   git
```

このエラーは、Git リポジトリではないディレクトリで `git` コマンドを実行したときに発生します。Git コマンドは `.git` ディレクトリを探して、それが見つからない場合にこのエラーを出します。

```
# 現在のディレクトリを確認する
pwd
```

```
# 正しいリポジトリのディレクトリに移動する
cd /path/to/your/repo

# または、現在のディレクトリをGitリポジトリとして初期化する
git init
```

ほとんどの場合、単にディレクトリを間違えているだけです。ターミナルの現在位置（`pwd` コマンドで確認できます）がリポジトリのルートディレクトリ、またはその中のサブディレクトリであることを確認しましょう。

24.9. 困ったときのコマンド集

トラブルの原因がわからないとき、まずは現在の状態を正確に把握することが重要です。以下のコマンドは「何が起きているか」を把握するためのものです。

```
# === 状態確認コマンド（安全にいつでも実行できる） ===

# 現在の状態を表示する（最初に実行すべきコマンド）
git status

# 最近のコミット履歴を確認する
git log --oneline -10

# リモートリポジトリの接続先を確認する
git remote -v

# 全てのブランチ（ローカル＋リモート）を一覧表示する
git branch -a

# 現在のブランチとリモートの差分を確認する
git fetch origin
git log HEAD..origin/main --oneline

# 設定を確認する
git config --list
```

上記のコマンドはいずれもリポジトリの状態を「読み取る」だけのコマンドであり、何も変更しません。安心して実行できます。

以下のコマンドは「元に戻す」ためのコマンドですが、**変更が失われる可能性がある**ため、慎重に使ってください。

```
# === 復元コマンド（注意して使用する） ===
```

```
# 特定のファイルの変更を元に戻す（未コミットの変更が消える）
git checkout -- ファイル名

# 全ファイルの変更を元に戻す（未コミットの全変更が消える）
git checkout .

# 未追跡ファイルを削除する
git clean -fd

# リモートの状態に完全にリセットする（ローカルの全変更が消える）
git fetch origin
git reset --hard origin/main
```

特に `git reset --hard` と `git clean -fd` は、未コミットの変更を完全に削除する不可逆的な操作です。実行前に、本当に変更を失ってよいか慎重に確認してください。心配な場合は、事前に `git stash` で変更を退避させておくとよいでしょう。

24.10. まとめ

この章では、Git や GitHub を使う際によく遭遇するトラブルとその解決法を網羅的に解説しました。

- **認証エラー**では、SSH 鍵の設定や PAT の使用が鍵であること、`gh auth login` が最も手軽な認証方法であることを学びました。
- **push/pull エラー**では、`non-fast-forward` は `git pull --rebase` で解決できること、`unrelated histories` はリポジトリの初期設定の不整合が原因であることを理解しました。
- **ブランチ関連**では、`git stash` による一時退避や `git fetch` によるリモート情報の取得が解決の鍵であることを学びました。
- **コミット関連**では、`git reset --soft` や `git commit --amend` を使って、プッシュ前のコミットを安全に修正する方法を学びました。
- **.gitignore** が効かない問題は、`git rm --cached` で追跡を解除することで解決できることを確認しました。
- **ネットワーク関連**では、ポート 443 への切り替えや shallow clone といった実用的な対処法を学びました。
- **detached HEAD** や**改行コード問題**といったその他のトラブルについても、原因と対処法を理解しました。

トラブルに遭遇したとき、最も大切なのは「慌てないこと」です。まず `git status` で現在の状

態を確認し、エラーメッセージをよく読み、この章で該当するパターンを探してください。Git の操作でデータが完全に失われることは稀で、ほとんどの場合は回復可能です。

ファイル拡張子とプログラミング言語

25.1. この章で学ぶこと

GitHub でリポジトリを眺めていると、`.py`、`.js`、`.yaml`、`.md` など、実にさまざまな拡張子を持つファイルが並んでいることに気づきます。プログラミング経験がない人にとって、これらのファイルが何を意味するのかを理解することは、リポジトリの全体像を把握するための第一歩です。

この章では、以下の内容を学びます。

- ファイル拡張子の基本的な役割と、GitHub での表示への影響
- 主要なプログラミング言語とそれに対応する拡張子
- データファイルや設定ファイルに使われる拡張子の種類と使い分け
- ドキュメントファイルの拡張子
- ビルドやパッケージ管理に使われるファイル
- GitHub 特有のファイルやディレクトリの役割
- GitHub がリポジトリの言語を判定する仕組み
- 文字コードの基礎知識
- 拡張子を持たないファイルの意味

この章は「プログラミング言語の入門」ではなく、「GitHub のリポジトリを読み解くための地図」として活用してください。それぞれの拡張子がどのような文脈で使われるのかを知ることで、初めて見るリポジトリでも「このプロジェクトは何の技術で作られているのか」が一目で判断できるようになります。

25.2. 拡張子とは何か

25.2.1. ファイル名の末尾の .xxx の役割

ファイルの**拡張子**とは、ファイル名の末尾にあるピリオド以降の部分のことです。たとえば `report.pdf` というファイル名であれば、`.pdf` が拡張子です。

```
main.py        ← .py = Pythonで書かれたプログラム
index.html     ← .html = Webページの構造を記述したファイル
style.css      ← .css = Webページの見た目を定義したファイル
README.md      ← .md = Markdownで書かれた文書
data.csv       ← .csv = カンマ区切りの表データ
```

拡張子の役割は、**このファイルがどのような種類のデータを含んでいるか**をコンピュータと人間の両方に伝えることです。OS はファイルの拡張子を見て、どのアプリケーションでファイルを開

くかを判断します。テキストエディタはファイルの拡張子を見て、どのプログラミング言語のルールで色分けするかを決定します。

25.2.2. GitHub でのシンタックスハイライト

GitHub では、ファイルの拡張子に基づいて**シンタックスハイライト**（構文の色分け表示）が自動的に適用されます。たとえば、`.py` ファイルを開くと、Python の予約語（`def`、`class`、`import` など）が青色で、文字列が緑色で、コメントがグレーで表示されます。

スクリーンショット: GitHub での Python ファイルのシンタックスハイライト表示

この色分けのおかげで、コードの構造が一目で把握しやすくなります。もし拡張子がないファイルや、GitHub が認識できない拡張子のファイルの場合は、色分けされずにプレーンテキストとして表示されます。

25.3. プログラミング言語と拡張子

25.3.1. Web 開発の拡張子

Web 開発は、複数の言語を組み合わせて行います。それぞれの言語が異なる役割を担っています。

拡張子	言語/技術	役割	具体例
`.html`	HTML	Web ページの構造（骨格）	`index.html`
`.css`	CSS	Web ページの見た目（装飾）	`style.css`
`.js`	JavaScript	Web ページの動的な振る舞い	`app.js`
`.ts`	TypeScript	JavaScript に型の安全性を追加した言語	`app.ts`
`.jsx`	JSX	React（UI ライブラリ）用の JavaScript	`App.jsx`
`.tsx`	TSX	React 用の TypeScript	`App.tsx`
`.vue`	Vue.js	Vue.js フレームワークのコンポーネント	`Header.vue`
`.svelte`	Svelte	Svelte フレームワークのコンポーネント	`App.svelte`
`.php`	PHP	サーバーサイドの Web 開発言語	`index.php`
`.scss` / `.sass`	Sass	CSS をより効率的に書くための拡張言語	`main.scss`

HTML は家の骨組み、CSS は壁紙や家具の配置、JavaScript は電気系統（スイッチを押すと照明が点く）にたとえることができます。Web 開発のリポジトリでこれらの拡張子を見かけたら、「ああ、Web サイトやアプリを作っているプロジェクトだな」と判断できます。

TypeScript（`.ts`）は JavaScript の拡張版で、変数の型を明示的に宣言できるようにした言語です。大規模なプロジェクトではバグの防止に役立つため、近年は `.js` よりも `.ts` を使うプロジェ

クトが増えています。

25.3.2. 汎用プログラミング言語の拡張子

Web 開発に限らず、さまざまな用途で使われる汎用的なプログラミング言語の一覧です。

拡張子	言語	特徴	主な用途
`.py`	Python	読みやすく書きやすい	AI/ML、データ分析、Web 開発、
`.java`	Java	エンタープライズ向けの堅牢な言語	業務システム、Android アプリ
`.go`	Go	Google が開発した高速な言語	サーバー、CLI、インフラツール
`.rs`	Rust	メモリ安全性と高速性を両立	システム開発、WebAssembly
`.rb`	Ruby	書きやすさを重視した言語	Web 開発（Ruby on Rails）
`.swift`	Swift	Apple が開発した言語	iOS / macOS アプリ
`.kt`	Kotlin	Java の後継となるモダンな言語	Android アプリ、サーバー
`.c`	C 言語	低レベルで高速な言語	OS、組み込みシステム
`.cpp` / `.cc`	C++	C 言語を拡張した言語	ゲームエンジン、システム開発
`.h`	C/C++ ヘッダー	関数や型の宣言を記述	ライブラリのインターフェース定
`.cs`	C#	Microsoft 製の言語	ゲーム（Unity）、Windows アプリ
`.dart`	Dart	Google 製の言語	Flutter（クロスプラットフォーム
`.scala`	Scala	JVM 上で動く関数型言語	ビッグデータ処理（Apache Spark
`.ex` / `.exs`	Elixir	並行処理に強い言語	高負荷 Web サーバー
`.lua`	Lua	軽量なスクリプト言語	ゲームへの組み込み
`.pl` / `.pm`	Perl	テキスト処理に強い言語	レガシーシステム

GitHub でリポジトリを見たとき、主に使われている拡張子からプロジェクトの性質を推測できます。`.py` ファイルが多ければ Python プロジェクト、`.go` ファイルが多ければ Go のプロジェクト、`.rs` ファイルが多ければ Rust のプロジェクトです。

25.3.3. データサイエンス・科学計算の拡張子

データサイエンスやバイオインフォマティクスの分野では、独自の拡張子を持つファイルが多く使われます。

`.ipynb`（Jupyter Notebook）は GitHub で特別に対応されており、ノートブックの内容（コード、出力結果、グラフ）がそのままレンダリングされて表示されます。これは、コードと実行結果を一緒に確認できるため、データサイエンスのプロジェクトでは特に重宝されます。

拡張子	言語/形式	用途
`.py`	Python	データ分析、機械学習
`.R` / `.r`	R 言語	統計分析、データの可視化
`.Rmd`	R Markdown	R のコードとレポートを統合した文書
`.ipynb`	Jupyter Notebook	対話型のデータ分析（Python/R など）
`.jl`	Julia	高速な科学計算
`.m`	MATLAB	数値計算（大学や研究機関で多用）
`.nf`	Nextflow	バイオインフォマティクスのパイプライン記述
`.wdl`	WDL	ワークフロー記述言語
`.qza` / `.qzv`	QIIME 2	メタゲノム解析のデータ（アーティファクト）

25.3.4. シェルスクリプトの拡張子

シェルスクリプトは、コマンドラインで実行する一連のコマンドをファイルにまとめたものです。OS の種類やシェルの種類によって拡張子が異なります。

拡張子	言語	用途
`.sh`	Bash / Shell	Linux / macOS での自動化スクリプト
`.bash`	Bash	Bash 専用のスクリプト
`.zsh`	Zsh	Zsh 専用のスクリプト（macOS のデフォルトシェル）
`.fish`	Fish	Fish シェル用のスクリプト
`.ps1`	PowerShell	Windows の高機能シェルスクリプト
`.bat` / `.cmd`	バッチファイル	Windows のコマンドプロンプト用スクリプト

GitHub のリポジトリで `.sh` ファイルを見かけたら、ビルドスクリプト、デプロイスクリプト、セットアップスクリプトなど、何らかの自動化処理が含まれていると考えてよいでしょう。

25.4. データ・設定ファイルの拡張子

プログラミング言語のファイル以外にも、データや設定を記述するためのファイル形式が多数あります。それぞれの形式には特徴と得意分野があり、適材適所で使い分けられています。

`.env` ファイルには特に注意が必要です。 `.env` ファイルにはパスワードや API キーなどの秘密情報が含まれることが多いため、**必ず `.gitignore` に追加して、Git の追跡対象から除外**してください。一度 GitHub にプッシュしてしまった秘密情報は、コミット履歴に残り続けるため、すべてのキーやパスワードを無効化して再発行する必要があります。

JSON と YAML は同じデータを異なる形式で表現できます。たとえば、同じデータを JSON 形式で書くと波括弧とダブルクォーテーションが多くなりますが、YAML 形式で書くとインデントだ

拡張子	形式名	特徴	主な用途
`.json`	JSON	構造化データの標準形式。プログラムから読み書きしやすい	API のデータ交
`.yaml` / `.yml`	YAML	人間にとって読み書きしやすい形式	GitHub Actions
`.toml`	TOML	セクション構造が明確で読みやすい	Rust（Cargo.to
`.xml`	XML	タグベースの構造化データ。冗長だが表現力が高い	Java プロジェク
`.csv`	CSV	カンマ区切りの表データ。最もシンプルな表形式	データの受け渡
`.tsv`	TSV	タブ区切りの表データ	バイオインフォ
`.ini` / `.cfg`	INI	セクションとキー=値のシンプルな形式	古い設定ファイ
`.env`	環境変数	KEY=VALUE 形式の環境変数定義	API キーなどの

けで構造を表現するためスッキリします。人間が直接編集するファイル（設定ファイルなど）には YAML が好まれ、プログラム間のデータ交換には JSON が好まれる傾向があります。

25.5. ドキュメントファイルの拡張子

ソフトウェアプロジェクトにはドキュメント（文書）が欠かせません。ドキュメントの形式もいくつかの種類があります。

拡張子	形式名	特徴	主な用途
`.md`	Markdown	シンプルな記法で見出し・リスト・リンクを表現できる	README、プロジ
`.rst`	reStructuredText	Markdown より表現力が高い	Python プロジェク
`.txt`	プレーンテキスト	書式なしの純粋なテキスト	LICENSE、簡単な
`.adoc`	AsciiDoc	高機能な文書作成形式	技術書、長い文書
`.tex`	LaTeX	数式や論文レイアウトに特化した組版言語	学術論文、技術報告
`.org`	Org Mode	Emacs 向けの文書形式	ノート、タスク管理

GitHub では `.md`（Markdown）ファイルが最もよく使われます。GitHub は Markdown ファイルを自動的にレンダリング（見出し・太字・リンクなどを反映して整形表示）するため、リポジトリのトップページに `README.md` を配置するのが標準的な慣習です。

25.6. ビルド・パッケージ管理ファイル

ソフトウェアプロジェクトには、プログラムのビルド方法や依存ライブラリを定義するファイルが含まれています。これらのファイルを見ると、そのプロジェクトがどの言語やツールを使っているかがわかります。

たとえば、リポジトリのルートに `package.json` があれば「Node.js（JavaScript/TypeScript）のプロジェクト」、`requirements.txt` か `pyproject.toml` があれば「Python のプロジェクト」、

ファイル名	言語/ツール	用途
`Makefile`	Make	ビルド手順の定義（C/C++プロジェクトに多
`CMakeLists.txt`	CMake	クロスプラットフォーム対応のビルド設定
`Dockerfile`	Docker	コンテナイメージの構築手順
`docker-compose.yml`	Docker Compose	複数のコンテナの構成を定義
`package.json`	npm (Node.js)	JavaScript プロジェクトの依存ライブラリと設
`requirements.txt`	pip (Python)	Python プロジェクトの依存ライブラリの一覧
`pyproject.toml`	Python	現代的な Python プロジェクトの設定と依存関
`Cargo.toml`	Cargo (Rust)	Rust プロジェクトの設定と依存関係
`go.mod`	Go Modules	Go プロジェクトの依存関係
`Gemfile`	Bundler (Ruby)	Ruby プロジェクトの依存ライブラリ
`build.gradle`	Gradle (Java/Kotlin)	Java/Kotlin プロジェクトのビルド設定
`pom.xml`	Maven (Java)	Java プロジェクトのビルドと依存関係
`Cargo.lock` / `package-lock.json`	各種	依存ライブラリの正確なバージョンを固定する

`Cargo.toml` があれば「Rust のプロジェクト」と判断できます。これらのファイルは、いわばプロジェクトの「名刺」のようなものです。

25.7. GitHub 特有のファイルとフォルダ

GitHub では、特定のファイル名やディレクトリ名を使うことで、GitHub の機能と連動する特別な動作を実現できます。

25.7.1. リポジトリルートの特別なファイル

ファイル名	役割
`README.md`	リポジトリのトップページに自動表示されるプロジェクト説明文
`LICENSE`	プロジェクトのライセンス。GitHub がライセンスを自動認識する
`.gitignore`	Git の追跡対象から除外するファイルのパターンを定義する
`.gitattributes`	ファイルの属性（改行コード、差分表示方法、LFS 対象など）を定義する
`CLAUDE.md`	Claude Code へのプロジェクト固有の指示を記述するファイル

`README.md` はリポジトリの「顔」であり、プロジェクトが何をするものなのか、どうやって使うのかを最初に伝える場所です。GitHub はリポジトリのトップページに `README.md` の内容を自動的にレンダリングして表示するため、丁寧に書かれた `README.md` は訪問者にとって非常に助けに

なります。

25.7.2. .github/ ディレクトリ内の各種ファイル

`.github/` ディレクトリは、GitHub の各種機能に関連する設定ファイルを格納する特別なディレクトリです。

ファイル/ディレクトリ	役割
`.github/workflows/`	GitHub Actions のワークフローファイル（YAML ファイル）を格納
`.github/ISSUE_TEMPLATE/`	Issue 作成時のテンプレートを格納する
`.github/pull_request_template.md`	PR 作成時に自動挿入されるテンプレート
`.github/CODEOWNERS`	ファイルやディレクトリごとのレビュー担当者を定義する
`.github/dependabot.yml`	Dependabot（依存ライブラリの自動更新）の設定
`.github/FUNDING.yml`	リポジトリのスポンサーボタンの設定

`.github/workflows/` ディレクトリに YAML ファイルが入っていれば、そのリポジトリでは GitHub Actions（自動テストやデプロイ）が設定されていることがわかります。`.github/CODEOWNERS` があれば、PR を作成したときに自動的に適切なレビュアーがアサインされる仕組みが導入されています。

25.8. GitHub の言語判定の仕組み

25.8.1. linguist による自動判定

GitHub のリポジトリページには、使用されているプログラミング言語の割合を示すカラーバーが表示されます。「Python 65%、JavaScript 25%、Shell 10%」のように、各言語が色分けされたバーで視覚的に表現されます。

スクリーンショット: *GitHub* リポジトリの言語バー

この言語判定は、**Linguist**（https://github.com/github-linguist/linguist）というオープンソースのツールによって行われています。Linguist はリポジトリ内の全てのファイルを解析し、拡張子やファイルの内容に基づいて使用言語を判定します。

ただし、Linguist の判定には以下のルールがあります。

- `vendor/` ディレクトリ内のファイルは「外部ライブラリ」とみなされ、統計から除外される
- `docs/` ディレクトリ内のファイルは「ドキュメント」とみなされ、統計から除外される
- 自動生成されたファイル（ミニファイされた JS など）は統計から除外される

25.8.2. .gitattributes でのカスタマイズ

Linguist の判定結果が意図と異なる場合、`.gitattributes` ファイルで上書きできます。たとえば、プロジェクトの主要言語が Python なのに、自動生成された JavaScript ファイルが大量にあるせいで JavaScript プロジェクトと判定されてしまう場合、以下のように設定できます。

```
# vendorディレクトリを統計から除外する
vendor/* linguist-vendored

# ドキュメントとして扱う（統計から除外される）
docs/* linguist-documentation

# 自動生成コードとして扱う（統計から除外される）
generated/* linguist-generated

# 特定のファイルを別の言語として認識させる
*.jsx linguist-language=JavaScript

# 特定のディレクトリのファイルを統計に含めない
static/js/* linguist-vendored
```

言語バーはリポジトリの「第一印象」に影響するため、プロジェクトの実態を正確に反映するように `.gitattributes` を設定することは、地味ですが重要です。

25.9. ファイルの文字コード

25.9.1. UTF-8 が標準

ファイルの**文字コード**（エンコーディング）とは、文字をどのようにバイト列として表現するかの規則のことです。現在、ソフトウェア開発における標準的な文字コードは **UTF-8** です。

```
UTF-8      ← 現在の世界標準。日本語、英語を含むあらゆる言語に対応
Shift_JIS  ← 日本語のレガシーな文字コード。古いWindowsソフトで使用
EUC-JP     ← 日本語のレガシーな文字コード。古いUnixシステムで使用
```

GitHub は UTF-8 でエンコードされたファイルを前提としています。Shift_JIS や EUC-JP でエンコードされたファイルを GitHub にプッシュすると、日本語が文字化けして表示されることがあります。特別な理由がない限り、すべてのファイルは UTF-8 で保存しましょう。

25.9.2. .editorconfig

チーム開発では、メンバーごとに異なるエディタや設定を使っていると、文字コードやインデントスタイルの不一致が発生しがちです。この問題を解決するのが **EditorConfig**（`.editorconfig` ファイル）です。

`.editorconfig` ファイルをリポジトリのルートに配置すると、EditorConfig に対応したエディタ（VS Code、JetBrains IDE、Vim など）が自動的にその設定に従います。

```
# .editorconfig
root = true

[*]
charset = utf-8
end_of_line = lf
indent_style = space
indent_size = 4
insert_final_newline = true
trim_trailing_whitespace = true

[*.md]
trim_trailing_whitespace = false

[*.{yml,yaml}]
indent_size = 2

[Makefile]
indent_style = tab
```

この設定では、すべてのファイルで UTF-8、LF 改行、スペース 4 つのインデントを標準としつつ、YAML ファイルではインデント 2 つ、Makefile ではタブインデントを使うように指定しています。Markdown ファイルでは行末の空白を残すように設定しています（Markdown では行末の空白 2 つが改行を意味するためです）。

25.10. 拡張子のないファイル

GitHub のリポジトリを見ていると、拡張子を持たないファイルがいくつか存在することに気づきます。これらは多くの場合、慣習的にファイル名が決まっている特別なファイルです。

また、ドットで始まる「隠しファイル」も拡張子がないことが多いです。

これらのファイルは拡張子がなくても、ファイル名自体が「何のファイルか」を示しています。Git ではこのような慣習が広く共有されているため、ファイル名を見ればその役割が推測できるようになっています。

ファイル名	用途
`Makefile`	ビルド手順を定義するファイル（make コマンドで実行）
`Dockerfile`	Docker コンテナイメージの構築手順を記述するファイル
`Procfile`	Heroku（PaaS）でのプロセス定義ファイル
`Vagrantfile`	Vagrant（仮想マシン管理ツール）の設定ファイル
`Brewfile`	Homebrew（macOS のパッケージマネージャー）の依存関係リスト
`Rakefile`	Ruby のタスク定義ファイル（rake コマンドで実行）
`Gemfile`	Ruby のライブラリ依存関係を定義するファイル
`LICENSE`	ライセンス条件を記述するファイル
`CHANGELOG`	プロジェクトの変更履歴を記録するファイル

ファイル名	用途
`.editorconfig`	エディタの設定を統一するファイル
`.prettierrc`	Prettier（コードフォーマッター）の設定ファイル
`.eslintrc`	ESLint（JavaScript リンター）の設定ファイル
`.gitignore`	Git の追跡除外パターンを定義するファイル
`.dockerignore`	Docker ビルド時に除外するファイルのパターン
`.env`	環境変数を定義するファイル（秘密情報を含むため要注意）

25.11. まとめ

この章では、GitHub のリポジトリでよく見かけるファイル拡張子と、それらが意味するものについて幅広く学びました。

- **拡張子**はファイルの種類を示すもので、GitHub のシンタックスハイライトにも影響します。
- **プログラミング言語**にはそれぞれ固有の拡張子があり、`.py`（Python）、`.js`（JavaScript）、`.go`（Go）、`.rs`（Rust）など、拡張子を見ればどの言語で書かれているかが一目でわかります。
- **データ・設定ファイル**には `.json`、`.yaml`、`.toml` などの形式があり、用途に応じて使い分けられています。特に `.env` ファイルは秘密情報を含む可能性があるため、`.gitignore` への追加が必須です。
- **GitHub 特有のファイル**（`README.md`、`.gitignore`、`.github/` ディレクトリ内のファイルなど）は、GitHub の各機能と連動する特別な役割を持っています。
- **Linguist** がリポジトリの言語判定を行っており、`.gitattributes` でカスタマイズできます。
- **UTF-8** が文字コードの標準であり、`.editorconfig` でチーム全体の設定を統一できます。

- `Makefile` や `Dockerfile` など、**拡張子のないファイル**もリポジトリでは重要な役割を果たしています。

リポジトリ内のファイルの拡張子を「読める」ようになることは、GitHub を使いこなすための大切な基礎力です。初めてのリポジトリを訪れたとき、ファイルの拡張子を見渡すだけで、そのプロジェクトの全体像をかなり正確に把握できるようになるでしょう。

GitHubアプリの比較 — Web・Desktop・CLI・Mobile・IDE

26.1. この章で学ぶこと

GitHubは単一のWebサイトではなく、さまざまな方法でアクセスし操作できるプラットフォームです。ブラウザでWebサイトを開く方法だけでなく、デスクトップアプリ、コマンドラインツール、スマートフォンアプリ、そしてVS Codeなどの IDEの拡張機能からもGitHubの機能を利用できます。

初心者にとっては「どれを使えばいいのか」がわかりにくいかもしれません。結論から言えば、それぞれの方法には得意分野と不得意分野があり、状況に応じて使い分けるのがベストです。プロの開発者も、ひとつのツールだけに頼るのではなく、複数のツールを組み合わせて使っています。

この章では、GitHubを操作する5つの方法を比較し、それぞれの特徴と向いている人を詳しく解説します。この章を読み終える頃には、自分に合ったツールの組み合わせが見つかるでしょう。

26.2. GitHubを操作する5つの方法の概観

まず、5つの方法の全体像を図で確認しましょう。いずれの方法も、最終的にはインターネット上のGitHubサーバーと通信してデータをやり取りするという点は共通しています。違いは、その「入り口」がどのようなインターフェースであるかです。

```
┌──────────────
```

機能	Web UI	Desktop	CLI (gh)	Mobile	IDE 拡張
コード閲覧	◎	○	△	○	◎
コード編集	○	◎	○	△	◎
コミット	○	◎	◎	–	◎
ブランチ操作	○	◎	◎	△	◎
PR 作成/レビュー	◎	○	◎	○	◎
Issue 管理	◎	–	◎	◎	△
Actions 管理	◎	–	◎	○	△
差分の視覚的比較	◎	◎	△	△	◎
コンフリクト解決	△	◎	△	–	◎
Settings 管理	◎	–	○	–	–
自動化/スクリプト	–	–	◎	–	△
オフライン作業	–	◎	◎	–	◎

この表を見ると、「全てが◎のツールはない」ことがわかります。だからこそ、複数のツールを組み合わせて使うことが大切なのです。それでは、各ツールを詳しく見ていきましょう。

26.3. GitHub Web UI

26.3.1. できること（全機能）

GitHub Web UI は、ブラウザで `https://github.com` にアクセスして使うインターフェースです。GitHub の全ての機能にアクセスできる、最も包括的な操作手段です。

Web UI でできることは多岐にわたります。リポジトリの作成・設定・削除、コードの閲覧と簡単な編集、Pull Request の作成・レビュー・マージ、Issue の作成・管理・ラベル付け、GitHub Actions のワークフロー設定と実行状況の監視、Organization やチームの管理、セキュリティ設定、Marketplace（アプリの追加）、プロフィールの編集など、GitHub のあらゆる機能が Web UI から操作できます。

特に、**リポジトリの Settings 画面や Organization の管理画面は Web UI からしかアクセスできない**ため、どのツールをメインに使うとしても、Web UI を完全に使わないということはありません。

26.3.2. Web 上でのコード編集

Web UI では、3 つの方法でコードを編集できます。

方法 1: ファイルの直接編集。リポジトリ内のファイルを開き、鉛筆アイコン（Edit this file）をクリックすると、ブラウザ上でファイルの内容を編集できます。編集後は「Commit changes」ボタンでコミットが作成されます。typo の修正やドキュメントの更新など、簡単な変更に適しています。

方法 2: github.dev（ブラウザ版 VS Code）。リポジトリのページでキーボードの `.`（ピリオド）キーを押すと、ブラウザ上で VS Code と同じインターフェースが起動します。複数ファイルの同時編集、検索・置換、拡張機能の一部が使えるため、方法 1 よりも快適に編集作業ができます。た

だし、ターミナルは使えないため、ビルドやテストの実行はできません。

スクリーンショット: *github.dev* の画面

方法 3: GitHub Codespaces（クラウド開発環境）。Codespaces は、クラウド上に完全な開発環境を構築するサービスです。ブラウザ上で VS Code が動作し、ターミナル、拡張機能、デバッガ、言語ランタイムなど、ローカル環境と同等の開発体験が得られます。無料プランでは月 60 時間まで利用できます。

26.3.3. 向いている人

- GitHub を始めたばかりの初心者（ソフトウェアのインストールが不要）
- リポジトリの設定変更や Issue/PR の管理が中心の人
- 他の人のリポジトリを閲覧・フォーク・コメントしたい人
- ちょっとした編集だけを行いたい人

26.4. GitHub Desktop

26.4.1. インストール

GitHub Desktop は、GitHub が公式に提供するデスクトップアプリケーションです。https://desktop.github.com/ からダウンロードでき、macOS と Windows に対応しています（Linux 版は公式には提供されていません）。

スクリーンショット: *GitHub Desktop* のメイン画面

26.4.2. 特徴（GUI で Git 操作）

GitHub Desktop の最大の特徴は、**Git コマンドを一切覚えなくても Git の操作ができる**ことです。ファイルの変更をステージングするのも、コミットを作成するのも、ブランチを切り替えるのも、すべてボタンのクリックやドラッグ&ドロップで行えます。

コマンドラインに不慣れな人にとって、Git の学習障壁を大幅に下げてくれるツールです。特に以下の点が初心者にとって助けになります。

- **差分の視覚的表示**: ファイルの変更箇所が緑色（追加）と赤色（削除）で色分け表示される。コマンドラインの `git diff` よりもはるかに直感的に変更内容を把握できる
- **コンフリクト解決の UI**: マージコンフリクトが発生したとき、GUI で「どちらの変更を採用するか」を選択できる

- **ブランチの視覚的管理**: ブランチの切り替えはドロップダウンメニューから選ぶだけ。ブランチの作成もダイアログ一つで完了する
- **履歴のビジュアル表示**: コミット履歴が時系列で一覧表示され、各コミットの変更内容をクリックひとつで確認できる

26.4.3. 画面の見方

GitHub Desktop の画面構成を理解しましょう。

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│  GitHub Desktop                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
│  Current Repository: [my-project ▼]                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
│  Current Branch: [main ▼]                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
│   ┌──────────────────────────────────────┐   ┌──────────────────────────────────────┐
│   │  Changes (3)                         │   │  History                             │
│   │                                      │   │                                      │
│   │  ☒ README.md                         │   │  abc1234 feat: 新機能                │
│   │  ☒ src/app.py                        │   │  def5678 fix: バグ修正               │
│   │  ☒ test.py                           │   │  ghi9012 docs: 更新                  │
│   │                                      │   │                                      │
│   │  [Summary]                           │   │                                      │
│   │  [Description]                       │   │                                      │
│   │  [Commit to main]                    │   │                                      │
│   └──────────────────────────────────────┘   └──────────────────────────────────────┘
│
│  [Push origin]  [Fetch origin]
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

画面上部には現在のリポジトリとブランチが表示されます。左側の「Changes」タブには変更されたファイルの一覧が表示され、チェックボックスでステージするファイルを選択します。右側の「History」タブでは過去のコミット一覧を確認できます。画面下部のボタンで Push（アップロード）や Fetch（ダウンロード）を行います。

26.4.4. 他の Git GUI ツールとの比較

GitHub Desktop 以外にも、GUI で Git を操作できるツールがあります。

ツール	料金	特徴
GitHub Desktop	無料	シンプルで使いやすい。GitHub 公式
Sourcetree	無料	高機能。Atlassian 製（Bitbucket との連携が得意）
GitKraken	有料（$4.95/月~）	美しい UI と直感的な操作性。ブランチグラフが秀逸
Fork	有料（$49.99 買い切り）	高速で軽量。macOS/Windows 対応
Tower	有料（$69/年~）	プロフェッショナル向け。高機能
Sublime Merge	有料（$99 買い切り）	高速検索と軽量動作が売り

初心者には GitHub Desktop が最適です。無料で、GitHub との統合がスムーズで、機能がシンプルなため迷うことが少ないです。より高度な機能（複雑なリベース操作、高度なブランチグラフの表示など）が必要になったら、他のツールへの移行を検討するとよいでしょう。

26.4.5. 向いている人

- **Git 初心者**: コマンドを覚えずに Git を使い始めたい人
- **ビジュアル派**: 差分やブランチを視覚的に確認したい人
- **非エンジニア**: デザイナー、ライター、研究者など、コマンドラインに馴染みのない人

26.5. GitHub CLI (gh)

26.5.1. git コマンドと gh コマンドの違い

GitHub CLI を理解する上で最も重要なのは、`git` **コマンドと** `gh` **コマンドは全く別のツール**であるということです。名前が似ているため混同しやすいですが、役割が明確に異なります。

`git` **コマンド**は、ローカルマシン上でのバージョン管理を行うツールです。ファイルのステージング（`git add`）、コミット（`git commit`）、ブランチの作成・マージ（`git branch`、`git merge`）、リモートとの通信（`git push`、`git pull`）など、Git の基本操作を担当します。GitHub がなくても使えるツールです。

`gh` **コマンド（GitHub CLI）**は、GitHub の Web 機能をコマンドラインから操作するためのツールです。リポジトリの作成（`gh repo create`）、PR の作成・マージ（`gh pr create`、`gh pr merge`）、Issue の管理（`gh issue create`）、GitHub Actions の監視（`gh run list`）など、通常は GitHub の Web サイトで行う操作をターミナルから実行できます。

```
# === git コマンド（ローカルのバージョン管理） ===
git add .                    # ファイルをステージング
git commit -m "メッセージ"    # コミットを作成
git push                     # リモートにアップロード
git branch feature           # ブランチを作成
git merge feature            # ブランチをマージ
```

```
# === gh コマンド（GitHubのWeb機能をCLIから操作） ===
gh repo create                # GitHubにリポジトリを作成
gh pr create                  # Pull Requestを作成
gh pr merge                   # Pull Requestをマージ
gh issue create               # Issueを作成
gh run list                   # GitHub Actionsの実行一覧
gh release create             # リリースを作成
```

両方のコマンドを組み合わせることで、ブラウザを一度も開くことなく、コーディングからPR作成・マージまでの全てのワークフローをターミナル内で完結させることができます。

26.5.2. インストール

```
# macOS（Homebrewを使用）
brew install gh

# Windows（wingetを使用）
winget install --id GitHub.cli

# Linux（Debian/Ubuntu）
sudo apt install gh

# インストール後、認証を行う
gh auth login
```

`gh auth login` を実行すると、対話的に認証方式を選択するプロンプトが表示されます。ブラウザが開き、GitHubアカウントでログインすると認証が完了します。

26.5.3. よく使うコマンド集

```
# --- 認証 ---
gh auth login                          # GitHubにログイン
gh auth status                         # 認証状態を確認

# --- リポジトリ ---
gh repo create my-project --public     # 新しいリポジトリを作成
gh repo clone owner/repo               # リポジトリをクローン
gh repo view --web                     # ブラウザでリポジトリを開く
gh repo list                           # 自分のリポジトリ一覧
```

```
# --- Pull Request ---
gh pr create --title "タイトル"          # PRを作成
gh pr list                              # PRの一覧
gh pr view 42                           # PR #42の詳細を表示
gh pr checkout 42                       # PR #42をローカルにチェック
   アウト
gh pr merge 42 --squash                 # PR #42をスカッシュマージ
gh pr diff 42                           # PR #42の差分を表示
gh pr review 42 --approve               # PR #42を承認

# --- Issue ---
gh issue create                         # 対話的にIssueを作成
gh issue list --assignee "@me"          # 自分にアサインされたIssue
gh issue close 42                       # Issue #42をクローズ
gh issue view 42                        # Issue #42の詳細を表示

# --- GitHub Actions ---
gh run list                             # ワークフロー実行の一覧
gh run view RUN_ID --log                # ログを表示
gh run rerun RUN_ID                     # 再実行
gh run watch RUN_ID                     # リアルタイムで監視

# --- リリース ---
gh release create v1.0.0                # リリースを作成
gh release list                         # リリース一覧
```

26.5.4. エイリアス

よく使うコマンドには**エイリアス**（ショートカット）を設定できます。

```
# エイリアスの設定
gh alias set prc 'pr create'          # gh prc で gh pr create と同
   じ
gh alias set prl 'pr list'            # gh prl で gh pr list と同じ
gh alias set il 'issue list'          # gh il で gh issue list と同
   じ

# エイリアスの一覧
gh alias list

# 使用例
gh prc --title "新機能を追加"          # gh pr create --title "新機
```

```
    能を追加" と同じ
```

26.5.5. 向いている人

- **ターミナルで全てを完結させたい人**: ブラウザとターミナルを行き来するのが面倒な人
- **操作を自動化・スクリプト化したい人**: `gh` コマンドはシェルスクリプトに組み込める
- **効率重視のパワーユーザー**: キーボードだけで素早く操作したい人

26.6. GitHub Mobile

26.6.1. インストール

GitHub Mobile は、GitHub が公式に提供するスマートフォンアプリです。

- **iOS**: App Store で「GitHub」を検索してインストール
- **Android**: Google Play で「GitHub」を検索してインストール

スクリーンショット: *GitHub Mobile* のメイン画面

26.6.2. できること（通知/Issue/PR/コード閲覧）

GitHub Mobile は「外出先で GitHub の状況を確認し、簡単な対応を行う」ことに特化したアプリです。

- **プッシュ通知**: PR へのコメントやレビューリクエスト、Issue へのメンション、Actions の失敗など、重要な通知をスマートフォンのプッシュ通知としてリアルタイムに受け取れる
- **Issue/PR の確認とコメント**: Issue や PR の詳細を確認し、コメントを投稿できる。移動中にレビューコメントに返信したり、質問に答えたりするのに便利
- **PR のマージ・承認**: PR を Approve（承認）したりマージしたりすることもできる。急ぎの対応が必要なときに、PC を開けなくてもスマートフォンから操作できる
- **コードの閲覧**: リポジトリ内のファイルをスマートフォンで確認できる。シンタックスハイライトも適用される
- **コントリビューション草の確認**: プロフィールのコントリビューショングラフ（草）を確認できる

26.6.3. できないこと（編集/設定）

GitHub Mobile は閲覧と簡単な操作に特化しており、以下のことはできません。

- コードの編集やコミットの作成
- リポジトリの設定変更（ブランチ保護ルール、Webhook 設定など）
- GitHub Actions のワークフローファイルの編集
- Organization の詳細な管理

これらの操作が必要な場合は、PC で Web UI や CLI を使ってください。

26.6.4. 向いている人

- **通知をすぐにチェックしたい人**: レビューリクエストやメンションにすぐ気づきたい人
- **外出先でレビュー対応が必要な人**: 移動中にコメントへの返信や緊急のマージを行いたい人
- **コントリビューションの草を毎日チェックしたい人**: 継続のモチベーション維持に

26.7. IDE 拡張機能

26.7.1. VS Code 標準 Git 機能の解説

VS Code（Visual Studio Code）には、拡張機能を追加しなくても最初から Git 連携機能が組み込まれています。これは VS Code の大きな強みです。

ソース管理パネル（`Cmd+Shift+G` / `Ctrl+Shift+G`）は、Git 操作のための統合的なインターフェースです。変更されたファイルの一覧表示、ファイルごとの差分確認、ステージング（`+` ボタン）、コミットメッセージの入力とコミット作成、Push/Pull/Sync など、日常的な Git 操作のほとんどをこのパネルから行えます。

ステータスバー（画面最下部の青いバー）には、現在のブランチ名と同期状態が表示されます。ブランチ名をクリックするとブランチの切り替えや新規作成ができます。

差分表示は、変更されたファイルをクリックすると左右分割で表示されます。左側が変更前、右側が変更後の状態で、追加された行は緑色、削除された行は赤色で表示されます。

エディタのガター（行番号の左側の余白）には、変更のある行が色付きのバーで表示されます。緑は追加、青は変更、赤は削除を意味します。

スクリーンショット: *VS Code* のソース管理パネルと差分表示

26.7.2. おすすめ拡張機能

VS Code の標準機能に加えて、以下の拡張機能をインストールすると、Git/GitHub 連携がさらに充実します。

拡張機能	用途
GitHub Pull Requests and Issues	VS Code 内で PR の作成・レビュー・マージ、Issue の閲覧・作成
GitLens	各行の最終変更者・日時を表示（インライン blame）、ファイルの
GitHub Copilot	AI によるコード補完。コメントやコンテキストに基づいてコード
GitHub Copilot Chat	AI との対話形式でコーディングの相談やコード生成を行う
Git Graph	ブランチの分岐・マージの履歴をグラフとして視覚的に表示する
GitHub Actions	ワークフローの実行状態を VS Code 内で確認できる

26.7.3. Claude Code

Claude Code は VS Code の拡張機能としても、独立した CLI ツールとしても使用できます。VS Code 内のサイドパネルに Claude Code のインターフェースが表示され、コードを書きながら AI に作業を依頼したり、相談したりできます。

Claude Code の特徴は、リポジトリの全体を理解した上で作業を行えることです。単にコードの補完をするだけでなく、ファイルの読み書き、テストの実行、Git 操作（コミットや PR 作成）まで対話的に行えます。

```
Claude Codeの操作イメージ:
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│ VS Code                                                                                      │
│ ┌──────────────────────────────┐   ┌──────────────────────────────────────────────────────┐
│ │ エディタ                     │   │ Claude Code                                          │
│ │                              │   │                                                      │
│ │ def calc(x):                 │   │ > このバグを直して                                   │
│ │   return x * 2               │   │                                                      │
│ │                              │   │ 分析しました。                                       │
│ │                              │   │ x * 2 ではなく                                       │
│ │                              │   │ x ** 2 が正しい                                      │
│ │                              │   │ ですね。修正します。                                 │
│ └──────────────────────────────┘   └──────────────────────────────────────────────────────┘
└──────────────────────────────────────────────────────────────────────────────────────────────┘
```

26.7.4. JetBrains IDE の Git 連携

IntelliJ IDEA、PyCharm、WebStorm、GoLand などの JetBrains 製 IDE にも、Git/GitHub 連携機能が標準搭載されています。

- **VCS メニュー**: Git 操作全般（コミット、プッシュ、プル、ブランチ操作など）がメニューから実行できる

- **Git ツールウィンドウ**: コミットログ、ブランチ一覧、リモートリポジトリの状態を確認できる
- **GitHub 連携**: プラグイン不要で PR の作成・レビューが可能。GitHub 上のリポジトリを IDE から直接クローンすることもできる
- **3 ペインマージツール**: コンフリクトが発生したとき、「自分の変更」「相手の変更」「マージ結果」の 3 つのペインを並べて、どの変更を採用するかを GUI で選択できる。これは全 Git GUI ツールの中でも最も優れたコンフリクト解決 UI のひとつ

26.7.5. 向いている人

- **コーディングと Git 操作を同じ画面で行いたい人**: エディタとターミナルを切り替える手間がなくなる
- **PR レビューをコードの横で行いたい人**: コードを読みながらレビューコメントを書ける
- **AI 支援を活用したい人**: GitHub Copilot や Claude Code を使って開発効率を上げたい人

26.8. どれを使うべきか

26.8.1. 初心者の場合

GitHub を始めたばかりの人には、以下のステップで段階的にツールを増やしていくことをおすすめします。

```
ステップ1: GitHub Web UI でアカウント作成、リポジトリの基本操作を学ぶ
    ↓
ステップ2: GitHub Desktop をインストールし、GUIでGitの概念（コミット、ブランチ）を体験する
    ↓
ステップ3: VS Code のソース管理パネルに移行し、コーディングとGit操作を統合する
    ↓
ステップ4: git コマンドと gh コマンドを覚え、ターミナルでの操作に慣れる
```

最初からコマンドラインを使おうとする必要はありません。GUI ツールで Git の概念を「目で見て」理解してからコマンドラインに移行すると、各コマンドが何をしているのかを具体的にイメージできるため、学習がスムーズに進みます。

26.8.2. 開発者の日常

プロの開発者がどのようにツールを使い分けているかの典型例を紹介します。

```
メインの作業:          VS Code（コード編集 + Git操作 + AI支援）
PR/Issueの管理:        gh CLI（ターミナルで素早く操作）
リポジトリの設定:       Web UI（Settingsは Web UIでしかアクセスできない）
外出先での対応:         GitHub Mobile（通知確認、緊急のコメント返信）
自動化スクリプト:       gh CLI（シェルスクリプトに組み込む）
複雑なコンフリクト解決: VS Code or JetBrains IDE（GUIのマージツール）
```

ポイントは、「ひとつのツールに固執しない」ことです。それぞれのツールの強みを活かし、状況に応じて最適なツールを使い分けることが、効率的な開発ワークフローにつながります。

26.8.3. レベル別おすすめ組み合わせ

レベル	おすすめの組み合わせ	理由
入門	Web UI + GitHub Desktop	インストール最小限で始められる
初級	VS Code + Web UI	コーディングと Git 操作を統合できる
中級	VS Code + gh CLI + Web UI	ターミナルから PR/Issue 操作ができ、効率が上が
上級	ターミナル（git + gh）+ VS Code + Mobile	全ての操作をキーボードで完結

どのレベルでも **Web UI は常に使います**。これはリポジトリの設定や Organization の管理が Web UI でしかできないためです。レベルが上がるにつれて、より多くのツールを使いこなすようになりますが、それは「以前のツールを捨てる」のではなく「使えるツールの選択肢が増える」ということです。

26.9. まとめ

この章では、GitHub を操作する 5 つの方法 – Web UI、Desktop、CLI、Mobile、IDE 拡張機能 – を詳しく比較しました。

- **GitHub Web UI** は全機能にアクセスできる包括的なインターフェースであり、github.dev や Codespaces によるブラウザ上での開発も可能です。
- **GitHub Desktop** は GUI で Git 操作ができる初心者向けのツールであり、差分表示やコンフリクト解決を視覚的に行えます。他の Git GUI ツール（Sourcetree、GitKraken、Fork 等）も選択肢として存在します。
- **GitHub CLI (gh)** はターミナルから GitHub の Web 機能を操作するツールであり、`git` コマンド（ローカル操作）と `gh` コマンド（GitHub 操作）を組み合わせることで、ブラウザを開かずに全てのワークフローを完結できます。
- **GitHub Mobile** は外出先での通知確認や簡単な操作に特化したスマートフォンアプリです。

- **IDE 拡張機能**は VS Code や JetBrains IDE に Git/GitHub 連携を統合し、コーディングと Git 操作を同じ画面で行えるようにします。Claude Code や GitHub Copilot などの AI 支援ツールとの連携も可能です。

最も重要なメッセージは、**「正解はひとつではない」**ということです。自分の作業スタイルやスキルレベルに合わせて、複数のツールを組み合わせて使い分けることが、GitHub を快適に使いこなすコツです。

Gitコマンド早見表

この付録の使い方

本書で解説してきたGitコマンドを、カテゴリ別に一覧で掲載しています。「あのコマンド、何だったかな」と思ったときに素早く引ける辞書としてお使いください。各コマンドの詳しい解説は、対応する章を参照してください。

また、末尾には「やりたいこと」から逆引きできる表も用意しています。

A.1. 初期設定（→ 第2章）

Gitを使い始めるときに最初に行う設定です。`--global` は全リポジトリ共通の設定、省略するとそのリポジトリだけの設定になります。

```
# ユーザー名の設定（コミットに記録される名前）
git config --global user.name "あなたの名前"

# メールアドレスの設定（コミットに記録されるメールアドレス）
git config --global user.email "your-email@example.com"

# デフォルトブランチ名をmainに統一する
git config --global init.defaultBranch main

# コミットメッセージ編集時に使うエディタを指定する
git config --global core.editor "code --wait"

# 現在の設定を一覧表示する
git config --list

# 特定の設定項目の値を確認する
git config user.name

# 設定ファイルをエディタで直接開く
git config --global --edit
```

A.2. リポジトリの作成と取得（→ 第5章）

```
# 現在のディレクトリをGitリポジトリとして初期化する
git init

# リモートリポジトリの完全なコピーをローカルに作成する
```

```
git clone URL

# 最新のコミットだけを取得する（高速だが履歴が不完全）
git clone --depth 1 URL

# 特定のブランチを指定してクローンする
git clone -b ブランチ名 URL

# サブモジュールも含めてクローンする
git clone --recursive URL
```

A.3. 基本操作（→ 第6章）

日常的に最も多く使うコマンド群です。add → commit → push の流れが基本サイクルになります。

```
# 現在のリポジトリの状態を確認する（最も頻繁に使うコマンド）
git status

# 短縮形式で状態を確認する
git status -s

# 特定のファイルをステージングエリアに追加する
git add ファイル名

# 複数のファイルをまとめてステージングする
git add file1.py file2.py

# カレントディレクトリ以下の全変更をステージングする
git add .

# 削除されたファイルも含めて全変更をステージングする
git add -A

# 変更を部分的にステージングする（対話形式で選択）
git add -p

# ステージングした変更をコミットする（メッセージ付き）
git commit -m "コミットメッセージ"

# 複数行のコミットメッセージを書く
git commit -m "1行目の要約
```

```
詳細な説明をここに書く"

# エディタを開いてコミットメッセージを書く
git commit

# ステージングとコミットを同時に行う（追跡済みファイルのみ）
git commit -am "コミットメッセージ"

# 直前のコミットメッセージを修正する（まだpushしていない場合のみ）
git commit --amend -m "新しいメッセージ"

# ローカルのコミットをリモートリポジトリに送信する
git push

# 初回プッシュ時に上流ブランチを設定する
git push -u origin ブランチ名

# リモートの変更をローカルに取り込む（fetch + merge）
git pull

# リベース形式でリモートの変更を取り込む（履歴がきれいになる）
git pull --rebase
```

A.4. 差分と履歴の確認（→ 第6章、第9章）

```
# ワーキングディレクトリの変更（未ステージ分）を表示する
git diff

# ステージ済みの変更を表示する
git diff --staged

# 特定のファイルの差分だけ表示する
git diff ファイル名

# ブランチ間の差分を表示する
git diff main..feature-branch

# 変更されたファイル名の一覧だけを表示する
git diff --name-only

# コミット履歴を表示する
git log
```

```
# コミット履歴を1行ずつ表示する
git log --oneline

# ブランチの分岐を視覚的に表示する
git log --oneline --graph --all

# 最新N件だけ表示する
git log -5

# 変更内容も含めて表示する
git log -p

# 統計情報（変更行数）を表示する
git log --stat

# 特定ファイルの履歴を表示する
git log -- ファイル名

# 著者で絞り込む
git log --author="名前"

# メッセージで検索する
git log --grep="キーワード"

# ファイルの各行が誰によって変更されたかを表示する
git blame ファイル名

# 特定のコミットの詳細を表示する
git show コミットID
```

A.5. ブランチ操作（→ 第7章）

```
# ローカルブランチの一覧を表示する
git branch

# リモートブランチも含めて一覧表示する
git branch -a

# 新しいブランチを作成する（切り替えはしない）
git branch ブランチ名
```

```
# ブランチを作成して切り替える
git switch -c ブランチ名

# ブランチを作成して切り替える（従来の書き方）
git checkout -b ブランチ名

# 既存のブランチに切り替える
git switch ブランチ名

# 直前にいたブランチに戻る
git switch -

# マージ済みのブランチを削除する
git branch -d ブランチ名

# 未マージでも強制削除する
git branch -D ブランチ名

# ブランチ名を変更する
git branch -m 新しい名前

# 別のブランチの変更を現在のブランチに統合する
git merge ブランチ名

# マージを中止する
git merge --abort

# 現在のブランチの分岐点を移動させる
git rebase ブランチ名

# リベースを中止する
git rebase --abort

# 特定のコミットだけを現在のブランチに取り込む
git cherry-pick コミットID
```

A.6. リモート操作（→ 第5章、第6章）

```
# 登録されているリモートリポジトリの一覧とURLを表示する
git remote -v

# リモートリポジトリを追加する
```

```
git remote add origin URL

# リモートリポジトリのURLを変更する
git remote set-url origin 新しいURL

# リモートリポジトリの登録を削除する
git remote remove origin

# リモートの最新情報を取得する（ローカルファイルは変更されない）
git fetch

# 削除されたリモートブランチの参照をクリーンアップする
git fetch --prune

# 特定のブランチをリモートにプッシュする
git push origin ブランチ名

# リモートブランチを削除する
git push origin --delete ブランチ名

# すべてのタグをリモートにプッシュする
git push --tags
```

A.7. 変更の取り消しと復元（→ 第9章）

```
# ファイルの変更を元に戻す（未ステージの変更を破棄）
git restore ファイル名

# ステージングを解除する（変更内容は保持される）
git restore --staged ファイル名

# コミットを安全に打ち消す（新しいコミットが作られる）
git revert コミットID

# 直前のコミットを取り消す（変更はステージに残る）
git reset --soft HEAD~1

# 直前のコミットとステージングを取り消す（変更はワーキングツリーに
  残る）
git reset HEAD~1

# 直前のコミットを完全に取り消す（変更が消える。要注意）
```

```
git reset --hard HEAD~1

# HEADの移動履歴を表示する（消えたコミットの復旧に使う）
git reflog
```

A.8. 一時退避（→ 第9章）

```
# 現在の変更を一時退避する
git stash

# メッセージ付きで退避する
git stash save "作業メモ"

# 未追跡ファイルも含めて退避する
git stash -u

# 退避の一覧を表示する
git stash list

# 最新の退避を復元する（退避リストからは削除される）
git stash pop

# 最新の退避を復元する（退避リストに残す）
git stash apply

# 特定の退避を復元する
git stash apply stash@{1}

# 特定の退避を削除する
git stash drop stash@{0}

# すべての退避を削除する
git stash clear
```

A.9. タグ（→ 第14章）

```
# タグの一覧を表示する
git tag

# 注釈付きタグを作成する（リリースバージョンに推奨）
```

```
git tag -a v1.0.0 -m "バージョン1.0.0"

# 軽量タグを作成する
git tag v1.0.0

# 特定のコミットにタグを付ける
git tag -a v1.0.0 -m "メッセージ" コミットID

# タグを削除する
git tag -d v1.0.0

# タグをリモートにプッシュする
git push origin v1.0.0

# すべてのタグをプッシュする
git push origin --tags

# リモートのタグを削除する
git push origin --delete v1.0.0
```

A.10. ファイル操作（→ 第6章）

```
# Gitの管理下でファイル名を変更する
git mv 旧ファイル名 新ファイル名

# Gitの追跡対象からファイルを削除する（ファイル自体も消える）
git rm ファイル名

# Gitの追跡だけを解除する（ファイルは残る）
git rm --cached ファイル名
```

A.11. GitHub CLI（gh）コマンド（→ 第23章）

GitHub CLI は、GitHub 固有の操作（PR、Issue、Actions、リリースなど）をターミナルから行うためのツールです。`git` コマンドがローカルのバージョン管理に使われるのに対し、`gh` コマンドは GitHub プラットフォームとの対話に使われます。

```
# 認証
gh auth login                          # GitHubにログインする
gh auth status                         # 認証状態を確認する
```

```
# リポジトリ
gh repo create 名前 --public            # リポジトリを作成する
gh repo clone owner/repo                # リポジトリをクローンする
gh repo list                            # 所有リポジトリの一覧を表示
   する
gh repo view --web                      # ブラウザでリポジトリを開く

# プルリクエスト
gh pr create --title "タイトル"          # PRを作成する
gh pr list                              # PRの一覧を表示する
gh pr view 番号                          # PRの詳細を表示する
gh pr checkout 番号                      # PRのブランチをローカルに
   取得する
gh pr merge 番号 --squash                # PRをスカッシュマージする
gh pr diff 番号                          # PRの差分を表示する

# Issue
gh issue create --title "タイトル"       # Issueを作成する
gh issue list                           # Issueの一覧を表示する
gh issue view 番号                       # Issueの詳細を表示する
gh issue close 番号                      # Issueをクローズする

# リリース
gh release create v1.0.0                # リリースを作成する
gh release list                         # リリースの一覧を表示する

# GitHub Actions
gh run list                             # ワークフロー実行の一覧を表
   示する
gh run view ID                          # 実行の詳細を表示する
gh run rerun ID                         # 失敗した実行を再実行する

# エイリアス
gh alias set prc 'pr create'            # ショートカットを設定する
gh alias list                           # 設定済みエイリアスの一覧を
   表示する
```

A.12. 逆引きリファレンス —「やりたいこと」から探す

日々の作業でよくある「こうしたい」という場面から、使うべきコマンドを逆引きできる表です。

やりたいこと	コマンド
ファイルの変更を元に戻したい	`git restore ファイル名`
ステージングを取り消したい	`git restore --staged ファイル名`
直前のコミットを取り消したい（変更は残す）	`git reset --soft HEAD~1`
特定のコミットの影響を打ち消したい	`git revert コミット ID`
今の作業を一旦脇に置きたい	`git stash`
退避した作業を元に戻したい	`git stash pop`
消してしまったコミットを復元したい	`git reflog` で履歴を探す
別ブランチの特定コミットだけ取り込みたい	`git cherry-pick コミット ID`
ファイルの各行を誰が変更したか知りたい	`git blame ファイル名`
リモートリポジトリの URL を変更したい	`git remote set-url origin URL`
`.gitignore` に追加したのに効かない	`git rm --cached ファイル名`
特定ファイルの変更履歴だけ見たい	`git log -- ファイル名`
間違ったブランチにコミットしてしまった	`git reset --soft HEAD~1` → `git stash` → 正しいブラン
ブランチ間の差分を確認したい	`git diff main..feature`
リモートの最新状態だけ確認したい	`git fetch`
PR を作りたい	`gh pr create`
Issue を作りたい	`gh issue create`
ワークフローの結果を確認したい	`gh run list`

用語集

この付録の使い方

本書に登場する Git・GitHub 関連の用語をアルファベット順に収録しています。本文を読んでいて知らない言葉に出会ったとき、あるいは用語の正確な意味を確認したいときにお使いください。各用語には、日本語の読みと、本書で解説している章への参照を記載しています。

A

Add（アド）

ファイルの変更をステージングエリアに追加する操作。`git add` コマンドで実行する。ステージングエリアに追加されたファイルだけが、次のコミットに含まれる。「コミットに入れる候補を選ぶ」作業だと考えるとわかりやすい。→ 第 6 章

Amend（アメンド）

直前のコミットを修正する操作。`git commit --amend` で実行する。コミットメッセージの修正や、追加し忘れたファイルを含めるために使う。ただし、既に push 済みのコミットに対して使うと履歴の不整合が生じるため、ローカルのコミットに対してのみ使うのが原則。→ 第 6 章

Artifact（アーティファクト）

GitHub Actions のワークフロー実行中に生成・保存されるファイルのこと。テスト結果のレポートやビルドされたバイナリなどが該当する。ワークフロー間でデータを受け渡すためにも使われる。→ 第 12 章

Assignee（アサイニー）

Issue や Pull Request の「担当者」として割り当てられた人。誰がその作業に責任を持っているのかを明示するために使う。→ 第 11 章

B

Base branch（ベースブランチ）

Pull Request において、変更のマージ先となるブランチ。多くの場合は `main` や `develop` がこれに当たる。PR の作成画面では「base」として表示される。→ 第 10 章

Blame（ブレイム）

ファイルの各行について、いつ・誰が・どのコミットで最後に変更したかを表示する機能。`git blame` コマンドで実行する。「犯人探し」と訳されることもあるが、実際にはコードの経緯を追跡するための正当な開発ツール。→ 第 9 章

Blob（ブロブ）

Git 内部でファイルの内容を表すオブジェクト。”Binary Large Object” の略。GitHub 上の URL で `/blob/` はファイルの表示を意味する（例: `github.com/owner/repo/blob/main/README.md`）。→ 第 19 章

Branch（ブランチ）

開発の流れを分岐させる仕組み。内部的には特定のコミットを指し示す軽量なポインタに過ぎないが、概念的には「独立した作業ライン」を表す。main ブランチに影響を与えずに新機能の開発やバグ修正を進められる。→ 第 7 章

C

Cherry-pick（チェリーピック）

特定のコミットだけを現在のブランチに取り込む操作。`git cherry-pick コミット ID` で実行する。別のブランチで行われた有用な変更を、ブランチ全体をマージせずに部分的に取り込みたいときに使う。→ 第 9 章

CI/CD（シーアイ/シーディー）

Continuous Integration（継続的統合）/ Continuous Deployment（継続的デプロイ）の略。コードの変更がリポジトリに push されるたびに、自動でテストを実行し（CI）、テストが通れば自動で本番環境にデプロイする（CD）一連の仕組み。→ 第 12 章

Clone（クローン）

リモートリポジトリの完全なコピーをローカルに作成する操作。`git clone` で実行する。ファイルだけでなく、全てのコミット履歴・ブランチ情報も含めてダウンロードされる。→ 第 5 章

CODEOWNERS（コードオーナーズ）

リポジトリ内のファイルやディレクトリの「所有者」（レビュー責任者）を定義するファイル。`.github/CODEOWNERS` に配置する。PR が作成されたとき、変更されたファイルの所有者が自動的にレビュアーとして追加される。→ 第 16 章

Collaborator（コラボレーター）

リポジトリに対する write（書き込み）権限を付与された共同作業者。リポジトリの Settings から招待できる。→ 第 5 章

Commit（コミット）

ファイルの変更を履歴に記録する操作、およびその記録自体を指す。著者・日時・変更内容・コミットメッセージ・一意のハッシュ値で構成される。ゲームの「セーブポイント」に例えられることが多い。→ 第 6 章

Conflict（コンフリクト）

2 つのブランチで同じファイルの同じ箇所が異なる変更をされたとき、Git が自動マージできずに人間の判断を求める状態。エラーではなく、正常な開発プロセスの一部。→ 第 8 章

Contributor（コントリビューター）

プロジェクトに何らかの貢献をした人。コードの提出、Issue 報告、ドキュメント改善、翻訳など、貢献の形は多岐にわたる。→ 第 15 章

D

Dependabot（ディペンダボット）

GitHub が提供する、依存パッケージの脆弱性を自動検出し、更新用の Pull Request を自動生成してくれるボット。→ 第 22 章

Deploy（デプロイ）

アプリケーションやサービスを本番環境に配置し、ユーザーが利用可能な状態にすること。GitHub Actions を使って自動化されることが多い。→ 第 12 章

Diff（ディフ）

2 つの状態（ファイル、コミット、ブランチ）の間の差分。追加された行は緑色、削除された行は赤色で表示されるのが一般的。→ 第 6 章

Draft PR（ドラフト PR）

まだレビュー準備が整っていないことを示す Pull Request。マージボタンが無効化され、レビュアーに「まだ見なくていいよ」というシグナルを送る。→ 第 10 章

F

Fast-forward（ファストフォワード）

マージの一種で、新しいマージコミットを作らずに、ブランチのポインタを前方に進めるだけで済むマージ方式。分岐先のブランチが分岐元から一直線に進んでいる場合にのみ可能。→ 第 7 章

Fetch（フェッチ）

リモートリポジトリの最新情報をローカルにダウンロードする操作。ただしローカルの作業ファイルには変更を加えない。「情報だけ取り寄せて、反映はまだしない」という安全な操作。→ 第 6 章

Fork（フォーク）

GitHub 上で他のユーザーのリポジトリを自分のアカウントにコピーする操作。Git 自体の機能ではなく、GitHub が提供する Web サービスの機能。オープンソースへの貢献の第一歩。→ 第 15 章

G

Git（ギット）

2005 年に Linus Torvalds が開発した分散型バージョン管理システム。世界の開発者の 95%以上が使用する事実上の標準。→ 第 1 章

GitHub（ギットハブ）

Git リポジトリのホスティングサービスおよび開発プラットフォーム。2008 年設立、2018 年に Microsoft が買収。2025 年現在、1 億 5,000 万人以上の開発者が利用。→ 第 1 章

GitHub Actions（ギットハブアクションズ）

GitHub に組み込まれた CI/CD プラットフォーム。YAML ファイルでワークフローを定義し、push や PR 作成などのイベントをトリガーにテスト・ビルド・デプロイを自動化する。→ 第 12 章

GitHub Pages（ギットハブページズ）

GitHub リポジトリから直接、静的 Web サイトを無料でホスティングできるサービス。ポートフォリオやプロジェクトドキュメントの公開に広く使われる。→ 第 13 章

.gitignore（ギットイグノア）

Git に追跡させたくないファイルやフォルダのパターンを記述するファイル。ビルド成果物、一時ファイル、秘密情報（.env）などを無視対象にする。→ 第 5 章

GPG（ジーピージー）

GNU Privacy Guard。暗号化と電子署名のためのソフトウェア。Git では、コミットが本当にその人物によるものであることを暗号的に証明するために使われる。署名されたコミットは GitHub 上で「Verified」バッジが表示される。→ 第 4 章

H

Hash（ハッシュ）

コミットを一意に識別するための文字列。SHA-1 アルゴリズムで生成される 40 文字の 16 進数（例: `a1b2c3d4e5f6...`）。同じ内容からは必ず同じハッシュ値が生成されるため、データの完全性を保証する仕組みでもある。→ 第 1 章

HEAD（ヘッド）

現在チェックアウトしているブランチの最新コミットを指すポインタ。`HEAD~1` は「HEAD の 1 つ前のコミット」を意味する。→ 第 7 章

Hook（フック）

特定の Git イベント（コミットの前後など）で自動的に実行されるスクリプト。`.git/hooks/` ディレクトリに配置する。コード品質チェックやコミットメッセージの検証などに使われる。→ 第 5 章

I

Index（インデックス）

ステージングエリアの別名。次のコミットに含める変更が一時的に保管される場所。→ 第 6 章

Issue（イシュー）

GitHub が提供するチケット管理システム。バグ報告、機能要望、タスク管理、議論など、プロジェクトに関するあらゆる「やるべきこと」を管理する。→ 第 11 章

L

Label（ラベル）

Issue や Pull Request に付ける色付きのカテゴリタグ。`bug`、`enhancement`、`priority:high` など。Issue のフィルタリングや分類に使う。→ 第 11 章

LFS（エルエフエス）

Git Large File Storage。大きなファイル（画像、動画、データセットなど）を効率的に管理するための Git 拡張機能。ファイル本体の代わりにポインタを Git で管理し、実体は別サーバーに保管する。→ 第 19 章

License（ライセンス）

ソフトウェアの利用・改変・再配布の条件を定めた法的文書。MIT、Apache 2.0、GPL v3 などが代表的。GitHub ではリポジトリ作成時にテンプレートから選択できる。→ 第 20 章

M

Main（メイン）

リポジトリのデフォルト（既定）ブランチの名前。2020 年以前は `master` が主流だったが、現在は `main` が業界標準。→ 第 2 章

Markdown（マークダウン）

テキストに簡単な記号を使って書式を付けられる軽量マークアップ言語。GitHub では GitHub Flavored Markdown（GFM）という拡張版が使われ、テーブル、タスクリスト、コードブロック、Mermaid 図などに対応している。→ 第 20 章

Merge（マージ）

あるブランチの変更を別のブランチに統合する操作。Fast-forward マージと 3-way マージ（マージコミットが作られる）の 2 種類がある。→ 第 7 章

Milestone（マイルストーン）

Issue をリリースや Sprint などの大きな目標でグループ化する機能。含まれる Issue の完了率がパーセンテージで表示される。→ 第 11 章

O

Open Source（オープンソース）

ソースコードが公開されており、誰でも自由に閲覧・利用・改変・再配布できるソフトウェア。Linux、Python、Git 自体もオープンソース。→ 第 1 章

Organization（オーガニゼーション）

GitHub における「組織」アカウント。チーム単位でリポジトリを管理し、メンバーの権限を細かく制御できる。→ 第 16 章

Origin（オリジン）

`git clone` でリポジトリを取得したとき、クローン元のリモートリポジトリに自動的に付けられる名前。Git の慣習であり、設定で変更も可能だが、ほぼ全てのプロジェクトでこの名前が使われる。→ 第 5 章

P

PAT（パット）

Personal Access Token。GitHub への HTTPS 認証に使用するトークン。パスワードの代わりとして使う。有効期限やスコープ（権限範囲）を設定できる。→ 第 4 章

PR / Pull Request（プルリクエスト）

ブランチの変更をメインブランチに取り込むことを「提案」する機能。コードレビューの場としても機能し、チーム開発の中核を担う。GitHub が洗練した仕組みで、Git 自体にはない概念。→ 第 10 章

Pull（プル）

リモートリポジトリの変更をローカルに取り込む操作。内部的には `git fetch`（情報取得）＋ `git merge`（統合）を連続で実行している。→ 第 6 章

Push（プッシュ）

ローカルのコミットをリモートリポジトリに送信する操作。push することで、他の開発者や GitHub 上から変更が見えるようになる。→ 第 6 章

R

Raw（ロー）

GitHub でファイルの「生データ」を取得するための URL 形式。`raw.githubusercontent.com` 経由でアクセスし、HTML レンダリングなしのファイル内容をそのまま取得できる。→ 第 19 章

Rebase（リベース）

ブランチの分岐点を移動させ、コミット履歴を直線的に整理する操作。コミットが「作り直される」ため、push 済みのブランチに対しては原則として使わない。→ 第 7 章

Reflog（リフログ）

HEAD の移動履歴を記録する Git の内部ログ。`git reset --hard` で消えたように見えるコミットも、reflog を辿れば復元できる場合がある。ローカル専用で、デフォルトでは 90 日間保持される。→ 第 9 章

Release（リリース）

タグに加えてリリースノート（変更履歴）やバイナリファイルを添付できる GitHub の機能。ソフトウェアのバージョンを正式に公開する際に使う。→ 第 14 章

Remote（リモート）

ネットワーク上にある別のリポジトリへの参照。`origin`（自分のリポジトリ）や `upstream`（Fork 元のリポジトリ）がよく使われる名前。→ 第 5 章

Repository（リポジトリ）

プロジェクトのファイル群とその全変更履歴を保存する場所。「repo」と略されることが多い。ローカルリポジトリ（自分の PC）とリモートリポジトリ（GitHub 等）がある。→ 第 5 章

Reset（リセット）

コミットやステージングを取り消す操作。`--soft`（コミットのみ取り消し）、`--mixed`（コミット+ステージ取り消し）、`--hard`（全て取り消し）の 3 モードがある。→ 第 9 章

Revert（リバート）

指定したコミットの変更を打ち消す新しいコミットを作成する操作。履歴を書き換えないため、push 済みのコミットにも安全に使える。→ 第 9 章

Review（レビュー）

Pull Request の変更内容を確認し、フィードバックを提供するプロセス。Comment（コメント）、Approve（承認）、Request changes（修正要求）の 3 種類がある。→ 第 17 章

Runner（ランナー）

GitHub Actions のワークフローを実行するサーバー（仮想マシン）。GitHub が提供するホステッドランナー（ubuntu-latest 等）と、自前で用意するセルフホステッドランナーがある。→ 第 12 章

S

Secret（シークレット）

GitHub Actions のワークフローで使用する秘密情報（API キー、パスワードなど）。暗号化されて保管され、ログに出力されないよう保護されている。→ 第 12 章、第 22 章

SHA（エスエイチエー）

Secure Hash Algorithm。Git がコミットやファイル内容を一意に識別するために使用するハッシュアルゴリズム。→ 第 1 章

SSH（エスエスエイチ）

Secure Shell。暗号化された通信プロトコル。GitHub との認証に公開鍵暗号方式で使用される。パスワード不要で安全な接続を実現する。→ 第 4 章

Squash（スカッシュ）

複数のコミットを 1 つにまとめる操作。Pull Request のマージ時に「Squash and merge」を選択すると、ブランチ内の全コミットが 1 つのコミットに集約されてマージされる。→ 第 10 章

Stage / Staging（ステージ / ステージング）

コミットに含めるファイルの変更を選択・準備する操作およびその領域。「ステージングエリア」は「インデックス」とも呼ばれる。→ 第 6 章

Stash（スタッシュ）

未コミットの変更を一時的に退避させる機能。作業途中で別のブランチに切り替える必要が生じたときなどに使う。→ 第 9 章

Submodule（サブモジュール）

Git リポジトリの内部に、別の Git リポジトリを埋め込む機能。共有ライブラリや共通コンポーネントを複数のプロジェクトで参照するために使う。→ 第 19 章

T

Tag（タグ）

特定のコミットに付ける名前。主にバージョン番号（`v1.0.0`）を付けるために使われる。軽量タグ（ポインタのみ）と注釈付きタグ（メッセージ・署名付き）の 2 種類がある。→ 第 14 章

Tree（ツリー）

Git 内部でディレクトリ構造を表すオブジェクト。GitHub 上の URL で `/tree/` はフォルダの表示を意味する。→ 第 19 章

U

Upstream（アップストリーム）

文脈によって 2 つの意味がある。(1) Fork 元のリモートリポジトリを指すリモート名として使う（`git remote add upstream URL`）。(2) ローカルブランチが追跡しているリモートブランチを指す（`git push -u` で設定する「上流ブランチ」）。→ 第 5 章、第 15 章

V

Verified（ベリファイド）

GPG 署名されたコミットに表示される緑色のバッジ。そのコミットが記載されている作者本人によるものであることを暗号的に証明している。→ 第 4 章

W

Webhook（ウェブフック）

特定のイベント（push、PR 作成など）が発生した際に、GitHub から外部の URL へ HTTP リクエストを自動送信する仕組み。外部サービスとの連携に使う。→ 第 23 章

Wiki（ウィキ）

リポジトリに付属するドキュメントページ群。Markdown で記述でき、通常の Git リポジトリとしてクローン・編集も可能。長いドキュメントの整理に適している。→ 第 20 章

Workflow（ワークフロー）

GitHub Actions における自動化処理の定義。`.github/workflows/` ディレクトリに YAML 形式のファイルとして配置する。→ 第 12 章

Working Directory / Working Tree（ワーキングディレクトリ / ワーキングツリー）

実際にファイルを編集する作業場所。リポジトリ内の `.git` フォルダを除いたすべてのファイル群。ステージングエリアに追加されるまで、ここでの変更は Git に記録されない。→ 第 6 章

GitHub完全ガイド

「コードを書いたことはあるけれど、Gitって何？」という方から、チームで本格的に開発を始めたい方まで。本書を一冊読み通せば、GitとGitHubの全体像が手に取るように分かります。